ZlibValidation

Generated on Sat Apr 5 2025 16:35:27 for ZlibValidation by Doxygen 1.9.5

Sat Apr 5 2025 16:35:27

1 ZlibValidation **1**

1.1 Description 1

1.2 Usage 1

1.3 Development Diary 1

1.3.1 2025-01-27 1

1.3.2 2025-01-28 2

1.3.3 2025-02-01 2

1.3.4 2025-02-10 2

1.3.5 2025-02-11 2

1.3.6 2025-02-14 3

1.3.7 2025-02-15 3

1.3.8 2025-02-17 3

1.3.9 2025-02-18 3

1.3.10 2025-02-19 3

1.3.11 2025-02-20 4

1.3.12 2025-02-25 4

1.3.13 2025-02-26 4

1.3.14 2025-02-27 4

1.3.15 2025-02-28 4

1.3.16 2025-03-01 5

1.3.17 2025-03-07 5

1.3.18 2025-03-10 5

1.3.19 2025-03-14 5

1.3.20 2025-03-15 6

1.3.21 2025-03-16 6

1.3.22 2025-03-17 7

1.3.23 2025-03-18 7

1.3.24 2025-03-19 8

1.3.25 2025-03-21 8

1.3.26 2025-03-24 8

1.3.27 2025-03-25 8

1.3.28 2025-03-26 9

1.3.29 2025-03-27 9

1.3.30 2025-03-28 10
1.3.31 2025-03-29 11
1.3.32 2025-03-30 11
1.3.33 2025-03-31 12
1.3.34 2025-04-01 13
1.3.35 2025-04-02 15
1.3.36 2025-04-05 16

2 Hierarchical Index 17

2.1 Class Hierarchy 17

3 Class Index 19

3.1 Class List 19

4 File Index 21

4.1 File List 21

5 Class Documentation 23

5.1 AttributesIterator Class Reference 23
5.1.1 Detailed Description 23
5.1.2 Constructor & Destructor Documentation 23
5.1.2.1 AttributesIterator() 24
5.1.2.2 ∼AttributesIterator() 24
5.1.3 Member Function Documentation 24
5.1.3.1 end() 24
5.1.3.2 get() 25
5.1.3.3 next() 25
5.1.4 Member Data Documentation 25
5.1.4.1 attr_ 25
5.1.4.2 attrs_ 26
5.1.4.3 err_ 26
5.2 CellExtractor Class Reference 26
5.2.1 Detailed Description 27
5.2.2 Constructor & Destructor Documentation 27
5.2.2.1 CellExtractor() 27

5.2.3 Member Function Documentation 28
5.2.3.1 foundTargetCell() 28
5.2.3.2 handle() 28
5.2.4 Member Data Documentation 28
5.2.4.1 foundTarget_ 28
5.2.4.2 targetCell_ 29
5.3 CellPrinter Class Reference 29
5.3.1 Detailed Description 30
5.3.2 Constructor & Destructor Documentation 30
5.3.2.1 CellPrinter() 30
5.3.3 Member Function Documentation 30
5.3.3.1 handle() 30
5.3.4 Member Data Documentation 31
5.3.4.1 foundTarget_ 31
5.3.4.2 out_ 31
5.3.4.3 targetCell_ 31
5.4 GateInfo Struct Reference 31
5.4.1 Detailed Description 32
5.4.2 Member Data Documentation 32
5.4.2.1 gateTypeName 32
5.4.2.2 inputSignals 32
5.4.2.3 kind 32
5.4.2.4 outputSignal 32
5.5 GroupsIterator Class Reference 33
5.5.1 Detailed Description 33
5.5.2 Constructor & Destructor Documentation 33
5.5.2.1 GroupsIterator() 33
5.5.2.2 ∼GroupsIterator() 33
5.5.3 Member Function Documentation 34
5.5.3.1 end() 34
5.5.3.2 get() 34
5.5.3.3 next() 35
5.5.4 Member Data Documentation 35

5.5.4.1 err_ 35
5.5.4.2 group_ 35
5.5.4.3 groups_ 35
5.6 LibAttribute Class Reference 36
5.6.1 Detailed Description 36
5.6.2 Constructor & Destructor Documentation 36
5.6.2.1 LibAttribute() 36
5.6.2.2 ∼LibAttribute() 37
5.6.3 Member Function Documentation 37
5.6.3.1 getBoolean() 37
5.6.3.2 getFloat() 37
5.6.3.3 getInt() 38
5.6.3.4 getName() 38
5.6.3.5 getString() 39
5.6.3.6 getValues() 39
5.6.3.7 isComplex() 39
5.6.4 Member Data Documentation 40
5.6.4.1 attr_ 40
5.6.4.2 err_ 40
5.7 LibFile Class Reference 40
5.7.1 Detailed Description 42
5.7.2 Constructor & Destructor Documentation 42
5.7.2.1 LibFile() 42
5.7.2.2 ∼LibFile() 42
5.7.3 Member Function Documentation 43
5.7.3.1 checkTimingArcMonotonicity() 43
5.7.3.2 generateRCLines() 44
5.7.3.3 logic() 45
5.7.3.4 modify() 47
5.7.3.5 modifySpiceNetlist() 47
5.7.3.6 mono() 49
5.7.3.7 parse() 51
5.7.3.8 read() 53

5.7.3.9 spice() 54
5.7.3.10 splitString() 56
5.7.3.11 supercell() 57
5.7.3.12 verilog() 59
5.7.3.13 writeJsonToFile() 61
5.7.4 Member Data Documentation 61
5.7.4.1 basename_ 61
5.7.4.2 err_ 62
5.7.4.3 filename_ 62
5.7.4.4 filepath_ 62
5.7.4.5 jsonname_ 62
5.7.4.6 lib_json_ 62
5.7.4.7 libname_ 63
5.7.4.8 logger_ 63
5.7.4.9 loggername_ 63
5.7.4.10 process_ 63
5.7.4.11 temperature_ 63
5.7.4.12 voltage_ 64
5.8 LibGroup Class Reference 64
5.8.1 Detailed Description 64
5.8.2 Constructor & Destructor Documentation 64
5.8.2.1 LibGroup() 65
5.8.2.2 ~LibGroup() 65
5.8.3 Member Function Documentation 65
5.8.3.1 getAttrs() 65
5.8.3.2 getGroups() 66
5.8.3.3 getName() 66
5.8.3.4 getType() 67
5.8.4 Member Data Documentation 67
5.8.4.1 err_ 67
5.8.4.2 group_ 67
5.9 LibraryComparator Class Reference 67
5.9.1 Detailed Description 68

5.9.2 Constructor & Destructor Documentation 69
5.9.2.1 LibraryComparator() 69
5.9.3 Member Function Documentation 70
5.9.3.1 compareCell() 70
5.9.3.2 compareLut() 71
5.9.3.3 comparePin() 73
5.9.3.4 compareTimingArc() 74
5.9.3.5 generateReport() 75
5.9.4 Member Data Documentation 76
5.9.4.1 abstol_ 76
5.9.4.2 comp_json_ 76
5.9.4.3 comp_lib_path_ 76
5.9.4.4 ref_json_ 76
5.9.4.5 ref_lib_path_ 77
5.9.4.6 reltol_ 77
5.10 LogicComparator Class Reference 77
5.10.1 Detailed Description 78
5.10.2 Constructor & Destructor Documentation 78
5.10.2.1 LogicComparator() 78
5.10.3 Member Function Documentation 78
5.10.3.1 compareCellLogic() 78
5.10.3.2 compareSingleExpressionPair() 80
5.10.3.3 extractVariables() 82
5.10.3.4 generateReport() 83
5.10.3.5 logic() 85
5.10.3.6 preprocessExpression() 86
5.10.4 Member Data Documentation 87
5.10.4.1 all_pin_results_ 87
5.10.4.2 cell_name_ 87
5.10.4.3 comp_outpin_map_ 88
5.10.4.4 ref_outpin_map_ 88
5.11 LogicExtractor Class Reference 88
5.11.1 Detailed Description 90

5.11.2 Constructor & Destructor Documentation 90
5.11.2.1 LogicExtractor() 90
5.11.3 Member Function Documentation 90
5.11.3.1 deriveLogicRecursive() 90
5.11.3.2 formatExpression() 92
5.11.3.3 getExtractedGates() 93
5.11.3.4 getInternalWires() 93
5.11.3.5 getLogicExpressions() 93
5.11.3.6 getPrimaryInputs() 94
5.11.3.7 getPrimaryOutputs() 95
5.11.3.8 handle() [1/5] 95
5.11.3.9 handle() [2/5] 95
5.11.3.10 handle() [3/5] 96
5.11.3.11 handle() [4/5] 97
5.11.3.12 handle() [5/5] 98
5.11.4 Member Data Documentation 98
5.11.4.1 gateOutputDrivers_ 98
5.11.4.2 inTargetModule_ 99
5.11.4.3 internalWires_ 99
5.11.4.4 logicCache_ 99
5.11.4.5 parsingComplete_ 99
5.11.4.6 portDirections_ 99
5.11.4.7 primaryInputs_ 100
5.11.4.8 primaryOutputs_ 100
5.11.4.9 targetCell_ 100
5.12 ModuleRewriter Class Reference 100
5.12.1 Detailed Description 101
5.12.2 Constructor & Destructor Documentation 102
5.12.2.1 ModuleRewriter() 102
5.12.3 Member Function Documentation 102
5.12.3.1 handle() [1/2] 102
5.12.3.2 handle() [2/2] 102
5.12.4 Member Data Documentation 103

5.12.4.1 cellName_ 103
5.12.4.2 depth_ 103
5.12.4.3 inputPins_ 103
5.12.4.4 instance_count_ 103
5.12.4.5 logger_ 104
5.12.4.6 moduleName_ 104
5.12.4.7 outputPins_ 104
5.12.4.8 portInfoMap_ 104
5.13 PinComparisonResult Struct Reference 104
5.13.1 Detailed Description 105
5.13.2 Member Data Documentation 105
5.13.2.1 are_equivalent 105
5.13.2.2 comp_compiles 105
5.13.2.3 comp_expr_processed 106
5.13.2.4 comp_expr_raw 106
5.13.2.5 comp_truth_table 106
5.13.2.6 comparison_possible 106
5.13.2.7 error_message 106
5.13.2.8 pin_name 107
5.13.2.9 ref_compiles 107
5.13.2.10 ref_expr_processed 107
5.13.2.11 ref_expr_raw 107
5.13.2.12 ref_truth_table 107
5.14 ValuesIterator Class Reference 108
5.14.1 Detailed Description 108
5.14.2 Constructor & Destructor Documentation 108
5.14.2.1 ValuesIterator() 108
5.14.2.2 ∼ValuesIterator() 109
5.14.3 Member Function Documentation 109
5.14.3.1 end() 109
5.14.3.2 next() 109
5.14.4 Member Data Documentation 110
5.14.4.1 bool_ 110

5.14.4.2 err_ 110
5.14.4.3 exprp_ 110
5.14.4.4 float_ 110
5.14.4.5 int_ 110
5.14.4.6 str_ 111
5.14.4.7 values_ 111
5.14.4.8 vtype_ 111
5.15 VerilogVisitor Class Reference 111
5.15.1 Detailed Description 112
5.15.2 Constructor & Destructor Documentation 112
5.15.2.1 VerilogVisitor() 113
5.15.3 Member Function Documentation 113
5.15.3.1 handle() [1/5] 113
5.15.3.2 handle() [2/5] 113
5.15.3.3 handle() [3/5] 113
5.15.3.4 handle() [4/5] 114
5.15.3.5 handle() [5/5] 114
5.15.4 Member Data Documentation 114
5.15.4.1 depth_ 115
5.15.4.2 inTargetModule_ 115
5.15.4.3 targetCell_ 115

6 File Documentation **117**
6.1 include/Iterators.hpp File Reference 117
6.2 Iterators.hpp 118
6.3 include/json_utils.hpp File Reference 119
6.3.1 Typedef Documentation 120
6.3.1.1 json 120
6.3.2 Function Documentation 121
6.3.2.1 generateCellJson() 121
6.3.2.2 generateLutJson() 123
6.3.2.3 generatePinJson() 124
6.3.2.4 generatePowerJson() 127
6.4 json_utils.hpp 128

6.5 include/LibAttribute.hpp File Reference 129
6.6 LibAttribute.hpp 130
6.7 include/LibFile.hpp File Reference 131
6.7.1 Typedef Documentation 132
6.7.1.1 json 132
6.8 LibFile.hpp 132
6.9 include/LibFileOperations.hpp File Reference 133
6.9.1 Function Documentation 134
6.9.1.1 compareLibFiles() 135
6.9.1.2 funcLibFile() 136
6.9.1.3 monoCheckLibFile() 138
6.9.1.4 parseLibFile() 140
6.9.1.5 printInfo() 142
6.9.1.6 spiceLibFile() 143
6.9.1.7 supercellLibFile() 144
6.9.1.8 verilogLibFile() 147
6.10 LibFileOperations.hpp 148
6.11 include/LibGroup.hpp File Reference 149
6.12 LibGroup.hpp 150
6.13 include/LibraryComparator.hpp File Reference 151
6.13.1 Typedef Documentation 152
6.13.1.1 json 152
6.14 LibraryComparator.hpp 152
6.15 include/LogicComparator.hpp File Reference 153
6.16 LogicComparator.hpp 154
6.17 include/LogicExtractor.hpp File Reference 155
6.17.1 Function Documentation 156
6.17.1.1 extractAndPrintNetlistInfo() 156
6.17.1.2 extractLogicFromVerilog() 157
6.18 LogicExtractor.hpp 158
6.19 include/verilog_utils.hpp File Reference 160
6.19.1 Function Documentation 161
6.19.1.1 getAST() 161

6.20 verilog_utils.hpp 161
6.21 include/version.h File Reference 163
6.21.1 Macro Definition Documentation 163
6.21.1.1 APP_AUTHOR 163
6.21.1.2 APP_CONTACT 164
6.21.1.3 APP_NAME 164
6.21.1.4 APP_VERSION 164
6.21.1.5 APP_VERSION_MAJOR 164
6.21.1.6 APP_VERSION_MINOR 164
6.21.1.7 APP_VERSION_PATCH 165
6.21.1.8 BUILD_TIMESTAMP 165
6.22 version.h 165
6.23 README.md File Reference 166
6.24 src/Iterators.cpp File Reference 166
6.25 Iterators.cpp 166
6.26 src/json_utils.cpp File Reference 167
6.26.1 Function Documentation 168
6.26.1.1 generateCellJson() 168
6.26.1.2 generateLutJson() 170
6.26.1.3 generatePinJson() 171
6.26.1.4 generatePowerJson() 174
6.26.1.5 generateTimingJson() 175
6.26.1.6 parseStringToVector() 177
6.27 json_utils.cpp 178
6.28 src/LibAtrribute.cpp File Reference 181
6.29 LibAtrribute.cpp 181
6.30 src/LibFile.cpp File Reference 182
6.31 LibFile.cpp 182
6.32 src/LibFileOperations.cpp File Reference 199
6.32.1 Function Documentation 200
6.32.1.1 compareLibFiles() 200
6.32.1.2 funcLibFile() 201
6.32.1.3 monoCheckLibFile() 204

6.32.1.4 parseLibFile() 206
6.32.1.5 printInfo() 208
6.32.1.6 spiceLibFile() 209
6.32.1.7 supercellLibFile() 210
6.32.1.8 verilogLibFile() 213
6.33 LibFileOperations.cpp 214
6.34 src/LibGroup.cpp File Reference 219
6.35 LibGroup.cpp 219
6.36 src/LibraryComparator.cpp File Reference 220
6.37 LibraryComparator.cpp 220
6.38 src/LogicComparator.cpp File Reference 225
6.38.1 Function Documentation 226
6.38.1.1 isIdentifier() 226
6.38.1.2 isOperator() 227
6.39 LogicComparator.cpp 227
6.40 src/LogicExtractor.cpp File Reference 242
6.40.1 Function Documentation 242
6.40.1.1 extractAndPrintNetlistInfo() 242
6.40.1.2 extractLogicFromVerilog() 243
6.41 LogicExtractor.cpp 244
6.42 src/main.cpp File Reference 253
6.42.1 Detailed Description 254
6.42.2 Function Documentation 255
6.42.2.1 main() 255
6.43 main.cpp 256
6.44 src/verilog_utils.cpp File Reference 261
6.44.1 Function Documentation 261
6.44.1.1 getAST() 261
6.45 verilog_utils.cpp 262

索引 **271**

Chapter 1

ZlibValidation

1.1 Description

Command line tool to validate standard cell libraries in `.lib` format.

1.2 Usage

```
ZlibValidation
Usage: ./zlibvalidation [OPTIONS] [SUBCOMMAND]
Options:
  -h,--help                    Print this help message and exit
  -v,--version                 Display program version information and exit
Subcommands:
  parse                        Parse the Liberty file and write JSON to a file
  mono                         Check the monotonicity of timing arc values
  compare                      Compare the comparison library against the reference one and report differences
  supercell                    Generate supercells for the given Liberty file
  zlibboost                    ZlibBoost - Multi-threaded Library Processing Tool
  clear                        Clear the log, JSON, map, markdown, Verilog, SPICE files in this directory
  verilog                      Generate Verilog file for given Liberty file
  spice                        Generate SPICE file for given Liberty file
  func                         Check functional equivalence of two Liberty files or Verilog files
```

1.3 Development Diary

1.3.1 2025-01-27

- 创建了一个C++空项目，使用CMake管理编译链，并手动添加了外部库 `libsi2dr_liberty.a`。
- 添加了 `CLI11` 库以处理命令行参数解析。
- 添加了 `spdlog` 日志库，实现高效的日志格式化输出。

1.3.2 2025-01-28

- 添加 nlohmann/json 库用于 JSON 数据存储和检索。
- 在 version.h 中存储项目作者、版本、构建日期等信息，由 CMake 自动维护。
- 简化了 --version 参数解析的代码实现，并添加了 --mode 选项以选择工作模式。
- 使用 spdlog 库的日志记录器将 debug 级别及以上的日志输出到文件，将 info 级别及以上的日志输出到控制台。
- 将库文件顶层封装为 LibFile 对象，提供了读取、解析和修改库文件的方法，包含名称和 PVT 信息作为公共属性。
  - 但仍然使用过程化方法循环迭代读取库文件名称和 PVT 信息。
- 按照 C++ 标准库、第三方库和本地库的顺序重新排列了 #include 语句。

1.3.3 2025-02-01

- 使用面向对象编程（OOP）重构了原先用C 语言循环遍历的代码，将简单属性的读取封装为 LibAttribute 类，
  - 并将函数返回的 char * 类型的属性值转换为 std::string 类型，便于外部使用。
- 将属性迭代过程封装为 AttributesIterator 类，在析构函数中处理迭代退出，实现资源获取即初始化（RAII）。

1.3.4 2025-02-10

- 将组的迭代过程封装为 GroupsIterator 类，同样在析构函数中处理迭代退出。
- 封装了 LibGroup 类，提供了读取组名、类别、属性、子组的方法。
- 将类的定义和实现分离，将类的实现放在 src 目录下，将类的定义放在 include 目录下，修改了 CMakeLists.txt 自动收集文件，便于模块化编译。

1.3.5 2025-02-11

- 类的头文件改为 hpp 后缀，修改了 CMakeLists.txt 中的文件收集规则，CMake 编译时间戳更新。
- 为 LibFile 类添加了输出同名 json 格式文件的方法 LibFile::writeJsonToFile，目前能输出 PVT、cell_name、cell_footprint、cell_area 信息。
- [x] voltage 浮点数误差未解决。

1.3.6 2025-02-14

- 在 json_utils.cpp 中，实现了以 JSON 数据结构循环迭代存储库 (lib) 信息的功能，具体包括 generateCellJson，generatePinJson，generatePowerJson，generateLutJson 等函数，并添加了同名的 hpp 头文件。
- [x] 时序 (timing) 信息解析功能待完成。
- 新增辅助函数 parseStringToVector，该函数可将以逗号分隔的字符串解析为浮点数向量，以便于读取 Look Up Table (LUT)。
- 为 LibFile 对象添加了私有属性 lib_json_，用于存储库 (lib) 的 JSON 对象。
- 封装了用于迭代复杂属性值的 ValuesIterator 类。
- 修改了 LibAttribute::isComplex() 函数的返回值类型，由原类型变更为 bool 类型。
- 重写了 LibGroup::getName() 函数，使其直接返回 std::string 类型的组名字。
- 重构了迭代器使用代码。移除了中间变量类似 si2drGroupsIdT sub_groups 的声明步骤。直接使用 lib_group.getGroups() 的返回值进行初始化 GroupsIterator，提升了代码的可读性和简洁性。

1.3.7 2025-02-15

- 实现 generateTimingJson 函数，用于全面解析时序信息。

1.3.8 2025-02-17

- 简化了迭代器在外部的调用流程，删除所有迭代器的 begin() 方法，改为在构造函数内部初始化。

1.3.9 2025-02-18

- 新建 LibFile::mono() 方法，准备实现库文件输出引脚的 values 单调递增性检查。
- 修改了 setupLogger() 函数，将日志文件名可作为参数传入，便于根据不同工作模式切换日志文件。

1.3.10 2025-02-19

- 修改了 mode == "mono"情况下的逻辑，使用 C++17 引入的 std::filesystem::exists() 函数检查同名 JSON 文件是否存在，如果 JSON 文件不存在，则先调用库文件解析功能，生成 JSON 文件，然后再执行单调性检查。
- 实现了 LibFile::mono() 方法，检查 cell_rise、cell_fall、rise_transition、fall_transition 这四种 timing 信息，针对其 LUT 数据结构，检查其 value 值在表格的每一行是否保持单调递增。

1.3.11 2025-02-20

- LibFile::mono() 方法增加结果统计功能，在程序运行的最后输出单调性检查中 passed (通过) 和 failed (失败) 的 cell 数量，输出格式参考 liberate_lv 工具的风格。
- 以 sl018_ff_3.96_-40.lib 为例，与 liberate_lv 工具对比测试，结果均为 205 out of 559 cells failed。
- 对于非单调信息警告，增加输出 when 信息，方便用户查看具体位置。

1.3.12 2025-02-25

- 重构命令行参数解析，使用 CLI11 库的子命令，在每个回调函数中实现 parse、mono 功能。

1.3.13 2025-02-26

- 子命令可自定义输出文件名、日志文件名，并设置了相应的默认值。
- 改为在 Logger 初始化时打印版本信息，避免在 -h，-v 时多余打印。
- LibFile::mono() 方法增加对 input_slew 的单调性检查，如果在输入时有指定，就检查 value 矩阵的每一列是否单调。经过测试，tcbn65lpbc.lib 输出与 liberate_lv 工具一致。

1.3.14 2025-02-27

- 多线程并行尝试，采用GDB 调试：
  - si2dr_liberty 库在解析 Liberty 文件时，可能使用了共享的数据结构（例如哈希表、字符串表）来存储解析结果或中间数据。
  - 在多线程并行解析多个文件时，不同的线程同时调用 si2dr_liberty 库的函数，并发地访问和操作这些共享数据结构。
  - si2dr_liberty 库可能没有采取足够的线程同步措施来保护这些共享数据结构，导致了数据竞争。
  - 数据竞争最终导致了内存损坏，使得 strcmp 函数在后续操作中访问了无效内存，触发了段错误。

1.3.15 2025-02-28

- 修改 LibFile 的构造函数与析构函数，read 方法改为私有，移入 parse 方法。将 si2dr 初始化与销毁放在 parse 方法中，使得 mono 方法与 si2dr 库解耦，从而实现多线程并行验证单调性。
- [x] 顶层 si2drPIGetGroups 未退出的 bug：WARNING: si2drPIQuit: GetGroups called 1 more times than IterQuit，内存泄漏？
- 学到了可以在 CMakeLists.txt 中使用 set(CMAKE_BUILD_TYPE Release) 减少编译器优化程度，配合GDB 等调试工具检查堆栈信息。

1.3.16 2025-03-01

- sub_groups_iter 是在每次 for 循环的迭代中声明的局部变量，在每次迭代结束后其作用域结束，析构函数被调用，释放相关资源。所以 si2drPIQuit 未发现内存泄漏。
- 顶层 group_iter 的析构函数发生在 parse 方法结束时，所以 parse 方法末尾的 si2drPIQuit 检测到了顶层 groups 未退出的情况。
- 综上，si2drPIQuit 的警告是由于当时顶层 groups 未退出导致的，在 parse 方法结束后能正确释放，不会造成实际上的内存泄漏。
- [ ] 顺序执行 parse，log 输出仍存在问题：解析第二第三个库的时候会多余打印之前库的PVT信息，时间上也比单独解析一个库的总和要慢 (parse 6s 左右，mono 1s)。并行执行 mono，log 输出会集中到最后一个库的文件。
- 给 LibFile 类的私有属性添加了初始值，避免了未初始化的问题，吗？

1.3.17 2025-03-07

- 实现了 compare 子命令的解析，能够对两个库文件依次进行解析，并生成对应的 JSON 文件。
- [ ] 仍然存在日志重复输出的问题，多文件处理时数据可能会相互干扰。

1.3.18 2025-03-10

- 新增 supercell 子命令和 LibFile::supercell 方法，用于生成超级单元以供验证。
- 集成了 zlibboost 子命令，支持调用开源库特征化工具 zlibboost。已与开发者取得联系。
- 调整了子命令输出文件的位置为当前目录，避免文件混淆。
- 添加了 clear 子命令，方便清理生成的 JSON 文件和日志文件。

1.3.19 2025-03-14

- 添加了 tabulate/table 库，通过 CMake FetchContent 进行管理，用于生成格式化的表格输出。
- 新建了 compare.cpp 和 compare.hpp 文件，创建了 LibraryComparator 类，用于比较两个库文件的差异。目前实现了读取参考库 JSON 文件的功能。

1.3.20 2025-03-15

- 使用 std::filesystem::path 重构了 LibFile 类的文件路径管理，方便文件路径的拼接和处理。LibFile 类添加了成员变量 basename_、filename_、libname_、jsonname_ 和 loggername_，用于存储文件名、库名、JSON 文件名和日志文件名。
- 通过作用域 (scope) 管理 LibFile::parse 方法中的顶层迭代器的生命周期，提前结束并调用 si2drPIQuit 保证资源释放，解决了分析多个库时日志重复输出和数据保存错误的问题。
- clear 子命令增加了删除 .map 和 .md 文件的功能。
- 实现了 mono 子命令的多线程并行。首先检查是否是多文件输入，如果是，就顺序检查是否准备好了每个文件的 JSON 文件，否则顺序进行多文件库解析。在所有文件的 JSON 准备就绪后，根据文件数量创建线程池，每个线程负责一个库的单调性检查。
- 重构了 spdlog 日志初始化，返回 logger 对象而不是设置全局默认 logger。保留名为 APP_NAME 的全局 logger，用于输出总体提示信息。
- 为 LibFile 类添加了独立的成员变量 logger，用于记录每个库文件的日志信息。
- 完成了 supercell 子命令的多线程并行，处理逻辑与 mono 子命令类似。
- LibraryComparator::generateReport() 方法测试了 tabulate 库的 markdown 表格输出功能，完善了报告的头部信息输出，包括参考库、比较库、相对容差、软件信息、报告生成时间和图例说明。

1.3.21 2025-03-16

- 针对 compare 子命令，增加了对报告文件名的检查，确保其以 .md 或 .txt 结尾。若不符合，则给出警告并自动添加 .md 后缀。
- 进一步完善了 LibraryComparator::generateReport() 方法。现在，该方法能够遍历比较库中的所有 cell，并在参考库中查找对应的 cell。如果找到，则调用 LibraryComparator::compareCell() 方法进行比较；否则，会记录 cell 未找到的警告信息。
- 新增了 LibraryComparator::compareCell() 方法，用于比较两个库中同名 cell 的输出引脚。该方法会遍历比较库 cell 的每个输出引脚，在参考库 cell 中寻找同名引脚。如果找到，则调用 comparePin 函数比较引脚；如果未找到，则记录警告信息。如果比较库 cell 没有输出引脚，则会记录一条信息级别的日志。
- 新增了 LibraryComparator::comparePin() 方法，用于比较两个库中同名 cell 的同名引脚。它会遍历比较库 pin 的每个 timing arc，并在参考库 pin 中查找相同类型的 timing arc。如果找到，则调用 compareTimingArc 函数进行比较；如果未找到，则记录警告信息。如果比较库 pin 没有 timing arc，则记录一条信息级别的日志。

- 修改了 LibraryComparator::compareTimingArc() 方法，用于比较两个 timing arc JSON 对象。该方法首先记录正在比较的 timing arc 的类型，然后提取 "related_pin" 属性。接着，它遍历预定义的 timing arc 名称列表（"cell_rise", "cell_fall", "rise_transition", "fall_transition"），并在比较库中查找这些 timing arc。如果找到，则在参考库中查找对应的 timing arc，并调用 compareLut 方法进行比较。如果参考库中未找到对应的 timing arc，则记录警告信息。
- 修改了 LibraryComparator::compareLut() 方法（原 compareValue 方法，已更正命名），用于比较两个 JSON 对象中具有相同名称的 value。它首先提取两个 JSON 对象中的 "index_1" 和 "index_2" 数组，如果这两个数组不相等，则记录错误信息并返回。如果索引匹配，则比较 LUT 中的实际 value 值，并根据相对容差 (reltol) 和绝对容差 (abstol) 标记差异。
- 重新设计了层级比较方法，现在可以逐层级地传入 cell_name、pin_name、timing_type、related_pin 和 arc_name，以便在最内层制表或提示时能够准确输出层级信息。
- 修复了 LibraryComparator::compareValue() 方法的命名错误，已更正为 LibraryComparator::compareLut
- 已经可以输出表格，数据数量和商用工具结果一致，但格式还需要进一步调整。
- 新增了 abstol 参数，默认值为 0.002ns，用于设置绝对容差，与库验证工具保持一致。

1.3.22 2025-03-17

- 新增 verilog 和 spice 子命令，用于生成 Verilog 和 SPICE 代码。
- 完善 LibraryComparator 报告生成：
  - 为每个 cell 增加表头，仅在存在表格数据时输出表格，避免冗余输出。
  - 增加 cell 结果统计功能，输出结果统计表格，目前仅支持 Timing 中 Delay 的比较。

1.3.23 2025-03-18

- 优化统计表格，包含 | Cell Name | Data Type | Failed Count | Avg Diff | Avg Diff% | Max Diff | Max Diff% | Outliers |。
- 优化报告对比表格，包含 | Pin Name | Reference | Comparison | Diff | Diff % | Type | Arc Name | Row # | Index_1 | Column # | Index_2 | Note |
- 报告表格的 stdout 输出增加表头加粗、黄色的格式化。
- 增加 si2drSimpleAttrGetBooleanValue 封装，用于获取布尔类型的属性值，以判断引脚是否是时钟引脚。
- LibFile::mono() 增加对 min_pulse_width 的单调性检查：
  - 针对包含 "input_pins" 的 cell，遍历每个输入引脚。
  - 如果引脚是时钟引脚，则检查其 "timing_arcs"。

  - 针对 "timing_type" 为 "min_pulse_width" 的 timing arc，对 "rise_constraint" 和 "fall_↩
    constraint" 调用 `checkTimingArcMonotonicity` 进行单调性检查。

- `checkTimingArcMonotonicity` 函数日志区分有无 when 信息，输出不同日志。
- `checkTimingArcMonotonicity` 核心比较功能取消了等于的情况，相等情况默认为通过。
- `checkTimingArcMonotonicity` 核心比较功能增加了判断 `related_pin` 是否等于当前 pin，不等于则跳过。最终改为：如果矩阵中当前的值小于前一个值，并且当前引脚不是相关引脚，或者当前值和前一个值都为零，那么就认为这些值不是单调递增的。

1.3.24 2025-03-19

- 完善 supercell 方法，如果有有时钟信号引脚，链式长度被设为 1，再生成超级单元。
- 解决 voltage 浮点数误差问题，std::round(voltage_ ∗ 100) / 100.0 保留两位小数。

1.3.25 2025-03-21

- 新增 func 子命令，用于检查库或者Verilog 文件的逻辑等价性。

1.3.26 2025-03-24

- 调研了 C++ 操作 Verilog 相关的库，找到了一个实用的 Verilog 库：`slang`，它可以解析出抽象语法树 (AST)。已修改 CMakeLists 文件，将 slang 集成到项目中，目前正在研究其 API。

1.3.27 2025-03-25

- 新增 `verilog_utils.cpp` 和 `verilog_utils.hpp` 文件，并创建了 `VerilogVisitor` 类，用于自定义遍历 Verilog AST。
- 验证了 slang 对不同类型逻辑单元的解析能力：
  - 简单组合逻辑（如 `AND2D0`）、复杂时序逻辑（如 `CMPE42D1`）和基本时序逻辑（如 `DFQD1`）均能被正确解析。
  - 用户自定义原语 (UDP)，例如 `tsmc_dff`，会被错误地识别为模块实例化，并产生 "Invalid instance declaration" 警告。
  - 成功提取了模块名、端口方向（`input`、`output`）及名称。
  - 能够解析命名端口连接的层次化实例化，并获取模块名、实例名称和端口映射关系。
  - 能够解析门级原语实例化，并获取门类型、实例名称、输出端口和输入端口。

1.3.28 2025-03-26

- 进一步完善了对 UDP（如 tsmc_dff）端口映射关系的解析，但其内部 Entry 尚未处理。
- 尝试了通过继承 slang::syntax::SyntaxRewriter 类创建自定义类 CellExtractor，实现了只保留目标模块、删除其他模块，并将修改后的语法树另存为新文件。使用 slang::syntax↩ ::SyntaxPrinter::printFile() 方法可以将 Verilog 代码输出到文件。
- 创建了 CellPrinter 类，继承自 slang::syntax::SyntaxVisitor 类。当访问到目标模块时，使用 module.toString() 方法也能将 Verilog 代码输出到文件，但发现输出结果中空行会被删除。
- 修改了 LibFile::supercell() 方法，该方法可以根据输入的 cell_names 生成指定的 supercell，并在遇到未找到的单元时提示警告信息。
- 优化了时钟引脚的处理方式，现在时钟引脚不再被视为 supercell 的输入引脚，而是仅记录为时序单元，并跳过存入 input_pins 集合的步骤。
- 引入了 Doxygen 文档生成工具，用于可视化分析项目，并生成了 Doxygen HTML 文档和 LaTeX 参考手册。
- [x] 尚未解决手册中文显示不正确的问题，可能需要自定义 LaTeX 头文件。

1.3.29 2025-03-27

- **重大失误**！误操作 Git 分支，导致部分代码修改丢失。务必牢记：** 及时提交代码！**
- 实现了 LibFile::verilog 方法的核心流程：
  - 首先调用 this->supercell() 生成超级单元。
  - 读取 .map 文件，提取单元映射关系到 std::pair<std::string, std::string> 中（原始单元名 -> 超级单元名/模块名）。
  - 从 lib_json_ 中提取单元的输入/输出端口信息。
  - 根据是否存在时钟引脚，确定 Verilog 模块中 instance 的生成数量（时序逻辑为 1，组合逻辑等于 chain length）。
  - 通过字符串操作生成 ANSI 风格的端口列表，并与模块名组合成完整的 fullModuleText。
  - 使用 slang::syntax::SyntaxTree::fromText 从 fullModuleText 生成语法树。
  - 将语法树传递给 ModuleRewriter 类，使用其 transform 方法遍历模块成员，并通过 handle() 方法进行处理。
  - 最后，使用 slang::syntax::SyntaxPrinter::printFile(*tree) 将处理后的语法树输出到文件。
- LibFile::verilog 目前能够正确输出模块名和 ANSI 风格的端口声明。

1.3.30 2025-03-28

- 实现了在模块成员列表中插入新节点的功能。具体而言，在 ModuleRewriter::handle(const slang::syntax::ModuleDeclarationSyntax &module) 方法中，通过 insertAtBack(module.members, newDataNode) 函数，成功地在模块成员的末尾插入了链式单元所需的中间变量声明，例如 wire OP_i;。
- 成功地在模块内部添加了关键连接网络变量。通过解析 CELLNAME__X#__CRITICALPORT__OUTPUTPORT 格式的字符串，提取出关键输入端口（critical input port）和输出端口(OUTPUTPORT)，用于后续的实例化端口连接。
- 增加了用于处理多输出接口的中间网络变量 P__UNUSEOUTPUTPORT，用于连接不需要考察的输出端口。
- 成功地在模块内添加了模块实例化所需的模块名和实例名称，完成了端口映射关系的连接处理。
- 验证了代码对简单组合逻辑单元（如 INVD0 反相器）和复杂组合逻辑单元（如 FA1D0 全加器）的输出能力，结果均符合预期。
- 为 ModuleRewriter 类添加了私有属性 std::map<std::string, std::string> portInfoMap_，用于存储端口映射关系，将端口名映射到其方向（input/output）。
- 完成了实例化端口连接的细节处理：
  - 如果是关键输入端口且为第一个实例，则直接连接到输入端口；否则，连接到 OP_(i-1) 这样的中间网络变量。
  - 如果是关键输出端口，且为最后一个实例，则连接到输出端口；否则，连接到 OP_i 中间网络变量。
  - 如果是其他输出端口且不是最后一个实例，则连接到 P_i__portName 这样的中间网络变量；否则，连接名等于端口名。
  - 其余输入端口，连接名等于端口名。
- 文档方面，GitHub 远程仓库已开放为公开访问，并配置了 GitHub Pages。可以通过 https://cedar17.github.io/ZlibValidation/ 访问项目文档，该页面包含 Doxygen 生成的 HTML 文档和仓库地址的链接。
- 配置了 GitHub Actions，实现了 Doxygen 文档和 Graphviz 图的自动化构建，以及 LaTeX 参考手册的自动编译（暂不支持中文）。每次在 dev 和 main 分支的提交都会触发文档和参考手册的生成，并发布到 gh-pages 分支。
- 将第三方库放置到 include_3rd_party 目录下，方便管理。修改了 CMakeLists 文件，将第三方库的头文件路径添加到 include_directories 中。修改了 Doxyfile 文件，将第三方库的头文件路径添加到 INPUT 中，便于文档生成工具理解第三方库函数、类的关系及使用方法。

1.3.31 2025-03-29

- 修复了 Doxygen 生成 LaTeX 参考手册时中文显示不正确的问题。通过修改 Doxyfile 文件，添加 `EXTRA_PACKAGES = ctex` 选项，并指定 `lualatex` 作为编译器，成功解决了中文显示问题。同时，从 `include_3rd_party` 目录中移除了 `Allsyntax.h` 文件，避免了由于 LaTeX 文档过大导致的 "TeX capacity exceeded, sorry [main memory size=5000000]" 内存不足问题。
- 实现了 Verilog 文件的顶层模块生成功能，采用非 ANSI 风格的端口声明方式，格式与 `liberate_lv` 工具生成的完全一致。实现过程如下：
  - 首先将各模块的 Verilog 代码存放到临时文件中。
  - 在处理模块的过程中，使用 `std::vector<std::string>` 收集每个模块的 `input_pins` 和 `output_pins`。
  - 遍历所有模块名称列表，拼接模块名和引脚名，生成顶层模块的 `all_input_ports` 和 `all_output_ports`。
  - 使用字符串拼接方式构建 `validate_top` 模块的完整代码。
  - 最后将临时文件内容与顶层模块代码合并，输出到最终的 Verilog 文件中。
- 更改 `deploy-docs.yml`，修复了 GitHub Action 生成的 LaTeX 参考手册中的图像显示问题。具体步骤如下：
  - 设置系统时区为 Asia/Shanghai (东八区)。
  - 安装 miniconda 并更新 conda。
  - 通过 conda 安装 1.9.5 版本的 Doxygen，并安装 graphviz 以解决依赖关系，确保每个 dot 图都能正确编译出 pdf 文件。
  - 使用 `doxygen` 命令生成 HTML 文档和 tex 文件。
  - 采用 action marketplace 提供的 `xu-cheng/texlive-action@v2` 安装全量的 texlive2024。
  - 进入 `./doc_doxygen/latex` 目录，运行 `make` 命令使用 lualatex 编译 tex 文件，生成 pdf 文件。
  - 使用 `JamesIves/github-pages-deploy-action@v4` 将生成的 pdf 文件上传到 GitHub Pages 发布，并保留 `README.md` 和 `index.html` 文件。

1.3.32 2025-03-30

- 完成了 `LibFile::logic(const std::string &cell_name)` 方法，该方法返回指定 cell_name 所有输出引脚对应的逻辑表达式，以 `std::map<std::string, std::string>` 的形式返回，存储输出引脚名称到其逻辑表达式字符串的映射关系.
- 实现了 `func` 子命令的基本框架，用于检查库或 Verilog 文件的逻辑等价性。
  - 完善了 `funcLibFile` 函数，能够处理 Liberty 文件和 Verilog 文件作为参考和比较对象。
  - 增加了文件类型判断，根据文件扩展名选择不同的处理方式。

  - 实现了对 `.lib` 文件的解析，并提取指定 cell 的逻辑表达式。
  - 目前仅支持 Liberty 文件的逻辑表达式提取，Verilog 文件的逻辑表达式提取功能尚未完成 (TODO)。
  - 增加了对 report 文件名的检查，确保其以 `.md` 或 `.txt` 结尾。若不符合，则给出警告并自动添加 `.md` 后缀。
  - 实现了从 Liberty 文件中提取逻辑表达式的功能，使用了 `LibFile::logic` 方法。
  - 增加了日志输出，记录每个 cell 的逻辑表达式提取和比较过程。
  - 逻辑比较部分目前为空，待实现（TODO）。

1.3.33 2025-03-31

- 实现了从 Verilog 结构网表中提取逻辑表达式的功能，使用了 `LogicExtractor` 类。
  - 设计了如何使用 Slang 库从 Verilog 结构网表中提取逻辑表达式的方案：
    * 使用 Slang 解析 Verilog 文件获取语法树 (AST)。
    * 遍历 AST 定位目标模块。
    * 在目标模块内构建网表的内部表示，识别输入、输出、线网及门级连接关系。
    * 对每个输出端口，递归反向追踪门和线网直至到达主输入端口。
    * 在回溯过程中根据遇到的门类型构建逻辑表达式。
    * 应用记忆化技术避免对相同线网/信号的重复计算。
  - 创建了 `LogicExtractor` 类，用于从 Verilog 代码中提取逻辑表达式。
    * `LogicExtractor` 类使用 Slang 库解析 Verilog 文件，并构建目标模块的网表表示。
    * 实现了 `handle(const slang::syntax::ModuleDeclarationSyntax &module)` 方法，用于定位目标模块，并初始化内部数据结构。
    * 实现了 `handle(const slang::syntax::PortDeclarationSyntax &portDecl)` 方法，用于提取端口信息，包括端口方向和名称。
    * 实现了 `handle(const slang::syntax::NetDeclarationSyntax &netDecl)` 方法，用于提取线网信息。
    * 实现了 `handle(const slang::syntax::PrimitiveInstantiationSyntax &primitiveInst)` 方法，用于提取门级单元信息，包括门类型、输入和输出信号。
    * 实现了 `getLogicExpressions()` 方法，用于根据提取的网表信息，递归地推导每个输出端口的逻辑表达式。
    * 实现了 `deriveLogicRecursive(const std::string &signalName)` 方法，用于递归地推导指定信号的逻辑表达式。
    * 实现了 `formatExpression(const GateInfo &gateInfo, const std::vector<std::string> &inputExprs)` 方法，用于根据门类型和输入表达式，生成逻辑表达式字符串。
  - 实现了 `extractAndPrintNetlistInfo(const std::string &verilog_file, const std::string &cell)` 函数，用于提取并打印网表信息，包括输入、输出、线网和门级单元。

- 实现了 extractLogicFromVerilog(const std::string &verilog_file, const std::string &cell) 函数，用于从 Verilog 文件中提取指定 cell 的逻辑表达式，并以 std::map<std::string, std::string> 的形式返回。
- 为了更好地组织代码结构，将 LogicExtractor 类从 verilog_utils.hpp 和 verilog_utils.cpp 中分离出来，并创建了独立的 LogicExtractor.hpp 和 LogicExtractor.cpp 文件。同时，统一了所有 hpp 和 cpp 文件的命名规范，使其与对应的类名保持一致。
- 新增 LogicComparator 类及其头文件、源文件，用于比较两个逻辑表达式的等价性，并生成比较报告。

1.3.34 2025-04-01

- 整理头文件引用，删除了不必要的引用，提高编译效率。
- 引入 exprtk.cpp 到第三方头文件目录，为后续的表达式解析做准备。
- 完善了 LogicComparator 类的实现，增加了其示例代码模板 template <typename T> void logic();，该模板能够对一个表达式遍历输入变量的所有组合，计算出逻辑值，并打印真值表，为逻辑等价性验证提供基础。
- 实现了 LogicComparator::preprocessExpression() 方法，用于预处理逻辑表达式，目的是简化表达式，使其更易于比较。预处理的逻辑如下：
  - ** 统一 AND 处理 **：将 * 和空格隐式 AND 都转换为带空格的 and，保证 AND 运算符的显式性。
  - ** 显式符号处理 **：
    - 在所有括号 () 周围添加空格，方便后续分词。
    - 将 + 替换为 or。
    - 将 ^ 替换为 xor。
    - 将 ! (非 != 中的 !) 替换为 not。
    - 将 * 替换为 and。
  - ** 分词 **：基于空格将处理后的字符串分割为 token 列表。
  - ** 精确插入隐式 AND**：
    - 遍历 token 列表。
    - 检查当前 token 和下一个 token，如果满足以下条件，则在当前 token 之后插入 and：
      - 当前 token 是一个标识符 (如 A, CIX) 或右括号)。
      - 下一个 token 是一个标识符或左括号 (。
      - 当前 token 不是一个显式逻辑运算符 (and, or, xor, not) 或左括号 (。
      - 下一个 token 不是一个显式逻辑运算符或右括号)。
  - ** 重组与清理 **：将处理后的 token 列表用单个空格连接起来，并进行最终的空格清理 (去除多余空格，修剪首尾空格)，得到最终的预处理表达式。

- 在 CMakeLists.txt 中，强制启用了 lld 链接器和 ccache，以提升链接和编译速度。
- 为了保持 main.cpp 的整洁，将 LibFile() 函数的相关操作剥离出来，并放入 LibFileOperations.cpp 文件中。
- 实现了 LogicComparator::extractVariables() 方法，用于从两个表达式中提取并验证变量名，确保它们一致。
  - 该方法使用正则表达式初步提取所有可能的标识符。
  - 对每个提取出的标识符，先用 isIdentifier 函数（检查大写开头等）进行验证。
  - 通过 isIdentifier 验证后，仍然将其转换为小写并与 keywords 集合比较，以过滤掉可能的关键词（这是一个保守但安全的操作）。
  - 只有通过这两步验证的标识符才被认为是有效变量，并以原始大小写形式添加到各自的 std::set 中。
  - 最后比较两个 std::set 是否相等，如果不等则报告差异并返回 false，如果相等则将变量列表（保留原始大小写）排序后存入 sorted_vars 并返回 true。
- 修复了 LogicComparator::preprocessExpression() 方法中对 not 操作符的处理。exprtk 库要求 not 关键字后必须跟着括号，如 not(A) 而非 not A。为此增加了专门的处理步骤：
  - 遍历预处理后的 token 列表，识别所有 not 关键字。
  - 当发现 not 后，检查其后是否跟随标识符（通过 isIdentifier() 函数判断）。
  - 如果后接标识符，自动将 not A 形式转换为 not(A)，具体做法是依次添加 not、(、标识符和)，并跳过已处理的标识符。
  - 如果后接其他元素（如 (、其他运算符或表达式结尾），保留原样，让 ExprTk 处理 not(表达式) 形式。
  - 对其他 token，直接添加到最终 token 列表，不做特殊处理。
- 实现了 LogicComparator::compareSingleExpressionPair() 方法，用于比较两个逻辑表达式的等价性：
  - 接收预处理后的表达式字符串和已排序的变量列表作为输入参数。
  - 为参考表达式和比较表达式分别创建 ExprTk 环境（符号表、表达式对象和解析器）。
  - 将所有变量添加到两个符号表中，确保变量一致性和顺序性。
  - 使用 ExprTk 解析器编译两个表达式，并检查可能的语法错误。
  - 通过二进制计数法，遍历所有可能的输入变量组合（2^N 种），其中 N 为变量数量。
  - 对每种组合：
    - 设置两个符号表中的变量值（0 或 1）。
    - 计算两个表达式的值，并处理可能的运行时错误。
    - 将计算结果转换为布尔值并存入结果向量。
    - 同步构建真值表，记录输入组合和对应的输出值。
  - 比较两个结果向量判断表达式是否等价。

  - 将生成的真值表存入 PinComparisonResult 对象的 std::optional 成员中，以便后续报告生成。
- 实现了 template <typename T> std::map<std::string, PinComparisonResult> LogicComparator::compareCellLogic();
  - 该方法用于比较两个 cell 的逻辑表达式，返回一个 std::map，键为 cell 名称，值为 PinComparisonResult 结构体。
  - 遍历在类私有属性里的整个独立输出引脚键，提取其值作为逻辑字符串表达式，首先使用 preprocessExpression 进行预处理，然后使用 extractVariables 提取变量向量。
  - 调用 LogicComparator::compareSingleExpressionPair() 方法进行比较，结果存入 PinComparisonResult 对象。
  - 最后将 PinComparisonResult 对象存入 std::map 中，键为 cell 名称，值为 PinComparisonResult 对象，返回。
- 实现了 LogicComparator::generateReport() 方法，报告格式仍需要调整。

1.3.35 2025-04-02

- 取消所有模板函数的 template <typename T> 声明，改为使用 double 作为参数类型，加快编译速度。
- 完善了 LogicComparator::generateReport() 方法，增加了对逻辑比较结果的详细报告输出，包括：
  - 元数据，例如：∗∗Performed by ZlibValidation v0.1.0 from Song Zixuan. on: Wed Apr 2 11:11:39 2025∗∗
  - 图例，包括参考 pin 名称、pin function，参考 pin 名称、pin function，符号解释表。
  - 参考 pin 的真值表
  - 比较 pin 的真值表
  - 比较结果表格，例如：

Property	Value
Status	[OK]
Reference (Raw)	`(!((!(A1+A2))+B))`
Comparison (Raw)	`!((!A1 * !A2) + B)`
Reference (Processed)	`(not((not(A1 or A2)) or B))`
Comparison (Processed)	`not((not(A1) and not(A2)) or B)`
Ref Expression Compiled	Yes
Comp Expression Compiled	Yes
Logically Equivalent	Yes

1.3.36 2025-04-05

- 新增了 `LibFile::spice()` 方法，为 spice 子命令添加了 SPICE 网表生成功能。 - 通过执行 `which v2lvs` 命令检测系统中是否安装了 V2LVS 工具。如果已安装，则调用该工具生成基本的 SPICE 网表。 - 添加了 `--vl` 和 `--sl` 命令行选项，允许用户指定 Verilog 库文件和 SPICE 库文件的路径。 - 目前已能生成基本的 SPICE 网表，后续需要进一步完善输出格式。

```
Generate SPICE file for given Liberty file
Usage: ./zlibvalidation spice [OPTIONS] library_path...
Positionals:
  library_path TEXT:FILE ... REQUIRED
                              Specify the library file to process
Options:
  -h,--help                   Print this help message and exit
  -l,--log TEXT               Specify the log file name. Default: <basename>.spice.log
  -c,--chain INT              Specify the chain length for SPICE generation. Default: 1
  --cells TEXT ...            Specify the cell names to generate SPICE for
  --vl TEXT                   Specify the location of the Verilog primitive library file
  --sl TEXT                   Specify the location of the SPICE library file to be included in the output
```

- 实现私有方法 `LibFile::splitString()`，用于将字符串按空格分割成多个子字符串，并返回一个 `std::vector<std::string>`。

- 实现了 `LibFile::generateRCLines()` 方法，用于为给定的 net 生成 RC lines。 - 该方法根据 `isFinalStage` 参数，生成不同的 RC 结构。 - 如果 `isFinalStage` 为 `false`，则生成 `R1-C1-R2` 结构。 - 如果 `isFinalStage` 为 `true`，则生成 `R1-C1` 结构。 - RC 的默认值参考目标 SPICE 文件。

- 实现了 `LibFile::modifySpiceNetlist()` 方法，用于修改 `v2lvs` 生成的 SPICE 网表。 - 该方法添加元数据注释，包括应用名称、版本、作者和时间戳。 - 在第一个 `.INCLUDE` 指令后插入指定的 global line。 - 修改 subcircuit，在 `.SUBCKT` 定义的端口列表中添加 `VDD VSS`。 - 对于 subcircuit 中的每个 instance line（以 ''X' 或'x' 开头），修改输出引脚名称，添加 VDD/VSS 连接，并调用 generateRCLines 生成 `R/C lines`。 - 保留原始注释，跳过空行、v2lvsheader `line` 和原始的.GLOBALline。

- 完全实现了 LibFile::spice()` 方法，能够生成 SPICE 网表并输出到指定文件中。SPICE 网表的生成过程如下： - 检查是否指定了 Verilog 库文件和 SPICE 库文件，如果没有指定，则使用默认路径。 - 调用 `v2lvs` 工具生成初步的 SPICE 网表。 - 调用 `LibFile::generateRCLines()` 方法，为 SPICE 网表中的每个 instance 生成对应的 RC lines。 - 调用 `LibFile::modifySpiceNetlist()` 方法，修改 SPICE 网表，添加元数据、插入 global line、调整 instance lines，并生成最终的 SPICE 网表。 - 将最终的 SPICE 网表输出到指定文件中。

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

AttributesIterator 23
GateInfo 31
GroupsIterator 33
LibAttribute 36
LibFile 40
LibGroup 64
LibraryComparator 67
LogicComparator 77
PinComparisonResult 104
slang::syntax::SyntaxRewriter
- CellExtractor 26
- ModuleRewriter 100

slang::syntax::SyntaxVisitor
- CellPrinter 29
- LogicExtractor 88
- VerilogVisitor 111

ValuesIterator 108

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

AttributesIterator . . . 23
CellExtractor . . . 26
CellPrinter . . . 29
GateInfo . . . 31
GroupsIterator . . . 33
LibAttribute . . . 36
LibFile . . . 40
LibGroup . . . 64
LibraryComparator . . . 67
LogicComparator . . . 77
LogicExtractor . . . 88
ModuleRewriter . . . 100
PinComparisonResult . . . 104
ValuesIterator . . . 108
VerilogVisitor . . . 111

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

include/Iterators.hpp 117
include/json_utils.hpp 119
include/LibAttribute.hpp 129
include/LibFile.hpp 131
include/LibFileOperations.hpp 133
include/LibGroup.hpp 149
include/LibraryComparator.hpp 151
include/LogicComparator.hpp 153
include/LogicExtractor.hpp 155
include/verilog_utils.hpp 160
include/version.h 163
src/Iterators.cpp 166
src/json_utils.cpp 167
src/LibAtrribute.cpp 181
src/LibFile.cpp 182
src/LibFileOperations.cpp 199
src/LibGroup.cpp 219
src/LibraryComparator.cpp 220
src/LogicComparator.cpp 225
src/LogicExtractor.cpp 242
src/main.cpp
This file contains the main function for the ZlibValidation tool 253
src/verilog_utils.cpp 261

Chapter 5

Class Documentation

5.1 AttributesIterator Class Reference

`#include <Iterators.hpp>`

Public Member Functions

- AttributesIterator (si2drAttrsIdT attrs, si2drErrorT &err)
- ~AttributesIterator ()
- void next ()
- bool end ()
- LibAttribute get ()

Private Attributes

- si2drAttrsIdT attrs_
- si2drAttrIdT attr_
- si2drErrorT & err_

5.1.1 Detailed Description

Definition at line 23 of file Iterators.hpp.

5.1.2 Constructor & Destructor Documentation

5.1.2.1 AttributesIterator()

```
AttributesIterator::AttributesIterator (
                si2drAttrsIdT attrs,
                si2drErrorT & err )
```

Definition at line 13 of file Iterators.cpp.

5.1.2.2 ∼AttributesIterator()

```
AttributesIterator::∼AttributesIterator ( )
```

Definition at line 17 of file Iterators.cpp.

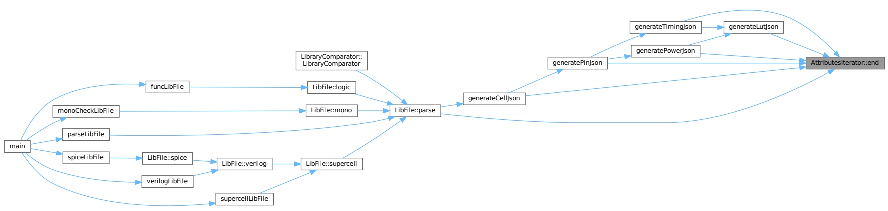
5.1.3 Member Function Documentation

5.1.3.1 end()

```
bool AttributesIterator::end ( )
```

Definition at line 20 of file Iterators.cpp.

Here is the caller graph for this function:

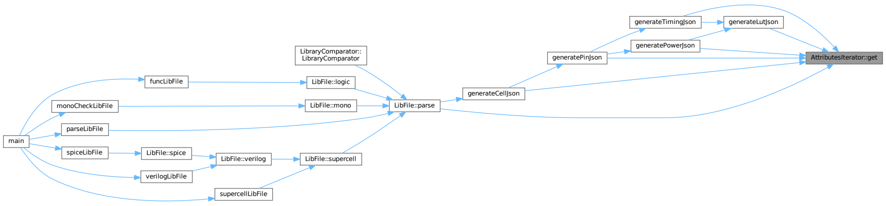


5.1.3.2 get()

```
LibAttribute AttributesIterator::get ( )
```

Definition at line 22 of file Iterators.cpp.

Here is the caller graph for this function:

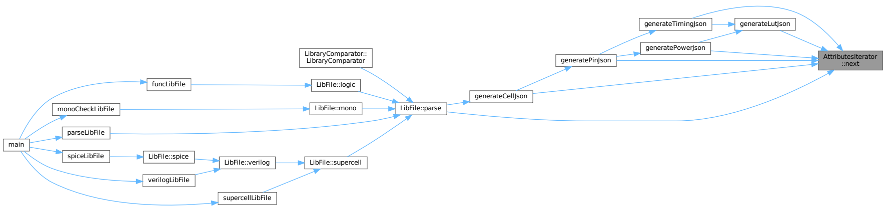


5.1.3.3 next()

```
void AttributesIterator::next ( )
```

Definition at line 19 of file Iterators.cpp.

Here is the caller graph for this function:



5.1.4 Member Data Documentation

5.1.4.1 attr_

```
si2drAttrIdT AttributesIterator::attr_  [private]
```

Definition at line 33 of file Iterators.hpp.

5.1.4.2 attrs_

```
si2drAttrsIdT AttributesIterator::attrs_ [private]
```

Definition at line 32 of file Iterators.hpp.

5.1.4.3 err_

```
si2drErrorT& AttributesIterator::err_ [private]
```

Definition at line 34 of file Iterators.hpp.

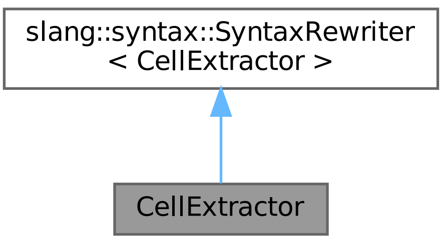
The documentation for this class was generated from the following files:

- include/Iterators.hpp
- src/Iterators.cpp

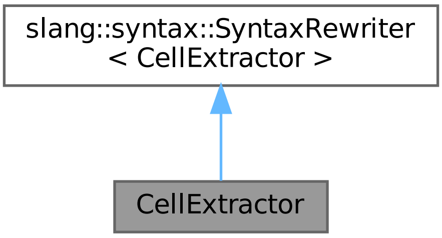
5.2 CellExtractor Class Reference

```
#include <verilog_utils.hpp>
```

Inheritance diagram for CellExtractor:



Collaboration diagram for CellExtractor:



Public Member Functions

- CellExtractor (const std::string &targetCell)
- void handle (const slang::syntax::ModuleDeclarationSyntax &module)
- bool foundTargetCell () const

Private Attributes

- const std::string & targetCell_
- bool foundTarget_

5.2.1 Detailed Description

Definition at line 33 of file verilog_utils.hpp.

5.2.2 Constructor & Destructor Documentation

5.2.2.1 CellExtractor()

```
CellExtractor::CellExtractor (
            const std::string & targetCell )  [inline], [explicit]
```

Definition at line 35 of file verilog_utils.hpp.

5.2.3 Member Function Documentation

5.2.3.1 foundTargetCell()

```
bool CellExtractor::foundTargetCell ( ) const
```

Definition at line 364 of file verilog_utils.cpp.

Here is the caller graph for this function:

5.2.3.2 handle()

```
void CellExtractor::handle (
        const slang::syntax::ModuleDeclarationSyntax & module )
```

Definition at line 348 of file verilog_utils.cpp.

5.2.4 Member Data Documentation

5.2.4.1 foundTarget_

```
bool CellExtractor::foundTarget_  [private]
```

Definition at line 42 of file verilog_utils.hpp.

5.2.4.2 targetCell_

```
const std::string& CellExtractor::targetCell_  [private]
```

Definition at line 41 of file verilog_utils.hpp.

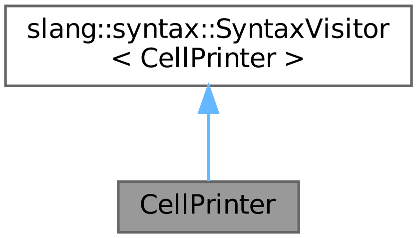
The documentation for this class was generated from the following files:

- include/verilog_utils.hpp
- src/verilog_utils.cpp

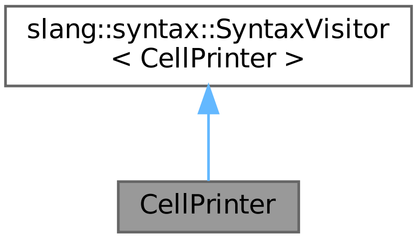
5.3 CellPrinter Class Reference

`#include <verilog_utils.hpp>`

Inheritance diagram for CellPrinter:



Collaboration diagram for CellPrinter:



Public Member Functions

- CellPrinter (const std::string &targetCell, std::ostream &out)
- void handle (const slang::syntax::ModuleDeclarationSyntax &module)

Private Attributes

- const std::string & targetCell_
- std::ostream & out_
- bool foundTarget_

5.3.1 Detailed Description

Definition at line 46 of file verilog_utils.hpp.

5.3.2 Constructor & Destructor Documentation

5.3.2.1 CellPrinter()

```
CellPrinter::CellPrinter (
        const std::string & targetCell,
        std::ostream & out ) [inline], [explicit]
```

Definition at line 48 of file verilog_utils.hpp.

5.3.3 Member Function Documentation

5.3.3.1 handle()

```
void CellPrinter::handle (
        const slang::syntax::ModuleDeclarationSyntax & module )
```

Definition at line 367 of file verilog_utils.cpp.

5.3.4 Member Data Documentation

5.3.4.1 foundTarget_

`bool CellPrinter::foundTarget_ [private]`

Definition at line 55 of file verilog_utils.hpp.

5.3.4.2 out_

`std::ostream& CellPrinter::out_ [private]`

Definition at line 54 of file verilog_utils.hpp.

5.3.4.3 targetCell_

`const std::string& CellPrinter::targetCell_ [private]`

Definition at line 53 of file verilog_utils.hpp.

The documentation for this class was generated from the following files:

- include/verilog_utils.hpp
- src/verilog_utils.cpp

5.4 GateInfo Struct Reference

`#include <LogicExtractor.hpp>`

Public Attributes

- slang::parsing::TokenKind kind
- std::string gateTypeName
- std::vector< std::string > inputSignals
- std::string outputSignal

5.4.1 Detailed Description

Definition at line 9 of file LogicExtractor.hpp.

5.4.2 Member Data Documentation

5.4.2.1 gateTypeName

`std::string GateInfo::gateTypeName`

Definition at line 11 of file LogicExtractor.hpp.

5.4.2.2 inputSignals

`std::vector<std::string> GateInfo::inputSignals`

Definition at line 12 of file LogicExtractor.hpp.

5.4.2.3 kind

`slang::parsing::TokenKind GateInfo::kind`

Definition at line 10 of file LogicExtractor.hpp.

5.4.2.4 outputSignal

`std::string GateInfo::outputSignal`

Definition at line 13 of file LogicExtractor.hpp.

The documentation for this struct was generated from the following file:

- include/LogicExtractor.hpp

5.5 GroupsIterator Class Reference

`#include <Iterators.hpp>`

Public Member Functions

- GroupsIterator (si2drGroupsIdT groups, si2drErrorT &err)
- ~GroupsIterator ()
- void next ()
- bool end ()
- LibGroup get ()

Private Attributes

- si2drGroupsIdT groups_
- si2drGroupIdT group_
- si2drErrorT & err_

5.5.1 Detailed Description

Definition at line 9 of file Iterators.hpp.

5.5.2 Constructor & Destructor Documentation

5.5.2.1 GroupsIterator()

```
GroupsIterator::GroupsIterator (
        si2drGroupsIdT groups,
        si2drErrorT & err )
```

Definition at line 3 of file Iterators.cpp.

5.5.2.2 ~GroupsIterator()

```
GroupsIterator::~GroupsIterator ( )
```

Definition at line 6 of file Iterators.cpp.

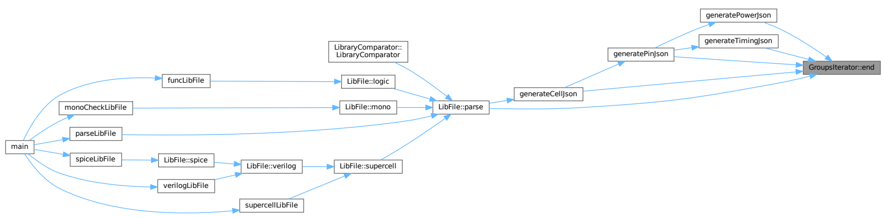
5.5.3 Member Function Documentation

5.5.3.1 end()

`bool GroupsIterator::end ( )`

Definition at line 9 of file Iterators.cpp.

Here is the caller graph for this function:

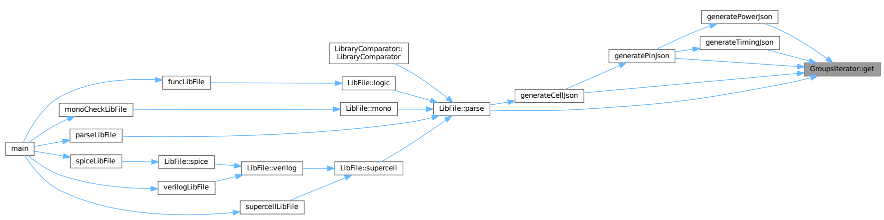


5.5.3.2 get()

`LibGroup GroupsIterator::get ( )`

Definition at line 11 of file Iterators.cpp.

Here is the caller graph for this function:

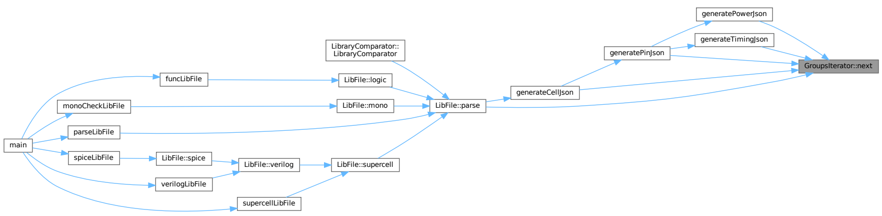


5.5.3.3 next()

```
void GroupsIterator::next ( )
```

Definition at line 8 of file Iterators.cpp.

Here is the caller graph for this function:



5.5.4 Member Data Documentation

5.5.4.1 err_

```
si2drErrorT& GroupsIterator::err_ [private]
```

Definition at line 20 of file Iterators.hpp.

5.5.4.2 group_

```
si2drGroupIdT GroupsIterator::group_ [private]
```

Definition at line 19 of file Iterators.hpp.

5.5.4.3 groups_

```
si2drGroupsIdT GroupsIterator::groups_ [private]
```

Definition at line 18 of file Iterators.hpp.

The documentation for this class was generated from the following files:

- include/Iterators.hpp
- src/Iterators.cpp

5.6 LibAttribute Class Reference

#include <LibAttribute.hpp>

Public Member Functions

- LibAttribute (si2drAttrIdT attr, si2drErrorT &err)
- ∼LibAttribute ()
- std::string getName ()
- bool isComplex ()
- si2drValuesIdT getValues ()
- long int getInt ()
- double getFloat ()
- std::string getString ()
- bool getBoolean ()

Private Attributes

- si2drAttrIdT attr_
- si2drErrorT & err_

5.6.1 Detailed Description

Definition at line 8 of file LibAttribute.hpp.

5.6.2 Constructor & Destructor Documentation

5.6.2.1 LibAttribute()

```
LibAttribute::LibAttribute (
        si2drAttrIdT attr,
        si2drErrorT & err )
```

Definition at line 3 of file LibAtrribute.cpp.

5.6.2.2 ∼LibAttribute()

`LibAttribute::∼LibAttribute ( )`

Definition at line 5 of file LibAtrribute.cpp.

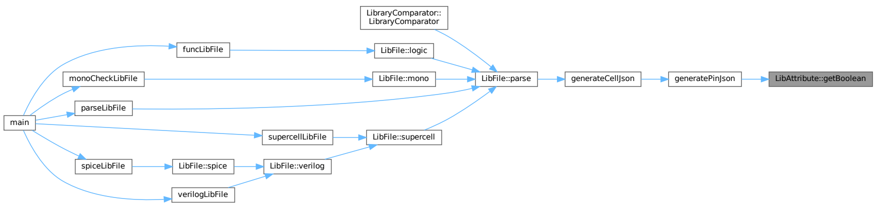
5.6.3 Member Function Documentation

5.6.3.1 getBoolean()

`bool LibAttribute::getBoolean ( )`

Definition at line 25 of file LibAtrribute.cpp.

Here is the caller graph for this function:

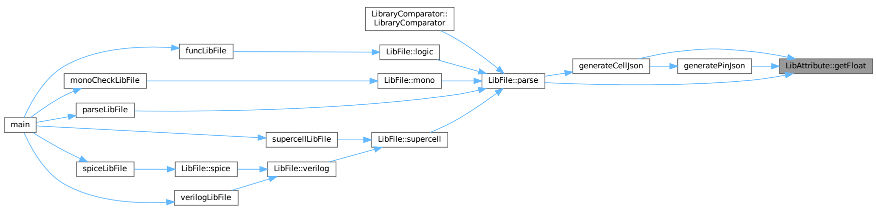


5.6.3.2 getFloat()

`double LibAttribute::getFloat ( )`

Definition at line 18 of file LibAtrribute.cpp.

Here is the caller graph for this function:

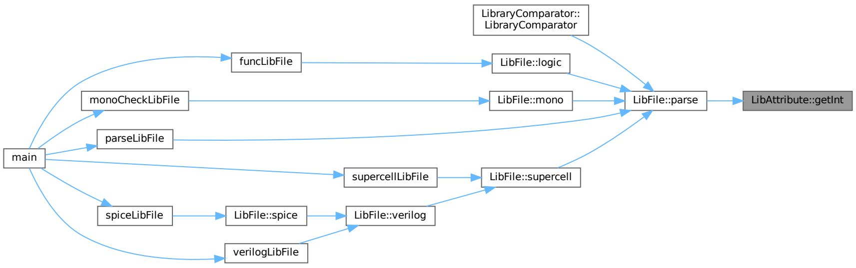


5.6.3.3 getInt()

```
long int LibAttribute::getInt ( )
```

Definition at line 16 of file LibAtrribute.cpp.

Here is the caller graph for this function:

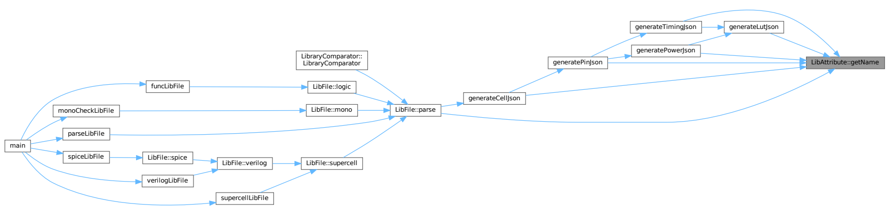


5.6.3.4 getName()

```
std::string LibAttribute::getName ( )
```

Definition at line 7 of file LibAtrribute.cpp.

Here is the caller graph for this function:

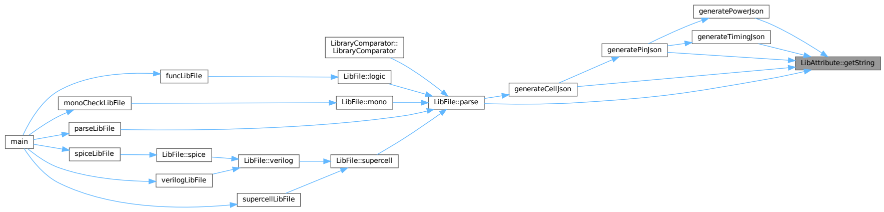


5.6.3.5 getString()

`std::string LibAttribute::getString ( )`

Definition at line 20 of file LibAtrribute.cpp.

Here is the caller graph for this function:

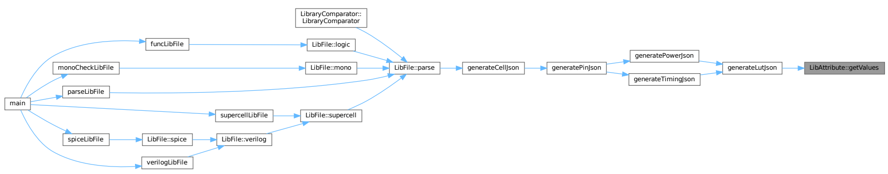


5.6.3.6 getValues()

`si2drValuesIdT LibAttribute::getValues ( )`

Definition at line 14 of file LibAtrribute.cpp.

Here is the caller graph for this function:



5.6.3.7 isComplex()

`bool LibAttribute::isComplex ( )`

Definition at line 12 of file LibAtrribute.cpp.

5.6.4 Member Data Documentation

5.6.4.1 attr_

```
si2drAttrIdT LibAttribute::attr_  [private]
```

Definition at line 21 of file LibAttribute.hpp.

5.6.4.2 err_

```
si2drErrorT& LibAttribute::err_  [private]
```

Definition at line 22 of file LibAttribute.hpp.

The documentation for this class was generated from the following files:

- include/LibAttribute.hpp
- src/LibAtrribute.cpp

5.7 LibFile Class Reference

`#include <LibFile.hpp>`

Public Member Functions

- LibFile (const std::string &filepath, const std::string &loggername)

    Constructs a LibFile object with specified file path and logger name.
- ~LibFile ()
- void writeJsonToFile ()

    Writes the JSON data stored in the object to a file.
- void parse ()

    Parses the Liberty file and extracts library information into a JSON structure.
- void modify ()
- void mono (const bool is_slew)

    Validates that lookup tables are monotonically increasing with respect to the output load.
- void supercell (const int chain_length, const std::vector< std::string > &cell_names)

Creates supercells based on standard cells in the library file.
- void verilog (const int chain_length, const std::vector< std::string > &cell_names)

  Generates Verilog files for cell validation based on the library file.
- void spice (const int chain_length, const std::vector< std::string > &cell_names, const std::string &verilog_lib_file, const std::string &spice_lib_file)

  Generates a SPICE netlist for the library based on a temporary Verilog file, using the V2LVS tool.
- std::map< std::string, std::string > logic (const std::string &cell_name)

  Retrieves the logic functions for the output pins of a specified cell within the Liberty file.

Public Attributes

- std::shared_ptr< spdlog::logger > logger_
- std::filesystem::path filepath_
- std::string basename_
- std::string filename_
- std::string libname_ = ""
- std::string jsonname_ = ""
- std::string loggername_ = ""
- json lib_json_ = json::object()

Private Member Functions

- void read ()

  Reads the Liberty file specified by the filename_ member variable.
- bool checkTimingArcMonotonicity (const json &cell, const json &pin, const json &arc, const std::string &type, bool is_slew)

  Checks if the values in a timing arc are monotonically increasing.
- std::vector< std::string > splitString (const std::string &s)

  Splits a string into a vector of tokens, using whitespace as the delimiter.
- void generateRCLines (std::ofstream &outFile, const std::string &netName, int instanceIndex, bool isFinalStage)

  Generates RC lines for a given net in either an intermediate or final stage.
- bool modifySpiceNetlist (const std::string &v2lvsSpiceFile, const std::string &finalSpiceFile, const std::string &targetGlobalLine)

  Modifies a SPICE netlist generated by v2lvs, adding metadata, inserting a global line, and adjusting instance lines within subcircuits to include VDD/VSS connections and generate corresponding R/C lines.

Private Attributes

- si2drErrorT err_
- int process_ = 0
- float voltage_ = 0.0
- int temperature_ = 0

5.7.1 Detailed Description

Definition at line 23 of file LibFile.hpp.

5.7.2 Constructor & Destructor Documentation

5.7.2.1 LibFile()

```
LibFile::LibFile (
        const std::string & filepath,
        const std::string & loggername )
```

Constructs a LibFile object with specified file path and logger name.

This constructor initializes a LibFile object with the given file path and creates a logger with the specified name. It sets up:

- Two logging sinks: a colored console sink and a file sink
- File path information including filename, basename, and a corresponding JSON filename
- Log levels (info for console, debug for file)

Parameters

filepath	The path to the file to be processed
loggername	The name for the logger and the log file

Definition at line 15 of file LibFile.cpp.

5.7.2.2 ∼LibFile()

```
LibFile::∼LibFile ( )
```

Definition at line 35 of file LibFile.cpp.

5.7.3 Member Function Documentation

5.7.3.1 checkTimingArcMonotonicity()

```
bool LibFile::checkTimingArcMonotonicity (
            const json & cell,
            const json & pin,
            const json & arc,
            const std::string & timing_arc_name,
            bool is_slew )  [private]
```

Checks if the values in a timing arc are monotonically increasing.

This function examines the values in the specified timing arc to ensure they are monotonically increasing. It checks monotonicity in two dimensions:

1. Across rows (by output load capacitance)
2. Across columns (by input slew, only if is_slew is true)

The function logs detailed information about any non-monotonic values found, including cell name, pin names, and conditional statements (when clause) if present.

Parameters

cell	The JSON object representing the cell being checked
pin	The JSON object representing the pin being checked
arc	The JSON object representing the timing arc being checked
timing_arc_name	The name of the timing arc to check (e.g., "cell_rise", "cell_fall")
is_slew	Boolean flag indicating whether to check monotonicity across input slew values

Returns

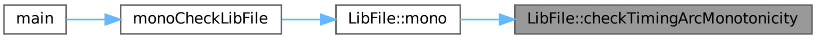
true if all values in the timing arc are monotonic, false otherwise

Exceptions

None,but	logs errors or warnings for invalid data formats or non-monotonic values

Definition at line 205 of file LibFile.cpp.

Here is the caller graph for this function:



5.7.3.2 generateRCLines()

```
void LibFile::generateRCLines (
        std::ofstream & outFile,
        const std::string & netName,
        int instanceIndex,
        bool isFinalStage ) [private]
```

Generates RC lines for a given net in either an intermediate or final stage.

This function generates SPICE-like resistor-capacitor (RC) lines and writes them to the provided output file stream. The generated RC structure differs based on whether it's an intermediate stage or the final stage. In the intermediate stage, an R1-C1-R2 structure is created. In the final stage, a simplified R1-C1 structure is used.

Parameters

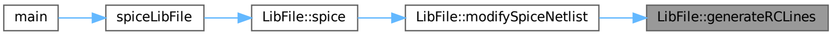
outFile	The output file stream to write the RC lines to.
netName	The name of the net to generate RC lines for. This name is used to construct node names.
instanceIndex	A unique index for the instance, used to create unique component names (R1, C1, R2).
isFinalStage	A boolean flag indicating whether this is the final stage. If true, a simplified RC structure is generated. If false, an intermediate RCR structure is generated.

- The function uses predefined default RC values (R1, C1, R2) that are specific to either the intermediate or final stage.
- The `CAP_GROUND` constant defines the ground node to which capacitors are connected.
- The output is formatted in scientific notation with a precision of 1 before writing the RC lines and then reset to default.
- Node names are constructed by appending ":1" or ":2" to the `netName` for intermediate nodes.

```
// Example usage:
std::ofstream outFile("rc_lines.sp");
LibFile::generateRCLines(outFile, "net123", 5, false); // Generate intermediate RC lines for net "net123", instance 5
LibFile::generateRCLines(outFile, "net123", 5, true);  // Generate final RC lines for net "net123", instance 5
outFile.close();
```

Definition at line 926 of file LibFile.cpp.

Here is the caller graph for this function:



5.7.3.3 logic()

```
std::map< std::string, std::string > LibFile::logic (
            const std::string & cell_name )
```

Retrieves the logic functions for the output pins of a specified cell within the Liberty file.

This method searches for a cell with the given name in the parsed Liberty file (either by parsing the file directly or reading from a JSON representation). It then iterates through the output pins of the cell, extracting the logic function associated with each pin. The logic functions are stored in a map, where the key is the pin name and the value is the logic function string.

If the JSON file does not exist, the Liberty file is parsed, and a JSON file is created. If the JSON file exists but cannot be opened or parsed, an error is logged, and an empty map is returned. If the specified cell is not found or if no logic functions are found for the cell, a warning is logged.

Parameters

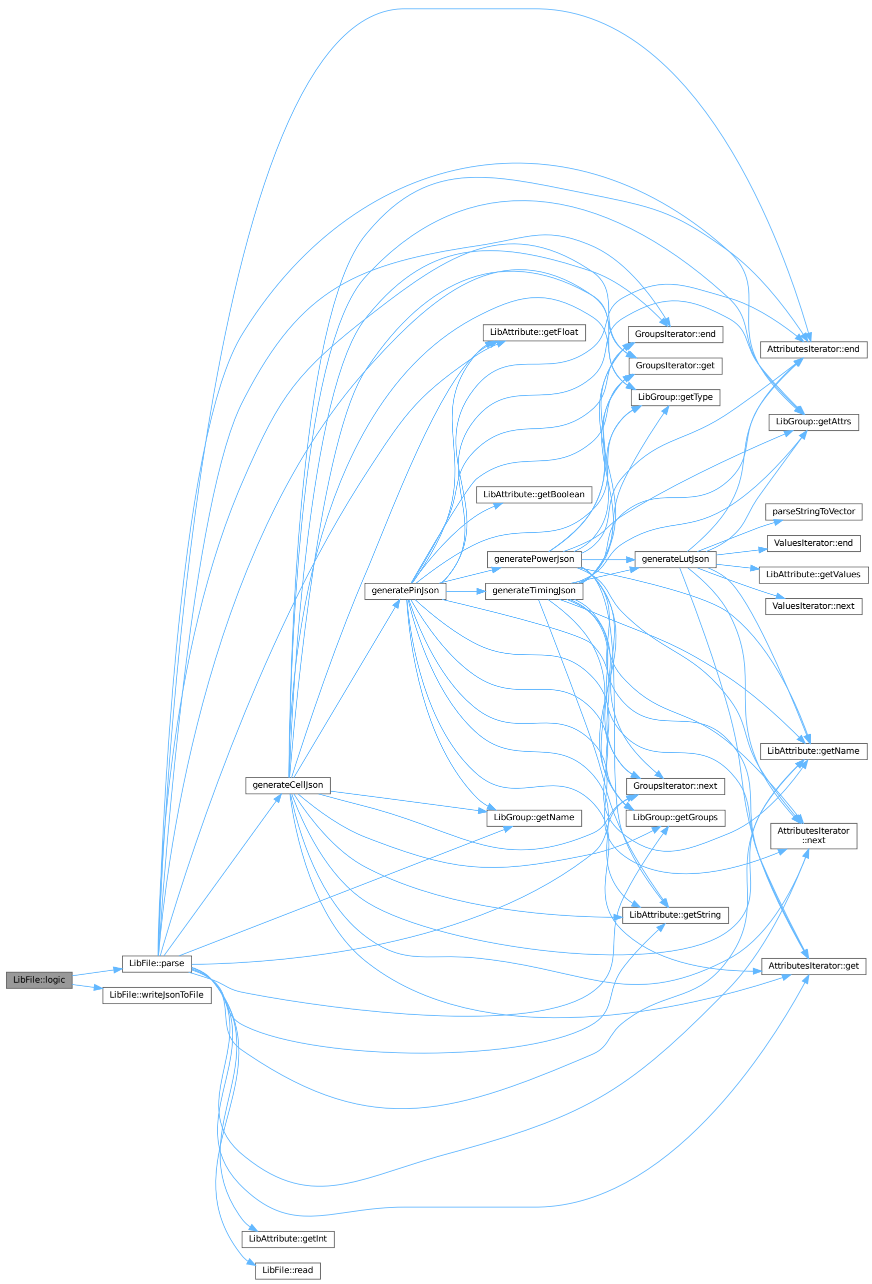
cell_name	The name of the cell for which to retrieve the logic functions.

Returns

A map containing the logic functions for the output pins of the specified cell. The keys of the map are the pin names, and the values are the corresponding logic function strings. Returns an empty map if no logic functions are found or if an error occurs during file processing.

Definition at line 1380 of file LibFile.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:

5.7.3.4 modify()

```
void LibFile::modify ( )
```

Definition at line 182 of file LibFile.cpp.

5.7.3.5 modifySpiceNetlist()

```
bool LibFile::modifySpiceNetlist (
            const std::string & v2lvsSpiceFile,
            const std::string & finalSpiceFile,
            const std::string & targetGlobalLine )  [private]
```

Modifies a SPICE netlist generated by v2lvs, adding metadata, inserting a global line, and adjusting instance lines within subcircuits to include VDD/VSS connections and generate corresponding R/C lines.

This function reads a SPICE netlist from `v2lvsSpiceFile`, modifies it according to the following rules, and writes the result to `finalSpiceFile`:

1. **Metadata Comments:** Adds a header with application name, version, author, and timestamp.
2. **Global Line Insertion:** Inserts a specified global line (`targetGlobalLine`) after the first `.INCLUDE` directive.
3. **Subcircuit Modifications:**
   - Adds `VDD VSS` to the port list of `.SUBCKT` definitions.
   - For each instance line (starting with 'X' or 'x') within a subcircuit:
     – Modifies the output pin name by appending ":1".
     – Inserts `VDD VSS` before the module name in the instance line.
     – Calls `generateRCLines` to generate and insert R/C lines for the instance.
4. **Comment Handling:** Preserves original comments, but processes any pending instance line before writing the comment.
5. **Empty/v2lvs Header Line Skipping:** Skips empty lines and lines starting with v2lvs header comments.
6. ∗∗.GLOBAL Line Skipping:∗∗ Skips original .GLOBAL lines.

The function handles case-insensitive `.SUBCKT` and `.ENDS` directives. It also manages a `previous←InstanceLine` buffer to handle instance lines that might need modification before the next line is processed.

Parameters

v2lvsSpiceFile	The path to the input SPICE netlist file generated by v2lvs.
finalSpiceFile	The path to the output SPICE netlist file after modification. This file will be overwritten if it exists.
targetGlobalLine	The string containing the global line to be inserted after the `.INCLUDE` directive.

Returns

`true` if the modification was successful, `false` otherwise (e.g., if file opening fails).

Note

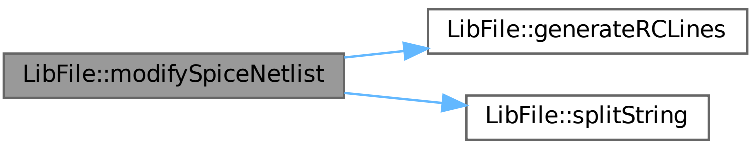
The function assumes that the second to last token in an instance line is the output net. It also assumes a specific format for instance lines where VDD/VSS insertion and R/C line generation are applicable. The `generateRCLines` method is expected to handle the actual generation of R/C components.

See also

generateRCLines

Definition at line 1008 of file LibFile.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:

5.7.3.6 mono()

```
void LibFile::mono (
            const bool is_slew )
```

Validates that lookup tables are monotonically increasing with respect to the output load.

This function checks the following timing data in the current library to ensure monotonicity:

- cell_rise and retaining_rise
- cell_fall and retaining_fall
- rise_transition and retain_rise_slew
- fall_transition and retain_fall_slew
- min_pulse_width constraints (rise_constraint, fall_constraint)

The function first checks if a parsed JSON representation exists. If not, it parses the Liberty file and creates the JSON file. It then evaluates each timing arc in all cells and reports the pass/fail status at the end.

Parameters

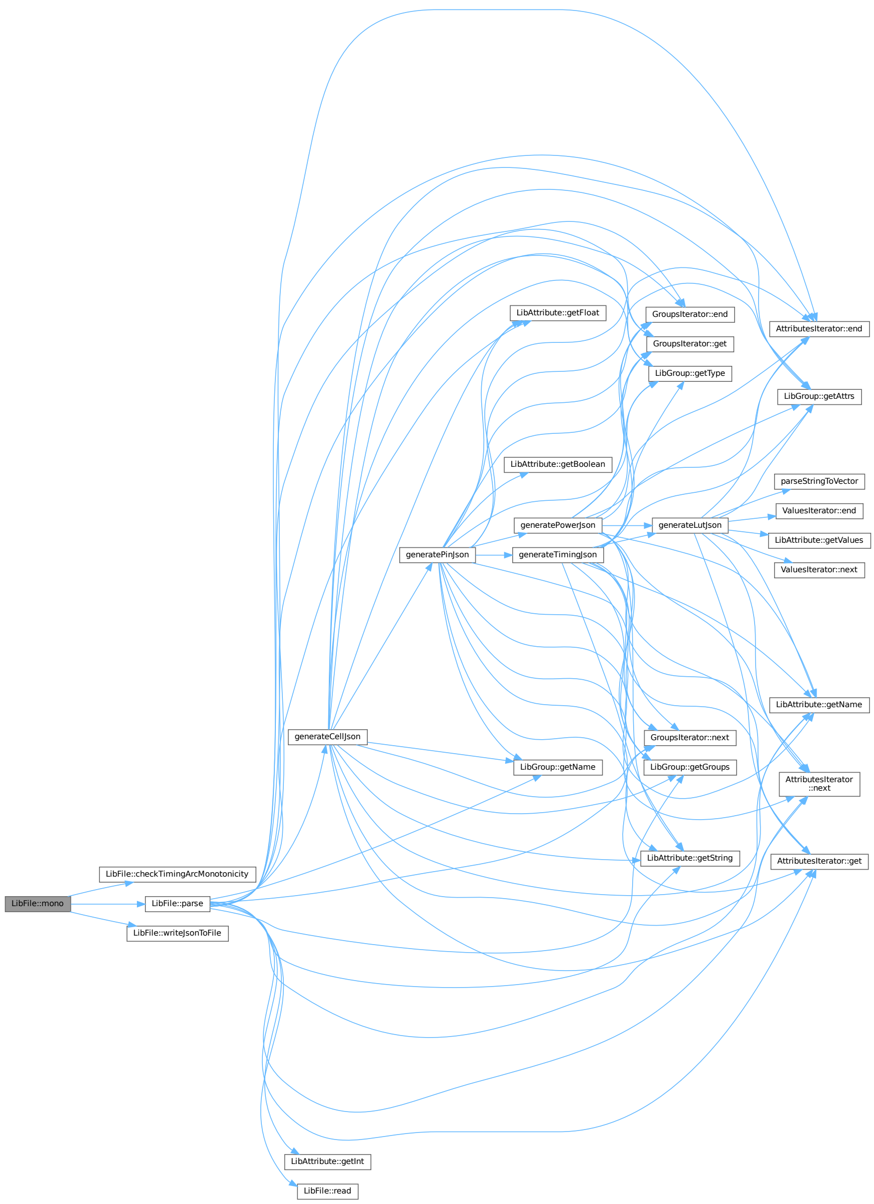
is_slew	Boolean flag indicating whether to check slew values rather than delay values

The function outputs:

- Number of cells that passed/failed the monotonicity check
- List of cells that failed the check (if any)

Definition at line 331 of file LibFile.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:

5.7.3.7 parse()

```
void LibFile::parse ( )
```

Parses the Liberty file and extracts library information into a JSON structure.

This method handles the parsing of a Liberty file by:

1. Initializing the SI2 parser interface error handler
2. Reading the Liberty file contents
3. Extracting the library name
4. Processing second-level groups including:
   - Cell definitions, which are added to the JSON structure
   - Operating conditions, extracting PVT (Process, Voltage, Temperature) values

The parsed data is stored in the lib_json_ member variable, with the following structure:

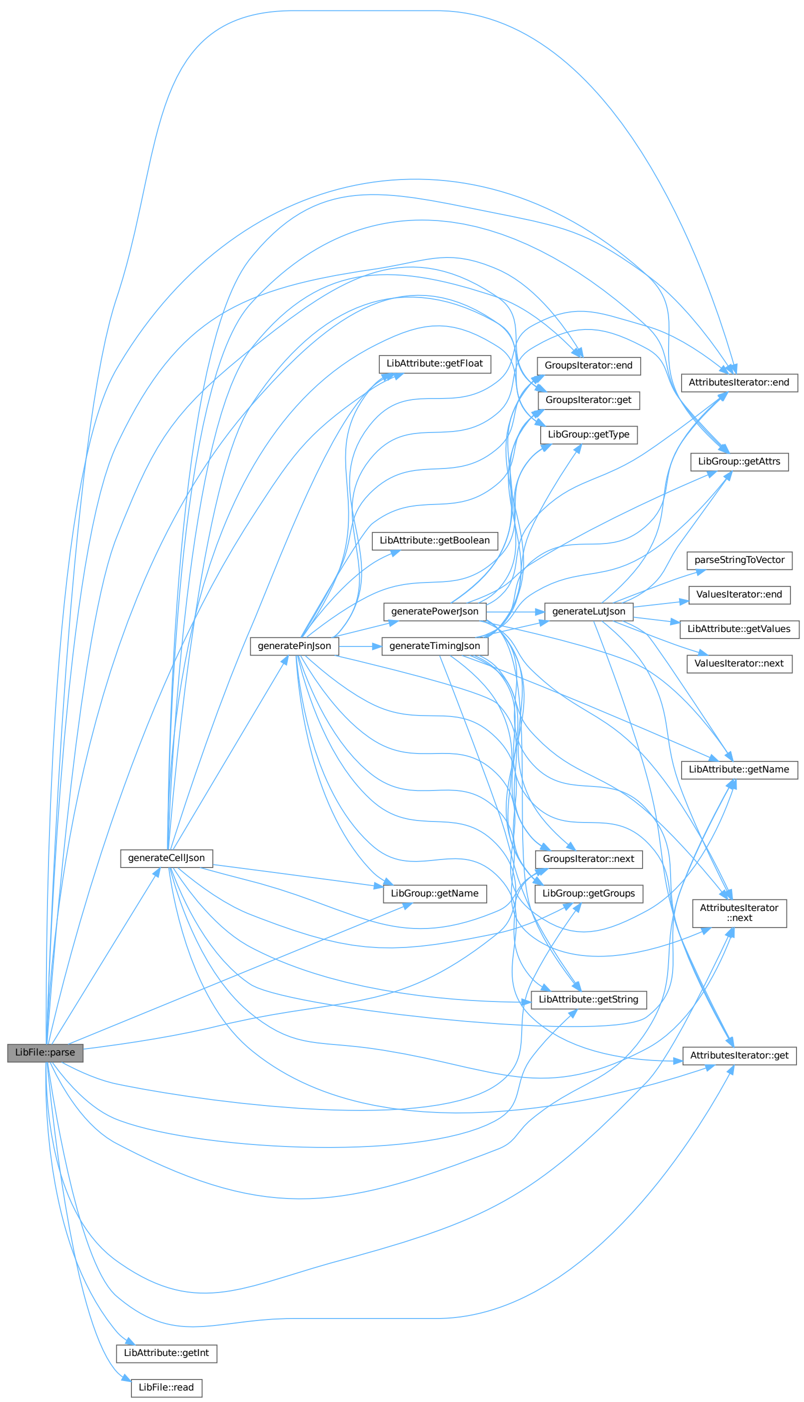
- "library_name": Name of the library
- "cells": Array of cell definitions
- "process": Process value
- "voltage": Voltage value (rounded to 2 decimal places)
- "temperature": Temperature value

Note

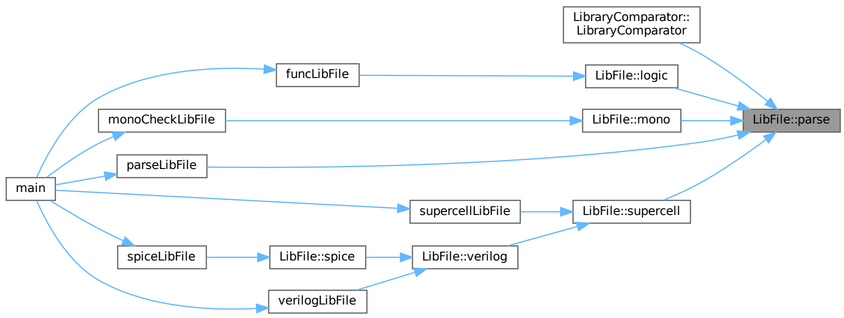
This method creates local scopes to ensure proper cleanup of SI2 Iterators.

Definition at line 116 of file LibFile.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



5.7.3.8 read()

```
void LibFile::read ( )  [private]
```

Reads the Liberty file specified by the filename_ member variable.

This function attempts to read a Liberty file and logs the process. It measures the time taken to read the file and logs any errors encountered during the read operation. If an error occurs, the function logs the error and terminates the program.

- Logs the start of the read operation.
- Measures the duration of the read operation.
- Checks for specific errors (invalid name or syntax error) and logs them.
- Terminates the program with specific exit codes if errors are detected.
- Logs the completion of the read operation and the time taken.

Note

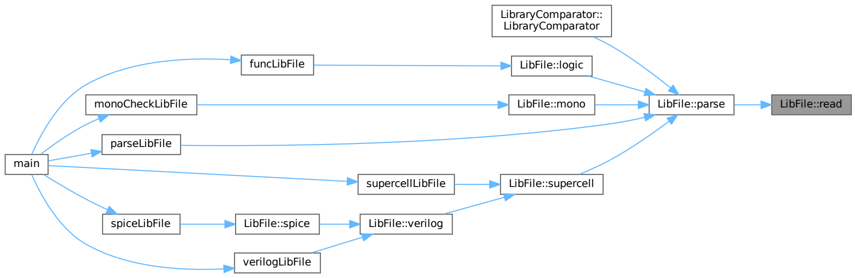
This function uses the spdlog library for logging and the si2drReadLibertyFile function for reading the Liberty file.

Exceptions

This	function will terminate the program with exit codes 301 or 401 if specific errors are encountered.

Definition at line 77 of file LibFile.cpp.

Here is the caller graph for this function:



5.7.3.9 spice()

```
void LibFile::spice (
        const int chain_length,
        const std::vector< std::string > & cell_names,
        const std::string & verilog_lib_file,
        const std::string & spice_lib_file )
```

Generates a SPICE netlist for the library based on a temporary Verilog file, using the V2LVS tool.

This function automates the process of converting a Verilog representation of the library into a SPICE netlist suitable for simulation. It involves the following steps:

1. Generates a temporary Verilog file using the `verilog` method.
2. Checks for the availability of the V2LVS tool in the system's PATH.
3. Constructs and executes the V2LVS command with appropriate options to generate an initial SPICE netlist.
4. Post-processes the generated SPICE netlist to adjust global signals using the `modifySpiceNetlist` method.
5. Logs the progress and any errors encountered during the process.

Parameters

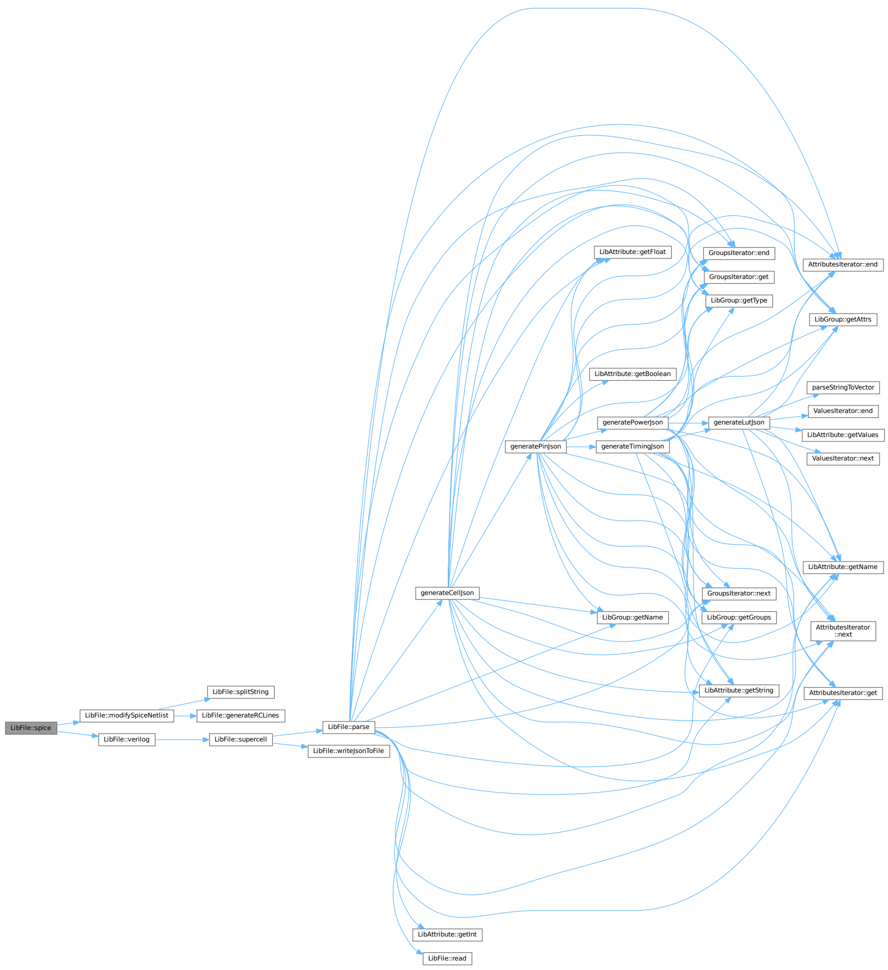
chain_length	The length of the transistor chain for Verilog generation.
cell_names	A vector of cell names to be included in the Verilog generation.
verilog_lib_file	The path to the Verilog library file used by V2LVS for pin order information.
spice_lib_file	The path to the SPICE library file to be included in the generated SPICE netlist.

Note

> The function assumes that the `verilog` method is available and correctly generates a temporary Verilog file.
>
> The V2LVS tool must be installed and accessible in the system's PATH for this function to work correctly.
>
> The generated SPICE netlist is post-processed to adjust global signals.
>
> Temporary files (Verilog and intermediate SPICE) are kept for debugging purposes.

Definition at line 1267 of file LibFile.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:

5.7.3.10 splitString()

```
std::vector< std::string > LibFile::splitString (
            const std::string & s )  [private]
```

Splits a string into a vector of tokens, using whitespace as the delimiter.

This function takes a string as input and splits it into a vector of strings, where each element in the vector represents a token from the original string. Whitespace characters (spaces, tabs, newlines, etc.) are used as delimiters to separate the tokens.

Parameters

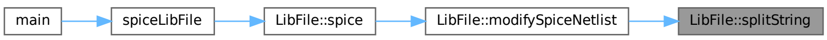
s	The string to be split.

Returns

A vector of strings, where each string is a token from the input string.

Definition at line 889 of file LibFile.cpp.

Here is the caller graph for this function:



5.7.3.11 supercell()

```
void LibFile::supercell (
        const int chain_length,
        const std::vector< std::string > & cell_names )
```

Creates supercells based on standard cells in the library file.

This method creates supercells by combining standard cells in chains. For each input-output pin pair of a cell, it creates a supercell entry with naming format: <cellname>**X**<**chain_length**><input_pin>↩
__<output_pin>. The results are written to a .map file.

For sequential cells (with clock pins), the chain length is always set to 1 regardless of the requested chain length.

Parameters

chain_length	The length of the chain for combinational cells
cell_names	Vector of specific cell names to process. If empty, all cells will be processed.

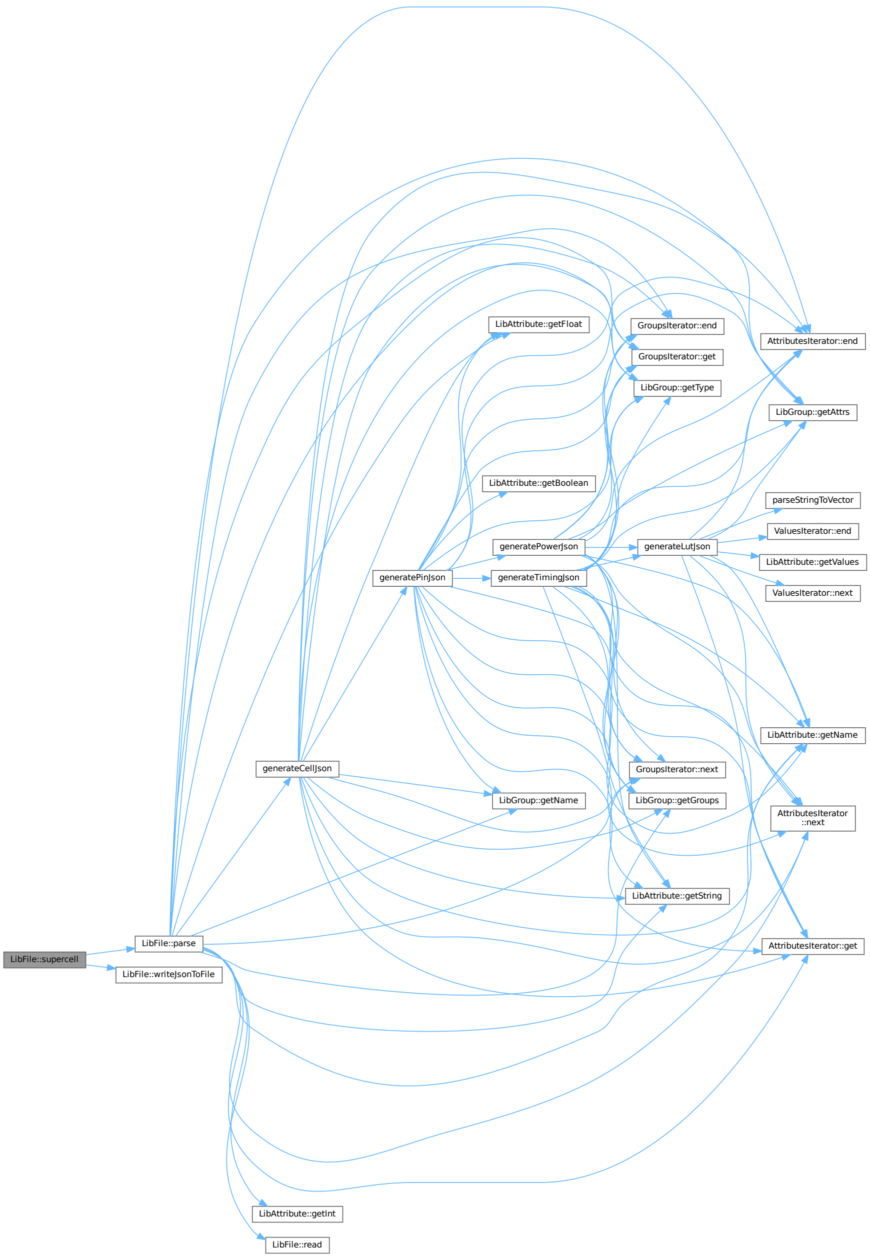
Note

Before creating supercells, this method checks for the existence of a JSON representation of the Liberty file and parses it if not found.

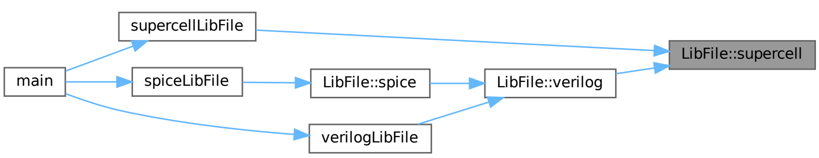
The method will report any requested cells that were not found in the library.

Definition at line 460 of file LibFile.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



5.7.3.12 verilog()

```
void LibFile::verilog (
            const int chain_length,
            const std::vector< std::string > & cell_names )
```

Generates Verilog files for cell validation based on the library file.

This function creates Verilog modules for each cell in the specified chain and generates a top-level module that connects them all together. The process involves:

1. Creating supercells based on the specified chain length and cell names
2. Reading the mapping file to identify supercell name relationships
3. Generating individual Verilog modules for each supercell, handling both sequential and combinational cells differently
4. Creating ANSI-style port definitions for each module
5. Using the slang library to build proper syntax trees for Verilog modules
6. Creating a top-level module that instantiates all the individual modules
7. Writing the complete Verilog output to the output file

Sequential cells are instantiated once, while combinational cells are instantiated multiple times based on the specified chain length.

Parameters

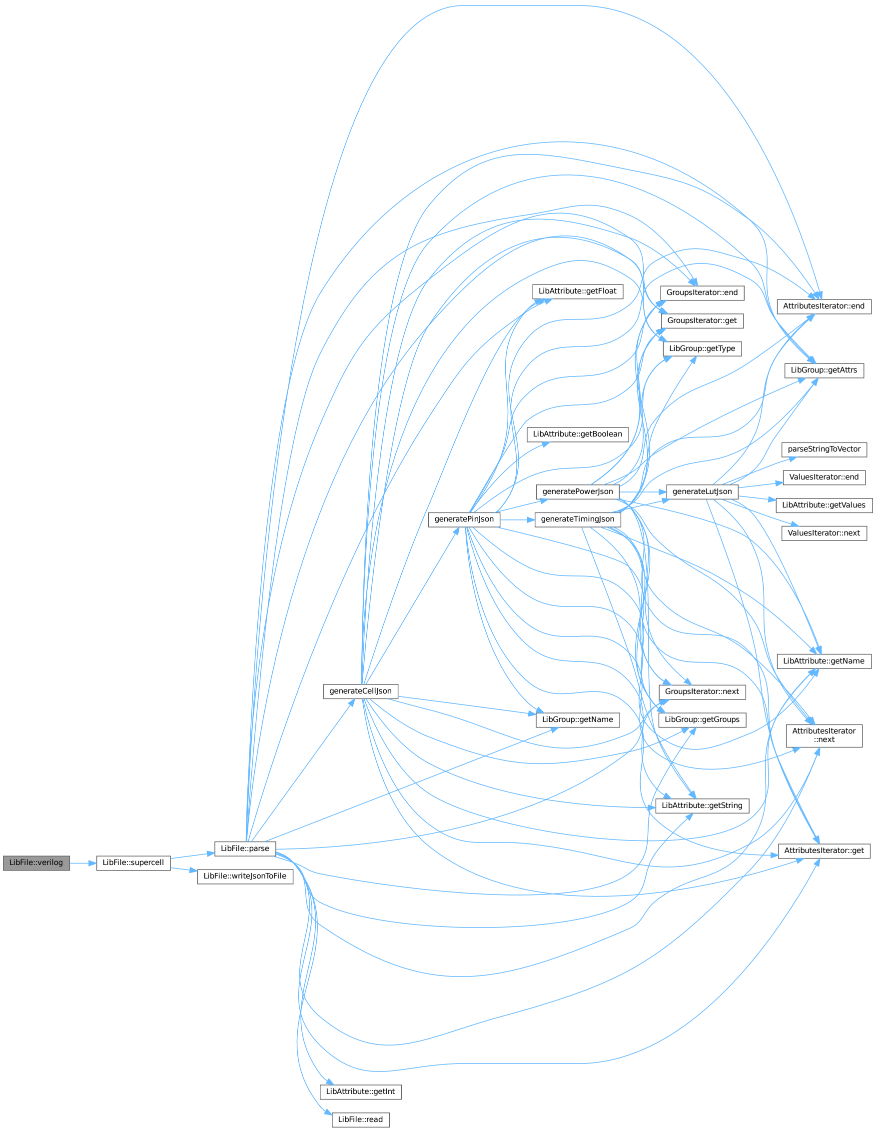
chain_length	The number of times to chain combinational cells together
cell_names	Vector of cell names to include in the Verilog generation

Note

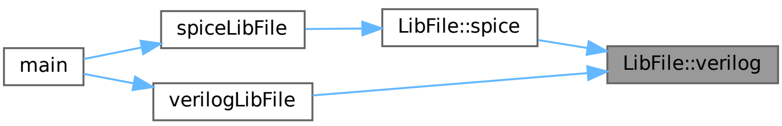
The function writes a temporary file during processing that might not be removed when the function completes (commented out cleanup code).

Definition at line 614 of file LibFile.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



5.7.3.13 writeJsonToFile()

```
void LibFile::writeJsonToFile ( )
```

Writes the JSON data stored in the object to a file.

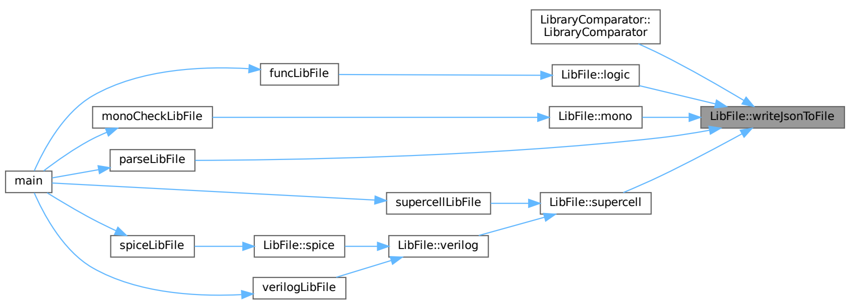
This method opens the file specified by jsonname_ and writes the content of lib_json_ with an indentation of 2 spaces. If the file cannot be opened, an error message is logged. Upon successful writing, an informational message is logged.

Exceptions

None	directly, but std::ofstream operations may throw exceptions

Definition at line 46 of file LibFile.cpp.

Here is the caller graph for this function:



5.7.4 Member Data Documentation

5.7.4.1 basename_

```
std::string LibFile::basename_
```

Definition at line 29 of file LibFile.hpp.

5.7.4.2 err_

```
si2drErrorT LibFile::err_ [private]
```

Definition at line 46 of file LibFile.hpp.

5.7.4.3 filename_

```
std::string LibFile::filename_
```

Definition at line 30 of file LibFile.hpp.

5.7.4.4 filepath_

```
std::filesystem::path LibFile::filepath_
```

Definition at line 28 of file LibFile.hpp.

5.7.4.5 jsonname_

```
std::string LibFile::jsonname_ = ""
```

Definition at line 32 of file LibFile.hpp.

5.7.4.6 lib_json_

```
json LibFile::lib_json_ = json::object()
```

Definition at line 34 of file LibFile.hpp.

5.7.4.7 libname_

```
std::string LibFile::libname_ = ""
```

Definition at line 31 of file LibFile.hpp.

5.7.4.8 logger_

```
std::shared_ptr<spdlog::logger> LibFile::logger_
```

Definition at line 27 of file LibFile.hpp.

5.7.4.9 loggername_

```
std::string LibFile::loggername_ = ""
```

Definition at line 33 of file LibFile.hpp.

5.7.4.10 process_

```
int LibFile::process_ = 0 [private]
```

Definition at line 47 of file LibFile.hpp.

5.7.4.11 temperature_

```
int LibFile::temperature_ = 0 [private]
```

Definition at line 49 of file LibFile.hpp.

5.7.4.12 voltage_

```
float LibFile::voltage_ = 0.0 [private]
```

Definition at line 48 of file LibFile.hpp.

The documentation for this class was generated from the following files:

- include/LibFile.hpp
- src/LibFile.cpp

5.8 LibGroup Class Reference

```
#include <LibGroup.hpp>
```

Public Member Functions

- LibGroup (si2drGroupIdT group, si2drErrorT &err)
- ~LibGroup ()
- std::string getName ()
- std::string getType ()
- si2drAttrsIdT getAttrs ()
- si2drGroupsIdT getGroups ()

Private Attributes

- si2drGroupIdT group_
- si2drErrorT & err_

5.8.1 Detailed Description

Definition at line 8 of file LibGroup.hpp.

5.8.2 Constructor & Destructor Documentation

5.8.2.1 LibGroup()

```
LibGroup::LibGroup (
                si2drGroupIdT group,
                si2drErrorT & err )
```

Definition at line 3 of file LibGroup.cpp.

5.8.2.2 ∼LibGroup()

```
LibGroup::∼LibGroup ( )
```

Definition at line 5 of file LibGroup.cpp.

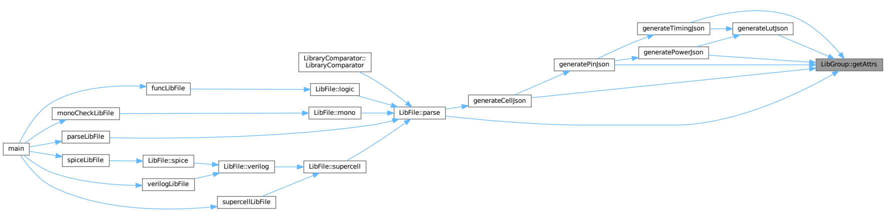
5.8.3 Member Function Documentation

5.8.3.1 getAttrs()

```
si2drAttrsIdT LibGroup::getAttrs ( )
```

Definition at line 19 of file LibGroup.cpp.

Here is the caller graph for this function:

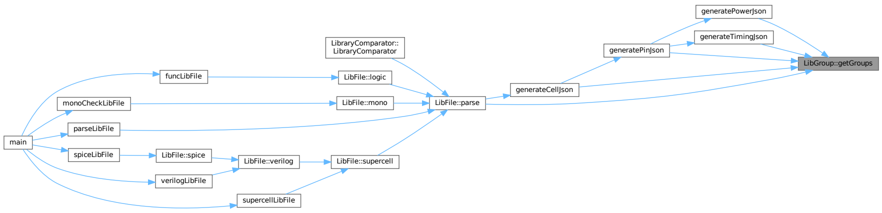


5.8.3.2 getGroups()

```
si2drGroupsIdT LibGroup::getGroups ( )
```

Definition at line 21 of file LibGroup.cpp.

Here is the caller graph for this function:

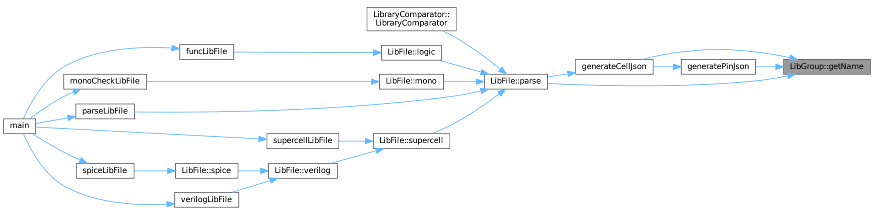


5.8.3.3 getName()

```
std::string LibGroup::getName ( )
```

Definition at line 7 of file LibGroup.cpp.

Here is the caller graph for this function:

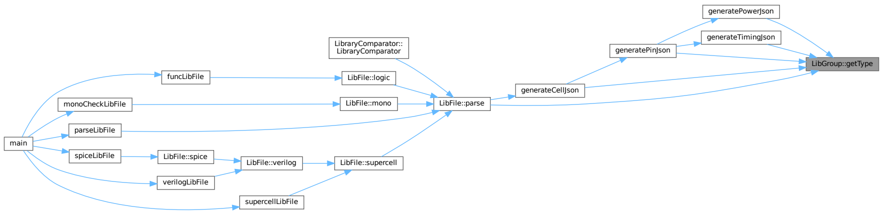


5.8.3.4 getType()

```
std::string LibGroup::getType ( )
```

Definition at line 14 of file LibGroup.cpp.

Here is the caller graph for this function:



5.8.4 Member Data Documentation

5.8.4.1 err_

```
si2drErrorT& LibGroup::err_ [private]
```

Definition at line 19 of file LibGroup.hpp.

5.8.4.2 group_

```
si2drGroupIdT LibGroup::group_ [private]
```

Definition at line 18 of file LibGroup.hpp.

The documentation for this class was generated from the following files:

- include/LibGroup.hpp
- src/LibGroup.cpp

5.9 LibraryComparator Class Reference

```
#include <LibraryComparator.hpp>
```

Public Member Functions

- LibraryComparator (LibFile &ref_libfile, LibFile &comp_libfile, double reltol, double abstol)

    Constructor for the LibraryComparator class.
- void generateReport (const std::string &output_file)

    Generates a comparison report between the reference and comparison libraries.

Public Attributes

- std::filesystem::path ref_lib_path_
- std::filesystem::path comp_lib_path_

Private Member Functions

- void compareCell (const std::string &cell_name, const json &ref_cell, const json &comp_cell, Table &table)

    Compares the output pins of a cell between a reference JSON and a comparison JSON.
- void comparePin (const std::string &cell_name, const std::string &pin_name, const json &ref_pin, const json &comp_pin, Table &table)

    Compares the timing arcs of a given pin between a reference JSON and a comparison JSON.
- void compareTimingArc (const std::string &cell_name, const std::string &pin_name, const std↩ ::string &timing_type, const json &ref_timing_arc, const json &comp_timing_arc, Table &table)

    Compares a timing arc between two JSON objects.
- void compareLut (const std::string &cell_name, const std::string &pin_name, const std::string &timing_type, const std::string &related_pin, const std::string &arc_name, const json &ref_lut, const json &comp_lut, Table &table)

    Compares lookup tables (LUTs) between reference and comparison libraries for a specific timing arc.

Private Attributes

- json ref_json_
- json comp_json_
- double reltol_
- double abstol_

5.9.1 Detailed Description

Definition at line 19 of file LibraryComparator.hpp.

5.9.2 Constructor & Destructor Documentation

5.9.2.1 LibraryComparator()

```
LibraryComparator::LibraryComparator (
        LibFile & ref_libfile,
        LibFile & comp_libfile,
        double reltol,
        double abstol )
```

Constructor for the LibraryComparator class.

Initializes a LibraryComparator object by loading JSON data from reference and comparison library files. If JSON files don't exist, parses the library files first and creates the JSON files.

The constructor performs the following steps:

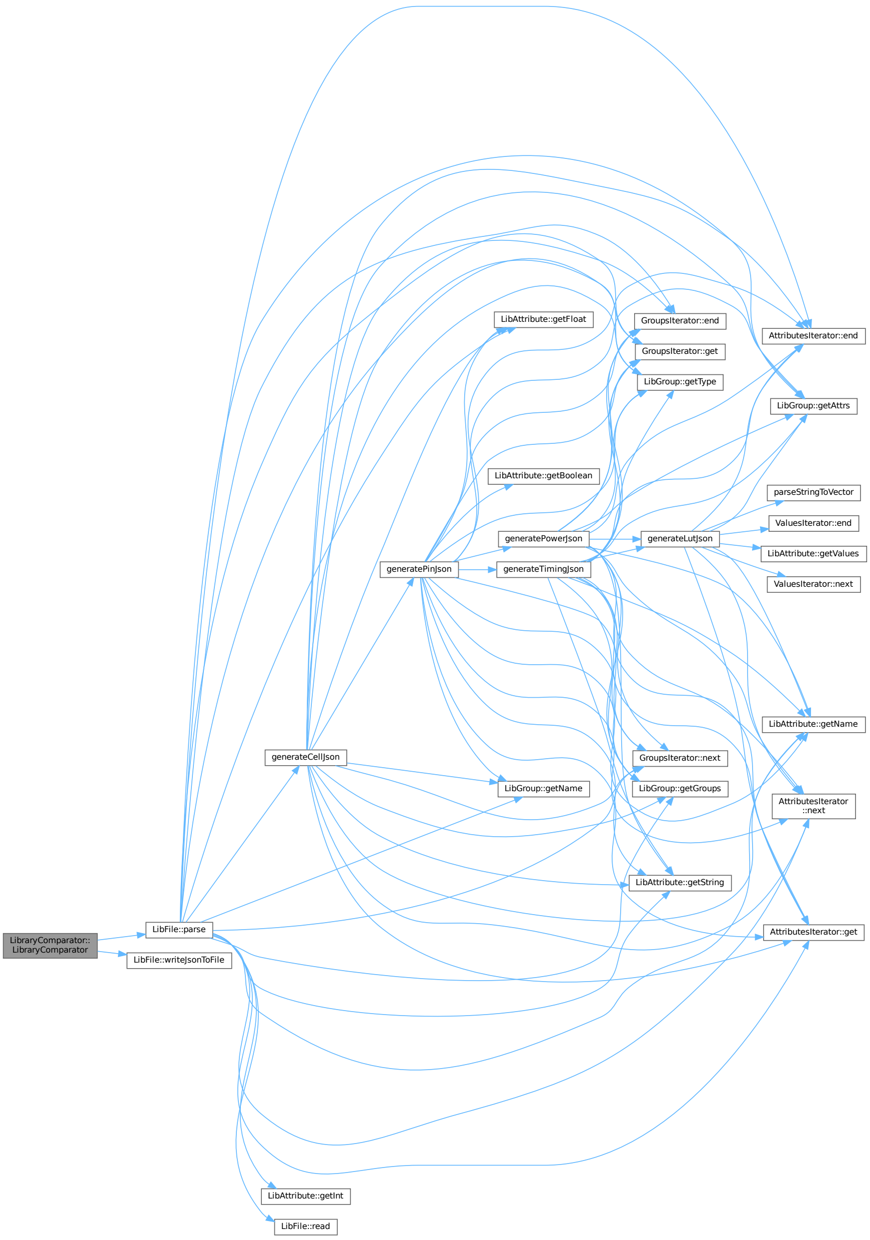
1. Looks for reference JSON file, parses library file if not found
2. Loads reference JSON data into ref_json_ member
3. Looks for comparison JSON file, parses library file if not found
4. Loads comparison JSON data into comp_json_ member
5. Stores file paths and initializes tolerance values for comparison

Parameters

ref_libfile	Reference library file object
comp_libfile	Comparison library file object
reltol	Relative tolerance for numerical comparisons (default defined elsewhere)
abstol	Absolute tolerance for numerical comparisons (default defined elsewhere)

Definition at line 21 of file LibraryComparator.cpp.

Here is the call graph for this function:



5.9.3 Member Function Documentation

5.9.3.1 compareCell()

```
void LibraryComparator::compareCell (
            const std::string & cell_name,
```

```
            const json & ref_cell,
            const json & comp_cell,
            Table & table ) [private]
```

Compares the output pins of a cell between a reference JSON and a comparison JSON.

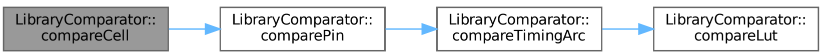
This function iterates through the output pins of the comparison cell and checks if each pin exists in the reference cell. If a pin is found in both cells, it calls the comparePin function to compare the details of the pin. If a pin is not found in the reference cell, a warning is logged. If the comparison cell does not contain any output pins, an informational message is logged.

Parameters

cell_name	The name of the cell being compared.
ref_cell	The reference JSON object containing the cell data.
comp_cell	The comparison JSON object containing the cell data.
table	The table object where the comparison results are stored.

Definition at line 301 of file LibraryComparator.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:

5.9.3.2 compareLut()

```
void LibraryComparator::compareLut (
            const std::string & cell_name,
            const std::string & pin_name,
```

```
const std::string & timing_type,
const std::string & related_pin,
const std::string & arc_name,
const json & ref_lut,
const json & comp_lut,
Table & table )  [private]
```

Compares lookup tables (LUTs) between reference and comparison libraries for a specific timing arc.

This function compares the lookup table values between reference and comparison libraries for a given timing arc, checking the consistency of indices and values. It verifies that the indices match between libraries and then compares each value in the LUT against relative and absolute tolerance thresholds. Any discrepancies that exceed the tolerance limits are recorded in the provided table.

Parameters

cell_name	The name of the cell containing the LUT.
pin_name	The name of the pin associated with the LUT.
timing_type	The timing type of the arc (e.g., "rise_transition", "fall_transition").
related_pin	The related pin for the timing arc.
arc_name	The name of the specific timing arc being compared.
ref_lut	The lookup table from the reference library (in JSON format).
comp_lut	The lookup table from the comparison library (in JSON format).
table	Table object to store comparison results for any mismatches found.

Note

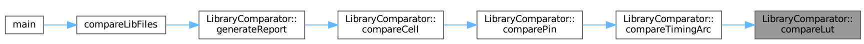
The function logs various information/warning/error messages:

- Logs index mismatches as warnings
- Logs value mismatches as debug messages
- Logs format errors in LUT structure as errors
- Records values exceeding tolerance in the provided table

The comparison uses both relative tolerance (reltol_) and absolute tolerance (abstol_) to determine if values differ significantly.

Definition at line 98 of file LibraryComparator.cpp.

Here is the caller graph for this function:



5.9.3.3 comparePin()

```
void LibraryComparator::comparePin (
        const std::string & cell_name,
        const std::string & pin_name,
        const json & ref_pin,
        const json & comp_pin,
        Table & table ) [private]
```

Compares the timing arcs of a given pin between a reference JSON and a comparison JSON.

This function iterates through the timing arcs of the comparison pin and looks for matching timing types in the reference pin. If a matching timing type is found, it calls the compareTimingArc function to compare the timing arcs. If a timing type is not found in the reference JSON, a warning is logged.

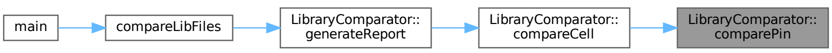
Parameters

cell_name	The name of the cell containing the pin.
pin_name	The name of the pin to compare.
ref_pin	The reference JSON object containing the pin's data.
comp_pin	The comparison JSON object containing the pin's data.
table	A reference to a Table object where comparison results are stored.

Definition at line 261 of file LibraryComparator.cpp.

Here is the call graph for this function:

Here is the caller graph for this function:



5.9.3.4 compareTimingArc()

```
void LibraryComparator::compareTimingArc (
        const std::string & cell_name,
        const std::string & pin_name,
        const std::string & timing_type,
        const json & ref_timing_arc,
        const json & comp_timing_arc,
        Table & table ) [private]
```

Compares a timing arc between two JSON objects.

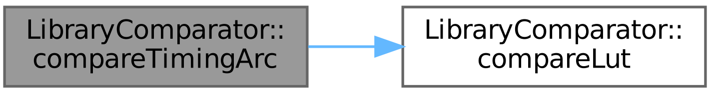
This function compares a specific timing arc (e.g., cell_rise, cell_fall, rise_transition, fall_transition) between a reference JSON object and a comparison JSON object. It extracts the related pin from the reference timing arc and iterates through a predefined list of arc names. If an arc name is found in the comparison timing arc, it checks if the same arc name exists in the reference timing arc. If both exist, it calls the compareLut function to compare the LUT data associated with the arc. If the arc name is found in the comparison JSON but not in the reference JSON, a warning message is logged.

Parameters

cell_name	The name of the cell.
pin_name	The name of the pin.
timing_type	The type of timing (e.g., "combinational", "sequential").
ref_timing_arc	The reference JSON object containing the timing arc information.
comp_timing_arc	The comparison JSON object containing the timing arc information.
table	The table to store comparison results.

Definition at line 220 of file LibraryComparator.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:

5.9.3.5 generateReport()

```
void LibraryComparator::generateReport (
            const std::string & output_file )
```

Generates a comparison report between the reference and comparison libraries.

This function generates a detailed comparison report between the reference library and the comparison library. The report includes metadata such as the reference and comparison library paths, absolute and relative tolerances, and the application details. It also includes a legend explaining various symbols used in the report.

The function iterates through each cell in the comparison library, compares it with the corresponding cell in the reference library, and generates a table of differences. If there are any differences, it calculates summary statistics such as the average and maximum differences, and the number of outliers. The results are written to the specified output file in either Markdown or plain text format.

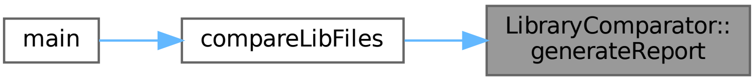
Parameters

output_file	The path to the output file where the report will be written.

Definition at line 343 of file LibraryComparator.cpp.

Here is the call graph for this function:

Here is the caller graph for this function:



5.9.4 Member Data Documentation

5.9.4.1 abstol_

`double LibraryComparator::abstol_ [private]`

Definition at line 30 of file LibraryComparator.hpp.

5.9.4.2 comp_json_

`json LibraryComparator::comp_json_ [private]`

Definition at line 28 of file LibraryComparator.hpp.

5.9.4.3 comp_lib_path_

`std::filesystem::path LibraryComparator::comp_lib_path_`

Definition at line 23 of file LibraryComparator.hpp.

5.9.4.4 ref_json_

`json LibraryComparator::ref_json_ [private]`

Definition at line 27 of file LibraryComparator.hpp.

5.9.4.5 ref_lib_path_

```
std::filesystem::path LibraryComparator::ref_lib_path_
```

Definition at line 22 of file LibraryComparator.hpp.

5.9.4.6 reltol_

```
double LibraryComparator::reltol_ [private]
```

Definition at line 29 of file LibraryComparator.hpp.

The documentation for this class was generated from the following files:

- include/LibraryComparator.hpp
- src/LibraryComparator.cpp

5.10 LogicComparator Class Reference

```
#include <LogicComparator.hpp>
```

Public Member Functions

- LogicComparator (const std::map< std::string, std::string > &ref_outpin_map, const std::map< std::string, std::string > &comp_outpin_map, const std::string &cell_name)
- void logic ()

  This function demonstrates the usage of the exprtk library to evaluate a boolean logic expression.
- std::string preprocessExpression (const std::string &input_expr)

  Preprocesses a logical expression string to prepare it for evaluation.
- bool extractVariables (const std::string &expr1, const std::string &expr2, std::vector< std::string > &sorted_vars)

  Extracts and validates variables from two expressions, ensuring they match.
- void compareSingleExpressionPair (const std::string &ref_expression_processed, const std::string &comp_expression_processed, const std::vector< std::string > &sorted_vars, PinComparisonResult &result)

  Compares two preprocessed logic expression strings for equivalence. Internal helper function.
- void compareCellLogic ()

  Compares logic for all output pins defined in the input maps, and stores results in all_pin_results_.
- void generateReport (const std::string &output_file)

  Generates a comparison report file based on cell logic comparison results.

Private Attributes

- std::map< std::string, std::string > ref_outpin_map_
- std::map< std::string, std::string > comp_outpin_map_
- std::string cell_name_
- std::map< std::string, PinComparisonResult > all_pin_results_

5.10.1 Detailed Description

Definition at line 38 of file LogicComparator.hpp.

5.10.2 Constructor & Destructor Documentation

5.10.2.1 LogicComparator()

```
LogicComparator::LogicComparator (
        const std::map< std::string, std::string > & ref_outpin_map,
        const std::map< std::string, std::string > & comp_outpin_map,
        const std::string & cell_name )
```

Definition at line 3 of file LogicComparator.cpp.

5.10.3 Member Function Documentation

5.10.3.1 compareCellLogic()

```
void LogicComparator::compareCellLogic ( )
```

Compares logic for all output pins defined in the input maps, and stores results in all_pin_results_.

Compares the logic expressions for each output pin of a cell between a reference and a comparison design.

This method performs a detailed comparison of the logic expressions associated with each output pin of a cell. It iterates through each unique pin name found in both the reference and comparison designs, retrieves their corresponding logic expressions, preprocesses them, extracts and validates the variables used, and then compares the processed expressions. The results of each pin comparison, including any errors or discrepancies, are stored in the `all_pin_results_` map.

The comparison process includes the following steps:

1. **Pin Collection:** Collects all unique pin names from both the reference (`ref_outpin_map_`) and comparison (`comp_outpin_map_`) maps.
2. **Iteration:** Iterates through each unique pin name.
3. **Expression Retrieval:** Retrieves the raw logic expressions for the current pin from both the reference and comparison maps. If a pin is missing in either map, an error is logged, and the comparison is marked as impossible for that pin.
4. **Preprocessing:** Preprocesses the raw expressions to simplify them and remove any irrelevant characters or formatting. If preprocessing results in an empty expression, an error is logged, and the comparison is marked as impossible.
5. **Variable Extraction and Validation:** Extracts the variables used in both expressions and validates that the sets of variables match. If the variable sets do not match, an error is logged, and the comparison is marked as impossible.
6. **Expression Comparison:** Compares the preprocessed expressions using the extracted variables. This step determines whether the logic functions represented by the expressions are equivalent.
7. **Result Storage:** Stores the detailed results of the pin comparison, including the raw and processed expressions, any errors encountered, and the comparison result, in the `all_pin_results_` map.

Note

The `spdlog` library is used for logging information, warnings, and errors throughout the comparison process.

The `preprocessExpression` method is used to simplify the raw logic expressions before comparison.

The `extractVariables` method is used to extract and validate the variables used in the logic expressions.

The `compareSingleExpressionPair` method is used to compare the preprocessed expressions and determine whether they are equivalent.

See also

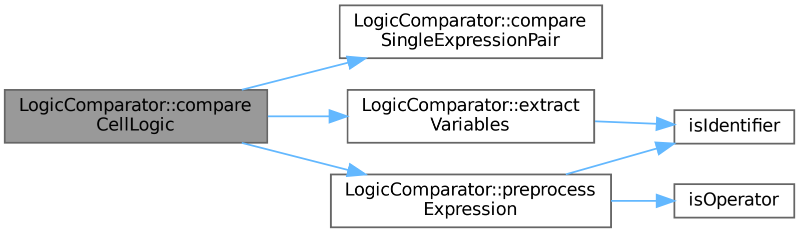
preprocessExpression

extractVariables

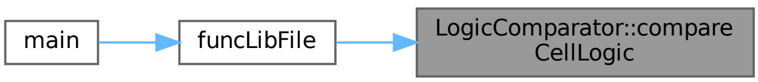
compareSingleExpressionPair

Definition at line 808 of file LogicComparator.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



5.10.3.2 compareSingleExpressionPair()

```
void LogicComparator::compareSingleExpressionPair (
            const std::string & ref_expression_processed,
            const std::string & comp_expression_processed,
            const std::vector< std::string > & sorted_vars,
            PinComparisonResult & result )
```

Compares two preprocessed logic expression strings for equivalence. Internal helper function.

Compares two boolean expressions for logical equivalence using a truth table.

Template Parameters

T	The numeric type used by ExprTk (e.g., double, float).

Parameters

ref_expression_processed	The preprocessed reference logic expression.

Parameters

comp_expression_processed	The preprocessed comparison logic expression.
sorted_vars	A sorted vector of unique variable names common to both expressions.
result	Reference to a PinComparisonResult object to store detailed results.

This function takes two processed boolean expressions (ref_expression_processed and comp_expression↩ _processed), a sorted list of variable names (sorted_vars), and a PinComparisonResult struct to store the results. It evaluates both expressions for all possible combinations of variable values and compares the resulting truth tables.

The function uses the ExprTk library to parse and evaluate the boolean expressions. It generates a truth table for each expression and compares the results. If the truth tables are identical, the expressions are considered logically equivalent.

Parameters

ref_expression_processed	The processed reference boolean expression.
comp_expression_processed	The processed comparison boolean expression.
sorted_vars	A sorted vector of variable names used in the expressions.
result	A PinComparisonResult struct to store the comparison results, including processed expressions, compilation status, equivalence, error messages, and the generated truth tables.

Note

The function limits the number of variables to 20 to prevent excessively large truth tables. It also performs overflow checking on the number of combinations.

The function uses spdlog for logging debug, warning, and error messages.

The function assumes that the input expressions are valid boolean expressions containing only the variables listed in sorted_vars.

Definition at line 549 of file LogicComparator.cpp.

Here is the caller graph for this function:

5.10.3.3 extractVariables()

```
bool LogicComparator::extractVariables (
        const std::string & expr1_raw,
        const std::string & expr2_raw,
        std::vector< std::string > & sorted_vars )
```

Extracts and validates variables from two expressions, ensuring they match.

This function parses two raw expression strings (`expr1_raw` and `expr2_raw`) to identify potential variable names. It uses a regular expression to find identifiers and then validates them against a set of rules:

1. The identifier must be a valid identifier as determined by the `isIdentifier` function.
2. The identifier must not be a keyword or function name defined in the `keywords` set.

The function compares the sets of validated variables from both expressions. If the sets are identical, the variables are extracted, sorted alphabetically, and stored in the `sorted_vars` vector. If the sets differ, an error is reported, and the differences between the sets are logged.

Parameters

expr1_raw	The raw string representation of the first expression.
expr2_raw	The raw string representation of the second expression.
sorted_vars	A vector to store the sorted list of unique variable names if the variable sets from both expressions match. This vector is cleared if the sets do not match.

Returns

`true` if the variable sets from both expressions match, indicating that the `sorted_vars` vector contains the sorted list of unique variable names. `false` if the variable sets do not match, indicating an error.

Note

The `isIdentifier` function is used to validate potential variable names.

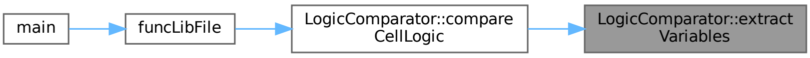
The `keywords` set contains a list of reserved words that cannot be used as variable names.

The function uses spdlog for logging debug, trace, info, and warning messages.

Definition at line 323 of file LogicComparator.cpp.

Here is the call graph for this function:

Here is the caller graph for this function:



5.10.3.4 generateReport()

```
void LogicComparator::generateReport (
            const std::string & output_file )
```

Generates a comparison report file based on cell logic comparison results.

Generates a comprehensive report comparing the logic equivalence of a cell's reference and comparison expressions.

Parameters

output_file	Path to the output report file.

This function performs the following steps:

1. **Initialization:** Sets up logging and determines the output format (Markdown or plain text) based on the file extension.
2. **File Handling:** Opens the specified output file in append mode, creating it if it doesn't exist. Handles potential file opening errors.

3. **Report Header:** Writes a header section to the report, including the cell name, application version, author, and timestamp.
4. **Legend Generation:** Creates a legend explaining the status symbols used in the report (e.g., [OK], [NO], [NA]).
5. **Table Generation:** Creates tables for reference pin functions, comparison pin functions, and the legend.
6. **Pin Result Processing:** Iterates through the results for each pin, generating detailed information including:
   - Pin Name
   - Reference and Comparison Truth Tables (if available)
   - Comparison Summary Table:
     - Status (Logic Equivalence)
     - Raw and Processed Expressions
     - Compilation Status
     - Error Messages (if any)
7. **Output:** Exports the generated tables and pin results to the output file in the determined format (Markdown or plain text).
8. **Closure:** Closes the output file and logs the completion of the report generation.

Parameters

output_file	The path to the output report file. If the file extension is ".md", the report will be generated in Markdown format; otherwise, it will be generated in plain text.

Note

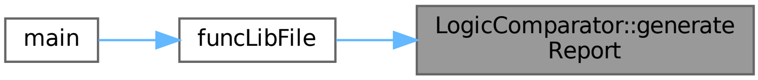
The function uses `spdlog` for logging and relies on several helper classes/structs:

- `Table`: For creating formatted tables.
- `MarkdownExporter`: For exporting tables to Markdown format.
- `LogicComparator::PinResult`: A struct containing the results of the logic comparison for a single pin.

The function appends to the output file if it already exists.

Definition at line 960 of file LogicComparator.cpp.

Here is the caller graph for this function:



5.10.3.5 logic()

```
void LogicComparator::logic ( )
```

This function demonstrates the usage of the exprtk library to evaluate a boolean logic expression.

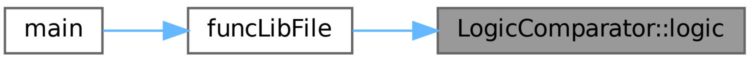
It defines a boolean expression "not(A and B) or C" and evaluates it for all possible combinations of boolean values for the variables A, B, and C. The results are then printed to the console in a tabular format.

The function utilizes the exprtk library for:

- Defining a symbol table to hold the variables A, B, and C.
- Creating an expression object and registering the symbol table with it.
- Compiling the boolean expression string into the expression object.
- Iterating through all possible boolean combinations for A, B, and C.
- Assigning the boolean values to the variables in the symbol table.
- Evaluating the expression using expression.value().
- Printing the input values and the result to the console.

Definition at line 24 of file LogicComparator.cpp.

Here is the caller graph for this function:



5.10.3.6 preprocessExpression()

```
std::string LogicComparator::preprocessExpression (
            const std::string & input_expr )
```

Preprocesses a logical expression string to prepare it for evaluation.

This function performs several steps to transform the input expression:

1. Trims leading/trailing whitespace.
2. Replaces the logical NOT operator '!' with " not " (except for '!=').
3. Adds spaces around operators (+, ^, *) and parentheses.
4. Replaces symbolic operators (+, ^, *) with their keyword equivalents (or, xor, and).
5. Tokenizes the expression based on spaces.
6. Inserts implied 'and' operators between operands where necessary. For example, "A B" becomes "A and B".
7. Handles 'not Identifier' sequences by converting them to 'not(Identifier)'.
8. Reconstructs the final expression string from the processed tokens, adding spaces appropriately.
9. Performs a final cleanup to consolidate multiple spaces and trim the result.

Parameters

input_expr	The input logical expression string.

Returns

The preprocessed logical expression string, ready for evaluation. Returns an empty string if the input is empty or if no tokens are found after processing.

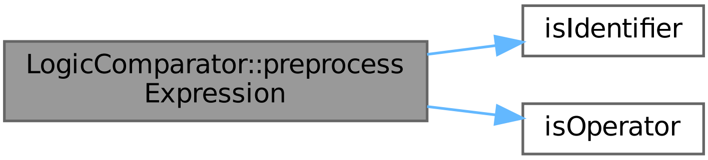
Note

The function uses spdlog for debugging and tracing.

The function assumes the existence of helper functions `isIdentifier` and `isOperator` to classify tokens.

Definition at line 124 of file LogicComparator.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:

5.10.4 Member Data Documentation

5.10.4.1 all_pin_results_

`std::map<std::string, PinComparisonResult> LogicComparator::all_pin_results_ [private]`

Definition at line 86 of file LogicComparator.hpp.

5.10.4.2 cell_name_

`std::string LogicComparator::cell_name_ [private]`

Definition at line 85 of file LogicComparator.hpp.

5.10.4.3 comp_outpin_map_

```
std::map<std::string, std::string> LogicComparator::comp_outpin_map_ [private]
```

Definition at line 84 of file LogicComparator.hpp.

5.10.4.4 ref_outpin_map_

```
std::map<std::string, std::string> LogicComparator::ref_outpin_map_ [private]
```

Definition at line 83 of file LogicComparator.hpp.

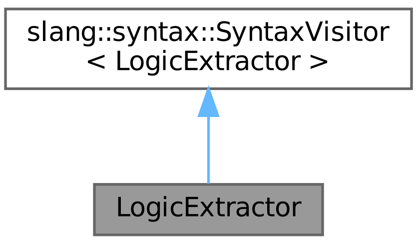
The documentation for this class was generated from the following files:

- include/LogicComparator.hpp
- src/LogicComparator.cpp

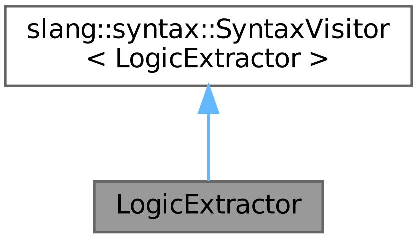
5.11 LogicExtractor Class Reference

```
#include <LogicExtractor.hpp>
```

Inheritance diagram for LogicExtractor:



Collaboration diagram for LogicExtractor:



Public Member Functions

- LogicExtractor (const std::string &targetCell)
- void handle (const slang::syntax::ModuleDeclarationSyntax &module)

  Handles a module declaration syntax node.
- void handle (const slang::syntax::PortDeclarationSyntax &portDecl)

  Handles a port declaration syntax node.
- void handle (const slang::syntax::NonAnsiPortListSyntax &portList)

  Handles a non-ANSI port list in a module.
- void handle (const slang::syntax::NetDeclarationSyntax &netDecl)

  Handles a net declaration syntax node.
- void handle (const slang::syntax::PrimitiveInstantiationSyntax &primitiveInst)

  Handles the extraction of logic gate information from a primitive instantiation syntax node.
- const std::unordered_map< std::string, GateInfo > & getExtractedGates () const
- const std::unordered_set< std::string > & getPrimaryInputs () const
- const std::unordered_set< std::string > & getPrimaryOutputs () const
- const std::unordered_set< std::string > & getInternalWires () const
- std::map< std::string, std::string > getLogicExpressions ()

  Extracts logic expressions for all primary output ports of the target cell.

Private Member Functions

- std::string deriveLogicRecursive (const std::string &signalName)

  Recursively derives the logic expression for a given signal.
- std::string formatExpression (const GateInfo &gateInfo, const std::vector< std::string > &input↩ Exprs)

  Formats a logic expression based on the gate type and input expressions.

Private Attributes

- const std::string & targetCell_
- bool inTargetModule_
- bool parsingComplete_
- std::unordered_set< std::string > primaryInputs_
- std::unordered_set< std::string > primaryOutputs_
- std::unordered_set< std::string > internalWires_
- std::unordered_map< std::string, std::string > portDirections_
- std::unordered_map< std::string, GateInfo > gateOutputDrivers_
- std::unordered_map< std::string, std::string > logicCache_

5.11.1 Detailed Description

Definition at line 17 of file LogicExtractor.hpp.

5.11.2 Constructor & Destructor Documentation

5.11.2.1 LogicExtractor()

```
LogicExtractor::LogicExtractor (
            const std::string & targetCell )  [inline], [explicit]
```

Definition at line 20 of file LogicExtractor.hpp.

5.11.3 Member Function Documentation

5.11.3.1 deriveLogicRecursive()

```
std::string LogicExtractor::deriveLogicRecursive (
            const std::string & signalName )  [private]
```

Recursively derives the logic expression for a given signal.

This function traces back the signal to its driving gates and primary inputs, constructing a logic expression that represents the signal's behavior. It uses memoization to cache previously computed expressions, avoiding redundant calculations.

Parameters

signalName	The name of the signal for which to derive the logic expression.

Returns

A string representing the logic expression for the signal.

Exceptions

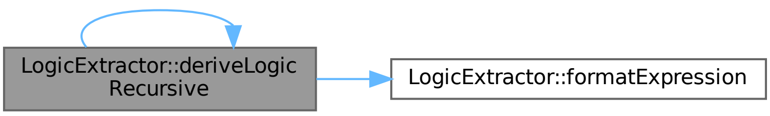
std::runtime_error	if the signal is not a primary input, a known wire, or driven by a recognized gate/assignment. Also thrown if an empty input signal name is encountered or if an internal wire has no identified driver.

The function operates in the following steps:

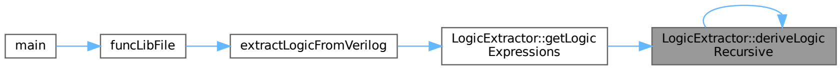
1. **Check Cache (Memoization):** If the logic expression for the signal is already cached in `logic↩ Cache_`, it is immediately returned.
2. **Base Case: Is it a primary input?** If the signal is a primary input (present in `primaryInputs_`), its logic expression is simply its own name.
3. **Recursive Step: Is it driven by a gate?** If the signal is driven by a gate (present in `gate↩ OutputDrivers_`), the function recursively derives the logic expressions for all input signals of the gate. These input expressions are then used to format the overall expression for the current signal based on the gate type.
4. **Handle Assign statements:** (Currently not implemented but reserved for future use).
5. **Error Case:** If the signal is not found in any of the above categories, it indicates an error. An exception is thrown, indicating that the signal is either an undriven internal wire or an unknown signal.

Definition at line 507 of file LogicExtractor.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



5.11.3.2 formatExpression()

```
std::string LogicExtractor::formatExpression (
            const GateInfo & gateInfo,
            const std::vector< std::string > & inputExprs )  [private]
```

Formats a logic expression based on the gate type and input expressions.

This function takes a GateInfo structure describing the gate and a vector of input expressions (strings) and constructs a logic expression string representing the gate's operation. It handles AND, NAND, OR, NOR, XOR, XNOR, NOT, and BUF gates. It uses '*', '+', '^', and '!' operators for AND, OR, XOR, and NOT respectively. If the gate type is unsupported, or if the number of inputs is incorrect for NOT/BUF gates, or if a gate requiring inputs receives none, an error string is returned, and a warning is logged.

Parameters

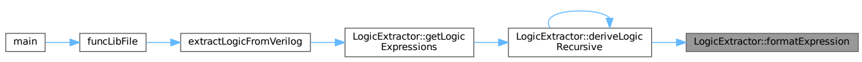
gateInfo	A GateInfo struct containing information about the gate, including its type (slang::parsing::TokenKind) and name.
inputExprs	A vector of strings representing the input expressions to the gate.

Returns

A string representing the formatted logic expression, or an error string if formatting fails due to unsupported gate type or incorrect number of inputs.

Definition at line 581 of file LogicExtractor.cpp.

Here is the caller graph for this function:

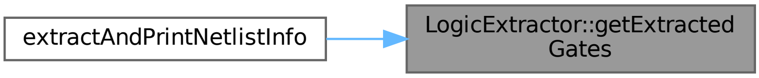


5.11.3.3 getExtractedGates()

const std::unordered_map< std::string, GateInfo > & LogicExtractor::getExtractedGates ( ) const [inline]

Definition at line 37 of file LogicExtractor.hpp.

Here is the caller graph for this function:

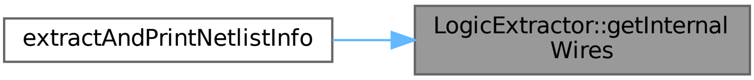


5.11.3.4 getInternalWires()

const std::unordered_set< std::string > & LogicExtractor::getInternalWires ( ) const [inline]

Definition at line 42 of file LogicExtractor.hpp.

Here is the caller graph for this function:



5.11.3.5 getLogicExpressions()

std::map< std::string, std::string > LogicExtractor::getLogicExpressions ( )

Extracts logic expressions for all primary output ports of the target cell.

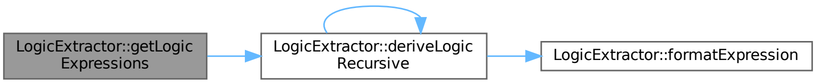
This method iterates through the primary output ports, derives the logic expression for each, and stores the result in a map. If an error occurs during the derivation of logic for a particular output, an error message is stored in the map instead.

Returns

A map where the key is the output port name (std::string) and the value is the corresponding logic expression (std::string). If AST parsing is not complete or the target module is not found, an empty map is returned. If an error occurs during logic derivation for a specific output, the corresponding value in the map will be an error message.

Definition at line 448 of file LogicExtractor.cpp.

Here is the call graph for this function:



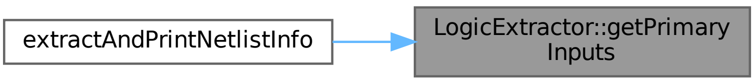
Here is the caller graph for this function:

5.11.3.6 getPrimaryInputs()

`const std::unordered_set< std::string > & LogicExtractor::getPrimaryInputs ( ) const [inline]`

Definition at line 40 of file LogicExtractor.hpp.

Here is the caller graph for this function:

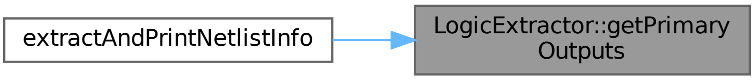


5.11.3.7 getPrimaryOutputs()

```
const std::unordered_set< std::string > & LogicExtractor::getPrimaryOutputs ( ) const  [inline]
```

Definition at line 41 of file LogicExtractor.hpp.

Here is the caller graph for this function:



5.11.3.8 handle() [1/5]

```
void LogicExtractor::handle (
            const slang::syntax::ModuleDeclarationSyntax & module )
```

Handles a module declaration syntax node.

This function is the entry point for processing a module declaration. It checks if the module is the target module and, if so, extracts relevant information such as primary inputs, primary outputs, and internal wires. It also visits the children of the module node to process ports, declarations, and instances.

The function ensures that the target module is only processed once and that parsing stops after the target module is found.

Parameters

module	A reference to the module declaration syntax node.

Definition at line 18 of file LogicExtractor.cpp.

5.11.3.9 handle() [2/5]

```
void LogicExtractor::handle (
            const slang::syntax::NetDeclarationSyntax & netDecl )
```

Handles a net declaration syntax node.

This function processes a net declaration syntax node, extracting the names of declared wires. It checks if the current scope is within the target module and avoids adding ports again if they were already declared as nets. The extracted wire names are added to the internal wires set.

Parameters

netDecl	A reference to the NetDeclarationSyntax node to handle.

Definition at line 277 of file LogicExtractor.cpp.

5.11.3.10 handle() [3/5]

```
void LogicExtractor::handle (
            const slang::syntax::NonAnsiPortListSyntax & portList )
```

Handles a non-ANSI port list in a module.

This method iterates through the ports in a non-ANSI port list, extracts the port name, determines its direction (input, output, or inout), and adds it to the appropriate sets (primaryInputs_, primaryOutputs↩_, internalWires_). It also handles different types of port declarations, including implicit, explicit, and empty ports.

Parameters

portList	A reference to the NonAnsiPortListSyntax object representing the port list.

- Skips processing if not in the target module (inTargetModule_ is false).
- Extracts port names from ImplicitNonAnsiPortSyntax and ExplicitNonAnsiPortSyntax.
- Handles PortConcatenationSyntax by logging a warning and skipping the port.
- Skips EmptyNonAnsiPortSyntax (placeholders).
- Uses the portDirections_ map to determine the direction of each port.
- Adds ports to primaryInputs_ and internalWires_ if the direction is "input".
- Adds ports to primaryOutputs_ and internalWires_ if the direction is "output".
- Adds ports to both primaryInputs_ and primaryOutputs_ if the direction is "inout".
- Logs warnings for ports with unknown or missing directions.

- Logs warnings if the port name cannot be determined.

Note

This method does not call visitDefault.

Definition at line 179 of file LogicExtractor.cpp.

5.11.3.11 handle() [4/5]

```
void LogicExtractor::handle (
            const slang::syntax::PortDeclarationSyntax & portDecl )
```

Handles a port declaration syntax node.

This method extracts information about port declarations within a SystemVerilog module, including the port's direction (input, output, or inout) and name. It updates internal data structures to track primary inputs, primary outputs, internal wires, and port directions.

Parameters

portDecl	A reference to the PortDeclarationSyntax node being processed.

- It first checks if the current processing context is within the target module. If not, the method returns early.
- It determines the port's direction by inspecting the port header (VariablePortHeaderSyntax or Net↩ PortHeaderSyntax). The direction is extracted using `valueText()` to handle keywords represented as text tokens.
- It iterates through the declarators in the port declaration to extract port names.
- If multiple direction declarations are found for the same port, a warning is logged, and the first declared direction is retained.
- Based on the port's direction, the port name is added to the appropriate sets:
  - `primaryInputs_`: For input and inout ports.
  - `primaryOutputs_`: For output and inout ports.
  - `internalWires_`: For all ports (input, output, inout, and ports with unknown directions).
- If a port has an unknown or missing direction, it's treated as an internal wire, and a warning is logged.

- The method avoids calling `visitDefault` to prevent unexpected revisits of syntax nodes.

Note

The `logicCache_` update is commented out, indicating it's part of a later processing step.

Warning

Logs warnings for port declarations without headers, port declarators without names, and multiple direction declarations for the same port.

Definition at line 92 of file LogicExtractor.cpp.

5.11.3.12 handle() [5/5]

```
void LogicExtractor::handle (
            const slang::syntax::PrimitiveInstantiationSyntax & primitiveInst )
```

Handles the extraction of logic gate information from a primitive instantiation syntax node.

This method processes a primitive instantiation, extracting the gate type, input signals, and output signal. It populates the `gateOutputDrivers_` map, which stores the driving gate information for each output signal. It also identifies internal wires and checks for multiple drivers on the same signal.

Parameters

primitiveInst	A reference to the PrimitiveInstantiationSyntax node representing the gate instance.

Definition at line 306 of file LogicExtractor.cpp.

5.11.4 Member Data Documentation

5.11.4.1 gateOutputDrivers_

```
std::unordered_map<std::string, GateInfo> LogicExtractor::gateOutputDrivers_ [private]
```

Definition at line 63 of file LogicExtractor.hpp.

5.11.4.2 inTargetModule_

`bool LogicExtractor::inTargetModule_ [private]`

Definition at line 50 of file LogicExtractor.hpp.

5.11.4.3 internalWires_

`std::unordered_set<std::string> LogicExtractor::internalWires_ [private]`

Definition at line 56 of file LogicExtractor.hpp.

5.11.4.4 logicCache_

`std::unordered_map<std::string, std::string> LogicExtractor::logicCache_ [private]`

Definition at line 66 of file LogicExtractor.hpp.

5.11.4.5 parsingComplete_

`bool LogicExtractor::parsingComplete_ [private]`

Definition at line 51 of file LogicExtractor.hpp.

5.11.4.6 portDirections_

`std::unordered_map<std::string, std::string> LogicExtractor::portDirections_ [private]`

Definition at line 60 of file LogicExtractor.hpp.

5.11.4.7 primaryInputs_

```
std::unordered_set<std::string> LogicExtractor::primaryInputs_ [private]
```

Definition at line 54 of file LogicExtractor.hpp.

5.11.4.8 primaryOutputs_

```
std::unordered_set<std::string> LogicExtractor::primaryOutputs_ [private]
```

Definition at line 55 of file LogicExtractor.hpp.

5.11.4.9 targetCell_

```
const std::string& LogicExtractor::targetCell_ [private]
```

Definition at line 49 of file LogicExtractor.hpp.

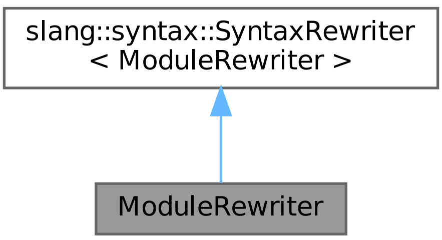
The documentation for this class was generated from the following files:

- include/LogicExtractor.hpp
- src/LogicExtractor.cpp

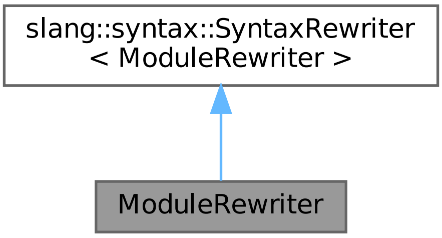
5.12 ModuleRewriter Class Reference

```
#include <verilog_utils.hpp>
```

Inheritance diagram for ModuleRewriter:



Collaboration diagram for ModuleRewriter:



Public Member Functions

- ModuleRewriter (const std::vector< std::string > &inputPins, const std::vector< std::string > &outputPins, const std::pair< std::string, std::string > &supercell_entry, int instance_count, std::shared_ptr< spdlog::logger > logger)
- void handle (const slang::syntax::SyntaxNode &node)

  Processes a syntax node in the module's AST.
- void handle (const slang::syntax::ModuleDeclarationSyntax &module)

Public Attributes

- std::shared_ptr< spdlog::logger > logger_

Private Attributes

- const std::vector< std::string > & inputPins_
- const std::vector< std::string > & outputPins_
- const std::string cellName_
- const std::string moduleName_
- std::map< std::string, std::string > portInfoMap_
- int depth_
- int instance_count_

5.12.1 Detailed Description

Definition at line 59 of file verilog_utils.hpp.

5.12.2 Constructor & Destructor Documentation

5.12.2.1 ModuleRewriter()

```
ModuleRewriter::ModuleRewriter (
        const std::vector< std::string > & inputPins,
        const std::vector< std::string > & outputPins,
        const std::pair< std::string, std::string > & supercell_entry,
        int instance_count,
        std::shared_ptr< spdlog::logger > logger ) [inline], [explicit]
```

Definition at line 61 of file verilog_utils.hpp.

5.12.3 Member Function Documentation

5.12.3.1 handle() [1/2]

```
void ModuleRewriter::handle (
        const slang::syntax::ModuleDeclarationSyntax & module )
```

Definition at line 406 of file verilog_utils.cpp.

5.12.3.2 handle() [2/2]

```
void ModuleRewriter::handle (
        const slang::syntax::SyntaxNode & node )
```

Processes a syntax node in the module's AST.

This method is called for each syntax node during the traversal of the AST. It logs debug information about the current node and continues processing its child nodes by recursively calling the appropriate visit methods.

Parameters

node	The syntax node to process

Definition at line 396 of file verilog_utils.cpp.

5.12.4 Member Data Documentation

5.12.4.1 cellName_

```
const std::string ModuleRewriter::cellName_ [private]
```

Definition at line 75 of file verilog_utils.hpp.

5.12.4.2 depth_

```
int ModuleRewriter::depth_ [private]
```

Definition at line 78 of file verilog_utils.hpp.

5.12.4.3 inputPins_

```
const std::vector<std::string>& ModuleRewriter::inputPins_ [private]
```

Definition at line 73 of file verilog_utils.hpp.

5.12.4.4 instance_count_

```
int ModuleRewriter::instance_count_ [private]
```

Definition at line 79 of file verilog_utils.hpp.

5.12.4.5 logger_

```
std::shared_ptr<spdlog::logger> ModuleRewriter::logger_
```

Definition at line 68 of file verilog_utils.hpp.

5.12.4.6 moduleName_

```
const std::string ModuleRewriter::moduleName_ [private]
```

Definition at line 76 of file verilog_utils.hpp.

5.12.4.7 outputPins_

```
const std::vector<std::string>& ModuleRewriter::outputPins_ [private]
```

Definition at line 74 of file verilog_utils.hpp.

5.12.4.8 portInfoMap_

```
std::map<std::string, std::string> ModuleRewriter::portInfoMap_ [private]
```

Definition at line 77 of file verilog_utils.hpp.

The documentation for this class was generated from the following files:

- include/verilog_utils.hpp
- src/verilog_utils.cpp

5.13 PinComparisonResult Struct Reference

```
#include <LogicComparator.hpp>
```

Public Attributes

- std::string pin_name
- std::string ref_expr_raw
- std::string comp_expr_raw
- std::string ref_expr_processed
- std::string comp_expr_processed
- bool comparison_possible = false
- bool are_equivalent = false
- bool ref_compiles = false
- bool comp_compiles = false
- std::optional< Table > ref_truth_table
- std::optional< Table > comp_truth_table
- std::string error_message

5.13.1 Detailed Description

Definition at line 23 of file LogicComparator.hpp.

5.13.2 Member Data Documentation

5.13.2.1 are_equivalent

```
bool PinComparisonResult::are_equivalent = false
```

Definition at line 30 of file LogicComparator.hpp.

5.13.2.2 comp_compiles

```
bool PinComparisonResult::comp_compiles = false
```

Definition at line 32 of file LogicComparator.hpp.

5.13.2.3 comp_expr_processed

```
std::string PinComparisonResult::comp_expr_processed
```

Definition at line 28 of file LogicComparator.hpp.

5.13.2.4 comp_expr_raw

```
std::string PinComparisonResult::comp_expr_raw
```

Definition at line 26 of file LogicComparator.hpp.

5.13.2.5 comp_truth_table

```
std::optional<Table> PinComparisonResult::comp_truth_table
```

Definition at line 34 of file LogicComparator.hpp.

5.13.2.6 comparison_possible

```
bool PinComparisonResult::comparison_possible = false
```

Definition at line 29 of file LogicComparator.hpp.

5.13.2.7 error_message

```
std::string PinComparisonResult::error_message
```

Definition at line 35 of file LogicComparator.hpp.

5.13.2.8 pin_name

```
std::string PinComparisonResult::pin_name
```

Definition at line 24 of file LogicComparator.hpp.

5.13.2.9 ref_compiles

```
bool PinComparisonResult::ref_compiles = false
```

Definition at line 31 of file LogicComparator.hpp.

5.13.2.10 ref_expr_processed

```
std::string PinComparisonResult::ref_expr_processed
```

Definition at line 27 of file LogicComparator.hpp.

5.13.2.11 ref_expr_raw

```
std::string PinComparisonResult::ref_expr_raw
```

Definition at line 25 of file LogicComparator.hpp.

5.13.2.12 ref_truth_table

```
std::optional<Table> PinComparisonResult::ref_truth_table
```

Definition at line 33 of file LogicComparator.hpp.

The documentation for this struct was generated from the following file:

- include/LogicComparator.hpp

5.14 ValuesIterator Class Reference

```
#include <Iterators.hpp>
```

Public Member Functions

- ValuesIterator (si2drValuesIdT values, si2drErrorT &err)
- ~ValuesIterator ()
- void next ()
- bool end ()

Public Attributes

- si2drValuesIdT values_
- si2drValueTypeT vtype_
- si2drInt32T int_
- si2drFloat64T float_
- si2drStringT str_
- si2drBooleanT bool_
- si2drExprT * exprp_
- si2drErrorT & err_

5.14.1 Detailed Description

Definition at line 37 of file Iterators.hpp.

5.14.2 Constructor & Destructor Documentation

5.14.2.1 ValuesIterator()

```
ValuesIterator::ValuesIterator (
        si2drValuesIdT values,
        si2drErrorT & err )
```

Definition at line 24 of file Iterators.cpp.

5.14.2.2 ∼ValuesIterator()

```
ValuesIterator::~ValuesIterator ( )
```

Definition at line 27 of file Iterators.cpp.

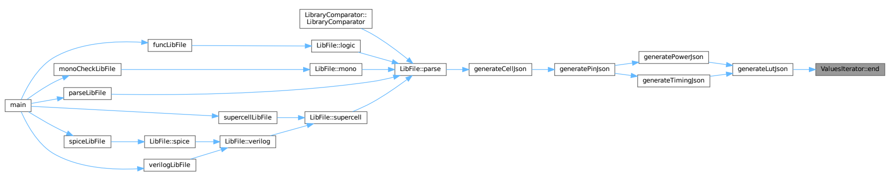
5.14.3 Member Function Documentation

5.14.3.1 end()

```
bool ValuesIterator::end ( )
```

Definition at line 32 of file Iterators.cpp.

Here is the caller graph for this function:

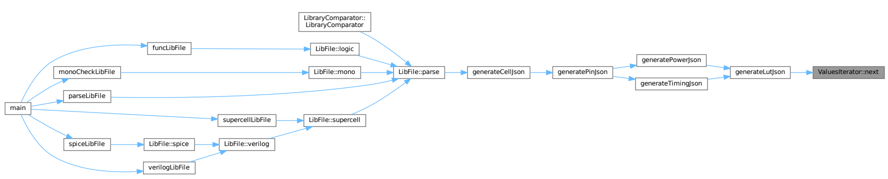


5.14.3.2 next()

```
void ValuesIterator::next ( )
```

Definition at line 29 of file Iterators.cpp.

Here is the caller graph for this function:



5.14.4 Member Data Documentation

5.14.4.1 bool_

`si2drBooleanT ValuesIterator::bool_`

Definition at line 49 of file Iterators.hpp.

5.14.4.2 err_

`si2drErrorT& ValuesIterator::err_`

Definition at line 51 of file Iterators.hpp.

5.14.4.3 exprp_

`si2drExprT* ValuesIterator::exprp_`

Definition at line 50 of file Iterators.hpp.

5.14.4.4 float_

`si2drFloat64T ValuesIterator::float_`

Definition at line 47 of file Iterators.hpp.

5.14.4.5 int_

`si2drInt32T ValuesIterator::int_`

Definition at line 46 of file Iterators.hpp.

5.14.4.6 str_

`si2drStringT ValuesIterator::str_`

Definition at line 48 of file Iterators.hpp.

5.14.4.7 values_

`si2drValuesIdT ValuesIterator::values_`

Definition at line 44 of file Iterators.hpp.

5.14.4.8 vtype_

`si2drValueTypeT ValuesIterator::vtype_`

Definition at line 45 of file Iterators.hpp.

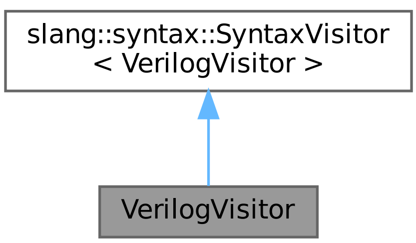
The documentation for this class was generated from the following files:

- include/Iterators.hpp
- src/Iterators.cpp

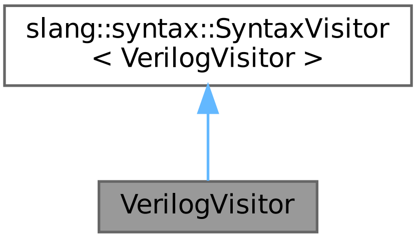
5.15 VerilogVisitor Class Reference

`#include <verilog_utils.hpp>`

Inheritance diagram for VerilogVisitor:



Collaboration diagram for VerilogVisitor:



Public Member Functions

- VerilogVisitor (const std::string &targetCell)
- void handle (const slang::syntax::SyntaxNode &node)
- void handle (const slang::syntax::ModuleDeclarationSyntax &module)
- void handle (const slang::syntax::PortDeclarationSyntax &portDecl)
- void handle (const slang::syntax::HierarchyInstantiationSyntax &hierarchyInst)
- void handle (const slang::syntax::SpecifyBlockSyntax &specifyBlock)

Private Attributes

- const std::string & targetCell_
- int depth_
- bool inTargetModule_

5.15.1 Detailed Description

Definition at line 15 of file verilog_utils.hpp.

5.15.2 Constructor & Destructor Documentation

5.15.2.1 VerilogVisitor()

```
VerilogVisitor::VerilogVisitor (
            const std::string & targetCell )  [inline], [explicit]
```

Definition at line 17 of file verilog_utils.hpp.

5.15.3 Member Function Documentation

5.15.3.1 handle() [1/5]

```
void VerilogVisitor::handle (
            const slang::syntax::HierarchyInstantiationSyntax & hierarchyInst )
```

Definition at line 78 of file verilog_utils.cpp.

5.15.3.2 handle() [2/5]

```
void VerilogVisitor::handle (
            const slang::syntax::ModuleDeclarationSyntax & module )
```

Definition at line 17 of file verilog_utils.cpp.

5.15.3.3 handle() [3/5]

```
void VerilogVisitor::handle (
            const slang::syntax::PortDeclarationSyntax & portDecl )
```

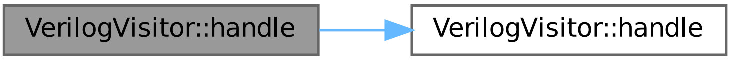
Definition at line 46 of file verilog_utils.cpp.

5.15.3.4 handle() [4/5]

```
void VerilogVisitor::handle (
            const slang::syntax::SpecifyBlockSyntax & specifyBlock )
```

Definition at line 258 of file verilog_utils.cpp.

Here is the call graph for this function:

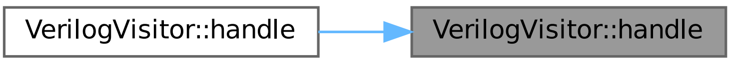


5.15.3.5 handle() [5/5]

```
void VerilogVisitor::handle (
            const slang::syntax::SyntaxNode & node )
```

Definition at line 6 of file verilog_utils.cpp.

Here is the caller graph for this function:



5.15.4 Member Data Documentation

5.15.4.1 depth_

```
int VerilogVisitor::depth_  [private]
```

Definition at line 28 of file verilog_utils.hpp.

5.15.4.2 inTargetModule_

```
bool VerilogVisitor::inTargetModule_  [private]
```

Definition at line 29 of file verilog_utils.hpp.

5.15.4.3 targetCell_

```
const std::string& VerilogVisitor::targetCell_  [private]
```

Definition at line 27 of file verilog_utils.hpp.

The documentation for this class was generated from the following files:

- include/verilog_utils.hpp
- src/verilog_utils.cpp

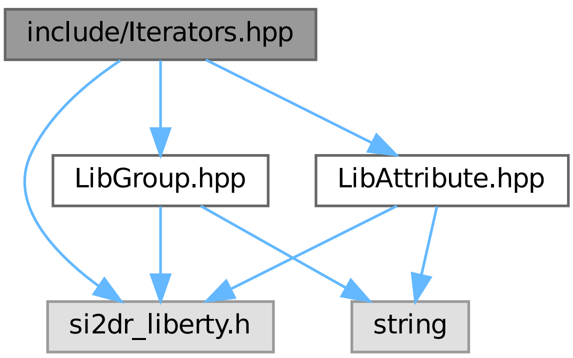
Chapter 6

File Documentation

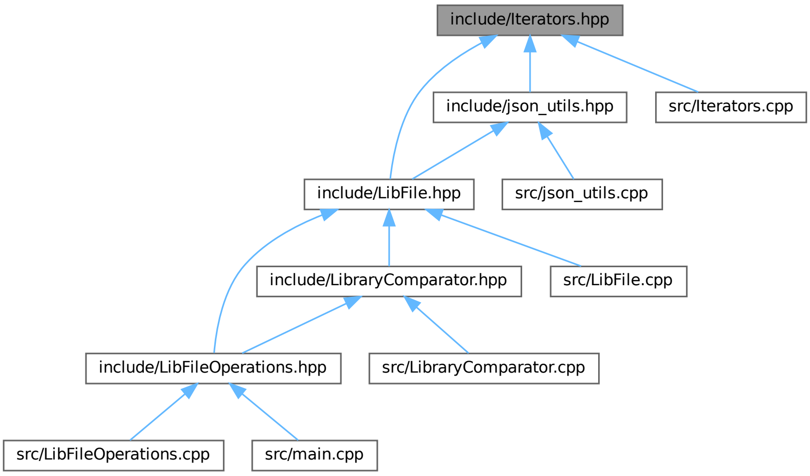
6.1 include/Iterators.hpp File Reference

`#include "si2dr_liberty.h"`
`#include "LibAttribute.hpp"`
`#include "LibGroup.hpp"`

Include dependency graph for Iterators.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class GroupsIterator
- class AttributesIterator
- class ValuesIterator

6.2 Iterators.hpp

Go to the documentation of this file.

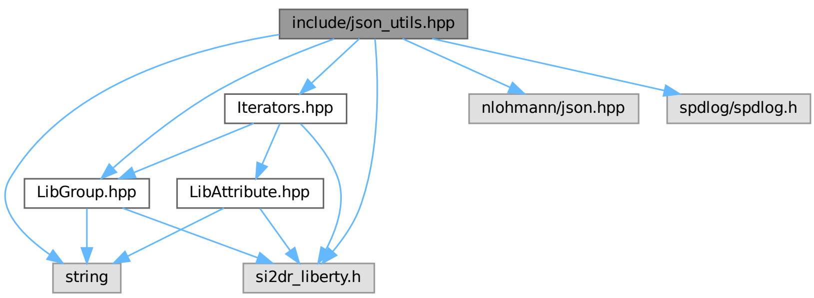
```
#ifndef ITERATORS_H
#define ITERATORS_H

#include "si2dr_liberty.h"

#include "LibAttribute.hpp"
#include "LibGroup.hpp"

class GroupsIterator {
public:
  GroupsIterator(si2drGroupsIdT groups, si2drErrorT &err);
  ~GroupsIterator();
  void next();
  bool end();
  LibGroup get();

private:
  si2drGroupsIdT groups_;
  si2drGroupIdT group_;
  si2drErrorT &err_;
};

class AttributesIterator {
public:
```

```
    AttributesIterator(si2drAttrsIdT attrs, si2drErrorT &err);
    ~AttributesIterator();
    void next();
    bool end();
    LibAttribute get();

private:
    si2drAttrsIdT attrs_;
    si2drAttrIdT attr_;
    si2drErrorT &err_;
};

class ValuesIterator {
public:
    ValuesIterator(si2drValuesIdT values, si2drErrorT &err);
    ~ValuesIterator();
    void next();
    bool end();

    si2drValuesIdT values_;
    si2drValueTypeT vtype_;
    si2drInt32T int_;
    si2drFloat64T float_;
    si2drStringT str_;
    si2drBooleanT bool_;
    si2drExprT *exprp_;
    si2drErrorT &err_;
};

#endif // ITERATORS_H
```

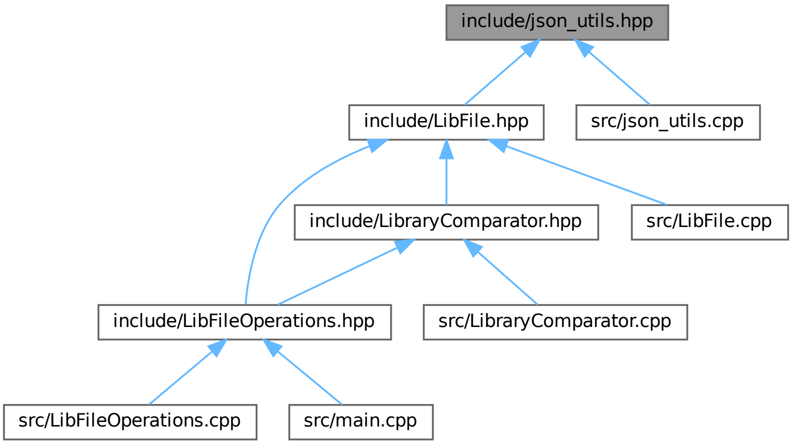
6.3 include/json_utils.hpp File Reference

```
#include <string>
#include "nlohmann/json.hpp"
#include "si2dr_liberty.h"
#include "spdlog/spdlog.h"
#include "Iterators.hpp"
#include "LibGroup.hpp"
```

Include dependency graph for json_utils.hpp:



This graph shows which files directly or indirectly include this file:



Typedefs

- using json = nlohmann::json

Functions

- json generateLutJson (LibGroup &lib_lut_group, si2drErrorT &err)

  Generates a JSON object representation of a look-up table (LUT) from a LibGroup object.
- json generatePowerJson (LibGroup &lib_power_group, si2drErrorT &err)

  Converts a Liberty power group into a JSON representation.
- std::pair< std::string, json > generatePinJson (LibGroup &lib_pin_group, si2drErrorT &err)

  Generates a JSON representation of a Liberty pin group.
- json generateCellJson (LibGroup &lib_cell_group, si2drErrorT &err)

  Generates a JSON representation of a cell from a LibGroup object.

6.3.1 Typedef Documentation

6.3.1.1 json

```
using json = nlohmann::json
```

Definition at line 13 of file json_utils.hpp.

6.3.2 Function Documentation

6.3.2.1 generateCellJson()

```
json generateCellJson (
        LibGroup & lib_cell_group,
        si2drErrorT & err )
```

Generates a JSON representation of a cell from a LibGroup object.

This function processes a LibGroup representing a cell and converts it into a JSON object. It extracts the cell name and processes specific attributes such as area, cell_leakage_power, and cell_footprint. It also processes pins within the cell by categorizing them based on their direction (input, output, internal, inout).

Parameters

lib_cell_group	The LibGroup object representing the cell
err	Reference to a si2drErrorT object for error handling

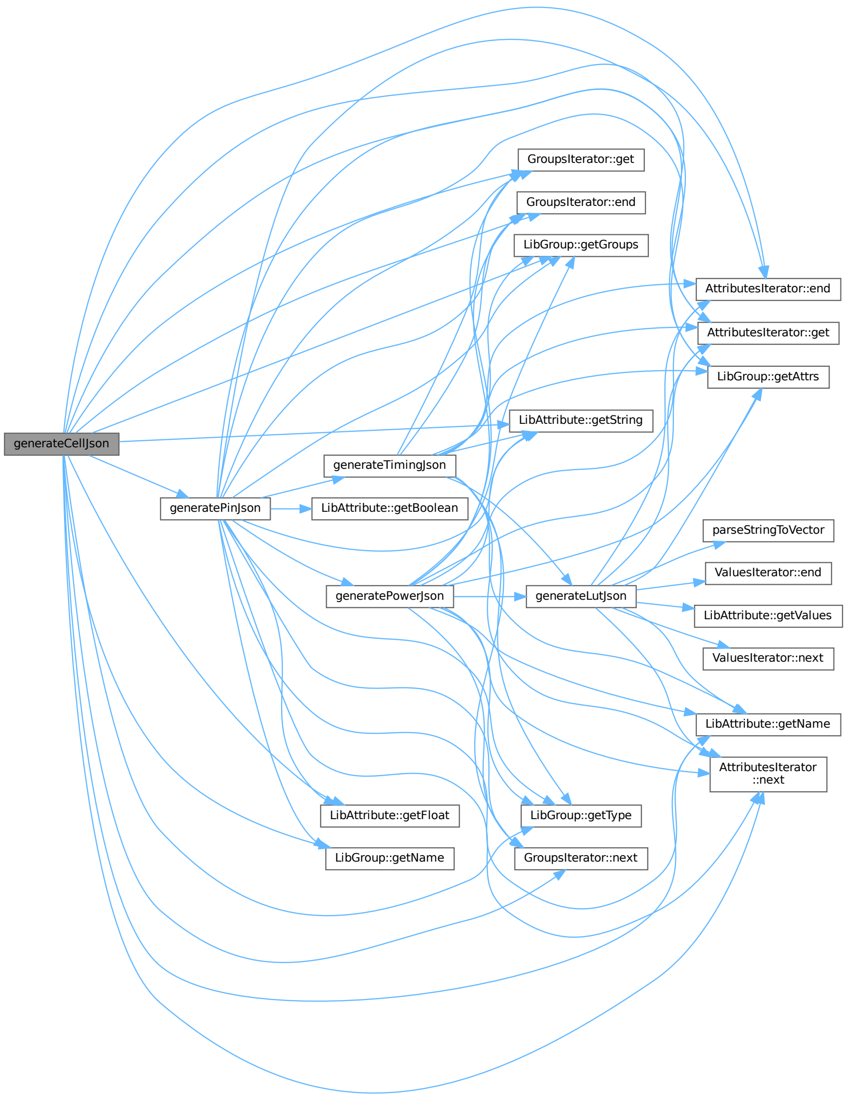
Returns

json A JSON object containing the cell's information with the following structure:

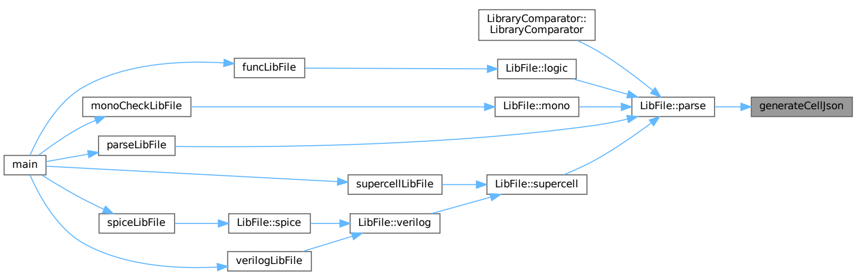
- "cell_name": string - name of the cell
- "area": float (optional) - cell area if present
- "cell_leakage_power": float (optional) - cell leakage power if present
- "cell_footprint": string (optional) - cell footprint if present
- "input_pins": array - list of input pins
- "output_pins": array - list of output pins
- "internal_pins": array - list of internal pins
- "inout_pins": array - list of inout pins

Definition at line 269 of file json_utils.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



6.3.2.2 generateLutJson()

```
json generateLutJson (
        LibGroup & lib_lut_group,
        si2drErrorT & err )
```

Generates a JSON object representation of a look-up table (LUT) from a LibGroup object.

This function iterates through the attributes of the provided LibGroup object representing a LUT and converts them into a JSON structure. It specifically handles the following attributes:

- "index_1": Converted to a vector and stored in the JSON
- "index_2": Converted to a vector and stored in the JSON
- "values": Each value is parsed into a vector and added to an array in the JSON

Any other attributes encountered will trigger a warning message.

Parameters

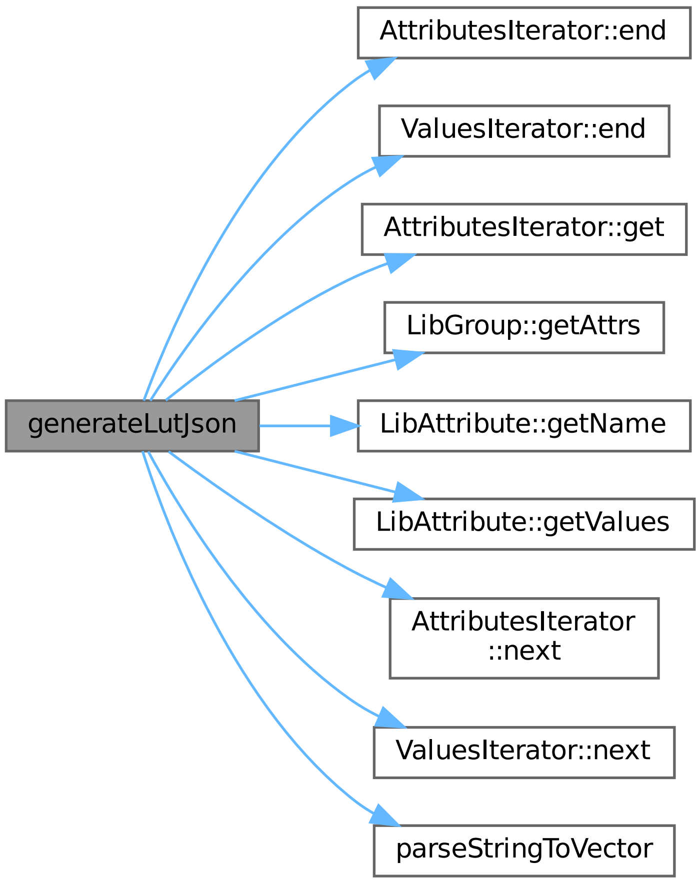
lib_lut_group	The LibGroup object containing the LUT data to be converted
err	Reference to an si2drErrorT object to track any errors during processing

Returns

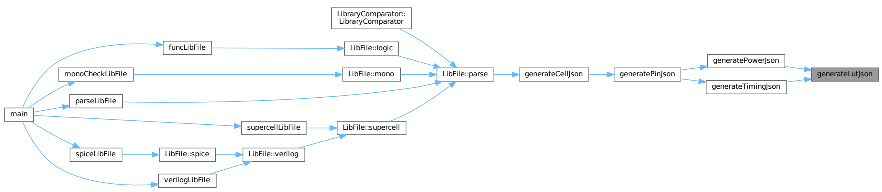
json A JSON object representing the LUT data with keys for "index_1", "index_2", and "values"

Definition at line 59 of file json_utils.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



6.3.2.3 generatePinJson()

```
std::pair< std::string, json > generatePinJson (
```

```
LibGroup & lib_pin_group,
si2drErrorT & err )
```

Generates a JSON representation of a Liberty pin group.

This function traverses a Liberty pin group, extracts relevant attributes and sub-groups, and converts them into a JSON object. It also determines the pin's direction.

Processes the following pin attributes:

- direction
- max_transition, capacitance, rise_capacitance, fall_capacitance, max_capacitance (float types)
- function, power_down_function, related_ground_pin, related_power_pin, three_state (string types)
- clock (boolean type)

Handles the following sub-groups:

- internal_power: Converted using generatePowerJson()
- timing: Converted using generateTimingJson()

Parameters

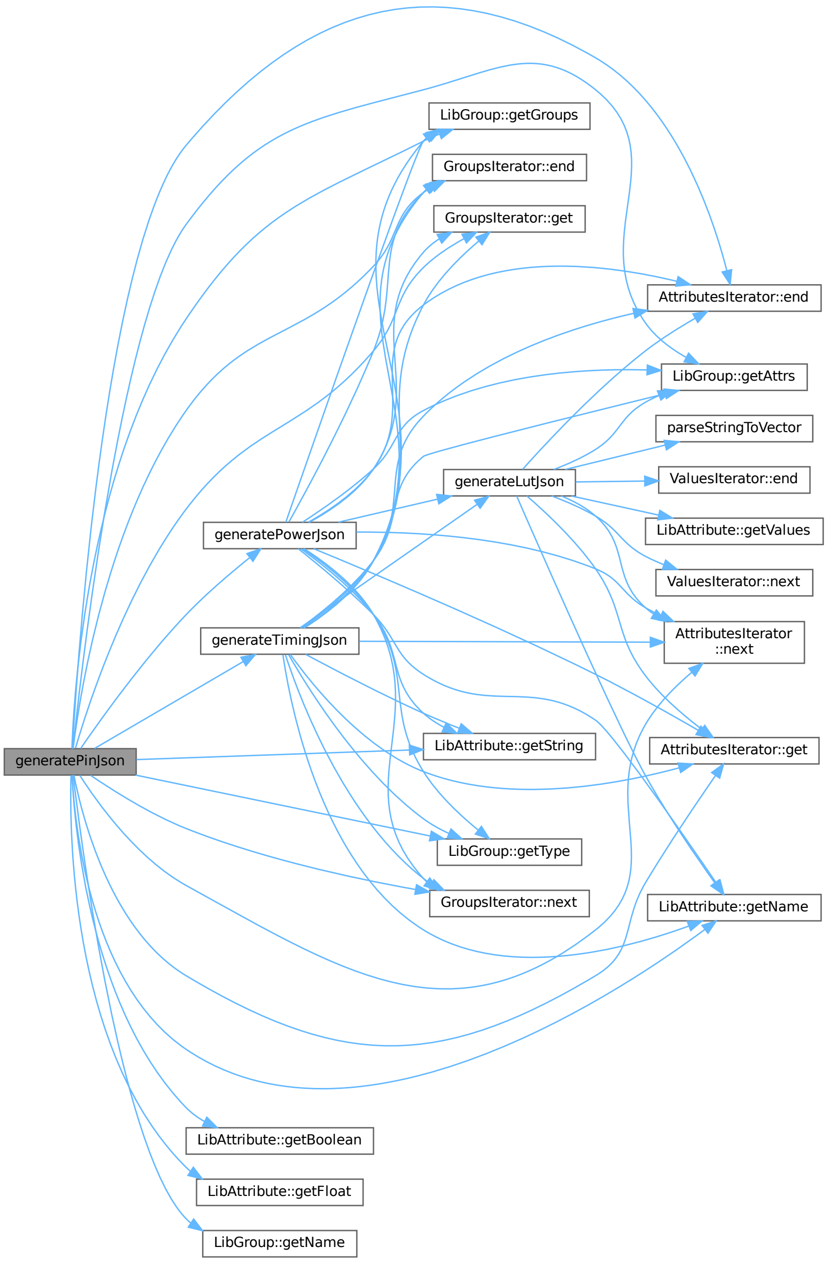
lib_pin_group	The Liberty pin group to process
err	Reference to an error object for tracking Liberty API errors

Returns

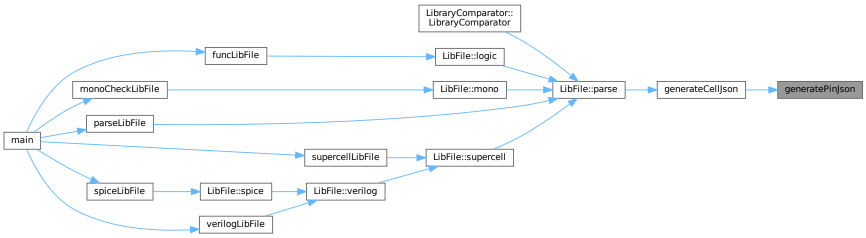
A pair containing the pin direction (string) and the JSON representation of the pin

Definition at line 203 of file json_utils.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



6.3.2.4 generatePowerJson()

```
json generatePowerJson (
            LibGroup & lib_power_group,
            si2drErrorT & err )
```

Converts a Liberty power group into a JSON representation.

This function processes a Liberty power group and converts its attributes and subgroups into a JSON object. It handles attributes like "when", "related_pin", and "related_pg_pin", as well as "rise_power" and "fall_power" subgroups.

Parameters

lib_power_group	The Liberty power group to convert
err	Reference to an si2drErrorT object for error handling

Returns

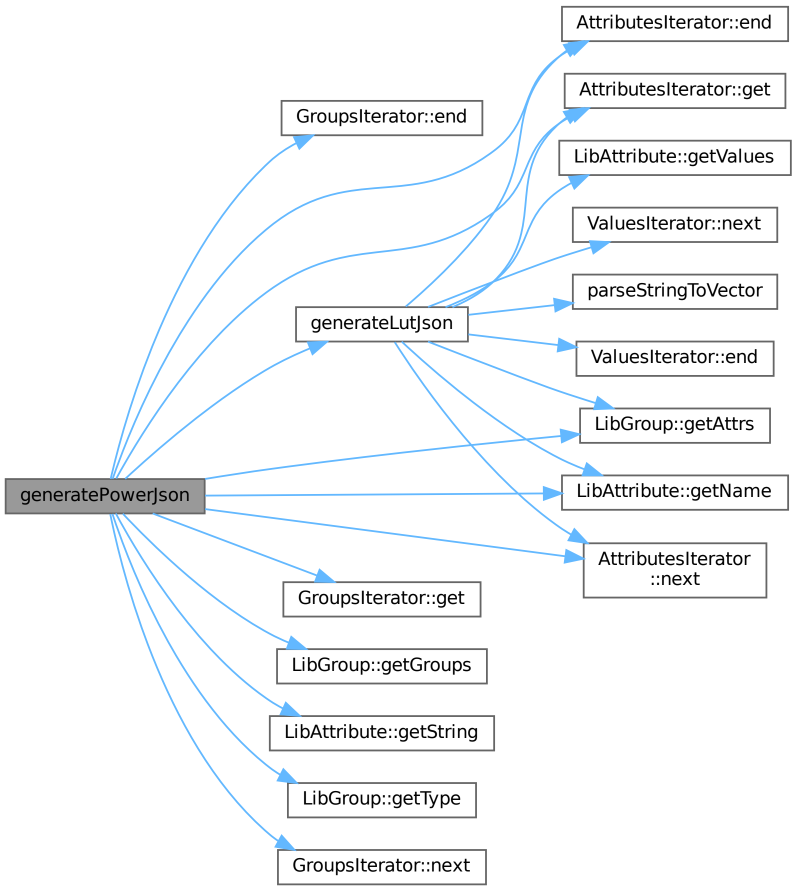
json A JSON object representing the power group data

The function:

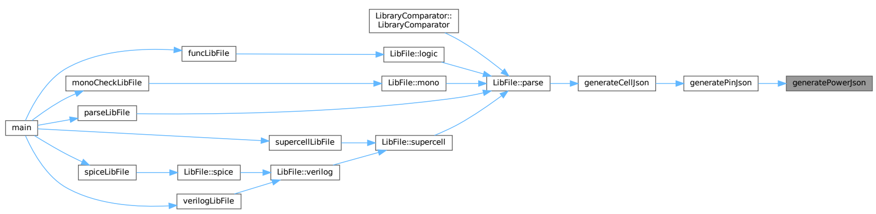
- Extracts string attributes (when, related_pin, related_pg_pin)
- Processes rise_power and fall_power subgroups by converting them to LUT JSON format
- Logs warnings for unknown power subgroup types

Definition at line 151 of file json_utils.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



6.4 json_utils.hpp

Go to the documentation of this file.

```
#ifndef JSON_UTILS_HPP
#define JSON_UTILS_HPP
```

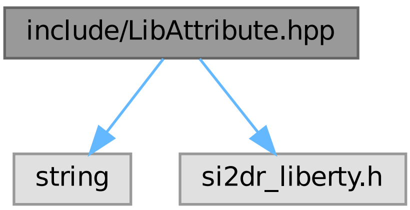
```

#include <string>

#include "nlohmann/json.hpp"
#include "si2dr_liberty.h"
#include "spdlog/spdlog.h"

#include "Iterators.hpp"
#include "LibGroup.hpp"

using json = nlohmann::json;

json generateLutJson(LibGroup &lib_lut_group, si2drErrorT &err);
json generatePowerJson(LibGroup &lib_power_group, si2drErrorT &err);
std::pair<std::string, json> generatePinJson(LibGroup &lib_pin_group, si2drErrorT &err);
json generateCellJson(LibGroup &lib_cell_group, si2drErrorT &err);

#endif // JSON_UTILS_HPP
```

6.5 include/LibAttribute.hpp File Reference

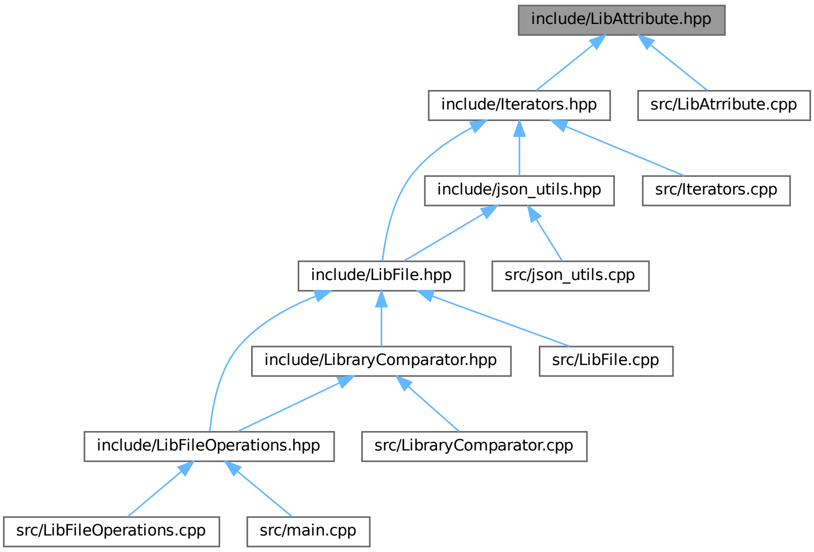
`#include <string>`

`#include "si2dr_liberty.h"`

Include dependency graph for LibAttribute.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class LibAttribute

6.6 LibAttribute.hpp

Go to the documentation of this file.

```
#ifndef LIB_ATTRIBUTE_H
#define LIB_ATTRIBUTE_H

#include <string>

#include "si2dr_liberty.h"

class LibAttribute {
public:
  LibAttribute(si2drAttrIdT attr, si2drErrorT &err);
  ~LibAttribute();
  std::string getName();
  bool isComplex();
  si2drValuesIdT getValues();
  long int getInt();
  double getFloat();
  std::string getString();
  bool getBoolean();

private:
  si2drAttrIdT attr_;
  si2drErrorT &err_;
};

#endif // LIB_ATTRIBUTE_H
```

6.7 include/LibFile.hpp File Reference

#include <chrono>

#include <filesystem>

#include <fstream>

#include <string>

#include <unordered_set>

#include "nlohmann/json.hpp"

#include "si2dr_liberty.h"

#include "spdlog/sinks/basic_file_sink.h"

#include "spdlog/sinks/stdout_color_sinks.h"

#include "spdlog/spdlog.h"

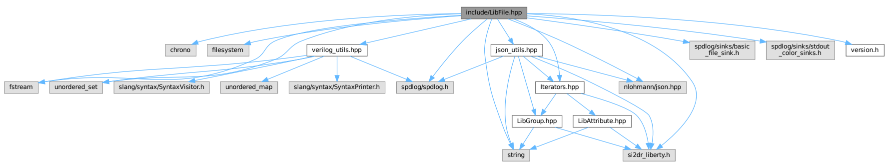
#include "Iterators.hpp"

#include "json_utils.hpp"

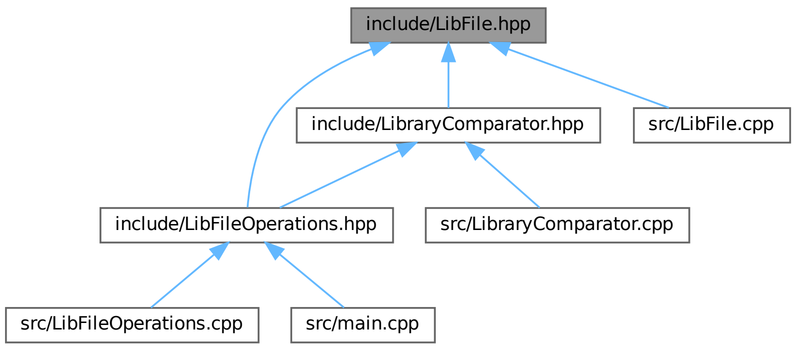
#include "verilog_utils.hpp"

#include "version.h"

Include dependency graph for LibFile.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class LibFile

Typedefs

- using json = nlohmann::json

6.7.1 Typedef Documentation

6.7.1.1 json

```
using json = nlohmann::json
```

Definition at line 21 of file LibFile.hpp.

6.8 LibFile.hpp

Go to the documentation of this file.

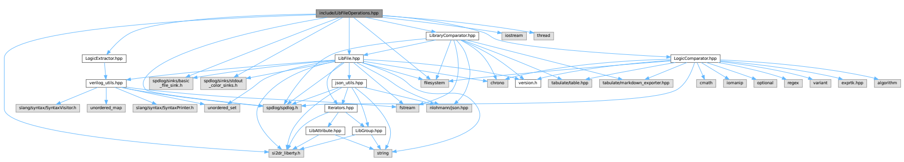
```
#ifndef LIB_FILE_H
#define LIB_FILE_H

#include <chrono>
#include <filesystem>
#include <fstream>
#include <string>
#include <unordered_set>

#include "nlohmann/json.hpp"
#include "si2dr_liberty.h"
#include "spdlog/sinks/basic_file_sink.h"
#include "spdlog/sinks/stdout_color_sinks.h"
#include "spdlog/spdlog.h"

#include "Iterators.hpp"
#include "json_utils.hpp"
#include "verilog_utils.hpp"
#include "version.h"

using json = nlohmann::json;

class LibFile {
public:
  LibFile(const std::string &filepath, const std::string &loggername);
  ~LibFile();
  std::shared_ptr<spdlog::logger> logger_;
  std::filesystem::path filepath_; // full path to the file
  std::string basename_;           // file name without extension
  std::string filename_;           // full file name with extension
  std::string libname_ = "";       // library name obtained through parsing
  std::string jsonname_ = "";      // json file name to store parsed data
  std::string loggername_ = "";    // log file name
  json lib_json_ = json::object();
  void writeJsonToFile();
  void parse();
  void modify();
  void mono(const bool is_slew);
```

```
  void supercell(const int chain_length, const std::vector<std::string> &cell_names);
  void verilog(const int chain_length, const std::vector<std::string> &cell_names);
  void spice(const int chain_length, const std::vector<std::string> &cell_names,
             const std::string &verilog_lib_file, const std::string &spice_lib_file);
  std::map<std::string, std::string> logic(const std::string &cell_name);

private:
  si2drErrorT err_;
  int process_ = 0;
  float voltage_ = 0.0;
  int temperature_ = 0;
  void read();
  bool checkTimingArcMonotonicity(const json &cell, const json &pin, const json &arc,
                                  const std::string &type, bool is_slew);
  // --- Private Helper Methods ---

  // Helper function to split a string by whitespace
  std::vector<std::string> splitString(const std::string &s);

  // Function to generate RC lines based on instance index and stage
  void generateRCLines(std::ofstream &outFile, const std::string &netName, int instanceIndex,
                       bool isFinalStage);

  // Function to modify the SPICE netlist generated by v2lvs
  bool modifySpiceNetlist(const std::string &v2lvsSpiceFile, // Input is the v2lvs output
                          const std::string &finalSpiceFile, // Output is the final file
                          const std::string &targetGlobalLine);
};

#endif // LIB_FILE_H
```

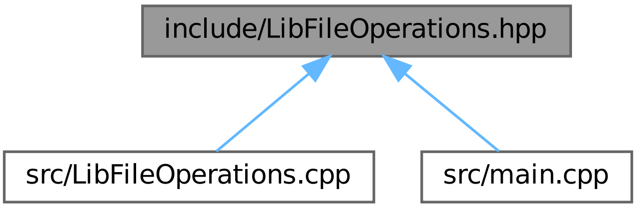
6.9 include/LibFileOperations.hpp File Reference

```
#include <filesystem>
#include <iostream>
#include <thread>
#include "si2dr_liberty.h"
#include "spdlog/sinks/basic_file_sink.h"
#include "spdlog/sinks/stdout_color_sinks.h"
#include "spdlog/spdlog.h"
#include "LibFile.hpp"
#include "LibraryComparator.hpp"
#include "LogicComparator.hpp"
#include "LogicExtractor.hpp"
```

Include dependency graph for LibFileOperations.hpp:



This graph shows which files directly or indirectly include this file:



Functions

- void printInfo ()

  Sets up and configures the global logger and prints application information.
- void parseLibFile (const std::string &library_path, const std::string log_file_name)

  Parses a library file and generates corresponding output.
- void monoCheckLibFile (const std::string &library_path, const std::string log_file_name, bool is_slew)

  Performs monotonicity check on a library file.
- void supercellLibFile (const std::string &library_path, const std::string &log_file_name, int chain_length, const std::vector< std::string > &cell_names)

  Creates supercell map structures from a Liberty library file.
- void verilogLibFile (const std::string &library_path, const std::string &log_file_name, int chain_length, const std::vector< std::string > &cell_names)

  Generates Verilog files from a library file for specified cells.
- void spiceLibFile (const std::string &library_path, const std::string &log_file_name, int chain_length, const std::vector< std::string > &cell_names, const std::string &verilog_lib_file, const std::string &spice_lib_file)

  Generates a SPICE library file from a given library file, applying a specified chain length and cell names.
- void compareLibFiles (const std::string &ref_lib, const std::string &comp_lib, const double reltol, const double abstol, std::string &report_file_name)

  Compares two library files and generates a detailed comparison report.
- void funcLibFile (const std::string &ref_file, const std::string &comp_file, const std::vector< std::string > &cell_names, std::string &report_file_name)

  Performs a functional equivalence check between two files (Liberty or Verilog) for a given set of cells.

6.9.1 Function Documentation

6.9.1.1 compareLibFiles()

```
void compareLibFiles (
        const std::string & ref_lib,
        const std::string & comp_lib,
        const double reltol,
        const double abstol,
        std::string & report_file_name )
```

Compares two library files and generates a detailed comparison report.

This function compares a reference library file with another library file, performing validation checks based on specified tolerance parameters. The comparison results are written to a report file in markdown or text format.

Parameters

ref_lib	Path to the reference library file to use as the baseline
comp_lib	Path to the library file to compare against the reference
reltol	Relative tolerance for numerical comparisons (must be >= 0.0)
abstol	Absolute tolerance for numerical comparisons
report_file_name	[in,out] Name of the file to write the comparison report to. If empty, defaults to [comp_lib_basename].cmp.md. If provided but doesn't end with .txt or .md, .md will be appended.

Note

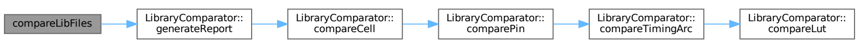
The function will log an error and return without comparing if reltol is invalid.

Log files will be created with the naming pattern [library_basename].cmp.log

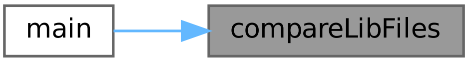
The function uses the LibraryComparator class to perform the actual comparison.

Definition at line 246 of file LibFileOperations.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



6.9.1.2 funcLibFile()

```
void funcLibFile (
        const std::string & ref_file,
        const std::string & comp_file,
        const std::vector< std::string > & cell_names,
        std::string & report_file_name )
```

Performs a functional equivalence check between two files (Liberty or Verilog) for a given set of cells.

This function compares the logic functions of specified cells in two files, which can be either in Liberty (.lib) or Verilog (.v) format. It extracts the logic functions for each cell from both files, compares them, and generates a report summarizing the comparison results.

Parameters

ref_file	The path to the reference file (Liberty or Verilog).
comp_file	The path to the comparison file (Liberty or Verilog).
cell_names	A vector of cell names to be checked for functional equivalence.
report_file_name	A string to store the name of the report file. If empty, a default name is generated. If the provided name does not end with ".txt" or ".md", ".md" is appended.

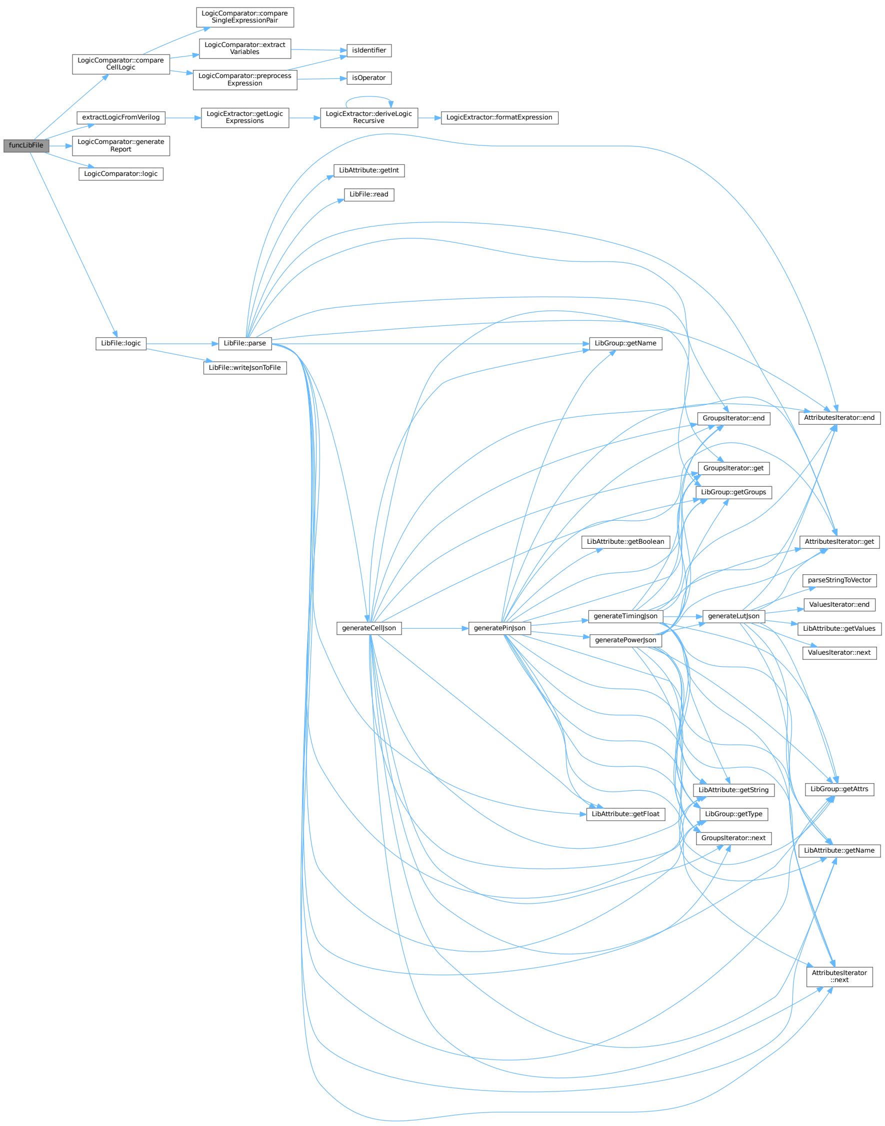
The function first checks the file extensions to determine the file format. It then extracts the logic functions for each specified cell from both files. The logic functions are then compared, and a report is generated, which includes the comparison results for each cell. The report is written to the specified report file.

Note

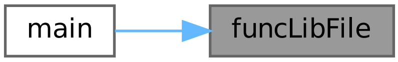
- If no cell names are provided, the function logs an error and returns.
- If the reference or comparison file format is not supported (i.e., not .lib or .v), the function logs an error and returns.
- The report file is cleared before writing the comparison results.
- The function uses spdlog for logging information, warnings, and errors.
- The function utilizes the LogicComparator class to perform the logic comparison and generate the report.
- Memory allocated for LibFile objects is managed using raw pointers and must be manually deallocated to prevent memory leaks.

Definition at line 299 of file LibFileOperations.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



6.9.1.3 monoCheckLibFile()

```
void monoCheckLibFile (
        const std::string & library_path,
        const std::string log_file_name,
        bool is_slew )
```

Performs monotonicity check on a library file.

This function validates the monotonicity of timing data in a library file. It creates a log file to record the results of the check and handles any exceptions that occur during the process.

Parameters

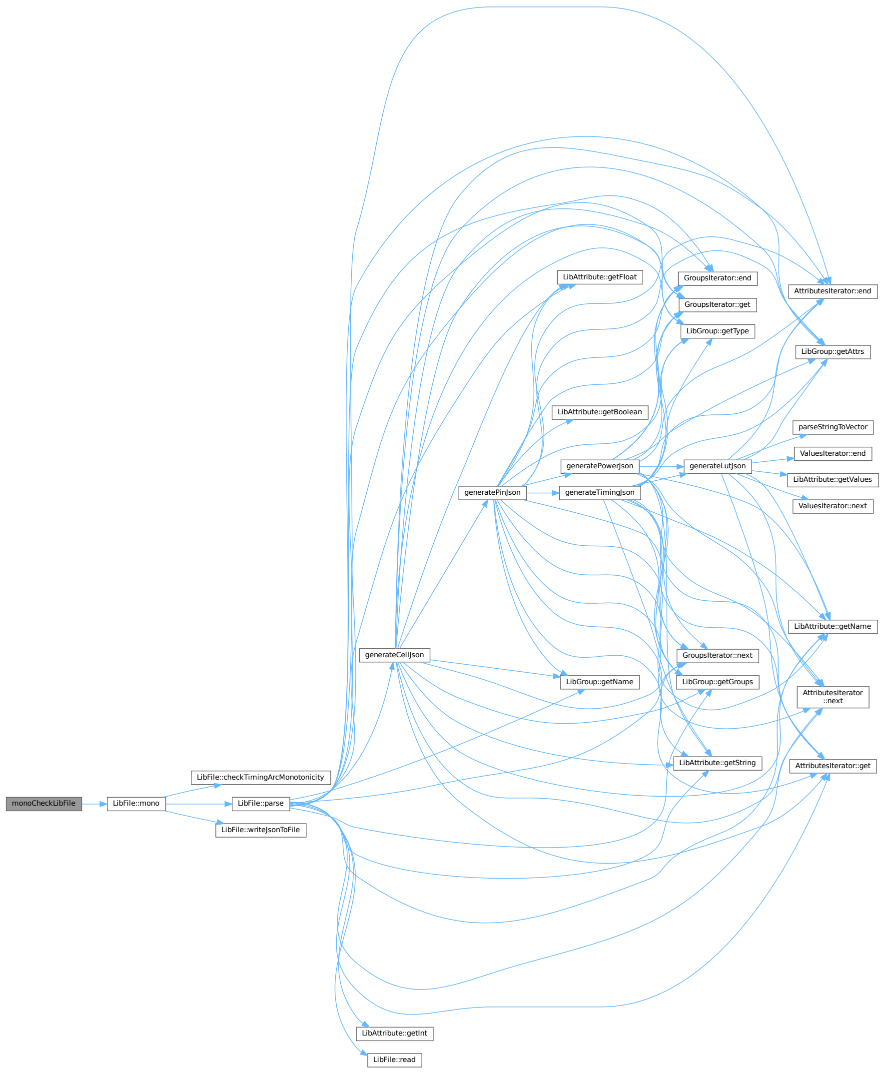
library_path	Path to the library file to check
log_file_name	Name of the log file to create (optional). If empty, a default name will be generated from the library file name
is_slew	Flag indicating whether to check input slew monotonicity (true) or output load monotonicity (false)

Exceptions

The	function catches and logs any exceptions but does not rethrow them

Definition at line 81 of file LibFileOperations.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:

6.9.1.4 parseLibFile()

```
void parseLibFile (
        const std::string & library_path,
        const std::string log_file_name )
```

Parses a library file and generates corresponding output.

This function processes the given library file, parsing its contents and generating a JSON output. It also logs the parsing process to a specified log file or creates a default log file if none is provided.

Parameters

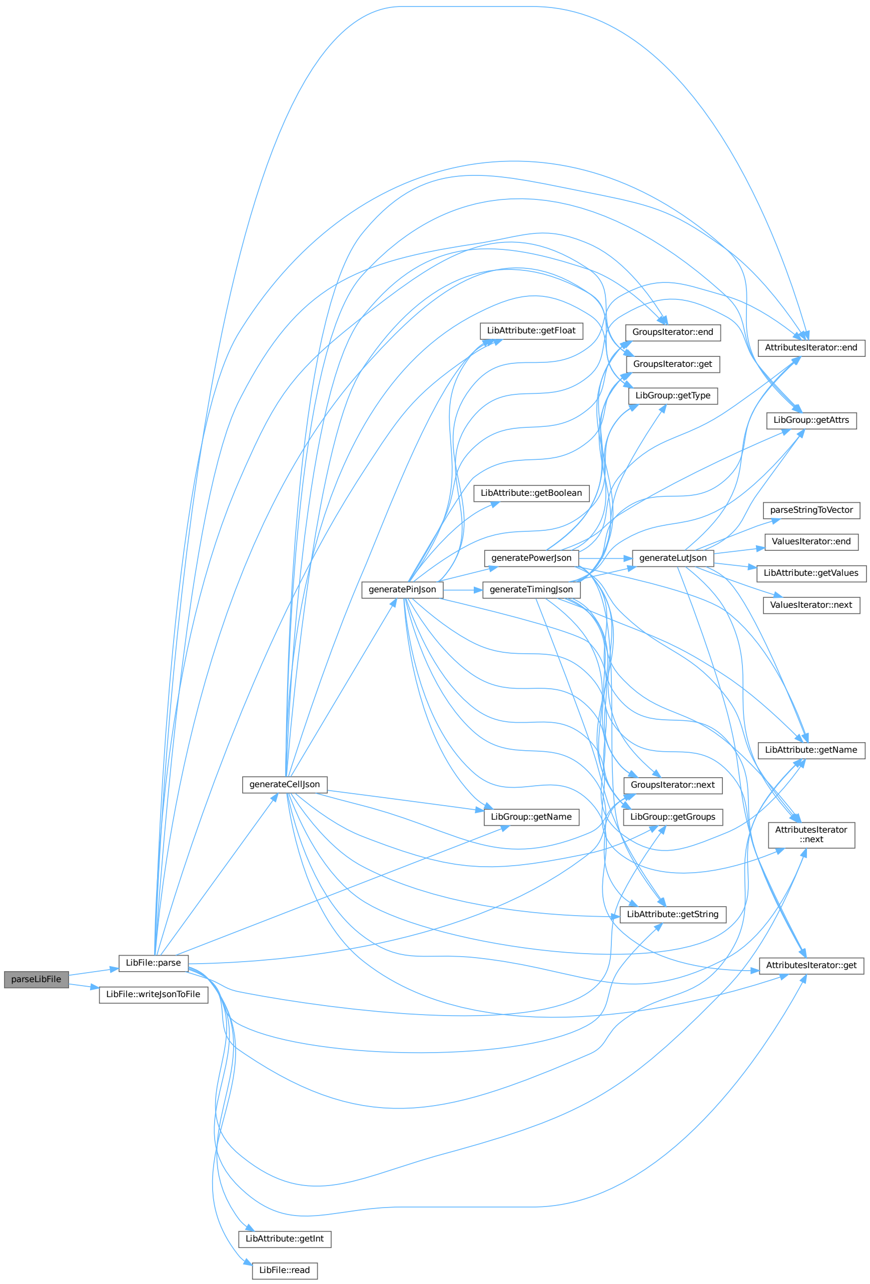
library_path	Path to the library file that needs to be parsed
log_file_name	Optional name for the log file. If empty, a default name is generated based on the library filename with ".parse.log" extension

Exceptions

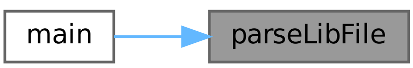
The	function catches and logs any exceptions but doesn't propagate them

Definition at line 49 of file LibFileOperations.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



6.9.1.5 printInfo()

```
void printInfo ( )
```

Sets up and configures the global logger and prints application information.

This function performs the following operations:

1. Creates a console sink for logging with level set to INFO
2. Creates a file sink for logging with level set to DEBUG, saving to [APP_NAME].log
3. Configures a logger with both sinks and sets it as the default logger
4. Outputs basic application information:
    - Version and build timestamp
    - Author information
    - Log file location

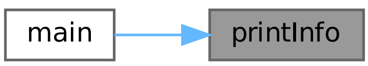
Note

Uses spdlog library for logging functionality

Depends on APP_NAME, APP_VERSION, BUILD_TIMESTAMP, APP_AUTHOR, and APP↩_CONTACT macros

Definition at line 18 of file LibFileOperations.cpp.

Here is the caller graph for this function:



6.9.1.6 spiceLibFile()

```
void spiceLibFile (
        const std::string & library_path,
        const std::string & log_file_name,
        int chain_length,
        const std::vector< std::string > & cell_names,
        const std::string & verilog_lib_file,
        const std::string & spice_lib_file )
```

Generates a SPICE library file from a given library file, applying a specified chain length and cell names.

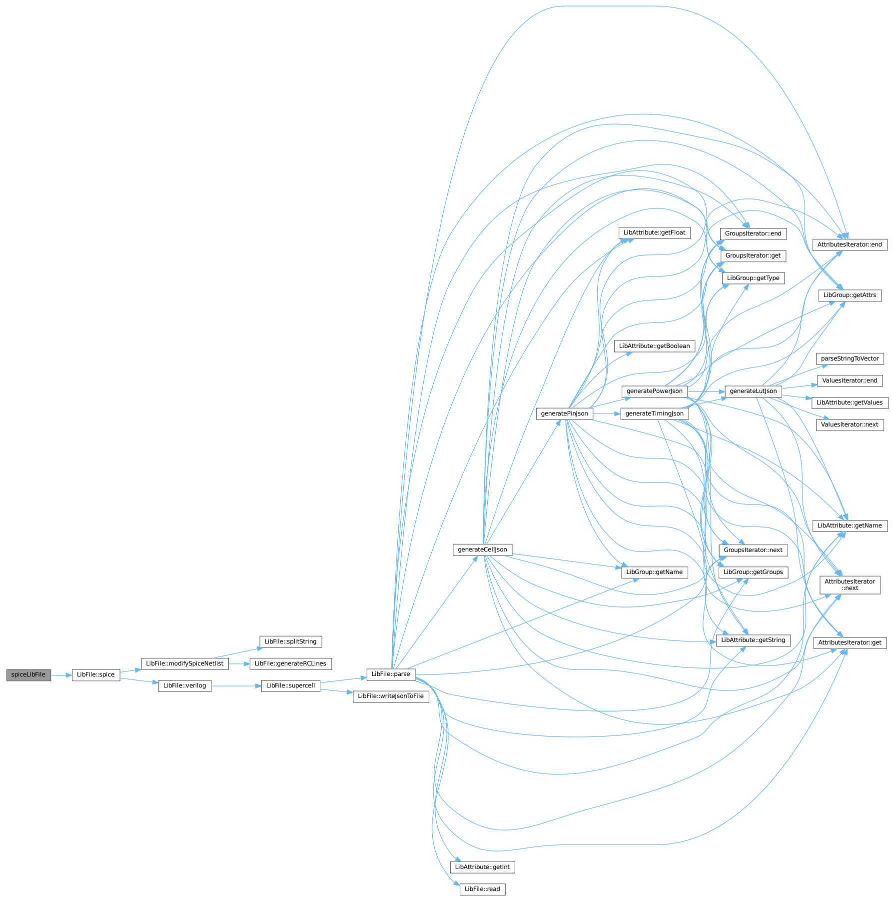
This function takes a library file path, a log file name, a chain length, a vector of cell names, a Verilog library file path, and a SPICE library file path as input. It initializes a LibFile object, validates the chain length, and then calls the spice method of the LibFile object to generate the SPICE library file. It logs the start and end of the SPICE generation process, as well as any errors that occur.

Parameters

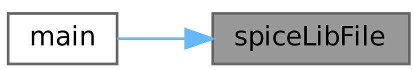
library_path	The path to the input library file.
log_file_name	The name of the log file. If empty, a default log file name is generated based on the library file name.
chain_length	The chain length to use during SPICE generation. Must be >= 1.
cell_names	A vector of cell names to include in the SPICE generation.
verilog_lib_file	The path to the Verilog library file.
spice_lib_file	The path to the output SPICE library file.

Definition at line 199 of file LibFileOperations.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



6.9.1.7 supercellLibFile()

```
void supercellLibFile (
        const std::string & library_path,
        const std::string & log_file_name,
```

```
int chain_length,
const std::vector< std::string > & cell_names )
```

Creates supercell map structures from a Liberty library file.

This function reads a Liberty file, creates supercell structures based on the specified chain length and cell names, and logs the process to a file.

Parameters

library_path	Path to the Liberty file to process
log_file_name	Name of the log file (if empty, defaults to "[library_name].supercell.log")
chain_length	The length of chains to create (must be >= 1)
cell_names	Vector of cell names to process for supercell generation

Exceptions

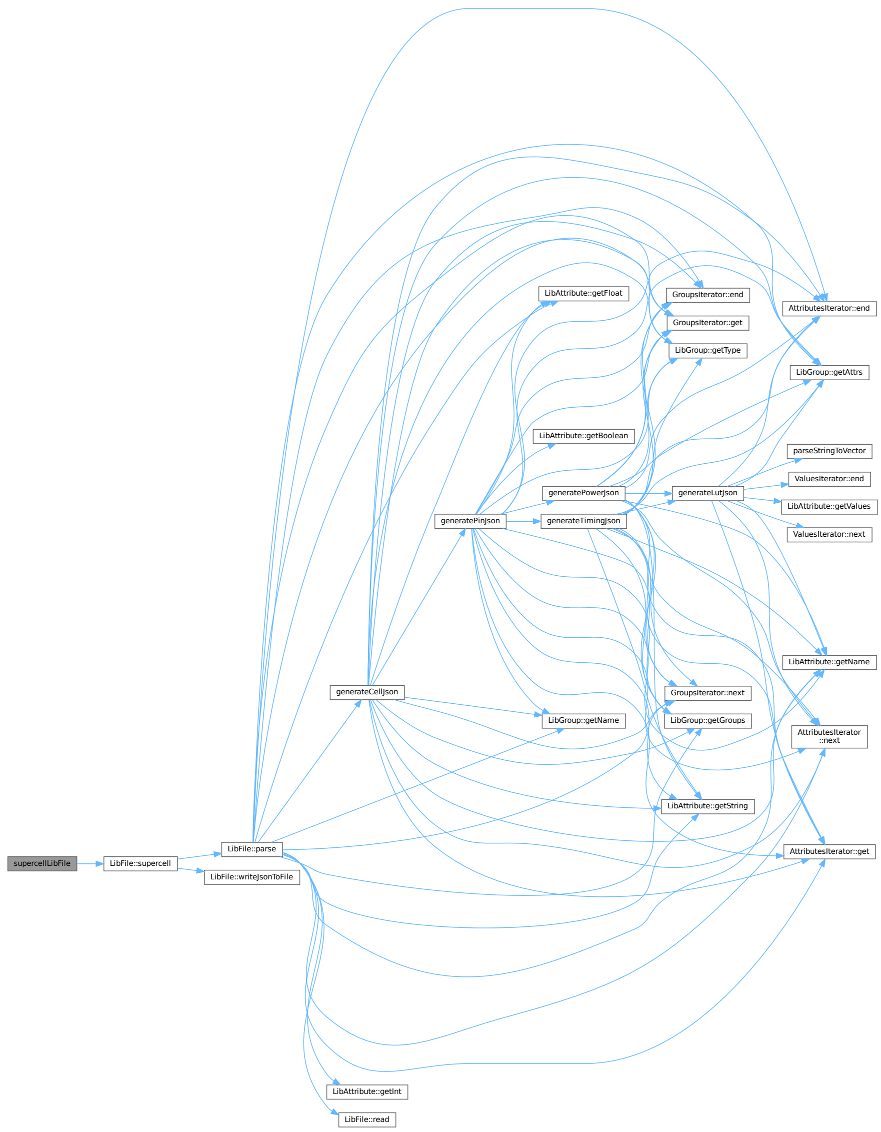
May	pass through exceptions from the LibFile::supercell method

Note

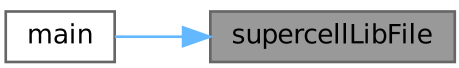
The function validates the chain length and logs all activities including errors that might occur during processing

Definition at line 116 of file LibFileOperations.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



6.9.1.8 verilogLibFile()

```
void verilogLibFile (
        const std::string & library_path,
        const std::string & log_file_name,
        int chain_length,
        const std::vector< std::string > & cell_names )
```

Generates Verilog files from a library file for specified cells.

This function processes a library file and generates Verilog representation for the specified cell names with a given chain length. The operation results are logged to a specified or default log file.

Parameters

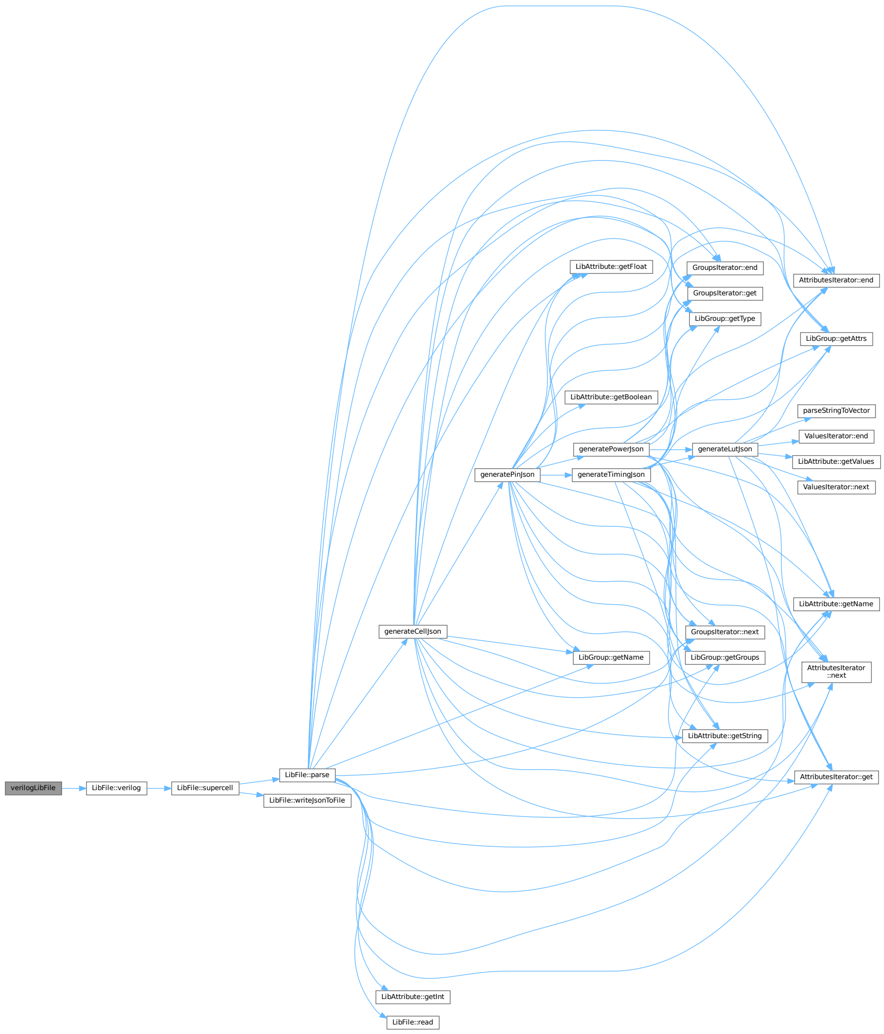
library_path	Path to the library file to process
log_file_name	Name for the log file (if empty, a default name will be generated)
chain_length	Number of cells to chain together, must be >= 1
cell_names	Vector of cell names to generate Verilog for

Exceptions

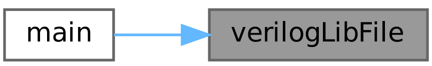
Catches	any exceptions from the verilog generation process and logs them

Definition at line 157 of file LibFileOperations.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



6.10 LibFileOperations.hpp

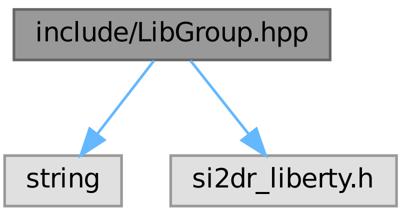
Go to the documentation of this file.

```
#ifndef LIBFILEOPERATIONS_H
#define LIBFILEOPERATIONS_H

#include <filesystem>
#include <iostream>
#include <thread>

#include "si2dr_liberty.h"
#include "spdlog/sinks/basic_file_sink.h"
#include "spdlog/sinks/stdout_color_sinks.h"
#include "spdlog/spdlog.h"

#include "LibFile.hpp"
#include "LibraryComparator.hpp"
#include "LogicComparator.hpp"
#include "LogicExtractor.hpp"

void printInfo();
void parseLibFile(const std::string &library_path, const std::string log_file_name);
void monoCheckLibFile(const std::string &library_path, const std::string log_file_name,
                       bool is_slew);
void supercellLibFile(const std::string &library_path, const std::string &log_file_name,
                       int chain_length, const std::vector<std::string> &cell_names);
void verilogLibFile(const std::string &library_path, const std::string &log_file_name,
                     int chain_length, const std::vector<std::string> &cell_names);
void spiceLibFile(const std::string &library_path, const std::string &log_file_name,
                   int chain_length, const std::vector<std::string> &cell_names,
                   const std::string &verilog_lib_file, const std::string &spice_lib_file);
void compareLibFiles(const std::string &ref_lib, const std::string &comp_lib, const double reltol,
                      const double abstol, std::string &report_file_name);
void funcLibFile(const std::string &ref_file, const std::string &comp_file,
                  const std::vector<std::string> &cell_names, std::string &report_file_name);

#endif // LIBFILEOPERATIONS_H
```

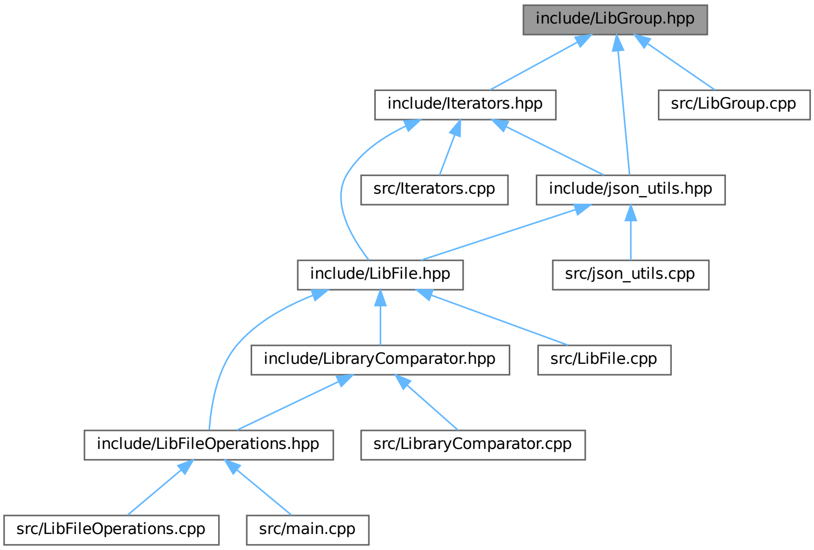
6.11 include/LibGroup.hpp File Reference

`#include <string>`
`#include "si2dr_liberty.h"`

Include dependency graph for LibGroup.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class LibGroup

6.12 LibGroup.hpp

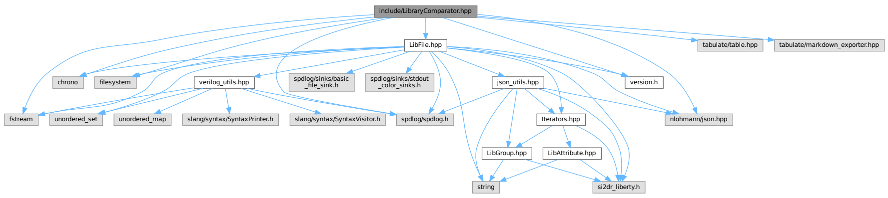
Go to the documentation of this file.

```
#ifndef LIB_GROUP_H
#define LIB_GROUP_H

#include <string>

#include "si2dr_liberty.h"

class LibGroup {
public:
  LibGroup(si2drGroupIdT group, si2drErrorT &err);
  ~LibGroup();
  std::string getName();
  std::string getType();
  si2drAttrsIdT getAttrs();
  si2drGroupsIdT getGroups();

private:
  si2drGroupIdT group_;
  si2drErrorT &err_;
};

#endif // LIB_GROUP_H
```

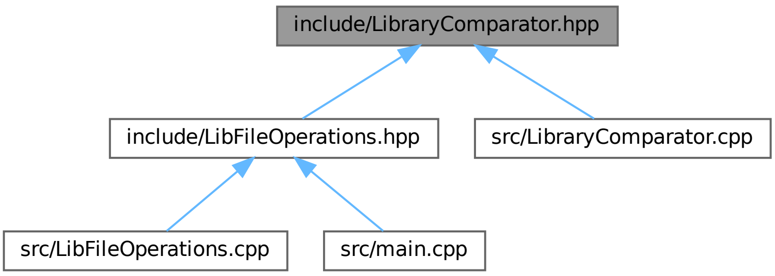
6.13 include/LibraryComparator.hpp File Reference

`#include <chrono>`
`#include <filesystem>`
`#include <fstream>`
`#include "nlohmann/json.hpp"`
`#include "spdlog/spdlog.h"`
`#include "tabulate/table.hpp"`
`#include <tabulate/markdown_exporter.hpp>`
`#include "LibFile.hpp"`
`#include "version.h"`

Include dependency graph for LibraryComparator.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class LibraryComparator

Typedefs

- using json = nlohmann::json

6.13.1 Typedef Documentation

6.13.1.1 json

```
using json = nlohmann::json
```

Definition at line 16 of file LibraryComparator.hpp.

6.14 LibraryComparator.hpp

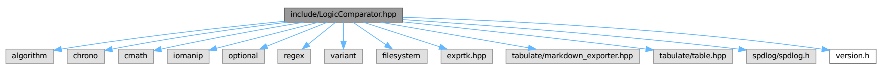
Go to the documentation of this file.

```
#ifndef COMPARE_HPP
#define COMPARE_HPP

#include <chrono>
#include <filesystem>
#include <fstream>

#include "nlohmann/json.hpp"
#include "spdlog/spdlog.h"
#include "tabulate/table.hpp"
#include <tabulate/markdown_exporter.hpp>

#include "LibFile.hpp"
#include "version.h"

using json = nlohmann::json;
using namespace tabulate;

class LibraryComparator {
public:
  LibraryComparator(LibFile &ref_libfile, LibFile &comp_libfile, double reltol, double abstol);
  std::filesystem::path ref_lib_path_;
  std::filesystem::path comp_lib_path_;
  void generateReport(const std::string &output_file);

private:
  json ref_json_;
  json comp_json_;
  double reltol_;
  double abstol_;

  void compareCell(const std::string &cell_name, const json &ref_cell, const json &comp_cell,
                   Table &table);
  void comparePin(const std::string &cell_name, const std::string &pin_name, const json &ref_pin,
                  const json &comp_pin, Table &table);
  void compareTimingArc(const std::string &cell_name, const std::string &pin_name,
                        const std::string &timing_type, const json &ref_timing_arc,
                        const json &comp_timing_arc, Table &table);
  void compareLut(const std::string &cell_name, const std::string &pin_name,
                  const std::string &timing_type, const std::string &related_pin,
                  const std::string &arc_name, const json &ref_lut, const json &comp_lut,
                  Table &table);
  // void configureTableFormat(Table &table);
};

#endif // COMPARE_HPP
```

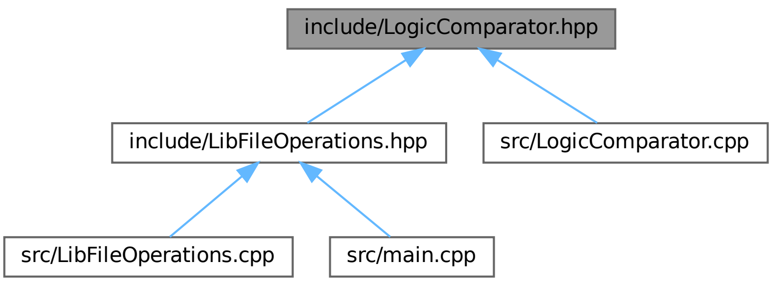
6.15 include/LogicComparator.hpp File Reference

#include <algorithm>
#include <chrono>
#include <cmath>
#include <iomanip>
#include <optional>
#include <regex>
#include <variant>
#include <filesystem>
#include "exprtk.hpp"
#include "tabulate/markdown_exporter.hpp"
#include "tabulate/table.hpp"
#include <spdlog/spdlog.h>
#include "version.h"

Include dependency graph for LogicComparator.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- struct PinComparisonResult
- class LogicComparator

6.16 LogicComparator.hpp

Go to the documentation of this file.

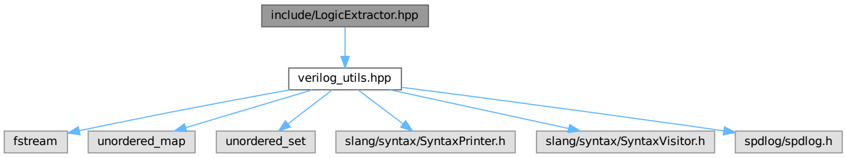
```
#ifndef LOGIC_COMPARATOR_HPP
#define LOGIC_COMPARATOR_HPP

#include <algorithm>
#include <chrono>
#include <cmath>
#include <iomanip>
#include <optional> // To store tables optionally
#include <regex>    // For regular expressions
#include <variant>
#include <filesystem>

#include "exprtk.hpp"
#include "tabulate/markdown_exporter.hpp"
#include "tabulate/table.hpp"
#include <spdlog/spdlog.h>

#include "version.h"

using namespace tabulate;

// Structure to hold results for a single pin comparison
struct PinComparisonResult {
  std::string pin_name;
  std::string ref_expr_raw;
  std::string comp_expr_raw;
  std::string ref_expr_processed;
  std::string comp_expr_processed;
  bool comparison_possible = false; // Was comparison attempted?
  bool are_equivalent = false;
  bool ref_compiles = false;
  bool comp_compiles = false;
  std::optional<Table> ref_truth_table; // Store tables only if needed/successful
  std::optional<Table> comp_truth_table;
  std::string error_message; // Store any error during comparison
};

class LogicComparator {
public:
  LogicComparator(const std::map<std::string, std::string> &ref_outpin_map,
                  const std::map<std::string, std::string> &comp_outpin_map,
                  const std::string &cell_name);

  // Example code from exprtk documentation
  void logic();

  // Preprocessing function
  std::string preprocessExpression(const std::string &input_expr);

  // Helper to extract unique sorted variables from TWO expressions
  bool extractVariables(const std::string &expr1, const std::string &expr2,
                        std::vector<std::string> &sorted_vars);

  void compareSingleExpressionPair(const std::string &ref_expression_processed,
                                   const std::string &comp_expression_processed,
                                   const std::vector<std::string> &sorted_vars,
                                   PinComparisonResult &result); // Pass result struct

  void compareCellLogic();

  void generateReport(const std::string &output_file);

private:
  std::map<std::string, std::string> ref_outpin_map_;
```

```
std::map<std::string, std::string> comp_outpin_map_;
std::string cell_name_;
std::map<std::string, PinComparisonResult> all_pin_results_;
};

#endif // LOGIC_COMPARATOR_HPP
```

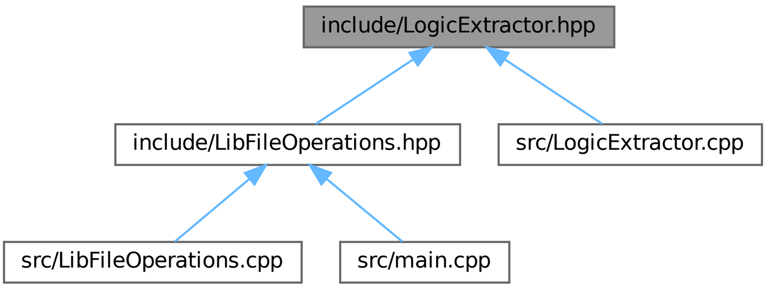
6.17 include/LogicExtractor.hpp File Reference

`#include "verilog_utils.hpp"`

Include dependency graph for LogicExtractor.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- struct GateInfo
- class LogicExtractor

Functions

- void extractAndPrintNetlistInfo (const std::string &verilog_file, const std::string &cell)

  Extracts and prints netlist information from a Verilog file for a specified cell.
- std::map< std::string, std::string > extractLogicFromVerilog (const std::string &verilog_file, const std::string &cell)

  Extracts logic expressions from a Verilog file for a specified cell.

6.17.1 Function Documentation

6.17.1.1 extractAndPrintNetlistInfo()

```
void extractAndPrintNetlistInfo (
            const std::string & verilog_file,
            const std::string & cell )
```

Extracts and prints netlist information from a Verilog file for a specified cell.

This function parses a Verilog file using the slang library, extracts information about the primary inputs, primary outputs, internal wires, and gate drivers within a specified cell (module). It then prints a summary of the extracted information to the console using spdlog.

Parameters

verilog_file	The path to the Verilog file to be parsed.
cell	The name of the cell (module) for which to extract netlist information.

Exceptions

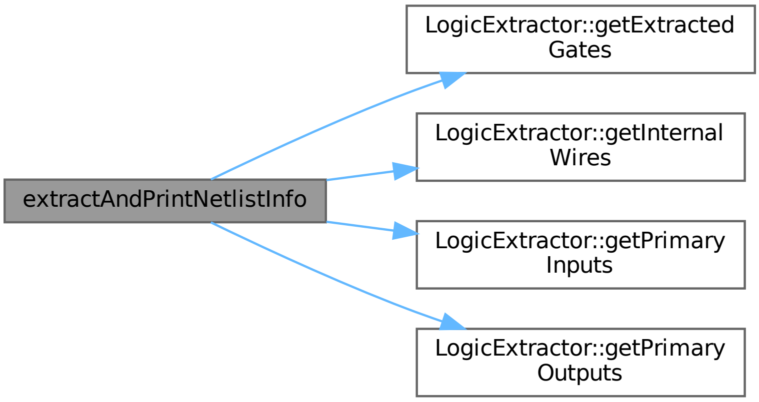
std::exception	If any error occurs during Verilog parsing or info extraction.

Note

The function uses the slang library for Verilog parsing and a custom LogicExtractor class to extract the desired information. Error messages are logged using spdlog.

Definition at line 673 of file LogicExtractor.cpp.

Here is the call graph for this function:



6.17.1.2 extractLogicFromVerilog()

```
std::map< std::string, std::string > extractLogicFromVerilog (
              const std::string & verilog_file,
              const std::string & cell )
```

Extracts logic expressions from a Verilog file for a specified cell.

This function parses a Verilog file using the slang library, identifies the specified cell, and extracts the logic expressions for its outputs. It returns a map where the keys are output signal names and the values are their corresponding logic expressions as strings.

Parameters

verilog_file	The path to the Verilog file to parse.
cell	The name of the cell (module) for which to extract logic expressions.

Returns

A map of output signal names to their logic expressions. Returns an empty map if parsing fails, the cell is not found, or no logic expressions can be derived.

Note

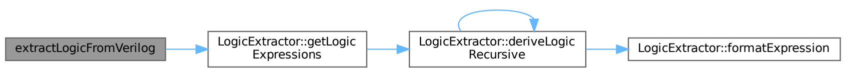
The function uses the slang library for Verilog parsing. Ensure that slang is properly installed and configured before using this function.

The logic extraction process involves traversing the syntax tree of the Verilog code and identifying relevant assignments and expressions within the specified cell.

Error messages and warnings are logged using the spdlog library.

Definition at line 752 of file LogicExtractor.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:

6.18 LogicExtractor.hpp

Go to the documentation of this file.

```
// include/LogicExtractor.hpp

#ifndef LOGIC_EXTRACTOR_HPP
#define LOGIC_EXTRACTOR_HPP

#include "verilog_utils.hpp"

// Structure to represent a gate instance's information
struct GateInfo {
  slang::parsing::TokenKind kind; // Store the TokenKind for reliable checks
  std::string gateTypeName;       // Store the string name like "and", "xor"
  std::vector<std::string> inputSignals;
```

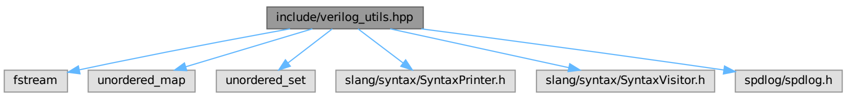
```
  std::string outputSignal;
};

// Visitor class to extract netlist information and derive logic expressions
class LogicExtractor : public slang::syntax::SyntaxVisitor<LogicExtractor> {
public:
  // Constructor takes the target cell name
  explicit LogicExtractor(const std::string &targetCell)
      : targetCell_(targetCell), inTargetModule_(false), parsingComplete_(false) {}

  // --- Visitor Handlers ---
  void handle(const slang::syntax::ModuleDeclarationSyntax &module);
  void handle(
      const slang::syntax::PortDeclarationSyntax &portDecl); // Handles direction/type declaration
  void
  handle(const slang::syntax::NonAnsiPortListSyntax &portList); // Handles port names list in header
  void
  handle(const slang::syntax::NetDeclarationSyntax &netDecl); // To find explicitly declared wires
  void handle(const slang::syntax::PrimitiveInstantiationSyntax
                  &primitiveInst); // Handles gate instantiation
  // Potentially handle ContinuousAssignSyntax if needed later:
  // void handle(const slang::syntax::ContinuousAssignSyntax& assign);

  // --- Access Extracted Info (for Step 1 debugging) ---
  const std::unordered_map<std::string, GateInfo> &getExtractedGates()const {
    return gateOutputDrivers_;
  }
  const std::unordered_set<std::string> &getPrimaryInputs()const { return primaryInputs_; }
  const std::unordered_set<std::string> &getPrimaryOutputs()const { return primaryOutputs_; }
  const std::unordered_set<std::string> &getInternalWires()const { return internalWires_; }

  // --- Logic Derivation (Commented out for Step 1) ---
  std::map<std::string, std::string> getLogicExpressions();

private:
  // --- Internal State ---
  const std::string &targetCell_;
  bool inTargetModule_;
  bool parsingComplete_; // Flag to indicate AST traversal is done

  // Netlist Information
  std::unordered_set<std::string> primaryInputs_;
  std::unordered_set<std::string> primaryOutputs_;
  std::unordered_set<std::string> internalWires_; // Includes gate outputs

  // Temporary map to store directions found in PortDeclarationSyntax
  // Key: port name, Value: direction ("input", "output", "inout", "unknown")
  std::unordered_map<std::string, std::string> portDirections_;

  // Map: output signal name -> GateInfo driving it
  std::unordered_map<std::string, GateInfo> gateOutputDrivers_;

  // Map: signal name -> Logic expression string (Memoization Cache) - (Used in Step 2)
  std::unordered_map<std::string, std::string> logicCache_;

  // --- Helper Methods ---
  // Recursive function to derive logic for a given signal (Used in Step 2)
  std::string deriveLogicRecursive(const std::string &signalName);
  // Helper to format expressions based on gate type (Used in Step 2)
  std::string formatExpression(const GateInfo &gateInfo,
                               const std::vector<std::string> &inputExprs);
};

// --- Function Declaration ---
// For Step 1, this function will just run the visitor and maybe print summary
void extractAndPrintNetlistInfo(const std::string &verilog_file, const std::string &cell);

// Function to be implemented in Step 2
std::map<std::string, std::string> extractLogicFromVerilog(const std::string &verilog_file,
```

```
                                                     const std::string &cell);

#endif // LOGIC_EXTRACTOR_HPP
```

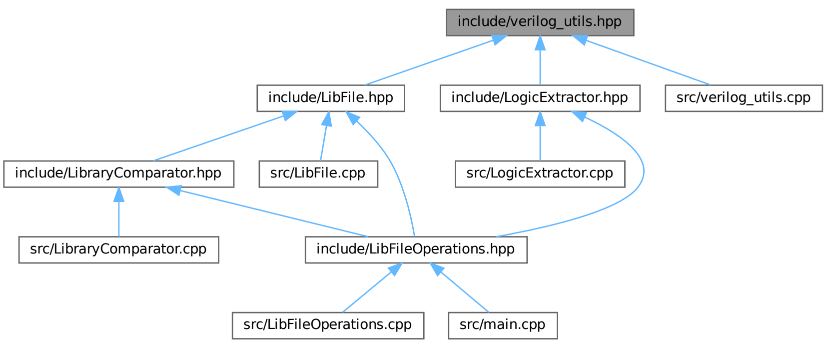
6.19 include/verilog_utils.hpp File Reference

#include <fstream>
#include <unordered_map>
#include <unordered_set>
#include "slang/syntax/SyntaxPrinter.h"
#include "slang/syntax/SyntaxVisitor.h"
#include "spdlog/spdlog.h"

Include dependency graph for verilog_utils.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class VerilogVisitor
- class CellExtractor
- class CellPrinter
- class ModuleRewriter

Functions

- void getAST (const std::string &verilog_file, const std::string &cell)

6.19.1 Function Documentation

6.19.1.1 getAST()

```
void getAST (
            const std::string & verilog_file,
            const std::string & cell )
```

Definition at line 544 of file verilog_utils.cpp.

Here is the call graph for this function:

6.20 verilog_utils.hpp

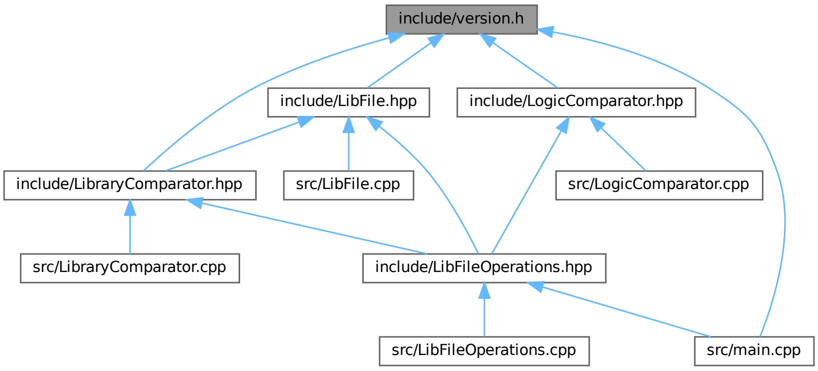
Go to the documentation of this file.

```
// verilog_utils.hpp

#ifndef VERILOG_UTILS_H
#define VERILOG_UTILS_H

#include <fstream>
#include <unordered_map>
#include <unordered_set>

#include "slang/syntax/SyntaxPrinter.h"
#include "slang/syntax/SyntaxVisitor.h"
#include "spdlog/spdlog.h" // Include spdlog

// Creating custom visitor class
class VerilogVisitor : public slang::syntax::SyntaxVisitor<VerilogVisitor> {
public:
  explicit VerilogVisitor(const std::string &targetCell)
      : targetCell_(targetCell), depth_(0), inTargetModule_(false) {}
  void handle(const slang::syntax::SyntaxNode &node);
  void handle(const slang::syntax::ModuleDeclarationSyntax &module);
```

```
  void handle(const slang::syntax::PortDeclarationSyntax &portDecl);
  void handle(const slang::syntax::HierarchyInstantiationSyntax &hierarchyInst);
  // void handle(const slang::syntax::PrimitiveInstantiationSyntax &primitiveInst);
  void handle(const slang::syntax::SpecifyBlockSyntax &specifyBlock);

private:
  const std::string &targetCell_;
  int depth_;
  bool inTargetModule_;
};

// Creating a Rewriter to extract a specific cell
class CellExtractor : public slang::syntax::SyntaxRewriter<CellExtractor> {
public:
  explicit CellExtractor(const std::string &targetCell)
      : targetCell_(targetCell), foundTarget_(false) {}
  void handle(const slang::syntax::ModuleDeclarationSyntax &module);
  bool foundTargetCell() const;

private:
  const std::string &targetCell_;
  bool foundTarget_;
};

// Print specific cell when visiting SyntaxTree
class CellPrinter : public slang::syntax::SyntaxVisitor<CellPrinter> {
public:
  explicit CellPrinter(const std::string &targetCell, std::ostream &out)
      : targetCell_(targetCell), out_(out), foundTarget_(false) {}
  void handle(const slang::syntax::ModuleDeclarationSyntax &module);

private:
  const std::string &targetCell_;
  std::ostream &out_;
  bool foundTarget_;
};

// Comprehensive module rewriter for adding ports, instances, and connections
class ModuleRewriter : public slang::syntax::SyntaxRewriter<ModuleRewriter> {
public:
  explicit ModuleRewriter(const std::vector<std::string> &inputPins,
                          const std::vector<std::string> &outputPins,
                          const std::pair<std::string, std::string> &supercell_entry,
                          int instance_count, std::shared_ptr<spdlog::logger> logger)
      : inputPins_(inputPins), outputPins_(outputPins), cellName_(supercell_entry.first),
        moduleName_(supercell_entry.second), logger_(logger), depth_(0),
        instance_count_(instance_count) {}
  std::shared_ptr<spdlog::logger> logger_;
  void handle(const slang::syntax::SyntaxNode &node);
  void handle(const slang::syntax::ModuleDeclarationSyntax &module);

private:
  const std::vector<std::string> &inputPins_;
  const std::vector<std::string> &outputPins_;
  const std::string cellName_;
  const std::string moduleName_;
  std::map<std::string, std::string> portInfoMap_; // Map from port name to direction
  int depth_;
  int instance_count_;
};

void getAST(const std::string &verilog_file, const std::string &cell);

#endif // VERILOG_UTILS_H
```

6.21 include/version.h File Reference

This graph shows which files directly or indirectly include this file:



Macros

- #define APP_NAME "ZlibValidation"
- #define APP_VERSION_MAJOR 0
- #define APP_VERSION_MINOR 1
- #define APP_VERSION_PATCH 1
- #define APP_VERSION "0.1.1"
- #define APP_AUTHOR "Song Zixuan"
- #define APP_CONTACT "cedar@zju.edu.cn"
- #define BUILD_TIMESTAMP "2025-04-05 15:48:24"

6.21.1 Macro Definition Documentation

6.21.1.1 APP_AUTHOR

```
#define APP_AUTHOR "Song Zixuan"
```

Definition at line 10 of file version.h.

6.21.1.2 APP_CONTACT

```
#define APP_CONTACT "cedar@zju.edu.cn"
```

Definition at line 11 of file version.h.

6.21.1.3 APP_NAME

```
#define APP_NAME "ZlibValidation"
```

Definition at line 5 of file version.h.

6.21.1.4 APP_VERSION

```
#define APP_VERSION "0.1.1"
```

Definition at line 9 of file version.h.

6.21.1.5 APP_VERSION_MAJOR

```
#define APP_VERSION_MAJOR 0
```

Definition at line 6 of file version.h.

6.21.1.6 APP_VERSION_MINOR

```
#define APP_VERSION_MINOR 1
```

Definition at line 7 of file version.h.

6.21.1.7 APP_VERSION_PATCH

```
#define APP_VERSION_PATCH 1
```

Definition at line 8 of file version.h.

6.21.1.8 BUILD_TIMESTAMP

```
#define BUILD_TIMESTAMP "2025-04-05 15:48:24"
```

Definition at line 12 of file version.h.

6.22 version.h

Go to the documentation of this file.

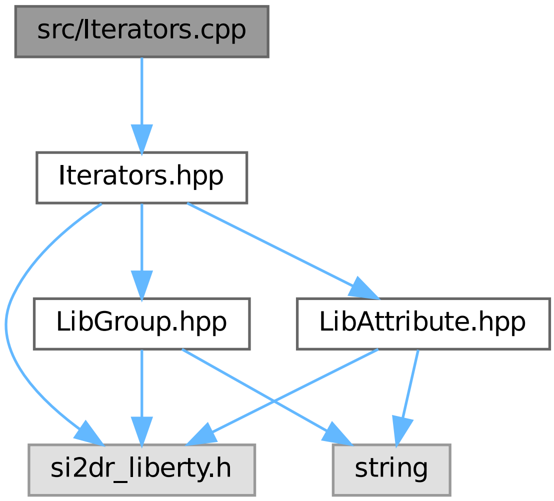
```
00001 // version.h.in
00002 #pragma once
00003
00004 // Auto-generated by CMake - DO NOT EDIT
00005 #define APP_NAME          "ZlibValidation"
00006 #define APP_VERSION_MAJOR 0
00007 #define APP_VERSION_MINOR 1
00008 #define APP_VERSION_PATCH 1
00009 #define APP_VERSION       "0.1.1"
00010 #define APP_AUTHOR        "Song Zixuan"
00011 #define APP_CONTACT       "cedar@zju.edu.cn"
00012 #define BUILD_TIMESTAMP   "2025-04-05 15:48:24"
```

6.23 README.md File Reference

6.24 src/Iterators.cpp File Reference

`#include "Iterators.hpp"`

Include dependency graph for Iterators.cpp:



6.25 Iterators.cpp

Go to the documentation of this file.

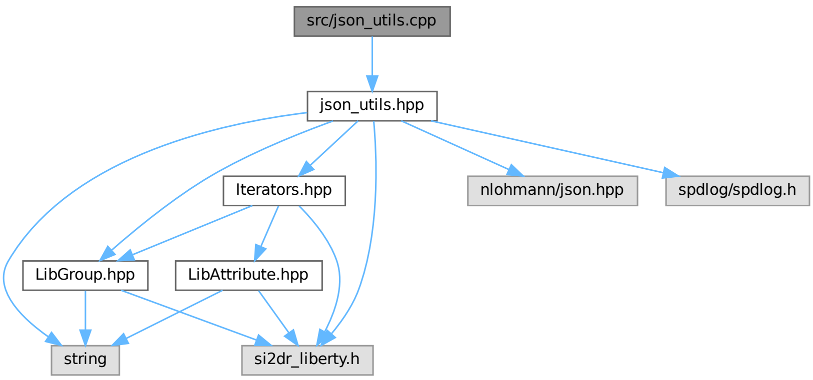
```
#include "Iterators.hpp"

GroupsIterator::GroupsIterator(si2drGroupsIdT groups, si2drErrorT &err) : groups_(groups), err_(err) {
  group_ = si2drIterNextGroup(groups_, &err_);
}
GroupsIterator::~GroupsIterator() { si2drIterQuit(groups_, &err_); }

void GroupsIterator::next() { group_ = si2drIterNextGroup(groups_, &err_); }
bool GroupsIterator::end() { return si2drObjectIsNull(group_, &err_); }

LibGroup GroupsIterator::get() { return LibGroup(group_, err_); }

AttributesIterator::AttributesIterator(si2drAttrsIdT attrs, si2drErrorT &err)
    : attrs_(attrs), err_(err) {
  attr_ = si2drIterNextAttr(attrs_, &err_);
}
AttributesIterator::~AttributesIterator() { si2drIterQuit(attrs_, &err_); }

void AttributesIterator::next() { attr_ = si2drIterNextAttr(attrs_, &err_); }
bool AttributesIterator::end() { return si2drObjectIsNull(attr_, &err_); }
```

```

LibAttribute AttributesIterator::get() { return LibAttribute(attr_, err_); }

ValuesIterator::ValuesIterator(si2drValuesIdT values, si2drErrorT &err) : values_(values), err_(err) {
  si2drIterNextComplexValue(values_, &vtype_, &int_, &float_, &str_, &bool_, &exprp_, &err_);
}
ValuesIterator::~ValuesIterator() { si2drIterQuit(values_, &err_); }

void ValuesIterator::next() {
  si2drIterNextComplexValue(values_, &vtype_, &int_, &float_, &str_, &bool_, &exprp_, &err_);
}
bool ValuesIterator::end() { return vtype_ == SI2DR_UNDEFINED_VALUETYPE; }
```

6.26 src/json_utils.cpp File Reference

`#include "json_utils.hpp"`

Include dependency graph for json_utils.cpp:



Functions

- std::vector< double > parseStringToVector (const std::string &str)

    Parses a string representation of a vector of doubles into a vector of doubles.

- json generateLutJson (LibGroup &lib_lut_group, si2drErrorT &err)

    Generates a JSON object representation of a look-up table (LUT) from a LibGroup object.

- json generateTimingJson (LibGroup &lib_timing_group, si2drErrorT &err)

    Generates a JSON representation of timing information from a library timing group.

- json generatePowerJson (LibGroup &lib_power_group, si2drErrorT &err)

    Converts a Liberty power group into a JSON representation.

- std::pair< std::string, json > generatePinJson (LibGroup &lib_pin_group, si2drErrorT &err)

    Generates a JSON representation of a Liberty pin group.

- json generateCellJson (LibGroup &lib_cell_group, si2drErrorT &err)

    Generates a JSON representation of a cell from a LibGroup object.

6.26.1 Function Documentation

6.26.1.1 generateCellJson()

```
json generateCellJson (
            LibGroup & lib_cell_group,
            si2drErrorT & err )
```

Generates a JSON representation of a cell from a LibGroup object.

This function processes a LibGroup representing a cell and converts it into a JSON object. It extracts the cell name and processes specific attributes such as area, cell_leakage_power, and cell_footprint. It also processes pins within the cell by categorizing them based on their direction (input, output, internal, inout).

Parameters

lib_cell_group	The LibGroup object representing the cell
err	Reference to a si2drErrorT object for error handling

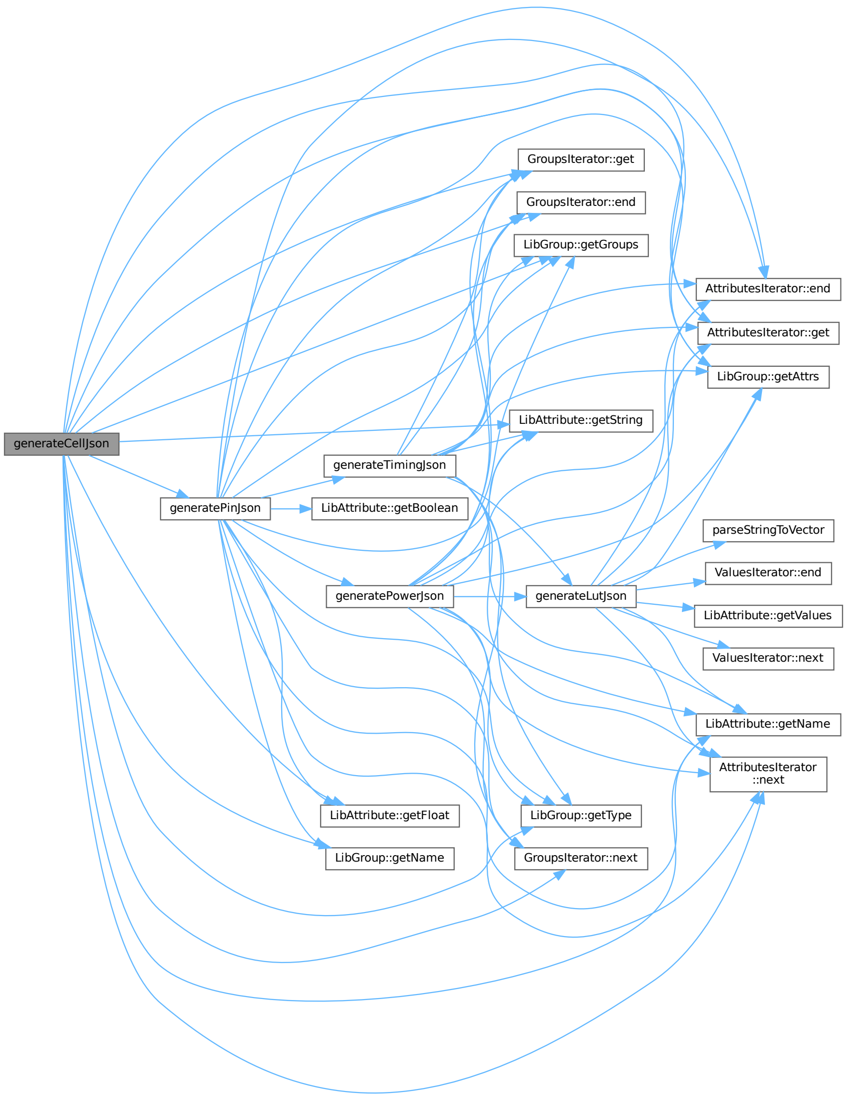
Returns

json A JSON object containing the cell's information with the following structure:

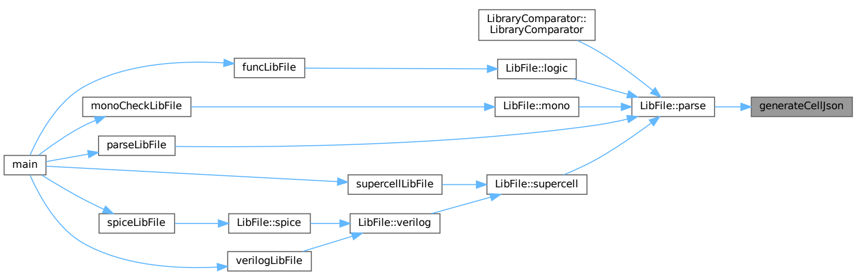
- "cell_name": string - name of the cell
- "area": float (optional) - cell area if present
- "cell_leakage_power": float (optional) - cell leakage power if present
- "cell_footprint": string (optional) - cell footprint if present
- "input_pins": array - list of input pins
- "output_pins": array - list of output pins
- "internal_pins": array - list of internal pins
- "inout_pins": array - list of inout pins

Definition at line 269 of file json_utils.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



6.26.1.2 generateLutJson()

```
json generateLutJson (
            LibGroup & lib_lut_group,
            si2drErrorT & err )
```

Generates a JSON object representation of a look-up table (LUT) from a LibGroup object.

This function iterates through the attributes of the provided LibGroup object representing a LUT and converts them into a JSON structure. It specifically handles the following attributes:

- "index_1": Converted to a vector and stored in the JSON
- "index_2": Converted to a vector and stored in the JSON
- "values": Each value is parsed into a vector and added to an array in the JSON

Any other attributes encountered will trigger a warning message.

Parameters

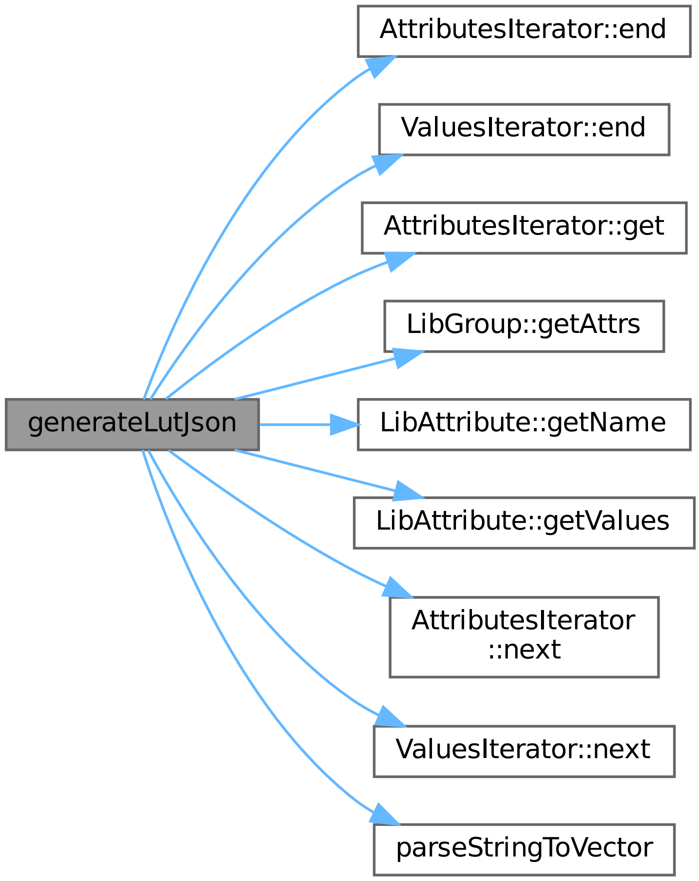
lib_lut_group	The LibGroup object containing the LUT data to be converted
err	Reference to an si2drErrorT object to track any errors during processing

Returns

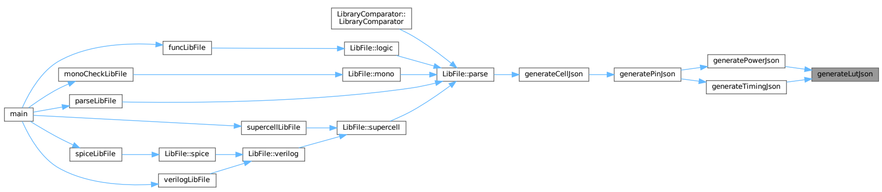
json A JSON object representing the LUT data with keys for "index_1", "index_2", and "values"

Definition at line 59 of file json_utils.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



6.26.1.3 generatePinJson()

std::pair< std::string, json > generatePinJson (

```
LibGroup & lib_pin_group,
si2drErrorT & err )
```

Generates a JSON representation of a Liberty pin group.

This function traverses a Liberty pin group, extracts relevant attributes and sub-groups, and converts them into a JSON object. It also determines the pin's direction.

Processes the following pin attributes:

- direction
- max_transition, capacitance, rise_capacitance, fall_capacitance, max_capacitance (float types)
- function, power_down_function, related_ground_pin, related_power_pin, three_state (string types)
- clock (boolean type)

Handles the following sub-groups:

- internal_power: Converted using generatePowerJson()
- timing: Converted using generateTimingJson()

Parameters

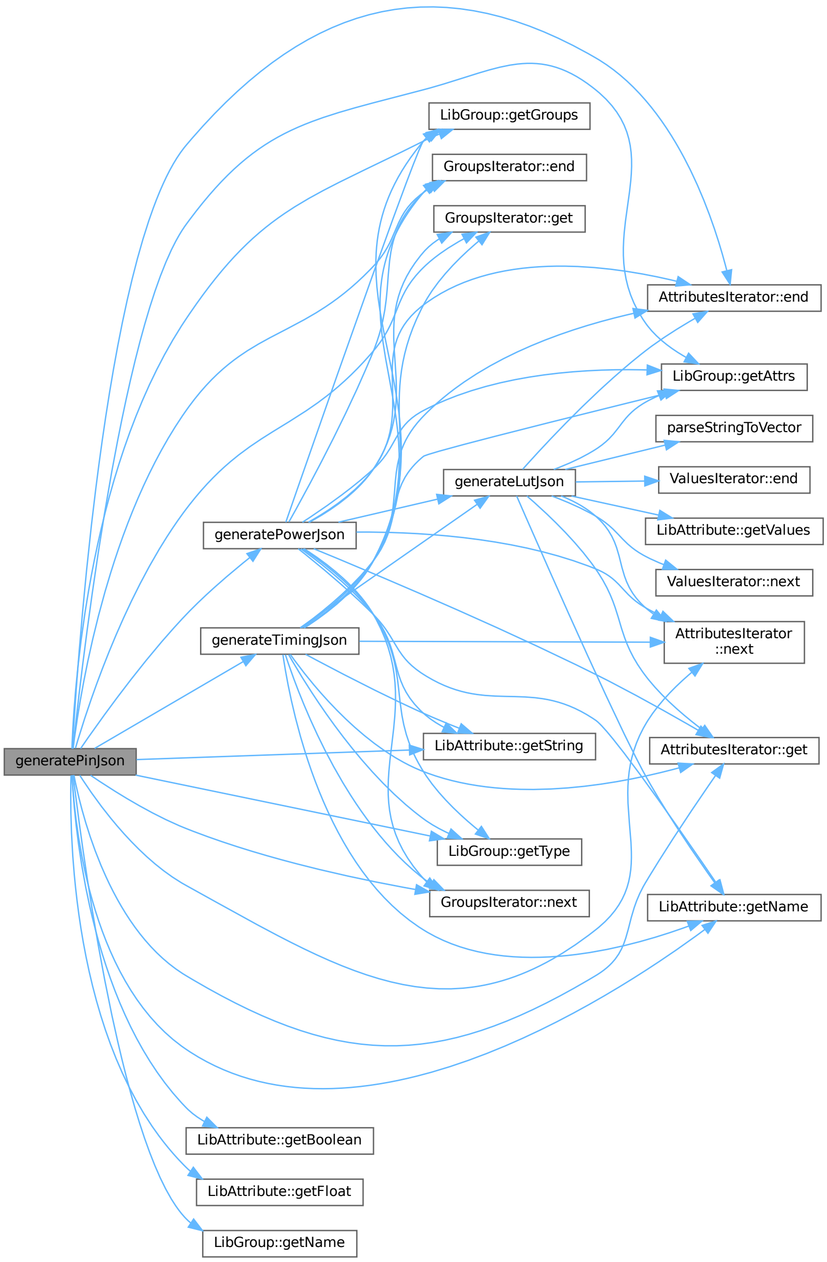
lib_pin_group	The Liberty pin group to process
err	Reference to an error object for tracking Liberty API errors

Returns

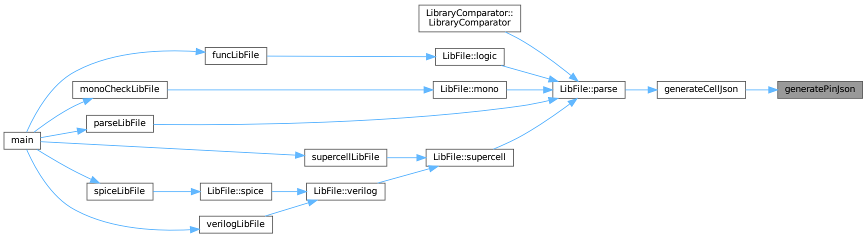
A pair containing the pin direction (string) and the JSON representation of the pin

Definition at line 203 of file json_utils.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



6.26.1.4 generatePowerJson()

```
json generatePowerJson (
        LibGroup & lib_power_group,
        si2drErrorT & err )
```

Converts a Liberty power group into a JSON representation.

This function processes a Liberty power group and converts its attributes and subgroups into a JSON object. It handles attributes like "when", "related_pin", and "related_pg_pin", as well as "rise_power" and "fall_power" subgroups.

Parameters

lib_power_group	The Liberty power group to convert
err	Reference to an si2drErrorT object for error handling

Returns

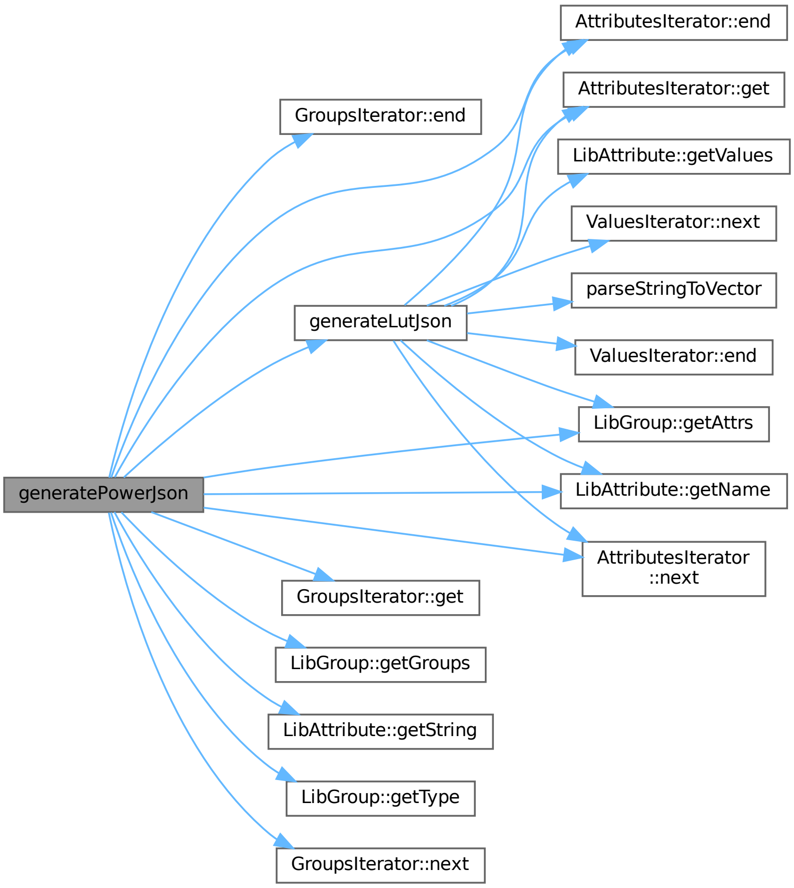
json A JSON object representing the power group data

The function:

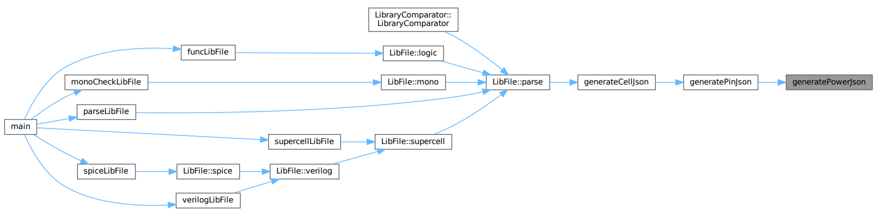
- Extracts string attributes (when, related_pin, related_pg_pin)
- Processes rise_power and fall_power subgroups by converting them to LUT JSON format
- Logs warnings for unknown power subgroup types

Definition at line 151 of file json_utils.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



6.26.1.5 generateTimingJson()

```
json generateTimingJson (
        LibGroup & lib_timing_group,
        si2drErrorT & err )
```

Generates a JSON representation of timing information from a library timing group.

This function extracts timing attributes and sub-groups from a Liberty timing group and organizes them into a JSON object. It processes standard timing attributes like 'related_pin', 'timing_type', 'timing_sense', and 'when', as well as timing tables such as 'cell_fall', 'cell_rise', 'fall_transition', 'rise_transition', 'fall_constraint', and 'rise_constraint'.

Parameters

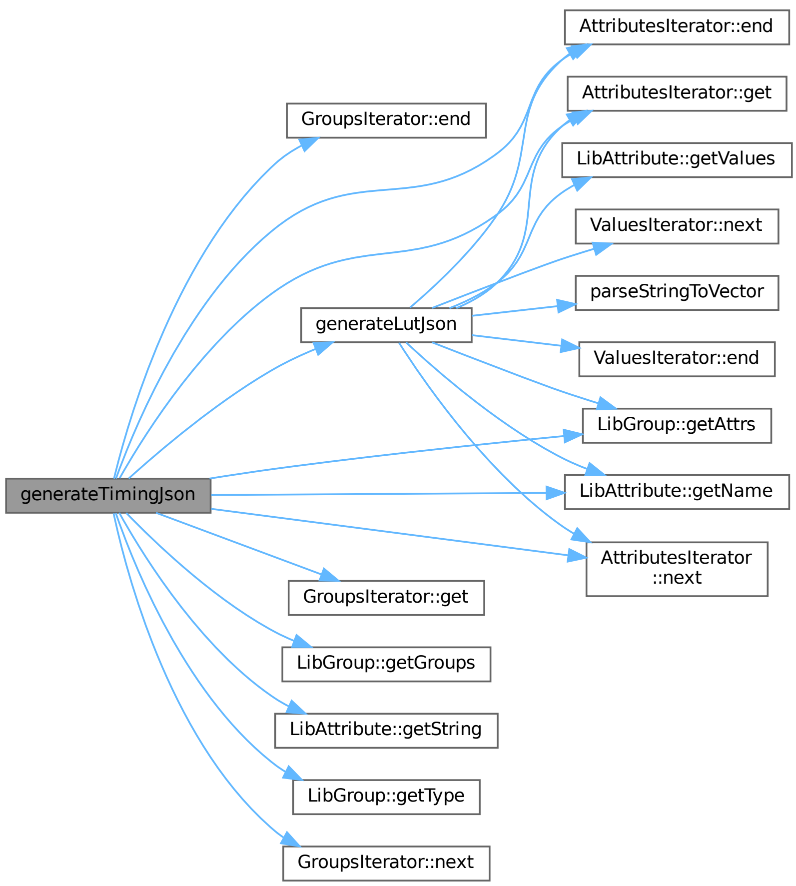
lib_timing_group	The Liberty timing group to process
err	Reference to an si2drErrorT object for error reporting

Returns

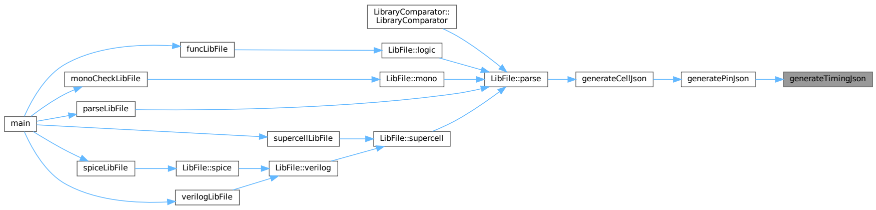
json A JSON object containing the extracted timing information

Definition at line 103 of file json_utils.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



6.26.1.6 parseStringToVector()

```
std::vector< double > parseStringToVector (
            const std::string & str )
```

Parses a string representation of a vector of doubles into a vector of doubles.

This function takes a string containing comma-separated numbers, cleans it by removing backslashes and newline characters, and then converts each comma-separated value into a double that is added to the resulting vector.

Parameters

str	The input string to be parsed, containing comma-separated numbers

Returns

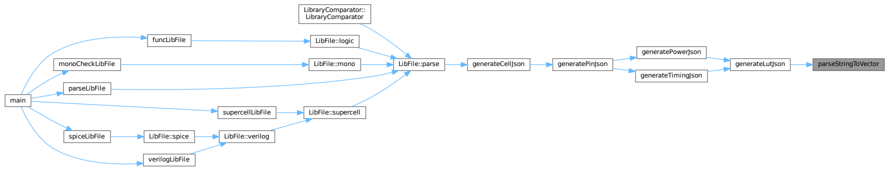
std::vector<double> A vector of doubles parsed from the input string

Exceptions

std::invalid_argument	If the string contains values that cannot be converted to double
std::out_of_range	If the string contains values that are out of range for double

Definition at line 25 of file json_utils.cpp.

Here is the caller graph for this function:



6.27 json_utils.cpp

Go to the documentation of this file.

```
#include "json_utils.hpp"

/*
json 类型的值可以是以下几种类型之一:
方括号 []: 用于包含一组有序的值 (数组)。
数组中的每个值可以是任意类型的 JSON 值，包括对象、数组、字符串、数字、布尔值或 null。
花括号 {}: 用于包含一组键值对 (对象)。
对象中的每个键都是一个字符串，值可以是任意类型的 JSON 值，包括对象、数组、字符串、数字、布尔值或
null。
*/

std::vector<double> parseStringToVector(const std::string &str) {
  std::vector<double> result;
  std::string cleaned_str;

  // Remove backslashes and newline characters
  for (char c : str) {
    if (c != '\\' && c != '\n') {
      cleaned_str += c;
    }
  }

  std::stringstream ss(cleaned_str);
  std::string item;
  while (std::getline(ss, item, ',')) {
    result.push_back(std::stod(item));
  }
  return result;
}

json generateLutJson(LibGroup &lib_lut_group, si2drErrorT &err) {
  json lut_json;

  AttributesIterator attr_iter(lib_lut_group.getAttrs(), err);
  for (; !attr_iter.end(); attr_iter.next()) {
    LibAttribute lib_attr = attr_iter.get();
    std::string lut_attr_name = lib_attr.getName();

    // spdlog::debug("LUT Attribute Name: {}", lib_attr.getName());
    // spdlog::debug("Is Complex? {}", lib_attr.isComplex());

    if (lut_attr_name == "index_1" || lut_attr_name == "index_2") {
      ValuesIterator values_iter(lib_attr.getValues(), err);
      for (; !values_iter.end(); values_iter.next()) {
        // spdlog::debug("Type: {}", int(values_iter.vtype_)); // 5 is string
        // spdlog::debug("Str: {}", values_iter.str_);
        lut_json[lut_attr_name] = parseStringToVector(std::string(values_iter.str_));
      }
    } else if (lut_attr_name == "values") {
```

```
      ValuesIterator values_iter(lib_attr.getValues(), err);
      for (; !values_iter.end(); values_iter.next()) {
        // spdlog::debug("{}", values_iter.str_);
        lut_json["values"].push_back(parseStringToVector(std::string(values_iter.str_)));
      }
    } else {
      spdlog::warn("Unknown LUT attribute name: {}", lut_attr_name);
    }
  }
  return lut_json;
}

json generateTimingJson(LibGroup &lib_timing_group, si2drErrorT &err) {
  json timing_json;

  AttributesIterator attr_iter(lib_timing_group.getAttrs(), err);
  for (; !attr_iter.end(); attr_iter.next()) {
    LibAttribute lib_attr = attr_iter.get();

    std::string attr_name = lib_attr.getName();
    if (attr_name == "related_pin" || attr_name == "timing_type" || attr_name == "timing_sense" ||
        attr_name == "when") {
      timing_json[attr_name] = lib_attr.getString();
    }
    // More timing attributes can be added here
  }

  GroupsIterator timing_sub_group_iter(lib_timing_group.getGroups(), err);
  for (; !timing_sub_group_iter.end(); timing_sub_group_iter.next()) {
    LibGroup lib_timing_sub_group = timing_sub_group_iter.get();

    std::string timing_sub_group_type = lib_timing_sub_group.getType();
    // std::string timing_sub_group_name = lib_timing_sub_group.getName();
    if (timing_sub_group_type == "cell_fall" || timing_sub_group_type == "cell_rise" ||
        timing_sub_group_type == "fall_transition" || timing_sub_group_type == "rise_transition" ||
        timing_sub_group_type == "fall_constraint" || timing_sub_group_type == "rise_constraint") {
      timing_json[timing_sub_group_type] = generateLutJson(lib_timing_sub_group, err);
    } else {
      spdlog::warn("Unknown timing sub group type: {}", timing_sub_group_type);
    }
  }
  return timing_json;
}

json generatePowerJson(LibGroup &lib_power_group, si2drErrorT &err) {
  json power_json;

  AttributesIterator attr_iter(lib_power_group.getAttrs(), err);
  for (; !attr_iter.end(); attr_iter.next()) {
    LibAttribute lib_attr = attr_iter.get();

    std::string attr_name = lib_attr.getName();
    if (attr_name == "when" || attr_name == "related_pin" || attr_name == "related_pg_pin") {
      power_json[attr_name] = lib_attr.getString();
    }
    // More power attributes can be added here
  }

  GroupsIterator power_sub_group_iter(lib_power_group.getGroups(), err);
  for (; !power_sub_group_iter.end(); power_sub_group_iter.next()) {
    LibGroup lib_power_sub_group = power_sub_group_iter.get();

    std::string power_sub_group_type = lib_power_sub_group.getType();
    // std::string power_sub_group_name = lib_power_sub_group.getName();
    if (power_sub_group_type == "rise_power") {
      // spdlog::debug("Rise Power: {}", power_sub_group_name);
      power_json["rise_power"] = generateLutJson(lib_power_sub_group, err);
    } else if (power_sub_group_type == "fall_power") {
      power_json["fall_power"] = generateLutJson(lib_power_sub_group, err);
```

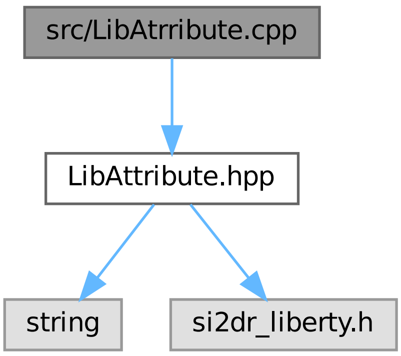
```
    } else {
      spdlog::warn("Unknown power sub group type: {}", power_sub_group_type);
    }
  }
  return power_json;
}

std::pair<std::string, json> generatePinJson(LibGroup &lib_pin_group, si2drErrorT &err) {
  json pin_json;
  std::string direction;
  pin_json["pin_name"] = lib_pin_group.getName();

  AttributesIterator attr_iter(lib_pin_group.getAttrs(), err);
  for (; !attr_iter.end(); attr_iter.next()) {
    LibAttribute lib_attr = attr_iter.get();

    std::string attr_name = lib_attr.getName();
    if (attr_name == "direction") {
      direction = lib_attr.getString();
    } else if (attr_name == "max_transition" || attr_name == "capacitance" ||
               attr_name == "rise_capacitance" || attr_name == "fall_capacitance" ||
               attr_name == "max_capacitance") {
      pin_json[attr_name] = lib_attr.getFloat();
    } else if (attr_name == "function" || attr_name == "power_down_function" ||
               attr_name == "related_ground_pin" || attr_name == "related_power_pin" ||
               attr_name == "three_state") {
      pin_json[attr_name] = lib_attr.getString();
    } else if (attr_name == "clock") {
      pin_json[attr_name] = lib_attr.getBoolean();
    }
    // More pin attributes can be added here
  }

  GroupsIterator pin_sub_group_iter(lib_pin_group.getGroups(), err);
  for (; !pin_sub_group_iter.end(); pin_sub_group_iter.next()) {
    LibGroup lib_pin_sub_group = pin_sub_group_iter.get();

    std::string pin_sub_group_type = lib_pin_sub_group.getType();
    // std::string pin_sub_group_name = lib_pin_sub_group.getName();
    if (pin_sub_group_type == "internal_power") {
      json power_json = generatePowerJson(lib_pin_sub_group, err);
      pin_json["power_arcs"].push_back(power_json);
    } else if (pin_sub_group_type == "timing") {
      // spdlog::debug("Has Timing: {}", lib_pin_group.getName());
      json timing_json = generateTimingJson(lib_pin_sub_group, err);
      pin_json["timing_arcs"].push_back(timing_json);
    } else {
      spdlog::warn("Unknown pin sub group type: {}", pin_sub_group_type);
    }
  }
  return std::make_pair(direction, pin_json);
}

json generateCellJson(LibGroup &lib_cell_group, si2drErrorT &err) {
  json cell_json;
  cell_json["cell_name"] = lib_cell_group.getName();

  AttributesIterator attr_iter(lib_cell_group.getAttrs(), err);
  for (; !attr_iter.end(); attr_iter.next()) {
    LibAttribute lib_attr = attr_iter.get();

    std::string attr_name = lib_attr.getName();
    if (attr_name == "area" || attr_name == "cell_leakage_power") {
      cell_json[attr_name] = lib_attr.getFloat();
    } else if (attr_name == "cell_footprint") {
      cell_json[attr_name] = lib_attr.getString();
    }
    // More cell attributes can be added here
  }
```

```

  GroupsIterator cell_sub_group_iter(lib_cell_group.getGroups(), err);
  for (; !cell_sub_group_iter.end(); cell_sub_group_iter.next()) {
    LibGroup lib_cell_sub_group = cell_sub_group_iter.get();

    std::string cell_sub_group_type = lib_cell_sub_group.getType();
    // std::string cell_sub_group_name = lib_cell_sub_group.getName();

    // spdlog::debug("Cell Sub Group Type: {}", cell_sub_group_type);
    // spdlog::debug("Cell Sub Group Name: {}", cell_sub_group_name);

    if (cell_sub_group_type == "pin") {
      auto [direction, pin_json] = generatePinJson(lib_cell_sub_group, err);
      if (direction == "input") {
        cell_json["input_pins"].push_back(pin_json);
      } else if (direction == "output") {
        cell_json["output_pins"].push_back(pin_json);
      } else if (direction == "internal") {
        cell_json["internal_pins"].push_back(pin_json);
      } else if (direction == "inout") {
        cell_json["inout_pins"].push_back(pin_json);
      } else {
        spdlog::warn("Unknown direction: {}", direction);
      }
    }
  }
  return cell_json;
}
```

6.28 src/LibAtrribute.cpp File Reference

`#include "LibAttribute.hpp"`

Include dependency graph for LibAtrribute.cpp:



6.29 LibAtrribute.cpp

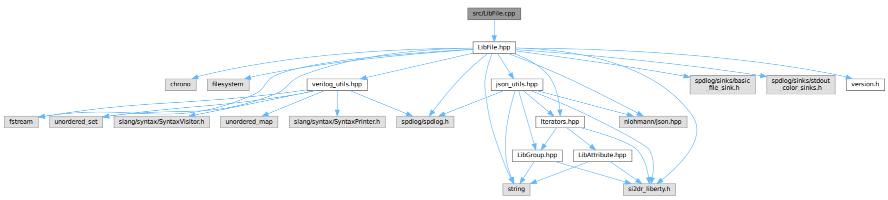
Go to the documentation of this file.

```cpp
#include "LibAttribute.hpp"

LibAttribute::LibAttribute(si2drAttrIdT attr, si2drErrorT &err) : attr_(attr), err_(err) {}

LibAttribute::~LibAttribute() {}

std::string LibAttribute::getName() {
  si2drStringT name = si2drAttrGetName(attr_, &err_);
  return name ? std::string(name) : std::string();
}

bool LibAttribute::isComplex() { return si2drAttrGetAttrType(attr_, &err_) ? 1 : 0; }

si2drValuesIdT LibAttribute::getValues() { return si2drComplexAttrGetValues(attr_, &err_); }

long int LibAttribute::getInt() { return si2drSimpleAttrGetInt32Value(attr_, &err_); }

double LibAttribute::getFloat() { return si2drSimpleAttrGetFloat64Value(attr_, &err_); }

std::string LibAttribute::getString() {
  si2drStringT str = si2drSimpleAttrGetStringValue(attr_, &err_);
  return str ? std::string(str) : std::string();
}

bool LibAttribute::getBoolean() { return si2drSimpleAttrGetBooleanValue(attr_, &err_) ? 1 : 0; }
```

6.30 src/LibFile.cpp File Reference

`#include "LibFile.hpp"`

Include dependency graph for LibFile.cpp:



6.31 LibFile.cpp

Go to the documentation of this file.

```cpp
#include "LibFile.hpp"

LibFile::LibFile(const std::string &filepath, const std::string &loggername)
    : filepath_(filepath), loggername_(loggername) {
  filename_ = filepath_.string();
  basename_ = filepath_.stem().string();
  jsonname_ = basename_ + ".json";

  auto console_sink = std::make_shared<spdlog::sinks::stdout_color_sink_mt>();
  console_sink->set_level(spdlog::level::info);

  auto file_sink = std::make_shared<spdlog::sinks::basic_file_sink_mt>(loggername_, true);
  file_sink->set_level(spdlog::level::debug);
```

```
  std::vector<spdlog::sink_ptr> sinks{console_sink, file_sink};
  logger_ = std::make_shared<spdlog::logger>(loggername_, sinks.begin(), sinks.end());
  logger_->set_level(spdlog::level::debug);

  logger_->info("Created LibFile object for '{}'", filename_);
  logger_->info("Debug log in: '{}'", loggername_);
}

LibFile::~LibFile() { logger_->info("Closing file: '{}'", filename_); }

void LibFile::writeJsonToFile() {
  std::ofstream out(jsonname_);
  if (!out.is_open()) {
    logger_->error("Could not open file '{}' for writing", jsonname_);
    return;
  }
  out << lib_json_.dump(2);
  out.close();
  logger_->info("JSON data written to '{}'", jsonname_);
}

void LibFile::read() {
  logger_->info("Reading '{}' ...", filename_);

  auto start = std::chrono::high_resolution_clock::now();
  si2drReadLibertyFile(const_cast<char *>(filepath_.c_str()), &err_);
  auto end = std::chrono::high_resolution_clock::now();
  std::chrono::duration<double> duration = end - start;

  if (err_ == SI2DR_INVALID_NAME) {
    logger_->error("Could not open file '{}' for parsing, quitting...", filename_);
    exit(301);
  } else if (err_ == SI2DR_SYNTAX_ERROR) {
    logger_->error("Syntax Errors were detected in the input file!");
    exit(401);
  } else {
    logger_->info("Done. Read time: {:.2f} seconds", duration.count());
  }
}

void LibFile::parse() {
  si2drPIInit(&err_); // Initialize private error handler
  this->read();       // Read the Liberty file

  logger_->info("Parsing '{}' ...", filename_);
  // Create a scope for the top-level groups iteration
  {
    GroupsIterator group_iter(si2drPIGetGroups(&err_), err_);
    for (; !group_iter.end(); group_iter.next()) {
      LibGroup lib_group = group_iter.get();

      if (lib_group.getName() != "") {
        libname_ = lib_group.getName();
        lib_json_["library_name"] = this->libname_;
        logger_->info("Library Name: {}", libname_);
      } else {
        logger_->warn("Library Name: <NONAME>");
      }
      // Second level groups
      GroupsIterator sub_group_iter(lib_group.getGroups(), err_);
      for (; !sub_group_iter.end(); sub_group_iter.next()) {
        LibGroup lib_sub_group = sub_group_iter.get();

        std::string sub_group_type = lib_sub_group.getType();
        std::string sub_group_name = lib_sub_group.getName();

        if (sub_group_type == "cell") {
          logger_->debug("Cell Name: {}", sub_group_name);
```

```
          // if (sub_group_name != "AN2D0") {
          //   continue;
          // }
          // Handle each cell
          json cell_json = generateCellJson(lib_sub_group, err_);
          lib_json_["cells"].push_back(cell_json);
        } else if (sub_group_type == "operating_conditions") {
          logger_->info("Operating Conditions: {}", sub_group_name);
          // Get PVT from Operating Conditions
          AttributesIterator attr_iter(lib_sub_group.getAttrs(), err_);
          for (; !attr_iter.end(); attr_iter.next()) {
            LibAttribute lib_attr = attr_iter.get();
            logger_->debug("Attribute Name: {}", lib_attr.getName());
            logger_->debug("Int Value: {}", lib_attr.getInt());
            logger_->debug("Float Value: {}", lib_attr.getFloat());
            logger_->debug("String Value: {}", lib_attr.getString());
            if (lib_attr.getName() == "process") {
              process_ = lib_attr.getInt();
              lib_json_["process"] = process_;
            } else if (lib_attr.getName() == "voltage") {
              voltage_ = lib_attr.getFloat();
              // Round to 2 decimal places to avoid floating-point precision issues
              lib_json_["voltage"] = std::round(voltage_ * 100) / 100.0;
            } else if (lib_attr.getName() == "temperature") {
              temperature_ = lib_attr.getInt();
              lib_json_["temperature"] = temperature_;
            }
          }
          logger_->info("P: {}, V: {}, T: {}", process_, voltage_, temperature_);
        }
        // sub_group_iter's lifetime ends here, and the destructor is called
      }
    }
    // group_iter's lifetime ends here, and the destructor is called
  }
  si2drPIQuit(&err_);
}

void LibFile::modify() { logger_->info("Modifying the file..."); }

bool LibFile::checkTimingArcMonotonicity(const json &cell, const json &pin, const json &arc,
                                         const std::string &timing_arc_name, const bool is_slew) {
  if (arc.contains("when") && !arc["when"].get<std::string>().empty()) {
    logger_->debug(
        "Checking cell: '{}', pin: '{}', related_pin: '{}', timing_arc: '{}', when: '{}'",
        cell["cell_name"].get<std::string>(), pin["pin_name"].get<std::string>(),
        arc["related_pin"].get<std::string>(), timing_arc_name, arc["when"].get<std::string>());
  } else {
    logger_->debug("Checking cell: '{}', pin: '{}', related_pin: '{}', timing_arc: '{}'",
                   cell["cell_name"].get<std::string>(), pin["pin_name"].get<std::string>(),
                   arc["related_pin"].get<std::string>(), timing_arc_name);
  }
  bool is_monotonic = true; // Assume the values are monotonic initially
  if (arc.contains(timing_arc_name) && arc[timing_arc_name].contains("values")) {
    std::vector<std::vector<double>> value_matrix;
    // Generate a matrix of values from the JSON array
    for (const auto &row_val : arc[timing_arc_name]["values"]) {
      std::vector<double> row_data;
      // Check if the value is an array
      if (!row_val.is_array()) {
        logger_->error("Invalid format: '{}' values should be an array of arrays in cell '{}', pin "
                       "'{}', related_pin '{}'",
                       timing_arc_name, cell["cell_name"].get<std::string>(),
                       pin["pin_name"].get<std::string>(), arc["related_pin"].get<std::string>());
        return false;
      }
      // Check for non-numeric values
      for (const auto &val : row_val) {
        if (val.is_number()) {
```

```
            row_data.push_back(val.get<double>());
          } else {
            logger_->warn("Non-numeric value found in '{}'.values, skipping value: {} in cell '{}', "
                          "pin '{}', related_pin '{}'",
                          timing_arc_name, val.dump(), cell["cell_name"].get<std::string>(),
                          pin["pin_name"].get<std::string>(), arc["related_pin"].get<std::string>());
          }
        }
        value_matrix.push_back(row_data);
      }
      // Check the matrix for monotonicity
      if (!value_matrix.empty() && !value_matrix[0].empty()) {
        // Check the monotonic incrementality of rows (by output load capacitance, index 2)
        for (size_t i = 0; i < value_matrix.size(); ++i) {
          for (size_t j = 1; j < value_matrix[i].size(); ++j) {
            if ((value_matrix[i][j] < value_matrix[i][j - 1] &&
                 pin["pin_name"] != arc["related_pin"]) ||
                (value_matrix[i][j] == 0 && value_matrix[i][j - 1] == 0)) {
              // If contains "when" key, log the when value
              if (arc.contains("when") && !arc["when"].get<std::string>().empty()) {
                logger_->warn("Non-monotonic (by load) '{}' values: ({}, {}) {} < ({}, {}) {} "
                              "for Cell: {} Pin: {}->{} when: \"{}\"",
                              timing_arc_name, i, j, value_matrix[i][j], i, j - 1,
                              value_matrix[i][j - 1], cell["cell_name"].get<std::string>(),
                              arc["related_pin"].get<std::string>(),
                              pin["pin_name"].get<std::string>(), arc["when"].get<std::string>());
              } else {
                logger_->warn("Non-monotonic (by load) '{}' values: ({}, {}) {} < ({}, {}) {} "
                              "for Cell: {} Pin: {}->{}",
                              timing_arc_name, i, j, value_matrix[i][j], i, j - 1,
                              value_matrix[i][j - 1], cell["cell_name"].get<std::string>(),
                              arc["related_pin"].get<std::string>(),
                              pin["pin_name"].get<std::string>());
              }
              is_monotonic = false;
            }
          }
        }
        if (is_slew) {
          // Check the monotonic incrementality of columns (by input slew, index 1)
          for (size_t j = 0; j < value_matrix[0].size(); ++j) {
            for (size_t i = 1; i < value_matrix.size(); ++i) {
              if ((value_matrix[i][j] < value_matrix[i - 1][j] &&
                   pin["pin_name"] != arc["related_pin"]) ||
                  (value_matrix[i][j] == 0 && value_matrix[i - 1][j] == 0)) {
                // If contains "when" key, log the when value
                if (arc.contains("when") && !arc["when"].get<std::string>().empty()) {
                  logger_->warn("Non-monotonic (by slew) '{}' values: ({}, {}) {} < ({}, {}) {} "
                                "for Cell: {} Pin: {}->{} when: \"{}\"",
                                timing_arc_name, i, j, value_matrix[i][j], i - 1, j,
                                value_matrix[i - 1][j], cell["cell_name"].get<std::string>(),
                                arc["related_pin"].get<std::string>(),
                                pin["pin_name"].get<std::string>(), arc["when"].get<std::string>());
                } else {
                  logger_->warn("Non-monotonic (by slew) '{}' values: ({}, {}) {} < ({}, {}) {} "
                                "for Cell: {} Pin: {}->{}",
                                timing_arc_name, i, j, value_matrix[i][j], i - 1, j,
                                value_matrix[i - 1][j], cell["cell_name"].get<std::string>(),
                                arc["related_pin"].get<std::string>(),
                                pin["pin_name"].get<std::string>());
                }
                is_monotonic = false;
              }
            }
          }
        }
      } else {
        logger_->warn(
            "Empty or invalid 'values' array found for cell: '{}', pin: '{}', related_pin: '{}', "
```

```
          "timing_arc: '{}'",
          cell["cell_name"].get<std::string>(), pin["pin_name"].get<std::string>(),
          arc["related_pin"].get<std::string>(), timing_arc_name);
    }
  }
  return is_monotonic;
}

void LibFile::mono(const bool is_slew) {
  /*
* Use this command to check the following data in the current library to ensure
* the tables are monotonically increasing with respect to output load:
* cell_rise retaining_rise
* cell_fall retaining_fall
* rise_transition retain_rise_slew
* fall_transition retain_fall_slew
* mpw
*/
  logger_->info("Monotonicity check of '{}' starting ...", filename_);

  if (!std::filesystem::exists(jsonname_)) {
    logger_->info("JSON file not found. Parsing Liberty file first.");
    this->parse();
    this->writeJsonToFile();
  } else {
    // Read the JSON file into json object
    std::ifstream in(jsonname_);
    if (!in.is_open()) {
      logger_->error("Could not open file '{}' for reading", jsonname_);
      return;
    }
    try {
      lib_json_ = json::parse(in);
    } catch (const json::parse_error &e) {
      logger_->error("JSON parsing error in file '{}': {}", jsonname_, e.what());
      in.close();
      return;
    }
    in.close();
  }

  std::map<std::string, bool> cell_monotonicity_status; // Track pass/fail status for each cell
  std::vector<std::string> failed_cells;                // List of failed cell names
  int total_cells = 0;
  int passed_cells = 0;

  // Check the monotonicity of delay values
  // The index 1 (time values) must be monotonically increasing and >= 0
  for (const auto &cell : lib_json_["cells"]) {
    total_cells++;
    bool cell_is_monotonic = true; // Assume cell is monotonic initially
    std::string cell_name = cell["cell_name"].get<std::string>();
    cell_monotonicity_status[cell_name] =
        true; // Initialize to pass, will be set to false if any check fails

    if (cell.contains("output_pins")) {
      for (const auto &pin : cell["output_pins"]) {
        if (pin.contains("timing_arcs")) {
          for (const auto &arc : pin["timing_arcs"]) {
            // Check 4 timing arc names and accumulate monotonicity status
            for (const auto &name :
                 {"cell_rise", "cell_fall", "rise_transition", "fall_transition"}) {
              if (!checkTimingArcMonotonicity(cell, pin, arc, name, is_slew)) {
                cell_is_monotonic = false;
              }
            }
          }
        }
      }
```

```
    }
    if (cell.contains("input_pins")) {
      for (const auto &pin : cell["input_pins"]) {
        if (!pin.contains("clock")) {
          continue; // Skip non-clock pins
        }
        // logger_->debug("Clock pin: {}", pin["pin_name"].get<std::string>());
        if (pin.contains("timing_arcs")) {
          for (const auto &arc : pin["timing_arcs"]) {
            if (arc.contains("timing_type")) {
              // logger_->debug("Timing type: {}", arc["timing_type"].get<std::string>());
              if (arc["timing_type"].get<std::string>() == "min_pulse_width") {
                for (const auto &name : {"rise_constraint", "fall_constraint"}) {
                  if (!checkTimingArcMonotonicity(cell, pin, arc, name, is_slew)) {
                    cell_is_monotonic = false;
                  }
                }
              }
            }
          }
        }
      }
    }
    // Update cell status based on all checks
    cell_monotonicity_status[cell_name] = cell_is_monotonic;
    if (cell_is_monotonic) {
      passed_cells++;
    } else {
      failed_cells.push_back(cell_name);
    }
  }
  logger_->info("Monotonicity check of '{}' completed.", filename_);

  // Output summary statistics
  logger_->info("validate_monotonicity : {} out of {} cells passed", passed_cells, total_cells);
  logger_->info("validate_monotonicity : {} out of {} cells failed", total_cells - passed_cells,
                total_cells);
  if (!failed_cells.empty()) {
    std::stringstream failed_cells_ss;
    for (size_t i = 0; i < failed_cells.size(); ++i) {
      failed_cells_ss << failed_cells[i];
      if (i < failed_cells.size() - 1) {
        failed_cells_ss << ", "; // Add comma if not the last element
      }
    }
    logger_->info("Failed cell list : {}", failed_cells_ss.str());
  }
}

void LibFile::supercell(const int chain_length, const std::vector<std::string> &cell_names) {
  /*
* Supercells are named as follows:
*   <cellname>__X<chain_length>__<input_pin>__<output_pin>
*/
  logger_->info("Creating supercells for '{}'", filename_);

  if (!std::filesystem::exists(jsonname_)) {
    logger_->info("JSON file not found. Parsing Liberty file first.");
    this->parse();
    this->writeJsonToFile();
  } else {
    // Read the JSON file into json object
    std::ifstream in(jsonname_);
    if (!in.is_open()) {
      logger_->error("Could not open file '{}' for reading", jsonname_);
      return;
    }
    try {
      lib_json_ = json::parse(in);
```

```
  } catch (const json::parse_error &e) {
    logger_->error("JSON parsing error in file '{}': {}", jsonname_, e.what());
    in.close();
    return;
  }
  in.close();
}

// write to .map file
std::ofstream out(basename_ + ".map");
if (!out.is_open()) {
  logger_->error("Could not open file '{}' for writing", basename_ + ".map");
  return;
}
// if celll_names is empty, create supercells for all cells
// else create supercells for only the specified cells

// Check if we're processing specific cells or all cells
bool process_all_cells = cell_names.empty();

if (process_all_cells) {
  logger_->info("Creating supercells for ALL cells in '{}'", filename_);
} else {
  logger_->info("Creating supercells for {} specified cells in '{}'", cell_names.size(),
                filename_);
}

// Create a set for faster lookups if we have specific cell names
std::unordered_set<std::string> cell_set;
std::unordered_set<std::string> found_cells;
if (!process_all_cells) {
  cell_set.insert(cell_names.begin(), cell_names.end());
}

for (const auto &cell : lib_json_["cells"]) {
  bool is_sequential = false;
  std::string cell_name = cell["cell_name"].get<std::string>();

  // Skip cells not in the specified list
  if (!process_all_cells && cell_set.find(cell_name) == cell_set.end()) {
    continue;
  }

  // Mark this cell as found
  if (!process_all_cells) {
    found_cells.insert(cell_name);
  }

  logger_->debug("Creating supercells for cell: '{}'", cell_name);

  std::vector<std::string> output_pins;
  std::vector<std::string> input_pins;
  // Extract input and output pins
  if (cell.contains("output_pins")) {
    for (const auto &pin : cell["output_pins"]) {
      output_pins.push_back(pin["pin_name"].get<std::string>());
    }
  }
  if (cell.contains("input_pins")) {
    for (const auto &pin : cell["input_pins"]) {
      if (pin.contains("clock")) {
        is_sequential = true;
        // clock pin not use for supercell
        continue;
      }
      input_pins.push_back(pin["pin_name"].get<std::string>());
    }
  }

```

```
    // create supercells for all combinations of input and output pins
    for (const auto &output_pin : output_pins) {
      for (const auto &input_pin : input_pins) {
        if (is_sequential) {
          const int chain_len = 1;
          std::string supercell_name =
              cell_name + "__X" + std::to_string(chain_len) + "__" + input_pin + "__" + output_pin;
          out << cell_name << " " << supercell_name << std::endl;
        } else {
          std::string supercell_name = cell_name + "__X" + std::to_string(chain_length) + "__" +
                                       input_pin + "__" + output_pin;
          out << cell_name << " " << supercell_name << std::endl;
        }
      }
    }
  }

  // Check for cells that were specified but not found
  if (!process_all_cells) {
    std::vector<std::string> not_found_cells;
    for (const auto &requested_cell : cell_names) {
      if (found_cells.find(requested_cell) == found_cells.end()) {
        not_found_cells.push_back(requested_cell);
      }
    }

    // Output warning for cells that weren't found
    if (!not_found_cells.empty()) {
      std::string missing_cells = not_found_cells[0];
      for (size_t i = 1; i < not_found_cells.size(); ++i) {
        missing_cells += ", " + not_found_cells[i];
      }
      logger_->warn("{} specified cell{} not found in the library: {}", not_found_cells.size(),
                    not_found_cells.size() > 1 ? "s were" : " was", missing_cells);
    }
  }

  out.close();
  logger_->info("Supercell creation complete in '{}'", basename_ + ".map");
}

void LibFile::verilog(const int chain_length, const std::vector<std::string> &cell_names) {
  logger_->info("Creating Verilog for '{}'", filename_);

  this->supercell(chain_length, cell_names);

  // Create a temporary file for individual modules
  std::string temp_file = basename_ + "_temp.v";
  std::ofstream out(temp_file);
  if (!out.is_open()) {
    logger_->error("Could not open temp file '{}' for writing", temp_file);
    return;
  }

  // Read the .map file into a vector to preserve all entries and their order
  std::vector<std::pair<std::string, std::string>> supercell_entries;
  std::ifstream in(basename_ + ".map");
  if (!in.is_open()) {
    logger_->error("Could not open file '{}' for reading", basename_ + ".map");
    return;
  }

  std::string line;
  while (std::getline(in, line)) {
    std::istringstream iss(line);
    std::string cell_name, supercell_name;
    if (!(iss >> cell_name >> supercell_name)) {
      logger_->warn("Invalid line format in map file: '{}'", line);
      continue;
```

```
    }
    supercell_entries.push_back({cell_name, supercell_name});
  }
  in.close();
  logger_->info("Read {} supercells from '{}'", supercell_entries.size(), basename_ + ".map");

  // Store module information for top-level creation
  std::vector<std::string> module_names;
  std::map<std::string, std::vector<std::string>> module_inputs;
  std::map<std::string, std::vector<std::string>> module_outputs;

  // Generate verilog module for each supercell
  for (const auto &supercell_entry : supercell_entries) {
    // Check if the cell is sequential
    bool is_sequential = false;
    // Count the number of instances will be created in verilog
    int instance_count = 0;
    // Get original cell name from pair
    const std::string &cell_name = supercell_entry.first;
    // Get supercell name(as well as module name) from pair
    const std::string &module_name = supercell_entry.second;

    // Store module name for top-level creation
    module_names.push_back(module_name);

    logger_->info("Creating Module: '{}' from Cell '{}'", module_name, cell_name);

    // Get the cell JSON object
    json cell_json;
    for (const auto &cell : lib_json_["cells"]) {
      if (cell["cell_name"].get<std::string>() == cell_name) {
        cell_json = cell;
        break;
      }
    }

    // Get input/output pins from the cell JSON object
    std::vector<std::string> input_pins;
    std::vector<std::string> output_pins;
    std::stringstream input_pins_ss;
    std::stringstream output_pins_ss;
    if (cell_json.contains("input_pins")) {
      for (const auto &pin : cell_json["input_pins"]) {
        if (pin.contains("clock")) {
          is_sequential = true;
        }
        std::string pin_name = pin["pin_name"].get<std::string>();
        input_pins.push_back(pin_name);
        input_pins_ss << pin_name << ", ";

        // Store input pin name for top-level creation
        module_inputs[module_name].push_back(pin_name);
      }
    }
    if (cell_json.contains("output_pins")) {
      for (const auto &pin : cell_json["output_pins"]) {
        std::string pin_name = pin["pin_name"].get<std::string>();
        output_pins.push_back(pin_name);
        output_pins_ss << pin_name << ", ";

        // Store output pin name for top-level creation
        module_outputs[module_name].push_back(pin_name);
      }
    }
    logger_->debug("Input pins set: {}", input_pins_ss.str());
    logger_->debug("Output pins set: {}", output_pins_ss.str());

    // Sequential cells will have only one instance
    // Combinational cells will have instances equal to chain length
```

```
    if (is_sequential) {
      instance_count = 1;
    } else {
      instance_count = chain_length;
    }

    // Create ANSI port list
    std::string port_list_text = "(";
    // Add input ports
    bool isFirstPort = true;
    for (const auto &input_pin : input_pins) {
      if (!isFirstPort) {
        port_list_text += ", ";
      }
      port_list_text += "input " + input_pin;
      isFirstPort = false;
    }
    // Add output ports
    for (const auto &output_pin : output_pins) {
      if (!isFirstPort) {
        port_list_text += ", ";
      }
      port_list_text += "output " + output_pin;
      isFirstPort = false;
    }
    port_list_text += ")";
    logger_->debug("ANSI Port list: {}", port_list_text);

    // Creat full module header text
    std::string fullModuleText = "module " + module_name + port_list_text + ";\nendmodule";
    logger_->debug("Full module text: {}", fullModuleText);

    // Generate the syntax tree for the module using slang library
    auto tree = slang::syntax::SyntaxTree::fromText(fullModuleText);
    if (tree) {
      // Add ports to the syntax tree
      ModuleRewriter rewriter(input_pins, output_pins, supercell_entry, instance_count, logger_);

      tree = rewriter.transform(tree);
      // Revisit the syntax tree to check architecture
      // rewriter.visit(tree->root());

      // Output the transformed syntax tree
      out << "`timescale 1ns/10ps" << std::endl;
      out << slang::syntax::SyntaxPrinter::printFile(*tree) << std::endl << std::endl;
    } else {
      logger_->error("Failed to create syntax tree for module '{}'", module_name);
      continue;
    }
  }

  out.close();

  // Create the top-level module
  std::ofstream final_out(basename_ + ".v");
  if (!final_out.is_open()) {
    logger_->error("Could not open file '{}' for writing", basename_ + ".v");
    return;
  }

  // Collect all port names separately for inputs and outputs
  std::vector<std::string> all_input_ports;
  std::vector<std::string> all_output_ports;
  std::stringstream top_instances;

  // First pass: collect all port names
  for (const auto &module_name : module_names) {
    // Collect input port names
    for (const auto &pin : module_inputs[module_name]) {
```

```
      all_input_ports.push_back(module_name + "__" + pin);
    }

    // Collect output port names
    for (const auto &pin : module_outputs[module_name]) {
      all_output_ports.push_back(module_name + "__" + pin);
    }

    // Create instance
    std::string instance_name = "I_" + module_name.substr(0, module_name.find("__X"));
    for (size_t i = module_name.find("__X") + 3; i < module_name.length(); i++) {
      if (module_name[i] == '_' && module_name[i + 1] == '_') {
        instance_name += "__" + module_name.substr(i + 2);
        break;
      }
    }

    // Generate port connections for this instance
    std::stringstream instance_ports;
    for (const auto &pin : module_inputs[module_name]) {
      instance_ports << "." << pin << "(" << module_name << "__" << pin << "), ";
    }
    for (const auto &pin : module_outputs[module_name]) {
      instance_ports << "." << pin << "(" << module_name << "__" << pin << "), ";
    }

    // Remove last comma and space
    std::string instance_ports_str = instance_ports.str();
    if (!instance_ports_str.empty()) {
      instance_ports_str = instance_ports_str.substr(0, instance_ports_str.length() - 2);
    }

    top_instances << "  " << module_name << "  " << instance_name << " (" << instance_ports_str
                  << ");\n";
  }

  // Now generate the port list with all inputs first, then all outputs
  std::stringstream top_ports;

  // Add all input ports first
  for (size_t i = 0; i < all_input_ports.size(); ++i) {
    top_ports << all_input_ports[i];
    if (i < all_input_ports.size() - 1 || !all_output_ports.empty()) {
      top_ports << ", ";
    }
  }

  // Then add all output ports
  for (size_t i = 0; i < all_output_ports.size(); ++i) {
    top_ports << all_output_ports[i];
    if (i < all_output_ports.size() - 1) {
      top_ports << ", ";
    }
  }

  // Generate input and output declarations
  std::stringstream top_inputs;
  std::stringstream top_outputs;

  // Generate input declarations
  for (const auto &port : all_input_ports) {
    top_inputs << "  input " << port << ";\n";
  }

  // Generate output declarations
  for (const auto &port : all_output_ports) {
    top_outputs << "  output " << port << ";\n";
  }

```

```
  // Write the top module
  final_out << "`timescale 1ns/10ps" << std::endl;
  final_out << "module validate_top ( " << top_ports.str() << " );" << std::endl;
  final_out << std::endl; // Add newline before port declarations
  final_out << top_inputs.str();
  final_out << top_outputs.str();
  final_out << std::endl;
  final_out << top_instances.str();
  final_out << "endmodule" << std::endl << std::endl;

  // Append the module definitions from the temporary file
  std::ifstream temp_in(temp_file);
  if (temp_in.is_open()) {
    final_out << temp_in.rdbuf();
    temp_in.close();
  } else {
    logger_->error("Could not open temp file '{}' for reading", temp_file);
  }

  final_out.close();

  // Remove temporary file
  // if (std::filesystem::exists(temp_file)) {
  //   std::filesystem::remove(temp_file);
  // }

  logger_->info("Verilog creation complete in '{}'", basename_ + ".v");
}

std::vector<std::string> LibFile::splitString(const std::string &s) {
  std::vector<std::string> tokens;
  std::stringstream ss(s);
  std::string token;
  while (ss >> token) {
    tokens.push_back(token);
  }
  return tokens;
}

void LibFile::generateRCLines(std::ofstream &outFile, const std::string &netName, int instanceIndex,
                              bool isFinalStage) {
  // --- Default RC Values (as observed in the target SPICE) ---
  const double R1_INTERMEDIATE = 0.01;
  const double C1_INTERMEDIATE = 5.3e-16;
  const double R2_INTERMEDIATE = 0.01;
  const double R1_FINAL = 1e-2;
  const double C1_FINAL = 1e-18;
  const std::string CAP_GROUND = "COREGND1"; // Assumed ground for capacitors

  std::string r1Name = "R1_" + std::to_string(instanceIndex);
  std::string c1Name = "C1_" + std::to_string(instanceIndex);
  std::string r2Name = "R2_" + std::to_string(instanceIndex);
  std::string node1 = netName + ":1"; // Intermediate node 1
  std::string node2 = netName + ":2"; // Intermediate node 2

  // Set precision for scientific notation output
  outFile << std::scientific << std::setprecision(1);

  if (!isFinalStage) {
    // Intermediate RCR structure (R1-C1-R2)
    outFile << r1Name << " " << node1 << " " << node2 << " " << R1_INTERMEDIATE << std::endl;
    outFile << c1Name << " " << node2 << " " << CAP_GROUND << " " << C1_INTERMEDIATE << std::endl;
    outFile << r2Name << " " << node2 << " " << netName << " " << R2_INTERMEDIATE << std::endl;
  } else {
    // Final RC structure (R1-C1 connected to the instance output node :1)
    // Note: The target SPICE connects R1 between node1 and the final port (netName).
    outFile << r1Name << " " << node1 << " " << netName << " " << R1_FINAL << std::endl;
    outFile << c1Name << " " << node1 << " " << CAP_GROUND << " " << C1_FINAL << std::endl;
  }
```

```
  // Reset precision/format to default
  outFile << std::defaultfloat << std::setprecision(6);
}

bool LibFile::modifySpiceNetlist(const std::string &v2lvsSpiceFile, // Input is the v2lvs output
                                const std::string &finalSpiceFile, // Output is the final file
                                const std::string &targetGlobalLine) {
  logger_->info("Post-processing SPICE netlist: {} -> {}", v2lvsSpiceFile, finalSpiceFile);

  std::ifstream inFile(v2lvsSpiceFile);
  // Write directly to the final output file, overwriting if it exists
  std::ofstream outFile(finalSpiceFile, std::ios::trunc);

  if (!inFile) {
    logger_->error("Could not open input v2lvs SPICE file for modification: {}", v2lvsSpiceFile);
    return false;
  }
  if (!outFile) {
    logger_->error("Could not open final output SPICE file for writing: {}", finalSpiceFile);
    inFile.close(); // Close input file if output fails
    return false;
  }

  // --- Add Metadata Comments ---
  outFile << "** Generated by " << APP_NAME << " v" << APP_VERSION << " from " << APP_AUTHOR;
  auto now = std::chrono::system_clock::now();
  std::time_t now_time_t = std::chrono::system_clock::to_time_t(now);
  // Use localtime_s on Windows, localtime_r on POSIX, or std::localtime (less safe)
  // std::tm now_tm;
  // localtime_s(&now_tm, &now_time_t); // Example for Windows
  std::tm *now_tm_ptr = std::localtime(&now_time_t); // Standard C++, potentially less thread-safe
  if (now_tm_ptr) {
    outFile << ". On: " << std::put_time(now_tm_ptr, "%c %Z") << " **\n" << std::endl;
  } else {
    outFile << ". Timestamp unavailable **\n" << std::endl;
  }
  // --- End Metadata ---

  std::string line;
  bool includeFound = false;
  bool globalInserted = false;
  bool inSubckt = false;
  std::string previousInstanceLine = "";
  int instanceIndex = 0; // Counter for R/C naming within a subckt

  while (std::getline(inFile, line)) {
    // Trim leading/trailing whitespace
    line.erase(0, line.find_first_not_of(" \t\n\r\f\v"));
    line.erase(line.find_last_not_of(" \t\n\r\f\v") + 1);

    // Skip empty lines and v2lvs header comments
    if (line.empty() || line.find("$ Spice netlist generated by v2lvs") == 0 ||
        line.find("$ v2") == 0 || // Catch version line too
        line.find("*.BUSDELIMITER") == 0) {
      continue; // Skip these lines entirely
    }

    // Write other comments directly after processing any pending instance
    if (line[0] == '*') {
      // Process previous instance before writing comment if needed
      if (!previousInstanceLine.empty()) {
        std::vector<std::string> tokens = this->splitString(previousInstanceLine);
        if (tokens.size() > 2 && (tokens[0][0] == 'X' || tokens[0][0] == 'x')) {
          std::string moduleName = tokens.back();
          std::string outputNet = tokens[tokens.size() - 2]; // Assumption
          bool isFinal = (outputNet == "CO" || outputNet == "S" || outputNet == "ZN");
          tokens[tokens.size() - 2] = outputNet + ":1"; // Modify output pin

          // Reconstruct line with correct VDD/VSS order
```

```
          for (size_t i = 0; i < tokens.size() - 1; ++i) { // Up to module name
            outFile << tokens[i] << " ";
          }
          outFile << "VDD VSS " << moduleName << std::endl; // Insert VDD VSS before module name

          this->generateRCLines(outFile, outputNet, instanceIndex - 1, isFinal);
        } else {
          logger_->warn("Previous line stored but not recognized as instance: {}",
                        previousInstanceLine);
          outFile << previousInstanceLine << std::endl;
        }
        previousInstanceLine = ""; // Clear after processing
      }
      outFile << line << std::endl; // Write the comment line
      continue;
    }

    // Handle .INCLUDE
    if (line.find(".INCLUDE") == 0) {
      outFile << line << std::endl;
      includeFound = true;
      continue; // Process next line to insert global
    }

    // Insert the target global line after .INCLUDE
    if (includeFound && !globalInserted) {
      outFile << targetGlobalLine << std::endl << std::endl; // Add extra newline for spacing
      globalInserted = true;
      // Fall through to process the current line
    }

    // Handle .SUBCKT
    if (line.find(".SUBCKT") == 0 || line.find(".subckt") == 0) { // Handle case-insensitivity
      if (!previousInstanceLine.empty()) {
        std::vector<std::string> tokens = this->splitString(previousInstanceLine);
        if (tokens.size() > 2 && (tokens[0][0] == 'X' || tokens[0][0] == 'x')) {
          std::string moduleName = tokens.back();
          std::string outputNet = tokens[tokens.size() - 2];
          bool isFinal = (outputNet == "CO" || outputNet == "S" || outputNet == "ZN");
          tokens[tokens.size() - 2] = outputNet + ":1";

          for (size_t i = 0; i < tokens.size() - 1; ++i) {
            outFile << tokens[i] << " ";
          }
          outFile << "VDD VSS " << moduleName << std::endl;

          this->generateRCLines(outFile, outputNet, instanceIndex - 1, isFinal);
        } else {
          logger_->warn("Previous line stored but not recognized as instance: {}",
                        previousInstanceLine);
          outFile << previousInstanceLine << std::endl;
        }
        previousInstanceLine = "";
      }

      inSubckt = true;
      instanceIndex = 0;                          // Reset R/C counter for new subcircuit
      outFile << line << " VDD VSS" << std::endl; // Add VDD VSS to ports
    }
    // Handle .ENDS
    else if (line.find(".ENDS") == 0 || line.find(".ends") == 0) { // Handle case-insensitivity
      inSubckt = false;
      // Process the last instance line *before* writing .ENDS
      if (!previousInstanceLine.empty()) {
        std::vector<std::string> tokens = this->splitString(previousInstanceLine);
        if (tokens.size() > 2 && (tokens[0][0] == 'X' || tokens[0][0] == 'x')) {
          std::string moduleName = tokens.back();
          std::string outputNet = tokens[tokens.size() - 2];
          bool isFinal = (outputNet == "CO" || outputNet == "S" || outputNet == "ZN");
```

```
          tokens[tokens.size() - 2] = outputNet + ":1";

          for (size_t i = 0; i < tokens.size() - 1; ++i) {
            outFile << tokens[i] << " ";
          }
          outFile << "VDD VSS " << moduleName << std::endl;

          this->generateRCLines(outFile, outputNet, instanceIndex - 1, isFinal);
        } else {
          logger_->warn("Previous line stored but not recognized as instance: {}",
                        previousInstanceLine);
          outFile << previousInstanceLine << std::endl;
        }
        previousInstanceLine = ""; // Clear after processing
      }
      outFile << line << std::endl << std::endl; // Add extra newline after .ENDS
    }
    // Skip original .GLOBAL lines
    else if (line.find(".GLOBAL") == 0 || line.find(".global") == 0) {
      continue;
    }
    // Handle Instance lines
    else if (inSubckt && (line[0] == 'X' || line[0] == 'x')) {
      if (!previousInstanceLine.empty()) {
        std::vector<std::string> tokens = this->splitString(previousInstanceLine);
        if (tokens.size() > 2 && (tokens[0][0] == 'X' || tokens[0][0] == 'x')) {
          std::string moduleName = tokens.back();
          std::string outputNet = tokens[tokens.size() - 2]; // Assumption
          bool isFinal = (outputNet == "CO" || outputNet == "S" || outputNet == "ZN");
          tokens[tokens.size() - 2] = outputNet + ":1"; // Modify output pin

          // Reconstruct line with correct VDD/VSS order
          for (size_t i = 0; i < tokens.size() - 1; ++i) { // Up to module name
            outFile << tokens[i] << " ";
          }
          outFile << "VDD VSS " << moduleName << std::endl; // Insert VDD VSS before module name

          this->generateRCLines(outFile, outputNet, instanceIndex - 1,
                                isFinal); // Use previous index
        } else {
          logger_->warn("Previous line stored but not recognized as instance: {}",
                        previousInstanceLine);
          outFile << previousInstanceLine << std::endl;
        }
      }
      previousInstanceLine = line;
      instanceIndex++;
    }
    // Handle other lines
    else {
      if (!previousInstanceLine.empty()) {
        std::vector<std::string> tokens = this->splitString(previousInstanceLine);
        if (tokens.size() > 2 && (tokens[0][0] == 'X' || tokens[0][0] == 'x')) {
          std::string moduleName = tokens.back();
          std::string outputNet = tokens[tokens.size() - 2];
          bool isFinal = (outputNet == "CO" || outputNet == "S" || outputNet == "ZN");
          tokens[tokens.size() - 2] = outputNet + ":1";

          for (size_t i = 0; i < tokens.size() - 1; ++i) {
            outFile << tokens[i] << " ";
          }
          outFile << "VDD VSS " << moduleName << std::endl;

          this->generateRCLines(outFile, outputNet, instanceIndex - 1, isFinal);
        } else {
          logger_->warn("Previous line stored but not recognized as instance: {}",
                        previousInstanceLine);
          outFile << previousInstanceLine << std::endl;
        }
```

```
        previousInstanceLine = ""; // Clear after processing
      }
      outFile << line << std::endl;
    }
  }

  // Process the very last instance line
  if (!previousInstanceLine.empty()) {
    std::vector<std::string> tokens = this->splitString(previousInstanceLine);
    if (tokens.size() > 2 && (tokens[0][0] == 'X' || tokens[0][0] == 'x')) {
      std::string moduleName = tokens.back();
      std::string outputNet = tokens[tokens.size() - 2];
      bool isFinal = (outputNet == "CO" || outputNet == "S" || outputNet == "ZN");
      tokens[tokens.size() - 2] = outputNet + ":1";

      for (size_t i = 0; i < tokens.size() - 1; ++i) {
        outFile << tokens[i] << " ";
      }
      outFile << "VDD VSS " << moduleName << std::endl;

      this->generateRCLines(outFile, outputNet, instanceIndex - 1, isFinal);
    } else {
      logger_->warn("Last stored line not recognized as instance: {}", previousInstanceLine);
      outFile << previousInstanceLine << std::endl;
    }
  }

  inFile.close();
  outFile.close();

  // No renaming needed, we wrote directly to the final file
  logger_->info("SPICE netlist post-processing successful for: {}", finalSpiceFile);
  return true;
}

void LibFile::spice(const int chain_length, const std::vector<std::string> &cell_names,
                    const std::string &verilog_lib_file, const std::string &spice_lib_file) {
  logger_->info("Creating SPICE for '{}'", filename_);

  // 1. Generate the temporary Verilog file first
  this->verilog(chain_length, cell_names);

  // Define input and output filenames
  std::string temp_verilog_file = basename_ + "_temp.v";
  std::string v2lvs_output_spice_file = basename_ + ".v2lvs.spi"; // v2lvs output
  std::string final_output_spice_file = basename_ + ".spi";       // Final processed output

  // Check if temporary Verilog file exists
  std::ifstream check_v_in(temp_verilog_file);
  if (!check_v_in.is_open()) {
    logger_->error("Temporary Verilog file {} not found or could not be opened. Verilog generation "
                   "might have failed.",
                   temp_verilog_file);
    return;
  }
  check_v_in.close(); // Close after checking

  // 2. Check if V2LVS tool is available
  logger_->debug("Checking for v2lvs tool...");
  std::string v2lvs_path_cmd = "which v2lvs";
  std::string v2lvs_path_result;
  std::array<char, 128> buffer;
  std::unique_ptr<FILE, decltype(&pclose)> pipe(popen(v2lvs_path_cmd.c_str(), "r"), pclose);
  if (!pipe) {
    logger_->error("Failed to run command to find v2lvs: {}", v2lvs_path_cmd);
    // Do NOT delete temp_verilog_file here if popen fails
    return;
  }
  while (fgets(buffer.data(), buffer.size(), pipe.get()) != nullptr) {
```

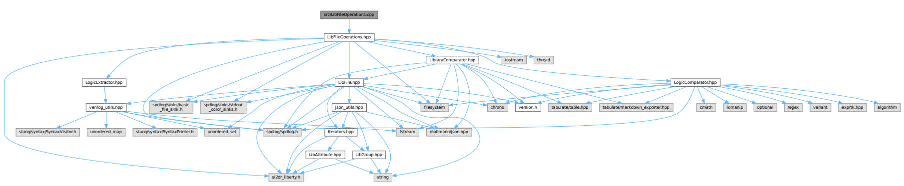
```
    v2lvs_path_result += buffer.data();
  }

  // Trim potential newline from the result
  if (!v2lvs_path_result.empty() && v2lvs_path_result.back() == '\n') {
    v2lvs_path_result.pop_back();
  }

  if (v2lvs_path_result.empty()) {
    logger_->error(
        "V2LVS tool not found in PATH. Please install Calibre or ensure v2lvs is accessible.");
    // Do NOT delete temp_verilog_file
    return;
  } else {
    logger_->info("V2LVS tool found at: {}", v2lvs_path_result);
  }

  // 3. Construct and Run V2LVS command
  logger_->info("Running V2LVS to generate initial SPICE netlist...");
  std::string v2lvs_cmd_full = "v2lvs";                // Start with the command name
  v2lvs_cmd_full += " -v " + temp_verilog_file;        // Input Verilog
  v2lvs_cmd_full += " -o " + v2lvs_output_spice_file; // Output SPICE
  v2lvs_cmd_full += " -s0 VSS";                        // Default ground net, generates .GLOBAL VSS
  v2lvs_cmd_full += " -s1 VDD";                        // Default power net, generates .GLOBAL VDD
  v2lvs_cmd_full += " -i";                             // Use positional pins (traditional SPICE)
  v2lvs_cmd_full += " -l " + verilog_lib_file;         // Verilog library for pin order
  v2lvs_cmd_full += " -s " + spice_lib_file;           // SPICE library to include

  logger_->debug("Executing V2LVS command: {}", v2lvs_cmd_full);
  int ret = system(v2lvs_cmd_full.c_str());

  // Do NOT delete temp_verilog_file

  if (ret != 0) {
    logger_->error("V2LVS command failed with exit code: {}. Command: {}", ret, v2lvs_cmd_full);
    // Attempt to remove potentially incomplete v2lvs output SPICE file
    std::remove(v2lvs_output_spice_file.c_str());
    return;
  }
  logger_->info("V2LVS completed successfully, output at '{}'.", v2lvs_output_spice_file);

  // 4. Post-process the generated SPICE file
  std::string target_global = ".global VSS GND COREGND1 VDD COREVDD1";

  // Call the modification function, reading from .v2lvs.spi and writing to .spi
  if (!this->modifySpiceNetlist(v2lvs_output_spice_file, final_output_spice_file, target_global)) {
    logger_->error("Failed to post-process the generated SPICE netlist: {}",
                   v2lvs_output_spice_file);
    // Keep both files for debugging if modification fails
    return;
  }

  // 5. Final success message
  logger_->info("Intermediate v2lvs output kept in '{}'", v2lvs_output_spice_file);
  logger_->info("Temporary Verilog input kept in '{}'", temp_verilog_file);
  logger_->info("SPICE generation and post-processing complete. Final output in '{}'",
                final_output_spice_file);
}

std::map<std::string, std::string> LibFile::logic(const std::string &cell_name) {
  std::map<std::string, std::string> logic_map = {};

  logger_->info("Getting output pin logic function for '{}'", filename_);

  if (!std::filesystem::exists(jsonname_)) {
    logger_->info("JSON file not found. Parsing Liberty file first.");
    this->parse();
    this->writeJsonToFile();
  } else {
```

```
    // Read the JSON file into json object
    std::ifstream in(jsonname_);
    if (!in.is_open()) {
      logger_->error("Could not open file '{}' for reading", jsonname_);
      return logic_map;
    }
    try {
      lib_json_ = json::parse(in);
    } catch (const json::parse_error &e) {
      logger_->error("JSON parsing error in file '{}': {}", jsonname_, e.what());
      in.close();
      return logic_map;
    }
    in.close();
  }

  // Check if the cell name exists in the JSON object
  for (const auto &cell : lib_json_["cells"]) {
    if (cell["cell_name"].get<std::string>() == cell_name) {
      logger_->debug("Found cell: '{}'", cell_name);

      // Tranverse all output pins
      if (cell.contains("output_pins")) {
        for (const auto &pin : cell["output_pins"]) {
          std::string pin_name = pin["pin_name"].get<std::string>();
          // Check if the pin has a "timing_arcs" attribute
          if (pin.contains("function")) {
            std::string function = pin["function"].get<std::string>();
            logic_map[pin_name] = function;
            logger_->debug("Pin: '{}' Logic Function: '{}'", pin_name, function);
          } else {
            logger_->warn("Pin: '{}' does not have a 'function' attribute", pin_name);
          }
        }
      }
    }
  }
  if (logic_map.empty()) {
    logger_->warn("No logic functions found for cell: '{}'", cell_name);
  } else {
    logger_->info("Found {} Logic functions for cell: '{}'", logic_map.size(), cell_name);
    // Print the logic functions
    for (const auto &pair : logic_map) {
      logger_->info("Pin -> Logic Function: {} -> {}", pair.first, pair.second);
    }
  }
  return logic_map;
}
```

6.32 src/LibFileOperations.cpp File Reference

`#include "LibFileOperations.hpp"`

Include dependency graph for LibFileOperations.cpp:



Functions

- void printInfo ()
  Sets up and configures the global logger and prints application information.
- void parseLibFile (const std::string &library_path, const std::string log_file_name)
  Parses a library file and generates corresponding output.
- void monoCheckLibFile (const std::string &library_path, const std::string log_file_name, bool is_slew)
  Performs monotonicity check on a library file.
- void supercellLibFile (const std::string &library_path, const std::string &log_file_name, int chain_length, const std::vector< std::string > &cell_names)
  Creates supercell map structures from a Liberty library file.
- void verilogLibFile (const std::string &library_path, const std::string &log_file_name, int chain_length, const std::vector< std::string > &cell_names)
  Generates Verilog files from a library file for specified cells.
- void spiceLibFile (const std::string &library_path, const std::string &log_file_name, int chain_length, const std::vector< std::string > &cell_names, const std::string &verilog_lib_file, const std::string &spice_lib_file)
  Generates a SPICE library file from a given library file, applying a specified chain length and cell names.
- void compareLibFiles (const std::string &ref_lib, const std::string &comp_lib, const double reltol, const double abstol, std::string &report_file_name)
  Compares two library files and generates a detailed comparison report.
- void funcLibFile (const std::string &ref_file, const std::string &comp_file, const std::vector< std::string > &cell_names, std::string &report_file_name)
  Performs a functional equivalence check between two files (Liberty or Verilog) for a given set of cells.

6.32.1 Function Documentation

6.32.1.1 compareLibFiles()

```
void compareLibFiles (
        const std::string & ref_lib,
        const std::string & comp_lib,
        const double reltol,
        const double abstol,
        std::string & report_file_name )
```

Compares two library files and generates a detailed comparison report.

This function compares a reference library file with another library file, performing validation checks based on specified tolerance parameters. The comparison results are written to a report file in markdown or text format.

Parameters

ref_lib	Path to the reference library file to use as the baseline
comp_lib	Path to the library file to compare against the reference
reltol	Relative tolerance for numerical comparisons (must be >= 0.0)
abstol	Absolute tolerance for numerical comparisons
report_file_name	[in,out] Name of the file to write the comparison report to. If empty, defaults to [comp_lib_basename].cmp.md. If provided but doesn't end with .txt or .md, .md will be appended.

Note

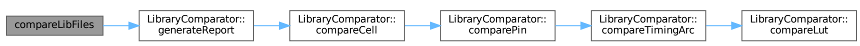
The function will log an error and return without comparing if reltol is invalid.

Log files will be created with the naming pattern [library_basename].cmp.log

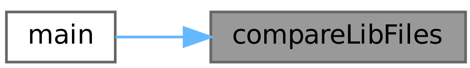
The function uses the LibraryComparator class to perform the actual comparison.

Definition at line 246 of file LibFileOperations.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



6.32.1.2 funcLibFile()

```
void funcLibFile (
        const std::string & ref_file,
        const std::string & comp_file,
```

```
const std::vector< std::string > & cell_names,
std::string & report_file_name )
```

Performs a functional equivalence check between two files (Liberty or Verilog) for a given set of cells.

This function compares the logic functions of specified cells in two files, which can be either in Liberty (.lib) or Verilog (.v) format. It extracts the logic functions for each cell from both files, compares them, and generates a report summarizing the comparison results.

Parameters

ref_file	The path to the reference file (Liberty or Verilog).
comp_file	The path to the comparison file (Liberty or Verilog).
cell_names	A vector of cell names to be checked for functional equivalence.
report_file_name	A string to store the name of the report file. If empty, a default name is generated. If the provided name does not end with ".txt" or ".md", ".md" is appended.

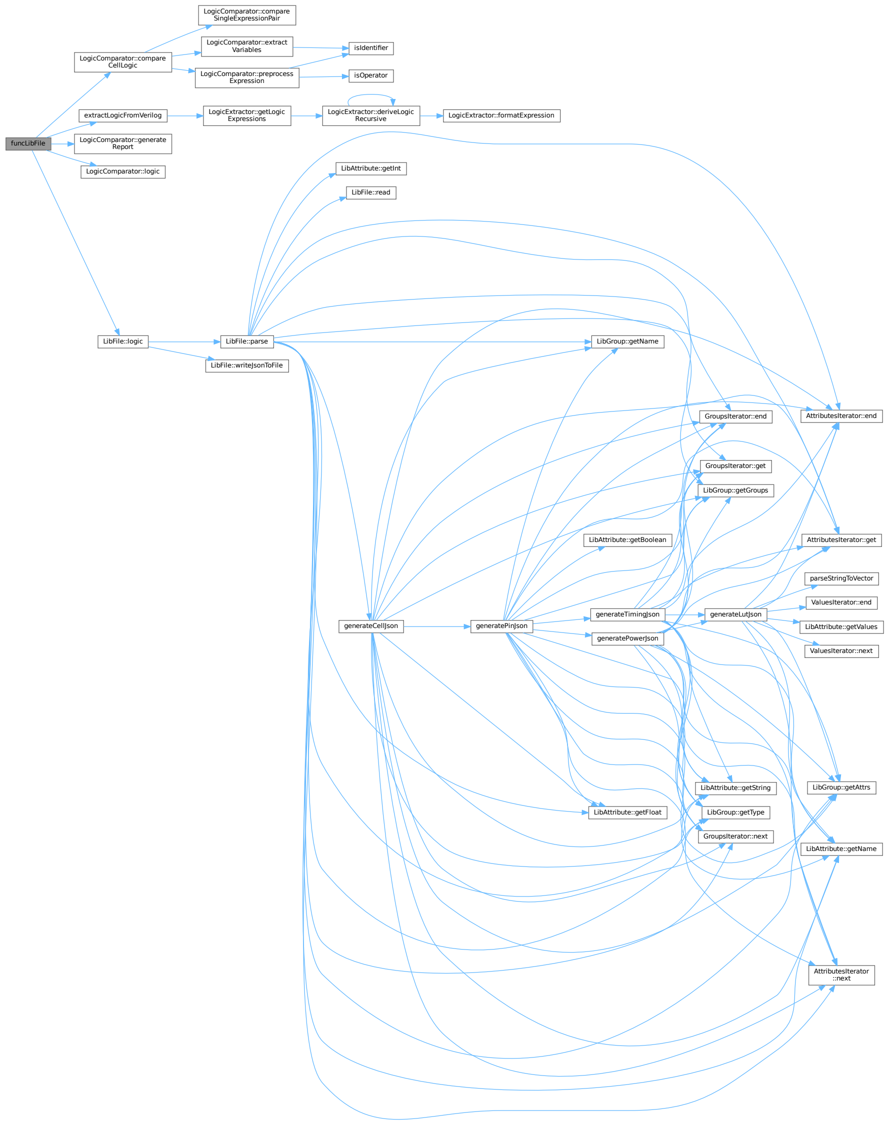
The function first checks the file extensions to determine the file format. It then extracts the logic functions for each specified cell from both files. The logic functions are then compared, and a report is generated, which includes the comparison results for each cell. The report is written to the specified report file.

Note

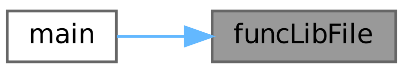
- If no cell names are provided, the function logs an error and returns.
- If the reference or comparison file format is not supported (i.e., not .lib or .v), the function logs an error and returns.
- The report file is cleared before writing the comparison results.
- The function uses spdlog for logging information, warnings, and errors.
- The function utilizes the LogicComparator class to perform the logic comparison and generate the report.
- Memory allocated for LibFile objects is managed using raw pointers and must be manually deallocated to prevent memory leaks.

Definition at line 299 of file LibFileOperations.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



6.32.1.3 monoCheckLibFile()

```
void monoCheckLibFile (
        const std::string & library_path,
        const std::string log_file_name,
        bool is_slew )
```

Performs monotonicity check on a library file.

This function validates the monotonicity of timing data in a library file. It creates a log file to record the results of the check and handles any exceptions that occur during the process.

Parameters

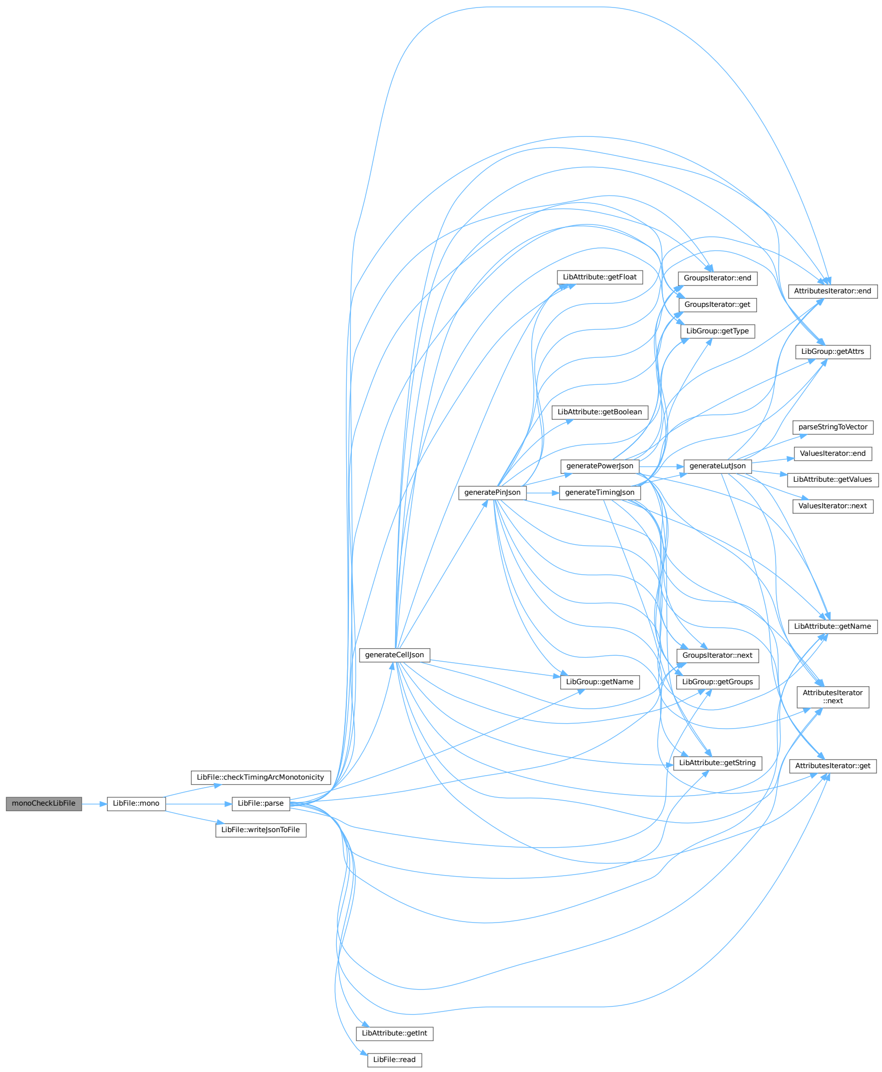
library_path	Path to the library file to check
log_file_name	Name of the log file to create (optional). If empty, a default name will be generated from the library file name
is_slew	Flag indicating whether to check input slew monotonicity (true) or output load monotonicity (false)

Exceptions

The	function catches and logs any exceptions but does not rethrow them

Definition at line 81 of file LibFileOperations.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:

6.32.1.4 parseLibFile()

```
void parseLibFile (
                const std::string & library_path,
                const std::string log_file_name )
```

Parses a library file and generates corresponding output.

This function processes the given library file, parsing its contents and generating a JSON output. It also logs the parsing process to a specified log file or creates a default log file if none is provided.

Parameters

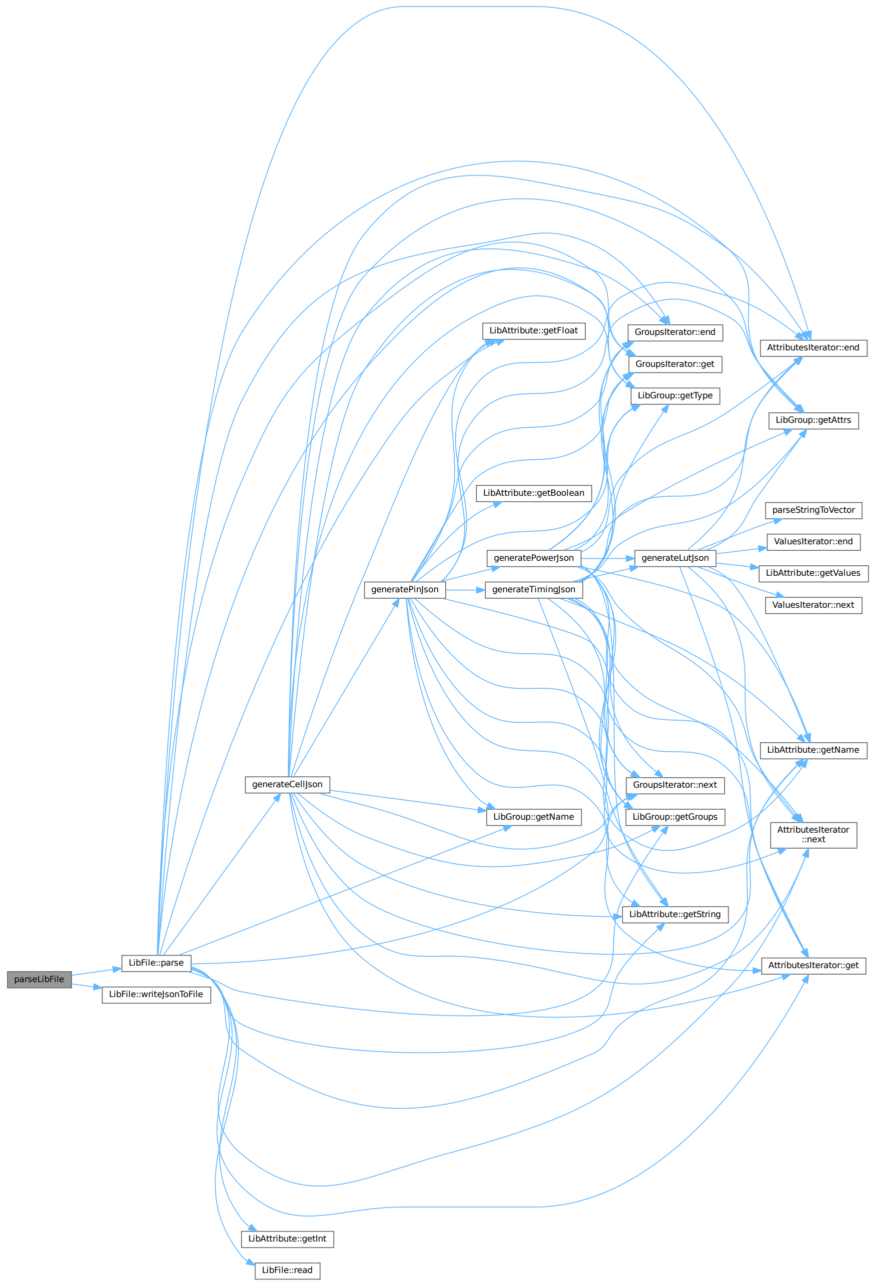
library_path	Path to the library file that needs to be parsed
log_file_name	Optional name for the log file. If empty, a default name is generated based on the library filename with ".parse.log" extension

Exceptions

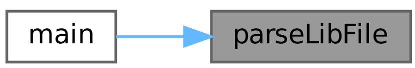
The	function catches and logs any exceptions but doesn't propagate them

Definition at line 49 of file LibFileOperations.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



6.32.1.5 printInfo()

```
void printInfo ( )
```

Sets up and configures the global logger and prints application information.

This function performs the following operations:

1. Creates a console sink for logging with level set to INFO
2. Creates a file sink for logging with level set to DEBUG, saving to [APP_NAME].log
3. Configures a logger with both sinks and sets it as the default logger
4. Outputs basic application information:
    - Version and build timestamp
    - Author information
    - Log file location

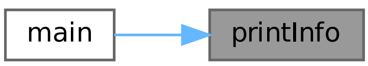
Note

Uses spdlog library for logging functionality

Depends on APP_NAME, APP_VERSION, BUILD_TIMESTAMP, APP_AUTHOR, and APP↩
_CONTACT macros

Definition at line 18 of file LibFileOperations.cpp.

Here is the caller graph for this function:



6.32.1.6 spiceLibFile()

```
void spiceLibFile (
        const std::string & library_path,
        const std::string & log_file_name,
        int chain_length,
        const std::vector< std::string > & cell_names,
        const std::string & verilog_lib_file,
        const std::string & spice_lib_file )
```

Generates a SPICE library file from a given library file, applying a specified chain length and cell names.

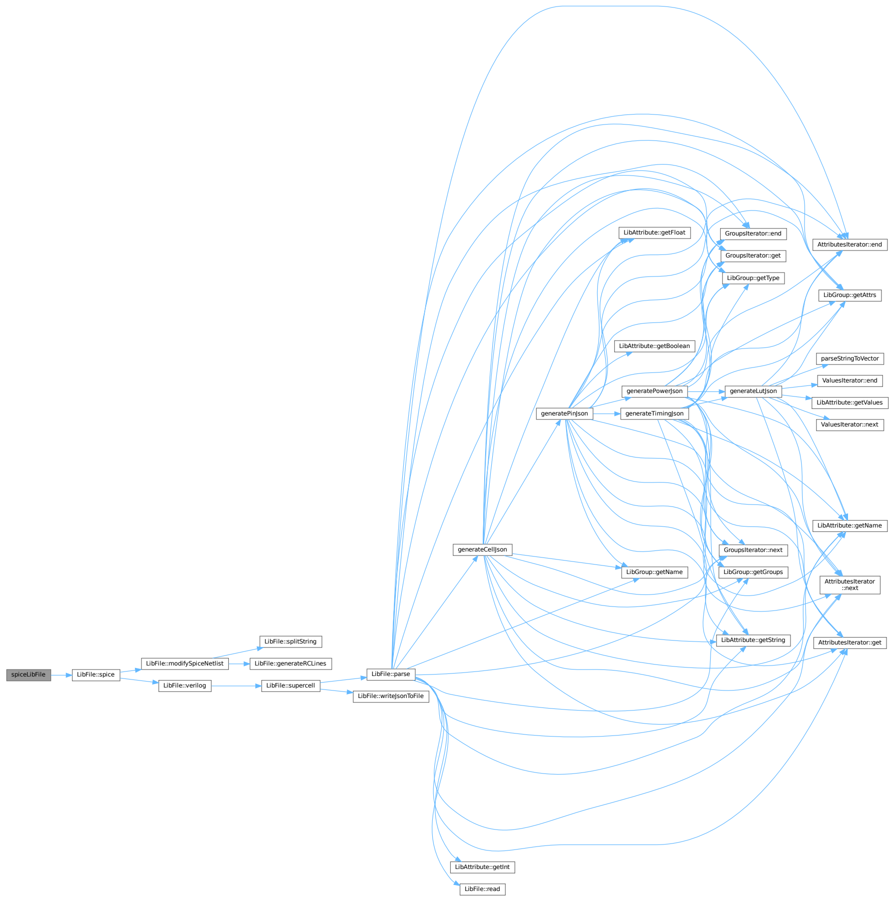
This function takes a library file path, a log file name, a chain length, a vector of cell names, a Verilog library file path, and a SPICE library file path as input. It initializes a LibFile object, validates the chain length, and then calls the spice method of the LibFile object to generate the SPICE library file. It logs the start and end of the SPICE generation process, as well as any errors that occur.

Parameters

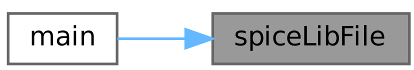
library_path	The path to the input library file.
log_file_name	The name of the log file. If empty, a default log file name is generated based on the library file name.
chain_length	The chain length to use during SPICE generation. Must be >= 1.
cell_names	A vector of cell names to include in the SPICE generation.
verilog_lib_file	The path to the Verilog library file.
spice_lib_file	The path to the output SPICE library file.

Definition at line 199 of file LibFileOperations.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



6.32.1.7 supercellLibFile()

```
void supercellLibFile (
        const std::string & library_path,
        const std::string & log_file_name,
```

```
int chain_length,
const std::vector< std::string > & cell_names )
```

Creates supercell map structures from a Liberty library file.

This function reads a Liberty file, creates supercell structures based on the specified chain length and cell names, and logs the process to a file.

Parameters

library_path	Path to the Liberty file to process
log_file_name	Name of the log file (if empty, defaults to "[library_name].supercell.log")
chain_length	The length of chains to create (must be >= 1)
cell_names	Vector of cell names to process for supercell generation

Exceptions

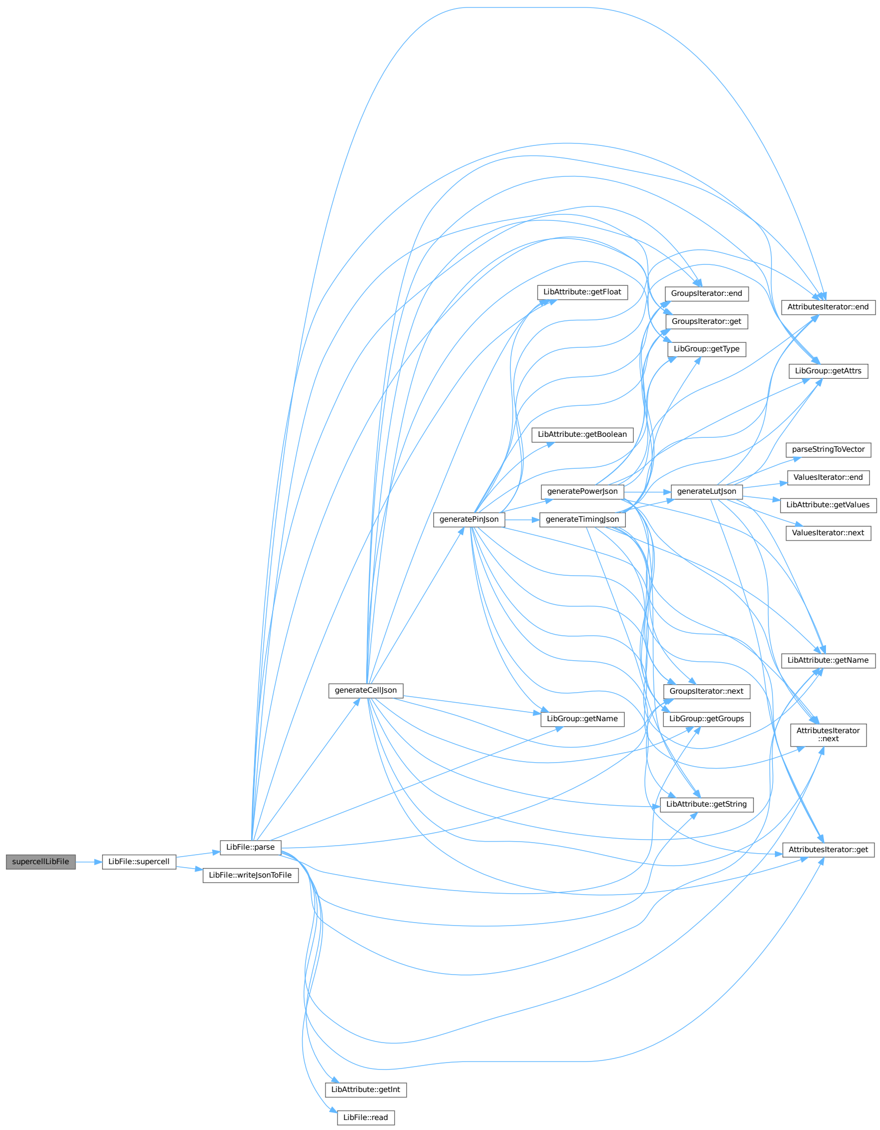
May	pass through exceptions from the LibFile::supercell method

Note

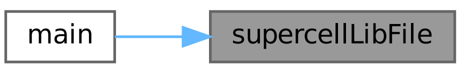
The function validates the chain length and logs all activities including errors that might occur during processing

Definition at line 116 of file LibFileOperations.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



6.32.1.8 verilogLibFile()

```
void verilogLibFile (
        const std::string & library_path,
        const std::string & log_file_name,
        int chain_length,
        const std::vector< std::string > & cell_names )
```

Generates Verilog files from a library file for specified cells.

This function processes a library file and generates Verilog representation for the specified cell names with a given chain length. The operation results are logged to a specified or default log file.

Parameters

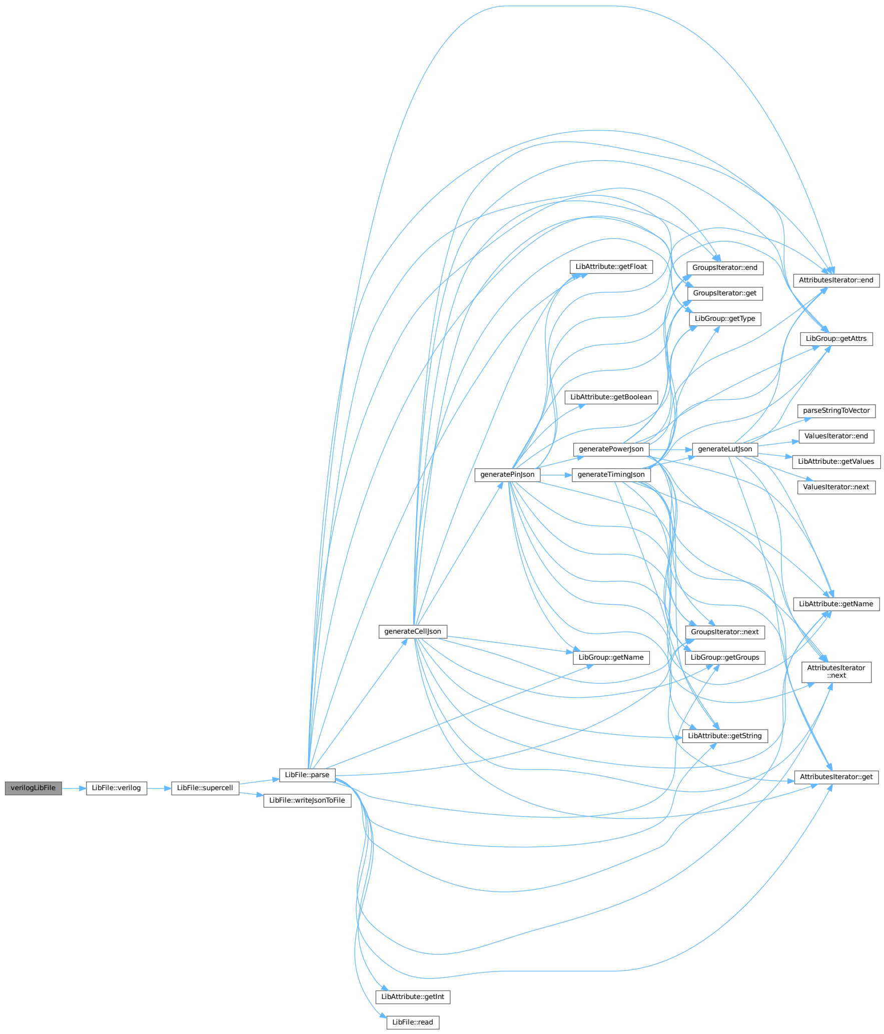
library_path	Path to the library file to process
log_file_name	Name for the log file (if empty, a default name will be generated)
chain_length	Number of cells to chain together, must be >= 1
cell_names	Vector of cell names to generate Verilog for

Exceptions

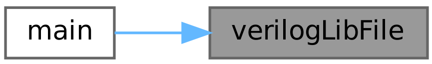
Catches	any exceptions from the verilog generation process and logs them

Definition at line 157 of file LibFileOperations.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



6.33 LibFileOperations.cpp

Go to the documentation of this file.

```
#include "LibFileOperations.hpp"

void printInfo() {
  // Set Global Logger
  auto console_sink = std::make_shared<spdlog::sinks::stdout_color_sink_mt>();
  console_sink->set_level(spdlog::level::info);
  std::string app_name = APP_NAME;
  auto file_sink = std::make_shared<spdlog::sinks::basic_file_sink_mt>(app_name + ".log", true);
  file_sink->set_level(spdlog::level::trace);

  std::vector<spdlog::sink_ptr> sinks{console_sink, file_sink};
  auto logger = std::make_shared<spdlog::logger>(APP_NAME, sinks.begin(), sinks.end());
  logger->set_level(spdlog::level::trace);
  spdlog::set_default_logger(logger);

  spdlog::info("Version {}, Built: {}", APP_VERSION, BUILD_TIMESTAMP);
  spdlog::info("Author: {}, Email: {}", APP_AUTHOR, APP_CONTACT);
  spdlog::info("Global log file in: '{}'", app_name + ".log");
}

void parseLibFile(const std::string &library_path, const std::string log_file_name) {
  std::string logname = log_file_name.empty()
                            ? std::filesystem::path(library_path).stem().string() + ".parse.log"
                            : log_file_name;
  LibFile libfile(library_path, logname);

  libfile.logger_->info("Starting parse for file: '{}' ...", library_path);

  try {
    libfile.parse();
    libfile.writeJsonToFile();
    libfile.logger_->info("Successfully parsed file: '{}'", library_path);
  } catch (const std::exception &e) {
    libfile.logger_->error("Error parsing file '{}': {}", library_path, e.what());
  }
}

void monoCheckLibFile(const std::string &library_path, const std::string log_file_name,
                      bool is_slew) {
  std::string logname = log_file_name.empty()
                            ? std::filesystem::path(library_path).stem().string() + ".mono.log"
                            : log_file_name;
  LibFile libfile(library_path, logname);

  libfile.logger_->info("Starting monotonicity check for file: '{}', input slew: {}", library_path,
                        is_slew);

  try {
    libfile.mono(is_slew);
    libfile.logger_->info("Successfully completed monotonicity check for file: '{}'", library_path);
  } catch (const std::exception &e) {
    libfile.logger_->error("Error during monotonicity check for file '{}': {}", library_path,
                           e.what());
  }
}

void supercellLibFile(const std::string &library_path, const std::string &log_file_name,
                      int chain_length, const std::vector<std::string> &cell_names) {
  std::string logname = log_file_name.empty()
                            ? std::filesystem::path(library_path).stem().string() + ".supercell.log"
                            : log_file_name;
  LibFile libfile(library_path, logname);

  // Check chain length validity
  if (chain_length < 1) {
    libfile.logger_->error("Invalid chain length: {}. Chain length must be >= 1.", chain_length);
    return;
  }
  std::stringstream ss;
```

```
  for (const auto &cell : cell_names) {
    ss << cell << ", ";
  }
  libfile.logger_->info("Starting supercell generation for file: '{}', chain length: {}, cells: {}",
                        library_path, chain_length, ss.str());
  try {
    libfile.supercell(chain_length, cell_names);
    libfile.logger_->info("Successfully generated supercells for file: '{}'", library_path);
  } catch (const std::exception &e) {
    libfile.logger_->error("Error during supercell generation for file '{}': {}", library_path,
                           e.what());
  }
}

void verilogLibFile(const std::string &library_path, const std::string &log_file_name,
                    int chain_length, const std::vector<std::string> &cell_names) {
  std::string logname = log_file_name.empty()
                            ? std::filesystem::path(library_path).stem().string() + ".verilog.log"
                            : log_file_name;
  LibFile libfile(library_path, logname);

  // Check chain length validity
  if (chain_length < 1) {
    libfile.logger_->error("Invalid chain length: {}. Chain length must be >= 1.", chain_length);
    return;
  }
  std::stringstream ss;
  for (const auto &cell : cell_names) {
    ss << cell << ", ";
  }
  libfile.logger_->info("Starting Verilog generation for file: '{}', chain length: {}, cells: {}",
                        library_path, chain_length, ss.str());
  try {
    libfile.verilog(chain_length, cell_names);
    libfile.logger_->info("Successfully generated Verilog for file: '{}'", library_path);
  } catch (const std::exception &e) {
    libfile.logger_->error("Error during Verilog generation for file '{}': {}", library_path,
                           e.what());
  }
}

void spiceLibFile(const std::string &library_path, const std::string &log_file_name,
                  int chain_length, const std::vector<std::string> &cell_names,
                  const std::string &verilog_lib_file, const std::string &spice_lib_file) {
  std::string logname = log_file_name.empty()
                            ? std::filesystem::path(library_path).stem().string() + ".spice.log"
                            : log_file_name;
  LibFile libfile(library_path, logname);

  // Check chain length validity
  if (chain_length < 1) {
    libfile.logger_->error("Invalid chain length: {}. Chain length must be >= 1.", chain_length);
    return;
  }
  std::stringstream ss;
  for (const auto &cell : cell_names) {
    ss << cell << ", ";
  }
  libfile.logger_->info("Starting SPICE generation for file: '{}', chain length: {}, cells: {}",
                        library_path, chain_length, ss.str());
  try {
    libfile.spice(chain_length, cell_names, verilog_lib_file, spice_lib_file);
    libfile.logger_->info("Successfully generated SPICE for file: '{}'", library_path);
  } catch (const std::exception &e) {
    libfile.logger_->error("Error during SPICE generation for file '{}': {}", library_path,
                           e.what());
  }
}

```

```
void compareLibFiles(const std::string &ref_lib, const std::string &comp_lib, const double reltol,
                     const double abstol, std::string &report_file_name) {
  std::string ref_logname = std::filesystem::path(ref_lib).stem().string() + ".cmp.log";
  std::string comp_logname = std::filesystem::path(comp_lib).stem().string() + ".cmp.log";
  LibFile ref_libfile(ref_lib, ref_logname), comp_libfile(comp_lib, comp_logname);

  // Check relative tolerance validity
  if (reltol < 0.0) {
    spdlog::error("Invalid relative tolerance: {}. Relative tolerance must be >= 0.0.", reltol);
    return;
  }

  // Check the report file name
  if (report_file_name.empty()) {
    report_file_name = comp_libfile.basename_ + ".cmp.md";
  } else if (std::filesystem::path(report_file_name).extension() != ".txt" &&
             std::filesystem::path(report_file_name).extension() != ".md") {
    // report file name not end in .txt or .md, warn user and append .md
    report_file_name = report_file_name + ".md";
    spdlog::warn("Report file name does not end in .txt or .md. Report will be written to '{}'",
                 report_file_name);
  }

  LibraryComparator comparator(ref_libfile, comp_libfile, reltol, abstol);
  comparator.generateReport(report_file_name);
  spdlog::info("Comparison completed. Report written to: '{}'", report_file_name);
}

void funcLibFile(const std::string &ref_file, const std::string &comp_file,
                 const std::vector<std::string> &cell_names, std::string &report_file_name) {
  // Check if the files are Liberty or Verilog
  std::string ref_ext = std::filesystem::path(ref_file).extension();
  std::string comp_ext = std::filesystem::path(comp_file).extension();
  std::string ref_logname = std::filesystem::path(ref_file).stem().string() + ".cmp.log";
  std::string comp_logname = std::filesystem::path(comp_file).stem().string() + ".cmp.log";

  // Pre cell names check
  if (cell_names.empty()) {
    spdlog::error("No cell names provided for functional equivalence check.");
    return;
  }

  // Check the report file name
  if (report_file_name.empty()) {
    report_file_name = std::filesystem::path(comp_file).stem().string() + ".logic.cmp.md";
  } else if (std::filesystem::path(report_file_name).extension() != ".txt" &&
             std::filesystem::path(report_file_name).extension() != ".md") {
    // report file name not end in .txt or .md, warn user and append .md
    report_file_name = report_file_name + ".md";
    spdlog::warn("Report file name does not end in .txt or .md. Report will be written to '{}'",
                 report_file_name);
  }

  // Clear the report file
  std::ofstream report_file(report_file_name);
  if (!report_file.is_open()) {
    spdlog::error("Failed to open report file: '{}'", report_file_name);
    return;
  }
  report_file.close();
  spdlog::info("Report file cleared: '{}'", report_file_name);

  // Declare LibFile objects as pointers, initialized to nullptr
  LibFile *ref_libfile = nullptr;
  LibFile *comp_libfile = nullptr;

  // Check the reference file extension
  if (ref_ext == ".v") {
    spdlog::info("Reference file in Verilog format.");
```

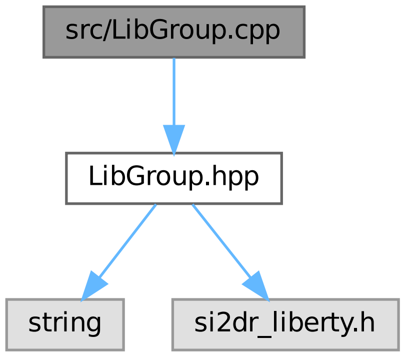
```
} else if (ref_ext == ".lib") {
  spdlog::info("Reference file in Liberty format.");
  ref_libfile = new LibFile(ref_file, ref_logname);
} else {
  spdlog::error("Unsupported reference file format: '{}'", ref_ext);
  return;
}
// Check the comparison file extension
if (comp_ext == ".v") {
  spdlog::info("Comparison file in Verilog format.");
} else if (comp_ext == ".lib") {
  spdlog::info("Comparison file in Liberty format.");
  comp_libfile = new LibFile(comp_file, comp_logname);
} else {
  spdlog::error("Unsupported comparison file format: '{}'", comp_ext);
  return;
}

// Tranverse the cell names and check functional equivalence
for (const auto &cell : cell_names) {
  spdlog::info("Checking functional equivalence for cell: '{}'", cell);
  std::map<std::string, std::string> ref_outpin_map;
  std::map<std::string, std::string> comp_outpin_map;

  if (ref_ext == ".v") {
    ref_outpin_map = extractLogicFromVerilog(ref_file, cell);
  } else if (ref_ext == ".lib") {
    ref_outpin_map = ref_libfile->logic(cell);
  }

  if (comp_ext == ".v") {
    // getAST(comp_file, cell);
    // extractAndPrintNetlistInfo(comp_file, cell);
    comp_outpin_map = extractLogicFromVerilog(comp_file, cell);
  } else if (comp_ext == ".lib") {
    comp_outpin_map = comp_libfile->logic(cell);
  }

  spdlog::info("Logic function expressions collected for cell: '{}'", cell);
  spdlog::info("Starting logic comparison ...");

  LogicComparator comparator(ref_outpin_map, comp_outpin_map, cell);
  // Easter egg
  if (cell == "easteregg") {
    spdlog::info("Easter egg  found!  Running ExprTk Eample 07...");
    comparator.logic();
    return;
  }

  // // Test for preprocessing
  // std::string ref_expr;
  // std::string comp_expr;
  // for (const auto &pin : ref_outpin_map) {
  //   spdlog::info("Pin -> Expression: {} -> {}", pin.first, pin.second);
  //   ref_expr = comparator.preprocessExpression(pin.second);
  // }
  // for (const auto &pin : comp_outpin_map) {
  //   spdlog::info("Pin => Expression: {} => {}", pin.first, pin.second);
  //   comp_expr = comparator.preprocessExpression(pin.second);
  // }
  // std::vector<std::string> sorted_vars;
  // comparator.extractVariables(ref_expr, comp_expr, sorted_vars);
  // struct PinComparisonResult pin_comparison_result;
  // // comp_expr = "not ((not(A1) and not(A2)) or B)";
  // comparator.compareSingleExpressionPair(ref_expr, comp_expr, sorted_vars,
  //                                                pin_comparison_result);

  // comparator.compareCellLogic();
  comparator.compareCellLogic();
```

```
    comparator.generateReport(report_file_name);

    spdlog::info("Functional equivalence check completed for cell: '{}'", cell);
  }
  spdlog::info("Functional equivalence check completed for all cells.");
  spdlog::info("Report written to: '{}'", report_file_name);
}
```

6.34 src/LibGroup.cpp File Reference

`#include "LibGroup.hpp"`

Include dependency graph for LibGroup.cpp:



6.35 LibGroup.cpp

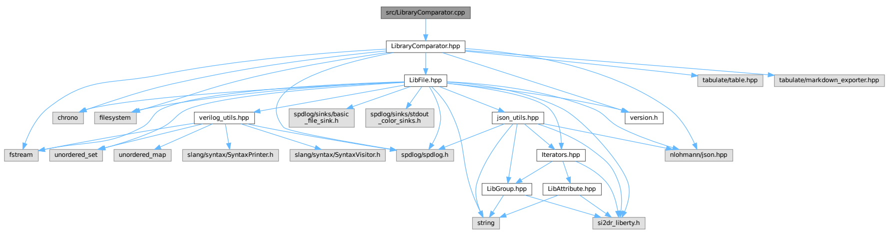
Go to the documentation of this file.

```
#include "LibGroup.hpp"

LibGroup::LibGroup(si2drGroupIdT group, si2drErrorT &err) : group_(group), err_(err) {}

LibGroup::~LibGroup() {}

std::string LibGroup::getName() {
  si2drNamesIdT names = si2drGroupGetNames(group_, &err_);
  si2drStringT name = si2drIterNextName(names, &err_);
  si2drIterQuit(names, &err_);
  return name ? std::string(name) : std::string();
}

std::string LibGroup::getType() {
  si2drStringT type = si2drGroupGetGroupType(group_, &err_);
  return type ? std::string(type) : std::string();
}

si2drAttrsIdT LibGroup::getAttrs() { return si2drGroupGetAttrs(group_, &err_); }

si2drGroupsIdT LibGroup::getGroups() { return si2drGroupGetGroups(group_, &err_); }
```

6.36 src/LibraryComparator.cpp File Reference

```
#include "LibraryComparator.hpp"
```

Include dependency graph for LibraryComparator.cpp:



6.37 LibraryComparator.cpp

Go to the documentation of this file.

```
#include "LibraryComparator.hpp"

LibraryComparator::LibraryComparator(LibFile &ref_libfile, LibFile &comp_libfile, double reltol,
                                     double abstol)
    : reltol_(reltol), abstol_(abstol) {
  std::string ref_json_name = ref_libfile.basename_ + ".json";
  if (!std::filesystem::exists(ref_json_name)) {
    spdlog::info("Reference JSON file {} not found. Parsing first.", ref_json_name);
    ref_libfile.parse();
    ref_libfile.writeJsonToFile();
    ref_json_ = ref_libfile.lib_json_;
  } else {
    // Read the JSON file into json object
    std::ifstream ref_in(ref_json_name);
    if (!ref_in.is_open()) {
      spdlog::error("Could not open file '{}' for reading", ref_json_name);
      return;
    }
    try {
      ref_json_ = json::parse(ref_in);
    } catch (const json::parse_error &e) {
      spdlog::error("Error parsing reference JSON file '{}': {}", ref_json_name, e.what());
      return;
    }
  }
  std::string comp_json_name = comp_libfile.basename_ + ".json";
  if (!std::filesystem::exists(comp_json_name)) {
    spdlog::info("Comparison JSON file {} not found. Parsing first.", comp_json_name);
    comp_libfile.parse();
    comp_libfile.writeJsonToFile();
    comp_json_ = comp_libfile.lib_json_;
  } else {
    // Read the JSON file into json object
    std::ifstream comp_in(comp_json_name);
    if (!comp_in.is_open()) {
      spdlog::error("Could not open file '{}' for reading", comp_json_name);
      return;
    }
    try {
      comp_json_ = json::parse(comp_in);
    } catch (const json::parse_error &e) {
```

```
      spdlog::error("Error parsing comparison JSON file '{}': {}", comp_json_name, e.what());
      return;
    }
  }

  ref_lib_path_ = ref_libfile.filepath_;
  comp_lib_path_ = comp_libfile.filepath_;
  spdlog::info("Successfully loaded JSON files for comparison");
}

void LibraryComparator::compareLut(const std::string &cell_name, const std::string &pin_name,
                                   const std::string &timing_type, const std::string &related_pin,
                                   const std::string &arc_name, const json &ref_lut,
                                   const json &comp_lut, Table &table) {
  spdlog::info("Comparing LUT of '{}' ...", arc_name);
  std::vector<double> ref_index_1 = ref_lut["index_1"].get<std::vector<double>>();
  std::vector<double> ref_index_2 = ref_lut["index_2"].get<std::vector<double>>();
  std::vector<double> comp_index_1 = comp_lut["index_1"].get<std::vector<double>>();
  std::vector<double> comp_index_2 = comp_lut["index_2"].get<std::vector<double>>();
  if (ref_index_1 != comp_index_1 || ref_index_2 != comp_index_2) {
    spdlog::warn("Mismatch in LUT indices for '{}' in cell: '{}', pin: '{}', related_pin: '{}', "
                 "timing_type: '{}'",
                 arc_name, cell_name, pin_name, related_pin, timing_type);
    return;
  } else {
    spdlog::debug("LUT indices match for '{}'", arc_name);

    std::vector<std::vector<double>> comp_value_matrix;
    // Generate a matrix of values from the JSON array for comparison library
    for (const auto &row_val : comp_lut["values"]) {
      std::vector<double> row_data;
      // Check if the row value is an array
      if (!row_val.is_array()) {
        spdlog::error(
            "Invalid LUT format: '{}' values should be an array of arrays in comp_lib '{}', "
            "cell '{}', pin '{}', "
            "related_pin '{}', timing_type '{}'",
            arc_name, this->comp_lib_path_.string(), cell_name, pin_name, related_pin, timing_type);
        return;
      }
      // Check if non-numeric values
      for (const auto &val : row_val) {
        if (val.is_number()) {
          row_data.push_back(val.get<double>());
        } else {
          spdlog::error("Non-numeric value found in '{}'.values, skipping value: {} in comp_lib "
                        "'{}', cell '{}', "
                        "pin '{}', related_pin '{}', timing_type '{}'",
                        arc_name, val.dump(), this->comp_lib_path_.string(), cell_name, pin_name,
                        related_pin, timing_type);
          return;
        }
      }
      comp_value_matrix.push_back(row_data);
    }
    // Generate a matrix of values from the JSON array for reference library
    std::vector<std::vector<double>> ref_value_matrix;
    for (const auto &row_val : ref_lut["values"]) {
      std::vector<double> row_data;
      // Check if the row value is an array
      if (!row_val.is_array()) {
        spdlog::error(
            "Invalid LUT format: '{}' values should be an array of arrays in ref_lib '{}', "
            "cell '{}', pin '{}', "
            "related_pin '{}', timing_type '{}'",
            arc_name, this->ref_lib_path_.string(), cell_name, pin_name, related_pin, timing_type);
        return;
      }
      // Check if non-numeric values
```

```
      for (const auto &val : row_val) {
        if (val.is_number()) {
          row_data.push_back(val.get<double>());
        } else {
          spdlog::error("Non-numeric value found in '{}'.values, skipping value: {} in ref_lib "
                        "'{}', cell '{}', "
                        "pin '{}', related_pin '{}', timing_type '{}'",
                        arc_name, val.dump(), this->ref_lib_path_.string(), cell_name, pin_name,
                        related_pin, timing_type);
          return;
        }
      }
      ref_value_matrix.push_back(row_data);
    }
    // Compare the matrix for relative tolerance
    if (!comp_value_matrix.empty() && !comp_value_matrix[0].empty()) {
      for (size_t i = 0; i < comp_value_matrix.size(); ++i) {
        for (size_t j = 0; j < comp_value_matrix[i].size(); ++j) {
          double ref_val = ref_value_matrix[i][j];
          double comp_val = comp_value_matrix[i][j];
          if (std::abs(ref_val - comp_val) > reltol_ * std::abs(ref_val) &&
              std::abs(ref_val - comp_val) > abstol_) {
            spdlog::debug("LUT value mismatch for '{}', index_1: {}, index_2: {}", arc_name,
                          ref_index_1[i], ref_index_2[j]);
            table.add_row({related_pin + "->" + pin_name, std::to_string(ref_val),
                           std::to_string(comp_val), std::to_string(comp_val - ref_val),
                           std::to_string((comp_val - ref_val) / ref_val * 100), timing_type,
                           arc_name, std::to_string(i + 1), std::to_string(ref_index_1[i]),
                           std::to_string(j + 1), std::to_string(ref_index_2[j]), "<"});
            spdlog::debug("table size: {}", table.size());
          } else {
            spdlog::debug("LUT value match for '{}', index_1: {}, index_2: {}", arc_name,
                          ref_index_1[i], ref_index_2[j]);
          }
        }
      }
    } else {
      spdlog::error(
          "Empty or invalid 'values' array found for cell: '{}', pin: '{}', related_pin: '{}', "
          "timing_type: '{}', arc: '{}'",
          cell_name, pin_name, related_pin, timing_type, arc_name);
    }
  }
}

void LibraryComparator::compareTimingArc(const std::string &cell_name, const std::string &pin_name,
                                         const std::string &timing_type, const json &ref_timing_arc,
                                         const json &comp_timing_arc, Table &table) {
  spdlog::info("Comparing timing type '{}' ...", timing_type);

  std::string related_pin = ref_timing_arc["related_pin"].get<std::string>();
  spdlog::debug("Related pin: '{}'", related_pin);

  std::vector<std::string> arc_names = {"cell_rise", "cell_fall", "rise_transition",
                                        "fall_transition"};
  for (auto arc_name : arc_names) {
    if (comp_timing_arc.contains(arc_name)) {
      spdlog::debug("Comparing timing arc: '{}'", arc_name);
      // Look for the arc in the reference JSON
      if (ref_timing_arc.contains(arc_name)) {
        // Found this arc
        spdlog::debug("Found timing arc: '{}'", arc_name);
        compareLut(cell_name, pin_name, timing_type, related_pin, arc_name,
                   ref_timing_arc[arc_name], comp_timing_arc[arc_name], table);
      } else {
        // arc not found in reference JSON
        spdlog::warn("Timing arc: '{}' not found in reference JSON", arc_name);
      }
    }
```

```cpp
  }
}

void LibraryComparator::comparePin(const std::string &cell_name, const std::string &pin_name,
                                   const json &ref_pin, const json &comp_pin, Table &table) {
  spdlog::info("Comparing pin '{}' ...", pin_name);

  if (comp_pin.contains("timing_arcs")) {
    for (const auto &comp_arc : comp_pin["timing_arcs"]) {
      std::string timing_type = comp_arc["timing_type"].get<std::string>();
      spdlog::debug("Comparing timing type: '{}'", timing_type);
      // Look for the timing type in the reference JSON
      auto ref_arc_it = std::find_if(
          ref_pin["timing_arcs"].begin(), ref_pin["timing_arcs"].end(),
          [&timing_type](const json &ref_arc) { return ref_arc["timing_type"] == timing_type; });
      if (ref_arc_it != ref_pin["timing_arcs"].end()) {
        // Found this timing type
        spdlog::debug("Found timing type: '{}'", timing_type);
        compareTimingArc(cell_name, pin_name, timing_type, *ref_arc_it, comp_arc, table);
      } else {
        // timing arc not found in reference JSON
        spdlog::warn("Timing arc: '{}' not found in reference JSON", timing_type);
      }
    }
  } else {
    spdlog::info("No timing arcs found for pin: '{}'", comp_pin["pin_name"].get<std::string>());
  }
}

void LibraryComparator::compareCell(const std::string &cell_name, const json &ref_cell,
                                    const json &comp_cell, Table &table) {
  spdlog::info("Comparing cell '{}' ...", cell_name);

  if (comp_cell.contains("output_pins")) {
    for (const auto &comp_pin : comp_cell["output_pins"]) {
      std::string pin_name = comp_pin["pin_name"].get<std::string>();
      spdlog::debug("Comparing pin: '{}'", pin_name);
      // Look for the pin in the reference JSON
      auto ref_pin_it = std::find_if(
          ref_cell["output_pins"].begin(), ref_cell["output_pins"].end(),
          [&pin_name](const json &ref_pin) { return ref_pin["pin_name"] == pin_name; });
      if (ref_pin_it != ref_cell["output_pins"].end()) {
        // Found this pin
        spdlog::debug("Found pin: '{}'", pin_name);
        comparePin(cell_name, pin_name, *ref_pin_it, comp_pin, table);
      } else {
        // pin not found in reference JSON
        spdlog::warn("Pin: '{}' not found in reference JSON", pin_name);
      }
    }
  } else {
    spdlog::info("No output pins found for cell: '{}'", cell_name);
  }
}

void LibraryComparator::generateReport(const std::string &output_file) {
  std::ofstream outfile(output_file);
  outfile << "# LIBRARY comparison\n" << std::endl;
  outfile << "**Reference library: " << ref_lib_path_ << "**\n" << std::endl;
  outfile << "**Comparison library: " << comp_lib_path_ << "**\n" << std::endl;
  outfile << "**Absolute tolerance: " << abstol_ << "**\n" << std::endl;
  outfile << "**Relative tolerance: " << reltol_ << "**\n" << std::endl;
  outfile << "**Performed by " << APP_NAME << " v" << APP_VERSION << " from " << APP_AUTHOR;
  auto now = std::chrono::system_clock::now();
  std::time_t now_time_t = std::chrono::system_clock::to_time_t(now);
  std::tm *now_tm = std::localtime(&now_time_t);
  outfile << ". on: " << std::put_time(now_tm, "%c") << "**\n" << std::endl;
  outfile << "> Legend: < outlier, * scaled, ! indices switched, ^ slews extrapolated, ~ loads "
             "extrapolated,\n";
```

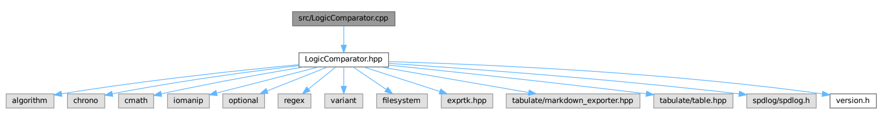
```
outfile << "> \n";
outfile << "> Legend: + padding added, /0 divide by zero, / slews interpolated, # loads "
           "interpolated\n";
outfile << "> \n";
outfile << "> Legend: << value is less but unknown, >> value is more but unknown\n\n";

spdlog::info("Starting comparison report generation ...");
for (const auto &comp_cell : comp_json_["cells"]) {
  std::string cell_name = comp_cell["cell_name"].get<std::string>();
  spdlog::debug("Comparing cell: '{}'", cell_name);
  // Look for the cell in the reference JSON
  auto ref_cell_it = std::find_if(
      ref_json_["cells"].begin(), ref_json_["cells"].end(),
      [&cell_name](const json &ref_cell) { return ref_cell["cell_name"] == cell_name; });
  if (ref_cell_it != ref_json_["cells"].end()) {
    // Found this cell
    spdlog::debug("Found cell: '{}'", cell_name);
    Table table;
    table.add_row({"Pin Name", "Reference", "Comparison", "Diff", "Diff %", "Type", "Arc Name",
                   "Row #", "Index_1", "Col #", "Index_2", "Note"});
    // Header formatting
    for (size_t i = 0; i < table[0].size(); ++i) {
      table[0][i].format().font_color(Color::yellow).font_style({FontStyle::bold});
    }
    compareCell(cell_name, *ref_cell_it, comp_cell, table);

    if (table.size() > 1) {
      // Data starts from row 1, row 0 is header
      size_t failed_count = table.size() - 1;
      double sum_diff = 0.0;
      double sum_diff_percent = 0.0;
      size_t outlier_count = 0;
      double max_diff = -1e9;
      double max_diff_percent = -1e9;
      std::vector<double> diff_values; // Store diff values for std deviation calculation

      // Index of columns
      const int diff_index = 3;
      const int diff_percent_index = 4;
      const int note_index = 11;

      for (size_t i = 1; i < table.size(); ++i) {
        try {
          double diff = std::stod(table[i][diff_index].get_text());
          double diff_percent = std::stod(table[i][diff_percent_index].get_text());

          sum_diff += diff;
          sum_diff_percent += diff_percent;
          diff_values.push_back(diff_percent);

          if (table[i][note_index].get_text() == "<") {
            outlier_count++;
          }
          max_diff = std::max(max_diff, diff);
          max_diff_percent = std::max(max_diff_percent, diff_percent);

        } catch (const std::invalid_argument &e) {
          spdlog::error("Could not convert value to double: {}", e.what());
          continue; // Skip this row if there's an error
        }
      }

      double avg_diff = sum_diff / failed_count;
      double avg_diff_percent = sum_diff_percent / failed_count;

      outfile << "## " << cell_name << "\n\n";
      outfile << "Delay Comparison in ns\n\n";
      if (output_file.substr(output_file.size() - 3) == ".md") {
        MarkdownExporter exporter;
```

```
          auto markdown = exporter.dump(table);
          outfile << markdown << std::endl;
        } else {
          outfile << table << std::endl;
        }
        outfile << std::endl;
        std::cout << table << std::endl;
        outfile << cell_name << " delay SUMMARY (abstol: " << abstol_ << ", reltol: " << reltol_
                << ")\n";

        // Create summary table
        Table summary_table;
        summary_table.add_row({"Cell Name", "Data Type", "Failed Count", "Avg Diff", "Avg Diff%",
                               "Max Diff", "Max Diff%", "Outliers"});
        summary_table.add_row({cell_name, "delay(ns)", std::to_string(failed_count),
                               std::to_string(avg_diff), std::to_string(avg_diff_percent) + "%",
                               std::to_string(max_diff), std::to_string(max_diff_percent) + "%",
                               std::to_string(outlier_count)});

        // Output summary table
        // Use std::filesystem to reliably get the extension
        std::filesystem::path file_path(output_file);
        if (file_path.has_extension() && file_path.extension() == ".md") {
          MarkdownExporter exporter;
          auto markdown = exporter.dump(summary_table);
          outfile << markdown << std::endl;
        } else {
          outfile << summary_table << std::endl;
        }
        outfile << "Worst delay outlier: Max Abs: " << max_diff << ", Max Rel: " << max_diff_percent
                << "%, Outliers: " << outlier_count << "\n\n";
      }
    } else {
      // cell not found in reference JSON
      spdlog::warn("Cell: '{}' not found in reference JSON", cell_name);
    }
  }

  outfile.close();
}
```

6.38 src/LogicComparator.cpp File Reference

`#include "LogicComparator.hpp"`

Include dependency graph for LogicComparator.cpp:



Functions

- bool isIdentifier (const std::string &token)

  Checks if a given string is a valid identifier.
- bool isOperator (const std::string &token)

  Checks if a given token is a logical operator.

6.38.1 Function Documentation

6.38.1.1 isIdentifier()

```
bool isIdentifier (
            const std::string & token )
```

Checks if a given string is a valid identifier.

A valid identifier must:

- Not be empty.
- Consist of alphanumeric characters and underscores.
- Start with an uppercase alphabetic character.
- Contain at least one alphabetic character.

Parameters

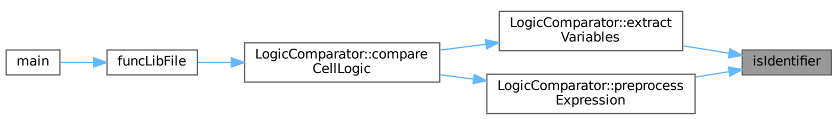
token	The string to check.

Returns

True if the string is a valid identifier, false otherwise.

Definition at line 74 of file LogicComparator.cpp.

Here is the caller graph for this function:



6.38.1.2 isOperator()

```
bool isOperator (
        const std::string & token )
```

Checks if a given token is a logical operator.

This function determines whether the input string `token` is one of the supported logical operators: "and", "or", "xor", or "not".

Parameters

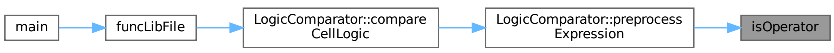
token	The string to check.

Returns

True if the token is a logical operator, false otherwise.

Definition at line 99 of file LogicComparator.cpp.

Here is the caller graph for this function:



6.39 LogicComparator.cpp

Go to the documentation of this file.

```
#include "LogicComparator.hpp"

LogicComparator::LogicComparator(const std::map<std::string, std::string> &ref_outpin_map,
                                 const std::map<std::string, std::string> &comp_outpin_map,
                                 const std::string &cell_name)
    : ref_outpin_map_(ref_outpin_map), comp_outpin_map_(comp_outpin_map), cell_name_(cell_name) {}
void LogicComparator::logic() {
  typedef exprtk::symbol_table<double> symbol_table_t;
  typedef exprtk::expression<double> expression_t;
  typedef exprtk::parser<double> parser_t;

  const std::string expression_string = "not(A and B) or C";

  symbol_table_t symbol_table;
  symbol_table.create_variable("A");
  symbol_table.create_variable("B");
  symbol_table.create_variable("C");

  expression_t expression;
  expression.register_symbol_table(symbol_table);
```

```

  parser_t parser;
  parser.compile(expression_string, expression);

  printf(" # | A | B | C | %s\n"
         "---+---+---+---+-%s\n",
         expression_string.c_str(), std::string(expression_string.size(), '-').c_str());

  for (int i = 0; i < 8; ++i) {
    symbol_table.get_variable("A")->ref() = double(i & 0x01 ? 1 : 0);
    symbol_table.get_variable("B")->ref() = double(i & 0x02 ? 1 : 0);
    symbol_table.get_variable("C")->ref() = double(i & 0x04 ? 1 : 0);

    const int result = static_cast<int>(expression.value());

    printf(" %d | %d | %d | %d | %d \n", i,
           static_cast<int>(symbol_table.get_variable("A")->value()),
           static_cast<int>(symbol_table.get_variable("B")->value()),
           static_cast<int>(symbol_table.get_variable("C")->value()), result);
  }
}

// --- Preprocessing Function ---

bool isIdentifier(const std::string &token) {
  if (token.empty())
    return false;
  // Check if the token is a valid identifier (upper alpha+ numeric + underscore)
  bool has_alpha = false;
  for (char c : token) {
    if (std::isalpha(c)) {
      has_alpha = true;
    } else if (!std::isalnum(c) && c != '_') {
      return false; // Contains invalid char
    }
  }
  // Must start with an uppercase alphabetic character
  return has_alpha && std::isupper(token[0]);
}

bool isOperator(const std::string &token) {
  static const std::set<std::string> operators = {"and", "or", "xor", "not"};
  return operators.count(token);
}

std::string LogicComparator::preprocessExpression(const std::string &input_expr) {
  std::string processed = input_expr;
  spdlog::debug("Preprocessing raw expression: {}", processed);

  // 0. Trim leading/trailing whitespace first
  processed = std::regex_replace(processed, std::regex("^\\s+|\\s+$"), "");
  if (processed.empty()) {
    spdlog::debug("Input expression is empty.");
    return ""; // Handle empty input early
  }

  // --- Helper function: build log string ---
  auto build_log_string = [](const auto &container) {
    std::ostringstream oss;
    for (auto it = container.begin(); it != container.end(); ++it) {
      oss << *it;
      if (std::next(it) != container.end()) {
        oss << ", ";
      }
    }
    return oss.str();
  };

  // 1. Handle '!' for NOT carefully BEFORE adding general spaces.
```

```
// Replace logical NOT '!' with " not ", ensuring not to replace '!='.
std::string temp_processed;
temp_processed.reserve(processed.length() * 1.2); // Pre-allocate a bit more space
for (size_t i = 0; i < processed.length(); ++i) {
  if (processed[i] == '!') {
    if (i + 1 < processed.length() && processed[i + 1] == '=') {
      temp_processed += "!="; // Keep inequality operator
      i++;                    // Skip the '='
    } else {
      temp_processed += " not "; // Replace logical NOT with spaced operator
    }
  } else {
    temp_processed += processed[i];
  }
}
processed = temp_processed;
spdlog::trace("After '!' to 'not' conversion: {}", processed);

// 2. Add spaces around other operators and parentheses that need them
processed = std::regex_replace(processed, std::regex("\\("), " ( ");
processed = std::regex_replace(processed, std::regex("\\)"), " ) ");
processed = std::regex_replace(processed, std::regex("\\+"), " + "); // OR
processed = std::regex_replace(processed, std::regex("\\^"), " ^ "); // XOR
processed = std::regex_replace(processed, std::regex("\\*"), " * "); // AND
// Consolidate multiple spaces into one and trim again
processed = std::regex_replace(processed, std::regex("\\s+"), " ");
processed = std::regex_replace(processed, std::regex("^\\s+|\\s+$"), "");
spdlog::trace("After adding spaces: {}", processed);

// 3. Replace symbolic operators with their keyword equivalents
//    Note: 'not' is already handled.
processed = std::regex_replace(processed, std::regex(" \\+ "), " or ");
processed = std::regex_replace(processed, std::regex(" \\^ "), " xor ");
processed = std::regex_replace(processed, std::regex(" \\* "), " and ");
spdlog::trace("After operator keyword replacement: {}", processed);

// 4. Tokenize based on spaces
std::stringstream ss(processed);
// Read tokens skipping whitespace
std::vector<std::string> tokens{std::istream_iterator<std::string>(ss),
                                std::istream_iterator<std::string>()};

if (tokens.empty()) {
  spdlog::debug("No tokens found after tokenization.");
  return "";
}
spdlog::trace("Tokens after initial processing: [{}]", build_log_string(tokens));

// 5. Insert implied 'and'
std::vector<std::string> tokens_with_implied_and;
if (!tokens.empty()) {
  tokens_with_implied_and.push_back(tokens[0]);
  for (size_t i = 0; i < tokens.size() - 1; ++i) {
    const std::string &current_token = tokens[i];
    const std::string &next_token = tokens[i + 1];

    // An operand ends if it's an identifier or a closing parenthesis.
    bool current_ends_operand = isIdentifier(current_token) || current_token == ")";
    // An operand starts if it's an identifier, an opening parenthesis, or 'not'.
    bool next_starts_operand =
        isIdentifier(next_token) || next_token == "(" || next_token == "not";

    // We should insert 'and' if the current token ends an operand AND the next token starts one,
    // UNLESS the current token is already an operator/opening bracket OR the next token is an
    // operator/closing bracket. Explicit check: Don't insert 'and' if an operator already exists
    // between them.
    bool current_is_op_or_open =
        isOperator(current_token) || current_token == "not" || current_token == "(";
    bool next_is_op_or_close = isOperator(next_token) || next_token == "not" || next_token == ")";
```

```
    if (current_ends_operand && next_starts_operand && !current_is_op_or_open &&
        !next_is_op_or_close) {
      spdlog::trace("Inserting implied 'and' between '{}' and '{}'", current_token, next_token);
      tokens_with_implied_and.push_back("and");
    }

    tokens_with_implied_and.push_back(next_token); // Add the next token regardless
  }
} else {
  // Handle case where initial tokenization yielded no tokens
  return "";
}

spdlog::trace("Tokens after implied 'and': [{}]", build_log_string(tokens_with_implied_and));

// 6. Handle 'not Identifier' ONLY during reconstruction (NEW LOGIC)
std::vector<std::string> final_tokens;
for (size_t i = 0; i < tokens_with_implied_and.size(); ++i) {
  const std::string &current_token = tokens_with_implied_and[i];

  if (current_token == "not") {
    // Check if the *next* token is an Identifier
    if (i + 1 < tokens_with_implied_and.size() && isIdentifier(tokens_with_implied_and[i + 1])) {
      // Found 'not Identifier', transform to 'not(Identifier)'
      final_tokens.push_back("not");
      final_tokens.push_back("(");
      final_tokens.push_back(tokens_with_implied_and[i + 1]); // The identifier
      final_tokens.push_back(")");
      i++; // Increment i to skip the identifier we just processed
    } else {
      // 'not' is followed by '(', another operator, or is at the end.
      // Add 'not' as is, let ExprTk parse 'not(...)' or handle errors.
      final_tokens.push_back("not");
    }
  } else {
    // Token is not 'not', just add it to the final list
    final_tokens.push_back(current_token);
  }
}
spdlog::trace("Tokens after 'not' parenthesis handling: [{}]", build_log_string(final_tokens));

// 7. Reconstruct the final expression string from tokens
std::string final_expr;
if (!final_tokens.empty()) {
  final_expr = final_tokens[0];
  for (size_t i = 1; i < final_tokens.size(); ++i) {
    const std::string &prev = final_tokens[i - 1];
    const std::string &curr = final_tokens[i];

    // Add space smartly: No space after '(', no space before ')',
    // no space after 'not' if followed by '(', no space before ',' (in function args, if
    // applicable)
    bool add_space = true;
    if (prev == "(" || curr == ")" || curr == ",") {
      add_space = false;
    }
    if (prev == "not" && curr == "(") {
      add_space = false; // Already handled by 'not(...)' structure
    }
    // Potentially add more rules if needed for other operators/functions

    if (add_space) {
      final_expr += " ";
    }
    final_expr += curr;
  }
}

```

```
  // 8. Final cleanup (consolidate any remaining multiple spaces and trim)
  final_expr = std::regex_replace(final_expr, std::regex("\\s+"), " ");
  final_expr = std::regex_replace(final_expr, std::regex("^\\s+|\\s+$"), "");

  spdlog::debug("Preprocessed expression result: {}", final_expr);
  return final_expr;
}

// --- Variable Extraction Helper ---
bool LogicComparator::extractVariables(const std::string &expr1_raw, const std::string &expr2_raw,
                                       std::vector<std::string> &sorted_vars) {
  // Define the set of ExprTk keywords/function names to filter out (lowercase)
  // Even though isIdentifier checks for uppercase, keep this filter as a safety measure
  static const std::set<std::string> keywords = {"abs",
                                                 "acos",
                                                 "acosh",
                                                 "and",
                                                 "asin",
                                                 "asinh",
                                                 "assert",
                                                 "atan",
                                                 "atan2",
                                                 "atanh",
                                                 "avg",
                                                 "break",
                                                 "case",
                                                 "ceil",
                                                 "clamp",
                                                 "continue",
                                                 "cosh",
                                                 "cos",
                                                 "cot",
                                                 "csc",
                                                 "default",
                                                 "deg2grad",
                                                 "deg2rad",
                                                 "else",
                                                 "equal",
                                                 "erfc",
                                                 "erf",
                                                 "exp",
                                                 "expm1",
                                                 "false",
                                                 "floor",
                                                 "for",
                                                 "frac",
                                                 "grad2deg",
                                                 "hypot",
                                                 "iclamp",
                                                 "if",
                                                 "ilike",
                                                 "in",
                                                 "inrange",
                                                 /*"in",*/ "like",
                                                 "log",
                                                 "log10",
                                                 "log1p",
                                                 "log2",
                                                 "logn",
                                                 "mand",
                                                 "max",
                                                 "min",
                                                 "mod",
                                                 "mor",
                                                 "mul",
                                                 "nand",
                                                 "ncdf",
                                                 "nor",
                                                 "not",
```

```
                                                "not_equal",
                                                /*"not",*/ "null",
                                                "or",
                                                "pow",
                                                "rad2deg",
                                                "repeat",
                                                "return",
                                                "root",
                                                "roundn",
                                                "round",
                                                "sec",
                                                "sgn",
                                                "shl",
                                                "shr",
                                                "sinc",
                                                "sinh",
                                                "sin",
                                                "sqrt",
                                                "sum",
                                                "swap",
                                                "switch",
                                                "tanh",
                                                "tan",
                                                "true",
                                                "trunc",
                                                "until",
                                                "var",
                                                "while",
                                                "xnor",
                                                "xor"};

// Regular expression to find potential identifiers (unchanged)
const std::regex identifier_regex("[a-zA-Z_][a-zA-Z0-9_]*");

// Store validated and filtered variable names
std::set<std::string> actual_vars1_set;
std::set<std::string> actual_vars2_set;

// --- Helper function: build log string ---
auto build_log_string = [](const auto &container) {
  std::ostringstream oss;
  oss << "[";
  for (auto it = container.begin(); it != container.end(); ++it) {
    oss << *it;
    if (std::next(it) != container.end()) {
      oss << ", ";
    }
  }
  oss << "]";
  return oss.str();
};

// --- Process the first expression ---
spdlog::debug("Extracting and validating identifiers from raw expr1: {}", expr1_raw);
auto words_begin1 = std::sregex_iterator(expr1_raw.begin(), expr1_raw.end(), identifier_regex);
auto words_end1 = std::sregex_iterator();
for (std::sregex_iterator i = words_begin1; i != words_end1; ++i) {
  std::string potential_var = i->str();
  spdlog::trace("Regex found in expr1: {}", potential_var);

  // 1. Validate using isIdentifier (now checks for uppercase etc.)
  if (isIdentifier(potential_var)) {
    // 2. Check if it's a keyword (still use lowercase comparison as a precaution)
    std::string lower_var = potential_var;
    std::transform(lower_var.begin(), lower_var.end(), lower_var.begin(),
                   [](unsigned char c) { return std::tolower(c); });

    if (keywords.find(lower_var) == keywords.end()) {
      actual_vars1_set.insert(potential_var); // Keep original case
```

```
        spdlog::trace("Kept variable from expr1: {}", potential_var);
      } else {
        spdlog::trace("Filtered keyword from expr1: {}", potential_var);
      }
    } else {
      spdlog::trace("Filtered invalid identifier from expr1: {}", potential_var);
    }
  }
  spdlog::debug("Validated variables found in expr1: {}", build_log_string(actual_vars1_set));

  // --- Process the second expression ---
  spdlog::debug("Extracting and validating identifiers from raw expr2: {}", expr2_raw);
  auto words_begin2 = std::sregex_iterator(expr2_raw.begin(), expr2_raw.end(), identifier_regex);
  auto words_end2 = std::sregex_iterator();
  for (std::sregex_iterator i = words_begin2; i != words_end2; ++i) {
    std::string potential_var = i->str();
    spdlog::trace("Regex found in expr2: {}", potential_var);

    // 1. Validate using isIdentifier
    if (isIdentifier(potential_var)) {
      // 2. Check if it's a keyword (still use lowercase comparison)
      std::string lower_var = potential_var;
      std::transform(lower_var.begin(), lower_var.end(), lower_var.begin(),
                     [](unsigned char c) { return std::tolower(c); });

      if (keywords.find(lower_var) == keywords.end()) {
        actual_vars2_set.insert(potential_var); // Keep original case
        spdlog::trace("Kept variable from expr2: {}", potential_var);
      } else {
        spdlog::trace("Filtered keyword from expr2: {}", potential_var);
      }
    } else {
      spdlog::trace("Filtered invalid identifier from expr2: {}", potential_var);
    }
  }
  spdlog::debug("Validated variables found in expr2: {}", build_log_string(actual_vars2_set));

  // --- Compare the two variable sets ---
  if (actual_vars1_set == actual_vars2_set) {
    // Sets are equal, extract the variable list
    sorted_vars.assign(actual_vars1_set.begin(), actual_vars1_set.end());
    std::sort(sorted_vars.begin(), sorted_vars.end()); // Sort alphabetically
    spdlog::info("Variable sets match. Found unique variables for comparison: {}",
                 build_log_string(sorted_vars));
    spdlog::info("Extracted variables done !");
    return true; // Variable sets are consistent
  } else {
    // Sets are not equal, report an error and return false
    spdlog::warn("Variable sets do not match between the two expressions!");
    // Calculate the differences for more detailed logging
    std::vector<std::string> diff1, diff2;
    std::set_difference(actual_vars1_set.begin(), actual_vars1_set.end(), actual_vars2_set.begin(),
                        actual_vars2_set.end(),
                        std::back_inserter(diff1)); // Vars only in expr1
    std::set_difference(actual_vars2_set.begin(), actual_vars2_set.end(), actual_vars1_set.begin(),
                        actual_vars1_set.end(),
                        std::back_inserter(diff2)); // Vars only in expr2

    if (!diff1.empty()) {
      spdlog::warn("Variables only in reference expression: {}", build_log_string(diff1));
    }
    if (!diff2.empty()) {
      spdlog::warn("Variables only in comparison expression: {}", build_log_string(diff2));
    }
    sorted_vars.clear(); // Clear the output variable list
    return false;        // Variable sets are inconsistent
  }
}
```

```
// --- Implementation of compareSingleExpressionPair ---
void LogicComparator::compareSingleExpressionPair(const std::string &ref_expression_processed,
                                                   const std::string &comp_expression_processed,
                                                   const std::vector<std::string> &sorted_vars,
                                                   PinComparisonResult &result) {
  // Store processed expressions in the result struct
  result.ref_expr_processed = ref_expression_processed;
  result.comp_expr_processed = comp_expression_processed;
  result.comparison_possible = true; // Assume comparison is possible initially

  spdlog::debug("Comparing processed expressions for pin '{}':", result.pin_name);
  spdlog::debug("  Ref : {}", ref_expression_processed);
  spdlog::debug("  Comp: {}", comp_expression_processed);

  // --- ExprTk Setup ---
  typedef exprtk::symbol_table<double> symbol_table_t;
  typedef exprtk::expression<double> expression_t;
  typedef exprtk::parser<double> parser_t;

  // Setup for Reference Expression
  symbol_table_t ref_symbol_table;
  expression_t ref_expression;
  parser_t ref_parser;

  for (const auto &var_name : sorted_vars) {
    // Create variable, ExprTk manages its memory within the table
    if (!ref_symbol_table.create_variable(var_name)) {
      result.error_message = "Failed to create variable '" + var_name + "' in ref symbol table.";
      spdlog::error(result.error_message);
      result.comparison_possible = false;
      return; // Cannot proceed
    }
  }
  ref_expression.register_symbol_table(ref_symbol_table);

  // Compile Reference Expression
  result.ref_compiles = ref_parser.compile(ref_expression_processed, ref_expression);
  if (!result.ref_compiles) {
    result.error_message = "Reference expression compilation failed: " + ref_parser.error();
    spdlog::warn("Pin '{}': {}", result.pin_name, result.error_message);
    result.comparison_possible = false;
    // Don't return yet, maybe the comparison one also fails
  } else {
    spdlog::debug("Pin '{}': Reference expression compiled successfully.", result.pin_name);
  }

  // Setup for Comparison Expression
  symbol_table_t comp_symbol_table;
  expression_t comp_expression;
  parser_t comp_parser;

  for (const auto &var_name : sorted_vars) {
    if (!comp_symbol_table.create_variable(var_name)) {
      // Append error if ref also failed, otherwise set it
      std::string comp_err = "Failed to create variable '" + var_name + "' in comp symbol table.";
      result.error_message += (result.error_message.empty() ? "" : "\n") + comp_err;
      spdlog::error(comp_err);
      result.comparison_possible = false; // Mark as impossible now
      return;                             // Cannot proceed if variable creation fails
    }
  }
  comp_expression.register_symbol_table(comp_symbol_table);

  // Compile Comparison Expression
  result.comp_compiles = comp_parser.compile(comp_expression_processed, comp_expression);
  if (!result.comp_compiles) {
    std::string comp_err = "Comparison expression compilation failed: " + comp_parser.error();
    result.error_message += (result.error_message.empty() ? "" : "\n") + comp_err;
    spdlog::warn("Pin '{}': {}", result.pin_name, comp_err);
```

```
    result.comparison_possible = false; // Mark as impossible if not already
  } else {
    spdlog::debug("Pin '{}': Comparison expression compiled successfully.", result.pin_name);
  }

  // If either failed to compile, comparison is not possible
  if (!result.ref_compiles || !result.comp_compiles) {
    result.comparison_possible = false;
    return;
  }

  // --- Truth Table Generation and Comparison ---
  size_t num_vars = sorted_vars.size();
  // Use unsigned long long for potentially large number of combinations
  unsigned long long num_combinations = 1ULL << num_vars;
  // Prevent excessively large tables (e.g., > 20 variables is 1M+ rows)
  const unsigned long long MAX_COMBINATIONS = 1ULL << 20; // Limit to 2^20 combinations
  if (num_vars > 20) {                                    // Check against var count for clarity
    result.error_message = "Too many variables (" + std::to_string(num_vars) +
                           "). Max supported for truth table is 20.";
    spdlog::error("Pin '{}': {}", result.pin_name, result.error_message);
    result.comparison_possible = false;
    return;
  }
  if (num_combinations == 0 && num_vars > 0) { // Overflow check
    result.error_message = "Number of combinations calculation overflowed for " +
                           std::to_string(num_vars) + " variables.";
    spdlog::error("Pin '{}': {}", result.pin_name, result.error_message);
    result.comparison_possible = false;
    return;
  }

  std::vector<bool> ref_results;
  std::vector<bool> comp_results;
  ref_results.reserve(static_cast<size_t>(num_combinations)); // Avoid reallocations
  comp_results.reserve(static_cast<size_t>(num_combinations));

  // --- Prepare Tabulate Tables ---
  Table ref_table;
  Table comp_table;
  Table::Row_t header_row;
  header_row.push_back("#"); // Row number column
  for (const auto &var_name : sorted_vars) {
    header_row.push_back(var_name);
  }
  header_row.push_back(ref_expression_processed); // Reference expression as last column header
  ref_table.add_row(header_row);

  // Adjust header for comparison table
  header_row.back() = comp_expression_processed; // Change last header to comparison expression
  comp_table.add_row(header_row);

  // Format header rows
  for (size_t i = 0; i < ref_table[0].size(); ++i) {
    ref_table[0][i].format().font_color(Color::yellow).font_style({FontStyle::bold});
    comp_table[0][i].format().font_color(Color::yellow).font_style({FontStyle::bold});
  }

  spdlog::debug("Pin '{}': Generating truth table with {} variables ({} combinations)...",
                result.pin_name, num_vars, num_combinations);

  bool evaluation_error = false;
  for (unsigned long long i = 0; i < num_combinations; ++i) {
    Table::Row_t ref_data_row;
    Table::Row_t comp_data_row;
    ref_data_row.push_back(std::to_string(i));
    comp_data_row.push_back(std::to_string(i));

    // Set variable values for this combination
```

```
  for (size_t j = 0; j < num_vars; ++j) {
    // The j-th bit of i determines the value of the j-th variable
    bool bit_value = ((i >> j) & 1ULL);
    double var_value = bit_value ? double(1.0) : double(0.0); // ExprTk uses floating point

    // Get variable reference and assign value
    // Need error checking here in theory, but create_variable should have caught issues
    ref_symbol_table.get_variable(sorted_vars[j])->ref() = var_value;
    comp_symbol_table.get_variable(sorted_vars[j])->ref() = var_value;

    // Add input value to table rows (as string "0" or "1")
    std::string bit_str = bit_value ? "1" : "0";
    ref_data_row.push_back(bit_str);
    comp_data_row.push_back(bit_str);
  }

  // Evaluate expressions
  double ref_val, comp_val;
  try {
    ref_val = ref_expression.value();
  } catch (const std::exception &e) {
    result.error_message +=
        "\nReference evaluation failed at combination " + std::to_string(i) + ": " + e.what();
    spdlog::error("Pin '{}': {}", result.pin_name, result.error_message);
    result.comparison_possible = false;
    evaluation_error = true;
    break; // Stop evaluation
  }
  try {
    comp_val = comp_expression.value();
  } catch (const std::exception &e) {
    result.error_message +=
        "\nComparison evaluation failed at combination " + std::to_string(i) + ": " + e.what();
    spdlog::error("Pin '{}': {}", result.pin_name, result.error_message);
    result.comparison_possible = false;
    evaluation_error = true;
    break; // Stop evaluation
  }

  // Store boolean results (commonly, non-zero is true in logic contexts)
  bool ref_bool_result = (ref_val != double(0.0));
  bool comp_bool_result = (comp_val != double(0.0));

  ref_results.push_back(ref_bool_result);
  comp_results.push_back(comp_bool_result);

  // Add output results to table rows (as string "0" or "1")
  ref_data_row.push_back(ref_bool_result ? "1" : "0");
  comp_data_row.push_back(comp_bool_result ? "1" : "0");

  // Add rows to tables
  ref_table.add_row(ref_data_row);
  comp_table.add_row(comp_data_row);

} // End of combination loop

if (evaluation_error) {
  return; // Don't proceed if evaluation failed
}

// --- Final Comparison ---
if (result.comparison_possible) {
  result.are_equivalent = (ref_results == comp_results);
  if (result.are_equivalent) {
    spdlog::info("Pin '{}': Expressions ARE logically equivalent.", result.pin_name);
  } else {
    spdlog::warn("Pin '{}': Expressions ARE NOT logically equivalent.", result.pin_name);
    result.error_message +=
        "\nTruth table outputs differ."; // Add specific reason for non-equivalence
```

```
    }

    // Store the generated tables in the result struct
    result.ref_truth_table = ref_table;
    result.comp_truth_table = comp_table;
  }
}

void LogicComparator::compareCellLogic() {

  // 1. Collect all unique pin names from both maps
  std::set<std::string> unique_pin_names;
  for (const auto &pair : ref_outpin_map_) {
    unique_pin_names.insert(pair.first);
  }
  for (const auto &pair : comp_outpin_map_) {
    unique_pin_names.insert(pair.first);
  }

  spdlog::info("Starting logic comparison for cell '{}' with {} unique output pins...", cell_name_,
               unique_pin_names.size());

  // 2. Iterate through each unique pin name
  for (const std::string &pin_name : unique_pin_names) {
    PinComparisonResult pin_result;
    pin_result.pin_name = pin_name;
    pin_result.comparison_possible = true; // Assume possible initially

    // 3. Get raw expressions
    auto ref_it = ref_outpin_map_.find(pin_name);
    auto comp_it = comp_outpin_map_.find(pin_name);

    if (ref_it == ref_outpin_map_.end()) {
      pin_result.error_message = "Pin not found in reference map.";
      spdlog::warn("Cell '{}', Pin '{}': {}", cell_name_, pin_name, pin_result.error_message);
      pin_result.comparison_possible = false;
      // Still try to get the comparison expression for reporting
      if (comp_it != comp_outpin_map_.end()) {
        pin_result.comp_expr_raw = comp_it->second;
      }
    } else {
      pin_result.ref_expr_raw = ref_it->second;
      spdlog::debug("Pin -> Expression: {} -> {}", pin_name, pin_result.ref_expr_raw);
    }

    if (comp_it == comp_outpin_map_.end()) {
      std::string comp_err = "Pin not found in comparison map.";
      pin_result.error_message += (pin_result.error_message.empty() ? "" : "\n") + comp_err;
      spdlog::warn("Cell '{}', Pin '{}': {}", cell_name_, pin_name, comp_err);
      pin_result.comparison_possible = false;
      // If ref existed, store it
      if (ref_it != ref_outpin_map_.end()) {
        pin_result.ref_expr_raw = ref_it->second; // Already stored if ref exists
      }
    } else {
      pin_result.comp_expr_raw = comp_it->second;
      spdlog::debug("Pin => Expression: {} => {}", pin_name, pin_result.comp_expr_raw);
    }

    // If pin missing in either, we can't compare, but store result and continue
    if (!pin_result.comparison_possible) {
      all_pin_results_[pin_name] = pin_result;
      continue;
    }

    // 4. Preprocess expressions
    std::string ref_processed = preprocessExpression(pin_result.ref_expr_raw);
    std::string comp_processed = preprocessExpression(pin_result.comp_expr_raw);

```

```
    if (ref_processed.empty()) {
      pin_result.error_message += "\nReference expression became empty after preprocessing.";
      spdlog::warn("Cell '{}', Pin '{}': {}", cell_name_, pin_name,
                   "Reference expression became empty after preprocessing.");
      pin_result.comparison_possible = false;
    }
    if (comp_processed.empty()) {
      std::string comp_err = "\nComparison expression became empty after preprocessing.";
      pin_result.error_message += (pin_result.error_message.empty() ? "" : "\n") + comp_err;
      spdlog::warn("Cell '{}', Pin '{}': {}", cell_name_, pin_name,
                   "Comparison expression became empty after preprocessing.");
      pin_result.comparison_possible = false; // Mark as impossible if not already
    }
    // If preprocessing failed for either, store and continue
    if (ref_processed.empty() || comp_processed.empty()) {
      pin_result.ref_expr_processed = ref_processed; // Store possibly empty strings
      pin_result.comp_expr_processed = comp_processed;
      all_pin_results_[pin_name] = pin_result;
      continue;
    }

    // 5. Extract and validate variables
    std::vector<std::string> sorted_vars;
    // Pass RAW expressions to extractVariables as it uses regex on original format
    bool variables_match =
        extractVariables(pin_result.ref_expr_raw, pin_result.comp_expr_raw, sorted_vars);

    if (!variables_match) {
      pin_result.error_message = "Variable sets do not match between expressions.";
      // extractVariables already logs details
      pin_result.comparison_possible = false;
      pin_result.ref_expr_processed = ref_processed; // Store processed even if vars mismatch
      pin_result.comp_expr_processed = comp_processed;
      all_pin_results_[pin_name] = pin_result;
      continue;
    }
    // --- Helper function: build log string ---
    auto build_log_string = [](const auto &container) {
      std::ostringstream oss;
      for (auto it = container.begin(); it != container.end(); ++it) {
        oss << *it;
        if (std::next(it) != container.end()) {
          oss << ", ";
        }
      }
      return oss.str();
    };
    spdlog::info("Pin '{}': Variable sets match. Found unique variables: {}", pin_name,
                 build_log_string(sorted_vars));

    // 6. Compare the single pair using the processed expressions
    // Pass pin_result by reference - it will be populated by the function
    compareSingleExpressionPair(ref_processed, comp_processed, sorted_vars, pin_result);

    // 7. Store the detailed result for this pin
    all_pin_results_[pin_name] = pin_result;

  } // End of pin loop

  spdlog::info("Logic comparison finished for cell '{}'.", cell_name_);
}

void LogicComparator::generateReport(const std::string &output_file) {
  // --- 0. Start Info ---
  spdlog::info("Generating report for cell '{}' to '{}'...", cell_name_, output_file);

  // --- 1. Determine Output Format ---
  bool output_markdown = false;
  try {
```

```
    // Use std::filesystem to reliably get the extension
    std::filesystem::path file_path(output_file);
    if (file_path.has_extension() && file_path.extension() == ".md") {
      output_markdown = true;
    }
  } catch (const std::exception &e) {
    spdlog::warn("Could not determine file extension for '{}': {}. Defaulting to plain text.",
                 output_file, e.what());
    output_markdown = false; // Default to plain text on error
  }

  // --- 2. Open Output File ---
  std::ofstream outfile(output_file,
                        std::ios::app); // Use app to append if file exists, or create new
  if (!outfile.is_open()) {
    spdlog::error("Failed to open output report file: {}", output_file);
    return;
  }
  spdlog::info("Generating report for cell '{}' to '{}' (Format: {})...", cell_name_, output_file,
               output_markdown ? "Markdown" : "Plain Text");

  // --- 3. Report Header ---
  outfile << "# Logic Equivalence Comparison Report\n\n";
  outfile << "**Cell Name: " << cell_name_ << "**\n\n";
  // Reference Pin Functions Table
  Table ref_pin_table;
  ref_pin_table.add_row({"Reference Pin", "Function"});
  ref_pin_table[0][0].format().font_style({FontStyle::bold});
  ref_pin_table[0][1].format().font_style({FontStyle::bold});
  for (const auto &[pin, function] : ref_outpin_map_) {
    ref_pin_table.add_row({pin, "`" + function + "`"}); // Add backticks to function
  }
  // Comparison Pin Functions Table
  Table comp_pin_table;
  comp_pin_table.add_row({"Comparison Pin", "Function"});
  comp_pin_table[0][0].format().font_style({FontStyle::bold});
  comp_pin_table[0][1].format().font_style({FontStyle::bold});
  for (const auto &[pin, function] : comp_outpin_map_) {
    comp_pin_table.add_row({pin, "`" + function + "`"}); // Add backticks to function
  }

  // --- 4. Report Metadata ---
  outfile << "**Performed by " << APP_NAME << " v" << APP_VERSION << " from " << APP_AUTHOR;
  auto now = std::chrono::system_clock::now();
  std::time_t now_time_t = std::chrono::system_clock::to_time_t(now);
  std::tm *now_tm = std::localtime(&now_time_t);
  outfile << ". on: " << std::put_time(now_tm, "%c") << "**\n" << std::endl;

  // --- 5. Legend Generation ---
  outfile << "## Legend\n\n";
  Table legend_table;
  legend_table.add_row({"Symbol", "Meaning"});
  legend_table[0][0].format().font_style({FontStyle::bold});
  legend_table[0][1].format().font_style({FontStyle::bold});
  legend_table.add_row({"[OK]", "Logically Equivalent"});
  legend_table.add_row({"[NO]", "Not Logically Equivalent"});
  legend_table.add_row({"[NA]", "Comparison Not Applicable (General)"}); // Changed from possible
  legend_table.add_row({"[VM]", "Variable Mismatch (Cannot Compare)"});
  legend_table.add_row({"[CE]", "Compilation Error (Cannot Compare)"});
  legend_table.add_row({"[EE]", "Evaluation Error (Cannot Compare)"});
  legend_table.add_row({"[PE]", "Preprocessing/Pin/Setup Error (Cannot Compare)"});

  // --- 6. Export Legend Table ---
  if (output_markdown) {
    MarkdownExporter exporter;
    outfile << exporter.dump(ref_pin_table) << "\n";  // Export ref pin table
    outfile << exporter.dump(comp_pin_table) << "\n"; // Export comp pin table
    outfile << exporter.dump(legend_table) << "\n";   // Export legend table
  } else {
```

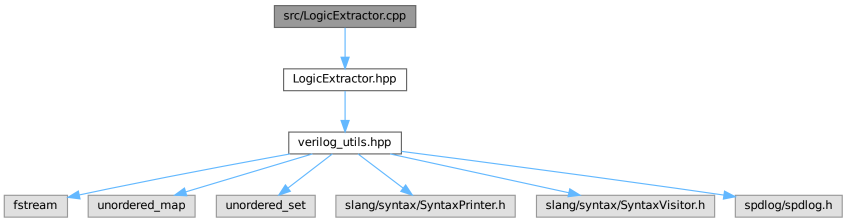
```
    outfile << ref_pin_table << "\n";  // Export ref pin table
    outfile << comp_pin_table << "\n"; // Export comp pin table
    outfile << legend_table << "\n";   // Export legend table (plain text)
  }

  // --- 7. Process Pin Results ---
  for (const auto &[pin_name, result] : all_pin_results_) { // Use passed argument
    outfile << "## Pin: `" << pin_name << "`\n\n";          // Use backticks for pin name

    // --- Output Truth Tables (Always if available) ---
    if (result.ref_truth_table.has_value()) {
      outfile << "### Reference Truth Table\n\n";
      Table ref_table = result.ref_truth_table.value();
      if (output_markdown) {
        MarkdownExporter exporter;
        outfile << exporter.dump(ref_table) << "\n";
      } else {
        outfile << ref_table << "\n";
      }
    } else {
      outfile << "*Reference truth table not generated (e.g., due to compilation error).*\n\n";
    }

    if (result.comp_truth_table.has_value()) {
      outfile << "### Comparison Truth Table\n\n";
      Table comp_table = result.comp_truth_table.value();
      if (output_markdown) {
        MarkdownExporter exporter;
        outfile << exporter.dump(comp_table) << "\n";
      } else {
        outfile << comp_table << "\n";
      }
    } else {
      outfile << "*Comparison truth table not generated (e.g., due to compilation error).*\n\n";
    }

    // --- Generate Summary Table ---
    outfile << "### Comparison Summary\n\n";
    Table summary_table;
    summary_table.add_row({"Property", "Value"});
    summary_table[0][0].format().font_style({FontStyle::bold});
    summary_table[0][1].format().font_style({FontStyle::bold});
    summary_table[0][0].format().font_color(Color::yellow);
    summary_table[0][1].format().font_color(Color::yellow);

    // Determine Status Symbol using ASCII codes
    std::string status_symbol;
    if (!result.comparison_possible) {
      status_symbol = "[NA]"; // Default Not Applicable
      if (!result.error_message.empty()) {
        if (result.error_message.find("Variable sets do not match") != std::string::npos) {
          status_symbol = "[VM]";
        } else if (!result.ref_compiles || !result.comp_compiles ||
                   result.error_message.find("compilation failed") != std::string::npos) {
          // Check flags first, then error message as fallback
          status_symbol = "[CE]";
        } else if (result.error_message.find("evaluation failed") != std::string::npos) {
          status_symbol = "[EE]";
        } else {
          // If comparison_possible is false for other reasons (e.g., pin missing, setup errors)
          status_symbol = "[PE]";
        }
      } else if (!result.ref_compiles || !result.comp_compiles) {
        // Handle cases where comparison_possible might be true but compilation failed (should
        // ideally not happen if logic is sound)
        status_symbol = "[CE]";
      } else {
        status_symbol = "[PE]"; // Fallback if no error message but still not possible
      }
```

```
    } else if (result.are_equivalent) {
      status_symbol = "[OK]";
    } else {
      status_symbol = "[NO]";
    }

    summary_table.add_row({"Status", status_symbol});
    summary_table.add_row({"Reference (Raw)", "`" + result.ref_expr_raw + "`"});
    summary_table.add_row({"Comparison (Raw)", "`" + result.comp_expr_raw + "`"});
    summary_table.add_row({"Reference (Processed)", "`" + result.ref_expr_processed + "`"});
    summary_table.add_row({"Comparison (Processed)", "`" + result.comp_expr_processed + "`"});
    summary_table.add_row({"Ref Expression Compiled", result.ref_compiles ? "Yes" : "No"});
    summary_table.add_row({"Comp Expression Compiled", result.comp_compiles ? "Yes" : "No"});
    summary_table.add_row({"Logically Equivalent", result.comparison_possible
                                                       ? (result.are_equivalent ? "Yes" : "No")
                                                       : "N/A"});
    summary_table[1][0].format().font_style({FontStyle::bold});
    summary_table[2][0].format().font_style({FontStyle::bold});
    summary_table[3][0].format().font_style({FontStyle::bold});
    summary_table[4][0].format().font_style({FontStyle::bold});
    summary_table[5][0].format().font_style({FontStyle::bold});
    summary_table[6][0].format().font_style({FontStyle::bold});
    summary_table[7][0].format().font_style({FontStyle::bold});
    summary_table[8][0].format().font_style({FontStyle::bold});

    if (!result.error_message.empty()) {
      std::string formatted_error = result.error_message;
      if (output_markdown) {
        // Basic newline replacement for Markdown
        size_t pos = 0;
        while ((pos = formatted_error.find('\n', pos)) != std::string::npos) {
          formatted_error.replace(pos, 1, "<br>");
          pos += 4; // Length of "<br>"
        }
      }
      summary_table.add_row({"Details/Error", formatted_error});
    }

    // Output Summary Table
    if (output_markdown) {
      MarkdownExporter exporter;
      outfile << exporter.dump(summary_table) << "\n";
    } else {
      outfile << summary_table << "\n";
    }
    std::cout << summary_table << "\n"; // Print to console for immediate feedback

    outfile << "---\n\n"; // Separator between pins

  } // End of pin results loop

  outfile.close();
  spdlog::info("Report generation complete for cell '{}'.", cell_name_);
}
```

6.40 src/LogicExtractor.cpp File Reference

`#include "LogicExtractor.hpp"`

Include dependency graph for LogicExtractor.cpp:



Functions

- void extractAndPrintNetlistInfo (const std::string &verilog_file, const std::string &cell)

  Extracts and prints netlist information from a Verilog file for a specified cell.
- std::map< std::string, std::string > extractLogicFromVerilog (const std::string &verilog_file, const std::string &cell)

  Extracts logic expressions from a Verilog file for a specified cell.

6.40.1 Function Documentation

6.40.1.1 extractAndPrintNetlistInfo()

```
void extractAndPrintNetlistInfo (
        const std::string & verilog_file,
        const std::string & cell )
```

Extracts and prints netlist information from a Verilog file for a specified cell.

This function parses a Verilog file using the slang library, extracts information about the primary inputs, primary outputs, internal wires, and gate drivers within a specified cell (module). It then prints a summary of the extracted information to the console using spdlog.

Parameters

verilog_file	The path to the Verilog file to be parsed.
cell	The name of the cell (module) for which to extract netlist information.

Exceptions

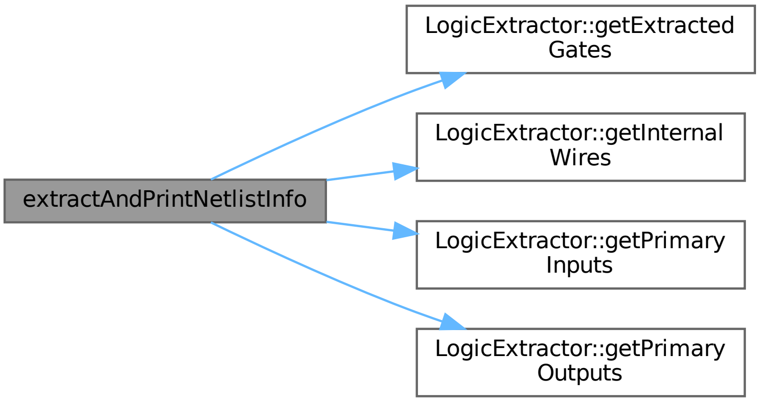
std::exception	If any error occurs during Verilog parsing or info extraction.

Note

The function uses the slang library for Verilog parsing and a custom LogicExtractor class to extract the desired information. Error messages are logged using spdlog.

Definition at line 673 of file LogicExtractor.cpp.

Here is the call graph for this function:



6.40.1.2 extractLogicFromVerilog()

```
std::map< std::string, std::string > extractLogicFromVerilog (
            const std::string & verilog_file,
            const std::string & cell )
```

Extracts logic expressions from a Verilog file for a specified cell.

This function parses a Verilog file using the slang library, identifies the specified cell, and extracts the logic expressions for its outputs. It returns a map where the keys are output signal names and the values are their corresponding logic expressions as strings.

Parameters

verilog_file	The path to the Verilog file to parse.
cell	The name of the cell (module) for which to extract logic expressions.

Returns

A map of output signal names to their logic expressions. Returns an empty map if parsing fails, the cell is not found, or no logic expressions can be derived.

Note

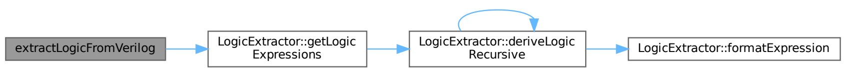
The function uses the slang library for Verilog parsing. Ensure that slang is properly installed and configured before using this function.

The logic extraction process involves traversing the syntax tree of the Verilog code and identifying relevant assignments and expressions within the specified cell.

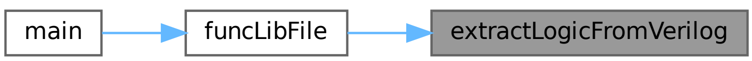
Error messages and warnings are logged using the spdlog library.

Definition at line 752 of file LogicExtractor.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



6.41 LogicExtractor.cpp

Go to the documentation of this file.

```
#include "LogicExtractor.hpp"

// --- Implementation for LogicExtractor ---
```

```
void LogicExtractor::handle(const slang::syntax::ModuleDeclarationSyntax &module) {
  if (parsingComplete_)
    return; // Don't re-process if called again

  if (module.header && module.header->name.valid()) {
    std::string_view moduleName = module.header->name.valueText();
    if (!targetCell_.empty() && moduleName == targetCell_) {
      spdlog::info("LogicExtractor: Found target module: {}", targetCell_);
      inTargetModule_ = true;

      // Reset state for the target module
      primaryInputs_.clear();
      primaryOutputs_.clear();
      internalWires_.clear();
      portDirections_.clear(); // Clear temporary direction map
      gateOutputDrivers_.clear();
      // logicCache_.clear(); // For Step 2

      // Visit children (ports, declarations, instances) IN ORDER
      // It's often better to visit declarations first, then the port list
      // Slang's default visitDefault might handle this, but be aware.
      visitDefault(module);

      // --- Finalize Ports after visiting declarations and port list ---
      // This part might be better placed *after* visiting NonAnsiPortList,
      // assuming the list visit confirms the names from the header.
      // Let's move the finalization logic to handle(NonAnsiPortListSyntax)

      // Mark that we finished processing the target module
      inTargetModule_ = false; // Important to prevent processing other modules
      parsingComplete_ = true; // Stop processing after finding the target
      spdlog::info("LogicExtractor: Finished visiting target module '{}'.", targetCell_);

    } else if (!parsingComplete_) {
      // If not the target module yet, continue searching
      visitDefault(module);
    }
  } else if (!parsingComplete_) {
    // Handle modules without valid headers if necessary, or just traverse
    visitDefault(module);
  }
}

// Handle Port Declarations (defines direction and type, usually inside module body)
void LogicExtractor::handle(const slang::syntax::PortDeclarationSyntax &portDecl) {
  if (!inTargetModule_)
    return;
  spdlog::debug("LogicExtractor: Handling Port Declaration");

  std::string direction = "unknown";
  if (portDecl.header) {
    // Determine direction (input/output/inout)
    // Important: Use valueText() as Slang typically represents keywords as text tokens
    if (auto varHeader = portDecl.header->as_if<slang::syntax::VariablePortHeaderSyntax>()) {
      if (varHeader->direction.valid()) {
        direction = std::string(varHeader->direction.valueText());
      }
    } else if (auto netHeader = portDecl.header->as_if<slang::syntax::NetPortHeaderSyntax>()) {
      if (netHeader->direction.valid()) {
        direction = std::string(netHeader->direction.valueText());
      }
    }
    // Add InterfacePortHeaderSyntax if needed
  } else {
    spdlog::warn("LogicExtractor: Port declaration without header found.");
  }

  for (const auto &declarator : portDecl.declarators) {
```

```
    if (declarator->name.valid()) {
      std::string portName = std::string(declarator->name.valueText());
      if (portDirections_.count(portName) && portDirections_[portName] != "unknown") {
        spdlog::warn("LogicExtractor: Multiple direction declarations for port '{}'. Keeping first "
                     "one ('{}'). New direction: '{}'",
                     portName, portDirections_[portName], direction);
      } else {
        spdlog::info("LogicExtractor: Storing direction '{}' for port '{}'", direction, portName);
        portDirections_[portName] = direction;
        if (direction == "input") {
          primaryInputs_.insert(portName);
          internalWires_.insert(portName); // Inputs are also technically 'wires' usable internally
        } else if (direction == "output") {
          primaryOutputs_.insert(portName);
          internalWires_.insert(portName); // Outputs are also wires driven by something
        } else if (direction == "inout") {
          spdlog::info("LogicExtractor: Inout port '{}' found. Adding to both inputs and outputs.",
                       portName);
          primaryInputs_.insert(portName);
          primaryOutputs_.insert(portName);
          internalWires_.insert(portName);
        } else {
          spdlog::warn("LogicExtractor: Port '{}' found in declaration but has unknown or missing "
                       "direction '{}'. Treating as internal wire.",
                       portName, direction);
          internalWires_.insert(portName);
        }
        // logicCache_[portName] = portName; // For Step 2
      }
    } else {
      spdlog::warn(
          "LogicExtractor: Port declarator without a name found in PortDeclarationSyntax.");
    }
  }
  // Don't call visitDefault here, as it might revisit things unexpectedly.
  // Let the main module visitor handle traversing into children if necessary.
}

// Handle Non-ANSI Port List (defines names, usually in module header)
void LogicExtractor::handle(const slang::syntax::NonAnsiPortListSyntax &portList) {
  if (!inTargetModule_)
    return;
  spdlog::debug("LogicExtractor: Handling NonAnsi Port List");

  for (const auto portSyntax : portList.ports) {
    if (!portSyntax)
      continue;

    std::string portName = "";
    // Most common case: ImplicitNonAnsiPortSyntax contains the expression (usually just the name)
    if (auto implicitPort = portSyntax->as_if<slang::syntax::ImplicitNonAnsiPortSyntax>()) {
      if (implicitPort->expr) {
        // The expression itself might be complex, try to get the simple name
        if (auto portRef = implicitPort->expr->as_if<slang::syntax::PortReferenceSyntax>()) {
          if (portRef->name.valid()) {
            portName = std::string(portRef->name.valueText());
          }
        } else if (auto portConcat =
                       implicitPort->expr->as_if<slang::syntax::PortConcatenationSyntax>()) {
          // Handle concatenations if necessary - complex
          spdlog::warn("LogicExtractor: Port concatenation found in NonAnsi port list - currently "
                       "not fully handled for logic extraction. Port: {}",
                       implicitPort->toString());
          continue; // Skip complex ports for now
        } else {
          // Fallback: try getting name from the expression directly (might be just identifier)
          portName = implicitPort->toString();
        }
      }
```

```
    }
    // Handle ExplicitNonAnsiPortSyntax (e.g., .A(A)) if needed
    else if (auto explicitPort = portSyntax->as_if<slang::syntax::ExplicitNonAnsiPortSyntax>()) {
      if (explicitPort->name.valid()) {
        portName = std::string(explicitPort->name.valueText());
        // Note: explicitPort->expr is the internal signal it connects to.
        // We are primarily interested in the port's own name (explicitPort->name) here.
      }
    }
    // Handle EmptyNonAnsiPortSyntax (commas for placeholders) if necessary
    else if (portSyntax->kind == slang::syntax::SyntaxKind::EmptyNonAnsiPort) {
      spdlog::debug("LogicExtractor: Skipping empty non-ANSI port placeholder.");
      continue;
    } else {
      spdlog::warn("LogicExtractor: Unhandled NonAnsi port syntax kind: {}",
                   slang::syntax::toString(portSyntax->kind));
      continue;
    }

    // Now we (hopefully) have the portName.
    if (!portName.empty()) {
      if (portDirections_.count(portName)) {
        const std::string &direction = portDirections_[portName];
        spdlog::debug("LogicExtractor: Finalizing port '{}' with direction '{}'", portName,
                      direction);

        if (direction == "input") {
          primaryInputs_.insert(portName);
          internalWires_.insert(portName); // Inputs are also technically 'wires' usable internally
                                           // logicCache_[portName] = portName; // For Step 2
        } else if (direction == "output") {
          primaryOutputs_.insert(portName);
          internalWires_.insert(portName); // Outputs are also wires driven by something
        } else if (direction == "inout") {
          spdlog::info("LogicExtractor: Inout port '{}' found. Adding to both inputs and outputs.",
                       portName);
          primaryInputs_.insert(portName);
          primaryOutputs_.insert(portName);
          internalWires_.insert(portName);
          // logicCache_[portName] = portName; // For Step 2
        } else {
          spdlog::warn("LogicExtractor: Port '{}' found in list but has unknown or missing "
                       "direction '{}'. Treating as internal wire.",
                       portName, direction);
          internalWires_.insert(portName);
        }
      } else {
        spdlog::debug("LogicExtractor: Port '{}' found in NonAnsi list, no direction declaration ",
                      portName);
      }
    } else {
      spdlog::warn("LogicExtractor: Could not determine port name from NonAnsi port list item: {}",
                   portSyntax->toString());
    }
  }
  // Don't call visitDefault here
}

// Handle explicit wire declarations
void LogicExtractor::handle(const slang::syntax::NetDeclarationSyntax &netDecl) {
  if (!inTargetModule_)
    return;

  for (const auto &declarator : netDecl.declarators) {
    if (declarator->name.valid()) {
      std::string wireName = std::string(declarator->name.valueText());
      spdlog::debug("LogicExtractor: Found wire declaration: {}", wireName);
      // Avoid adding ports again if they were also declared as nets (common)
      if (!primaryInputs_.count(wireName) && !primaryOutputs_.count(wireName)) {
```

```
      internalWires_.insert(wireName);
    } else {
      spdlog::trace("LogicExtractor: Wire '{}' is already known as a port.", wireName);
    }
  }
}
// Don't call visitDefault here
}

// Handle primitive gate instantiations (MOST IMPORTANT PART)
void LogicExtractor::handle(const slang::syntax::PrimitiveInstantiationSyntax &primitiveInst) {
  if (!inTargetModule_)
    return;

  if (!primitiveInst.type.valid()) {
    spdlog::warn("LogicExtractor: Primitive instance without a type token found.");
    return;
  }

  // Use token kind for reliable checking, text for name storage
  slang::parsing::TokenKind gateKind = primitiveInst.type.kind;
  std::string gateTypeName = std::string(primitiveInst.type.valueText());

  spdlog::debug("LogicExtractor: Found Primitive Instance of Type: {} (Kind: {})", gateTypeName,
                slang::parsing::toString(gateKind));

  for (const auto &instance : primitiveInst.instances) {
    // if (!instance || !instance->connections.isInitialized()) { // Check if connections are valid
    //   spdlog::warn("LogicExtractor: Skipping primitive instance of type {} due to null pointer or
    //   "
    //                "uninitialized connections.",
    //                gateTypeName);
    //   continue;
    // }

    GateInfo currentGateInfo;
    currentGateInfo.gateTypeName = gateTypeName;
    currentGateInfo.kind = gateKind;

    // Primitives usually have ordered connections.
    // The FIRST connection is typically the OUTPUT.
    // The REST are INPUTS.
    if (instance->connections.empty()) {
      spdlog::warn("LogicExtractor: Gate instance of type {} has no connections.", gateTypeName);
      continue;
    }

    // Extract Output Signal
    // Ensure the connection itself is not null
    if (!instance->connections[0]) {
      spdlog::error("LogicExtractor: First connection (output) is null for primitive {}",
                    gateTypeName);
      continue;
    }
    if (auto firstConn =
            instance->connections[0]->as_if<slang::syntax::OrderedPortConnectionSyntax>()) {
      currentGateInfo.outputSignal =
          firstConn->expr->toString(); // Get the output signal name from the connection
                                       // remove extra spaces in the signal name
      currentGateInfo.outputSignal.erase(
          std::remove_if(currentGateInfo.outputSignal.begin(), currentGateInfo.outputSignal.end(),
                         [](unsigned char x) { return std::isspace(x); }),
          currentGateInfo.outputSignal.end());
      if (!currentGateInfo.outputSignal.empty()) {
        spdlog::debug("  Output Signal: {}", currentGateInfo.outputSignal);
        // Gate outputs are internal signals (unless they are module outputs)
        // Add to internal wires if not already a primary output.
        if (!primaryOutputs_.count(currentGateInfo.outputSignal)) {
          internalWires_.insert(currentGateInfo.outputSignal);
```

```
        }
      } else {
        spdlog::error(
            "LogicExtractor: Could not determine output signal name for gate instance of type {}",
            gateTypeName);
        continue; // Skip this instance if output is unknown
      }
    } else {
      spdlog::error("LogicExtractor: Expected OrderedPortConnectionSyntax for output of primitive "
                    "{}, but got kind: {}. Conn: {}",
                    gateTypeName, slang::syntax::toString(instance->connections[0]->kind),
                    instance->connections[0]->toString());
      continue; // Skip this instance
    }

    // Extract Input Signals
    for (size_t i = 1; i < instance->connections.size(); ++i) {
      // Ensure the connection itself is not null
      if (!instance->connections[i]) {
        spdlog::warn("LogicExtractor: Input connection {} is null for primitive {}", i,
                     gateTypeName);
        continue;
      }
      if (auto conn =
              instance->connections[i]->as_if<slang::syntax::OrderedPortConnectionSyntax>()) {
        std::string inputSig =
            conn->expr->toString(); // Get the input signal name from the connection
        // remove extra spaces in the signal name
        inputSig.erase(std::remove_if(inputSig.begin(), inputSig.end(),
                                      [](unsigned char x) { return std::isspace(x); }),
                       inputSig.end());
        if (!inputSig.empty()) {
          currentGateInfo.inputSignals.push_back(inputSig);
          spdlog::debug("  Input Signal {}: {}", i, inputSig);
        } else {
          spdlog::warn("LogicExtractor: Could not determine input signal name for input {} of gate "
                       "instance type {}. Conn: {}",
                       i, gateTypeName, conn->toString());
          // Decide whether to skip or continue with partial inputs
        }
      } else {
        spdlog::warn("LogicExtractor: Expected OrderedPortConnectionSyntax for input {} of "
                     "primitive {}, but got kind: {}. Conn: {}",
                     i, gateTypeName, slang::syntax::toString(instance->connections[i]->kind),
                     instance->connections[i]->toString());
      }
    }

    // Store the gate information, mapping the output signal to its driving gate
    if (!currentGateInfo.outputSignal.empty()) {
      if (gateOutputDrivers_.count(currentGateInfo.outputSignal)) {
        // This is a critical warning - indicates multiple drivers for the same net!
        spdlog::error("LogicExtractor: Multiple drivers found for signal '{}'! Previous driver: "
                      "{}, New driver: {}. Netlist is likely invalid.",
                      currentGateInfo.outputSignal,
                      gateOutputDrivers_[currentGateInfo.outputSignal].gateTypeName, gateTypeName);
        // Keep the first one found for now, or decide on error handling
      } else {
        spdlog::info("  Storing driver for '{}': Gate Type '{}'", currentGateInfo.outputSignal,
                     gateTypeName);
        gateOutputDrivers_[currentGateInfo.outputSignal] = currentGateInfo;
      }
    }
  }
  // Don't call visitDefault here
}

// --- Logic Derivation Implementation ---

```

```cpp
// Public method called after visiting the tree
std::map<std::string, std::string> LogicExtractor::getLogicExpressions() {
  std::map<std::string, std::string> result_map;
  if (!parsingComplete_) {
    spdlog::error("LogicExtractor: AST parsing did not complete or target module '{}' not found. "
                  "Cannot extract logic.",
                  targetCell_);
    return result_map; // Return empty map
  }

  spdlog::info("LogicExtractor: Deriving logic expressions for {} output ports...",
               primaryOutputs_.size());

  for (const std::string &outputPort : primaryOutputs_) {
    spdlog::debug("LogicExtractor: Deriving logic for output: {}", outputPort);
    try {
      result_map[outputPort] = deriveLogicRecursive(outputPort);
      spdlog::info("  Output: {} => {}", outputPort, result_map[outputPort]);
    } catch (const std::runtime_error &e) {
      spdlog::error("LogicExtractor: Error deriving logic for output '{}': {}", outputPort,
                    e.what());
      result_map[outputPort] = "/* Error deriving logic */";
    } catch (...) {
      spdlog::error("LogicExtractor: Unknown error deriving logic for output '{}'", outputPort);
      result_map[outputPort] = "/* Unknown error deriving logic */";
    }
  }
  return result_map;
}

// Recursive function with memoization
std::string LogicExtractor::deriveLogicRecursive(const std::string &signalName) {
  // 1. Check Cache (Memoization)
  if (logicCache_.count(signalName)) {
    return logicCache_.at(signalName);
  }

  // 2. Base Case: Is it a primary input?
  if (primaryInputs_.count(signalName)) {
    // Already cached during port handling, but double-check
    if (!logicCache_.count(signalName)) {
      logicCache_[signalName] = signalName;
    }
    return signalName;
  }

  // 3. Recursive Step: Is it driven by a gate?
  if (gateOutputDrivers_.count(signalName)) {
    const GateInfo &driverGate = gateOutputDrivers_.at(signalName);
    std::vector<std::string> inputExpressions;
    spdlog::debug("    Tracing signal '{}', driven by {} gate", signalName,
                  driverGate.gateTypeName);

    // Recursively find expressions for all inputs of this gate
    for (const std::string &inputSig : driverGate.inputSignals) {
      if (inputSig.empty()) {
        throw std::runtime_error("Empty input signal name encountered for gate driving " +
                                 signalName);
      }
      spdlog::debug("      Recursing for input: {}", inputSig);
      inputExpressions.push_back(deriveLogicRecursive(inputSig));
    }

    // Format the expression based on gate type and input expressions
    std::string currentExpr = formatExpression(driverGate, inputExpressions);

    // Cache the result
    logicCache_[signalName] = currentExpr;
    return currentExpr;
```

```
  }

  // 4. Handle Assign statements (if implemented)
  // if (assignDrivers_.count(signalName)) { ... }

  // 5. Error Case: Signal not found or not driven by known element
  // Check if it's just an internal wire that wasn't driven?
  if (internalWires_.count(signalName)) {
    throw std::runtime_error(
        "Signal '" + signalName +
        "' is an internal wire but has no identified driver (gate or assign).");
  } else {
    throw std::runtime_error(
        "Signal '" + signalName +
        "' is not a primary input, known wire, or driven by a recognized gate/assignment.");
  }
}

// Helper to format the expression string based on gate type
std::string LogicExtractor::formatExpression(const GateInfo &gateInfo,
                                             const std::vector<std::string> &inputExprs) {
  if (inputExprs.empty() && gateInfo.kind != slang::parsing::TokenKind::NotKeyword &&
      gateInfo.kind != slang::parsing::TokenKind::BufKeyword) {
    // Gates like AND/OR/XOR need inputs
    spdlog::warn("Gate type {} requires inputs, but none were provided/derived for output {}",
                 gateInfo.gateTypeName, gateInfo.outputSignal);
    return "/*<Error: Missing Inputs for " + gateInfo.gateTypeName + ">*/";
  }

  std::string result = "";

  // Use gateInfo.type (enum) for reliable checking
  switch (gateInfo.kind) {
  case slang::parsing::TokenKind::AndKeyword:
    result = "(" + inputExprs[0];
    for (size_t i = 1; i < inputExprs.size(); ++i)
      result += " * " + inputExprs[i]; // Use * for AND
    result += ")";
    break;
  case slang::parsing::TokenKind::NandKeyword:
    result = "!(" + inputExprs[0];
    for (size_t i = 1; i < inputExprs.size(); ++i)
      result += " * " + inputExprs[i];
    result += ")";
    break;
  case slang::parsing::TokenKind::OrKeyword:
    result = "(" + inputExprs[0];
    for (size_t i = 1; i < inputExprs.size(); ++i)
      result += " + " + inputExprs[i]; // Use + for OR
    result += ")";
    break;
  case slang::parsing::TokenKind::NorKeyword:
    result = "!(" + inputExprs[0];
    for (size_t i = 1; i < inputExprs.size(); ++i)
      result += " + " + inputExprs[i];
    result += ")";
    break;
  case slang::parsing::TokenKind::XorKeyword:
    result = "(" + inputExprs[0];
    for (size_t i = 1; i < inputExprs.size(); ++i)
      result += " ^ " + inputExprs[i]; // Use ^ for XOR
    result += ")";
    break;
  case slang::parsing::TokenKind::XnorKeyword:
    result = "!(" + inputExprs[0];
    for (size_t i = 1; i < inputExprs.size(); ++i)
      result += " ^ " + inputExprs[i];
    result += ")";
    break;
```

```
  case slang::parsing::TokenKind::NotKeyword: // NOT gate
    if (inputExprs.size() != 1) {
      spdlog::warn("NOT gate expects 1 input, got {}", inputExprs.size());
      return "/*<Error: Incorrect Inputs for NOT>*/";
    }
    result = "!" + inputExprs[0]; // Use ! for NOT
    break;
  case slang::parsing::TokenKind::BufKeyword: // BUF gate
    if (inputExprs.size() != 1) {
      spdlog::warn("BUF gate expects 1 input, got {}", inputExprs.size());
      return "/*<Error: Incorrect Inputs for BUF>*/";
    }
    result = inputExprs[0]; // Output is the same as input
    break;
  // Add cases for bufif0, bufif1, notif0, notif1, pullup, pulldown, cmos, nmos, pmos, tran etc. if
  // needed
  default:
    spdlog::warn("Unsupported primitive gate type for logic expression generation: {}",
                 gateInfo.gateTypeName);
    result = "/*<Unsupported Gate: " + gateInfo.gateTypeName + ">*/";
    break;
  }
  return result;
}

// --- New Function Implementation (Step 1: Visit and Print) ---

void extractAndPrintNetlistInfo(const std::string &verilog_file, const std::string &cell) {
  spdlog::info("--- Step 1: Starting Netlist Info Extraction ---");
  spdlog::info("Verilog file: '{}', Target cell: '{}'", verilog_file, cell);

  try {
    auto result = slang::syntax::SyntaxTree::fromFile(verilog_file);
    if (!result) { // Check if result is valid
      spdlog::error("Error parsing Verilog file '{}'. Cannot extract info.", verilog_file);
      return;
    }

    spdlog::info("Successfully parsed Verilog file.");
    std::shared_ptr<slang::syntax::SyntaxTree> tree = result.value();

    LogicExtractor extractor(cell);
    tree->root().visit(extractor); // Populate extractor's internal state

    // --- Print Summary (Debug for Step 1) ---
    spdlog::info("--- Extraction Summary for Cell '{}': ---", cell);

    const auto &inputs = extractor.getPrimaryInputs();
    spdlog::info("Found {} Primary Inputs:", inputs.size());
    for (const auto &name : inputs) {
      spdlog::info("  - {}", name);
    }

    const auto &outputs = extractor.getPrimaryOutputs();
    spdlog::info("Found {} Primary Outputs:", outputs.size());
    for (const auto &name : outputs) {
      spdlog::info("  - {}", name);
    }

    const auto &wires = extractor.getInternalWires();
    spdlog::info("Found {} Internal Wires:", wires.size());
    for (const auto &name : wires) {
      spdlog::info("  - {}", name);
    }

    const auto &gates = extractor.getExtractedGates();
    spdlog::info("Found {} Gate Drivers:", gates.size());
    for (const auto &pair : gates) {
      const std::string &outputNet = pair.first;
```

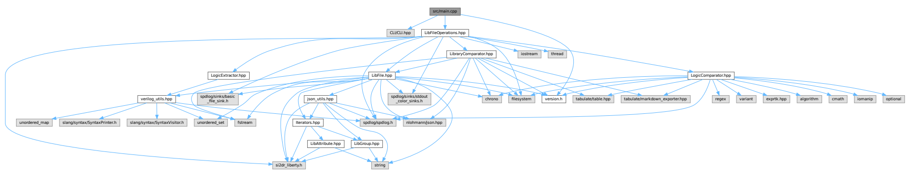
```
      const GateInfo &info = pair.second;
      std::string inputsStr = "";
      for (size_t i = 0; i < info.inputSignals.size(); ++i) {
        inputsStr += info.inputSignals[i] + (i == info.inputSignals.size() - 1 ? "" : ", ");
      }
      spdlog::info("  - Output: {} <= Driven by: {} ({}) Inputs: [{}]", outputNet,
                   info.gateTypeName, slang::parsing::toString(info.kind), inputsStr);
    }
    spdlog::info("--- End Netlist Info Extraction ---");

  } catch (const std::exception &e) {
    spdlog::error("Exception during Verilog parsing or info extraction: {}", e.what());
  } catch (...) {
    spdlog::error("Unknown exception during Verilog parsing or info extraction.");
  }
}

// --- (Step 2: Extract Logic Expressions) ---

std::map<std::string, std::string> extractLogicFromVerilog(const std::string &verilog_file,
                                                           const std::string &cell) {
  spdlog::info("--- Starting Logic Expression Extraction for cell: '{}' ---", cell);
  std::map<std::string, std::string> logicMap; // Default empty map

  try {
    auto result = slang::syntax::SyntaxTree::fromFile(verilog_file);
    if (!result) {
      spdlog::error("Error parsing Verilog file '{}'. Cannot extract logic.", verilog_file);
      return logicMap;
    }

    spdlog::info("Successfully parsed Verilog file.");
    std::shared_ptr<slang::syntax::SyntaxTree> tree = result.value();

    LogicExtractor extractor(cell);
    tree->root().visit(extractor); // Populate extractor's internal state

    // Now, call the method to derive and get the expressions
    logicMap = extractor.getLogicExpressions();

  } catch (const std::exception &e) {
    spdlog::error("Exception during Verilog parsing or logic extraction: {}", e.what());
  } catch (...) {
    spdlog::error("Unknown exception during Verilog parsing or logic extraction.");
  }

  if (logicMap.empty()) {
    spdlog::warn("Logic extraction finished, but no expressions were derived for cell '{}'. Check "
                 "if cell exists and is correctly defined.",
                 cell);
  } else {
    spdlog::info("Logic extraction completed for cell '{}'. Found {} output expression strings",
                 cell, logicMap.size());
  }

  spdlog::info("--- End Logic Expression Extraction ---");
  return logicMap;
}
```

6.42 src/main.cpp File Reference

This file contains the main function for the ZlibValidation tool.

```
#include "CLI/CLI.hpp"
#include "LibFileOperations.hpp"
#include "version.h"
```

Include dependency graph for main.cpp:



Functions

- int main (int argc, char *argv[])

6.42.1 Detailed Description

This file contains the main function for the ZlibValidation tool.

The ZlibValidation tool is a command-line application that provides several functionalities for processing and validating Liberty files, including parsing, monotonicity checking, comparison, supercell generation, Verilog/SPICE netlist generation, functional equivalence check, and a utility to clear generated files. It uses the CLI11 library for command-line argument parsing and spdlog for logging.

The main function parses command-line arguments using CLI11 to determine the desired operation and its parameters. It then calls the appropriate functions to perform the requested task. The tool supports processing multiple Liberty files in parallel using multi-threading for monotonicity checking, supercell generation, Verilog generation, and SPICE generation.

Parameters

argc	The number of command-line arguments.
argv	An array of command-line argument strings.

Returns

0 if the program executes successfully, or an error code otherwise.

The main function performs the following steps:

1. Initializes command-line argument parsing using CLI11.

2. Defines subcommands for each supported operation:
   - `parse`: Parses a Liberty file and writes JSON to a file.
   - `mono`: Checks the monotonicity of timing arc values in a Liberty file.
   - `compare`: Compares two Liberty files and reports differences.
   - `supercell`: Generates supercells for a given Liberty file.
   - `zlibboost`: Runs the ZlibBoost tool for multi-threaded library processing.
   - `clear`: Clears log, JSON, map, markdown, Verilog, and SPICE files in the current directory.
   - `verilog`: Generates Verilog netlist for a given Liberty file.
   - `spice`: Generates SPICE netlist for a given Liberty file.
   - `func`: Checks functional equivalence of two Liberty or Verilog files.
3. Parses the command-line arguments using CLI11.
4. Calls the appropriate function based on the selected subcommand.
5. Logs the program's exit status and timestamp.

Note

The tool relies on external libraries such as CLI11, spdlog, and potentially others depending on the specific operation being performed. Ensure that these libraries are properly installed and configured before running the tool.

Definition in file main.cpp.

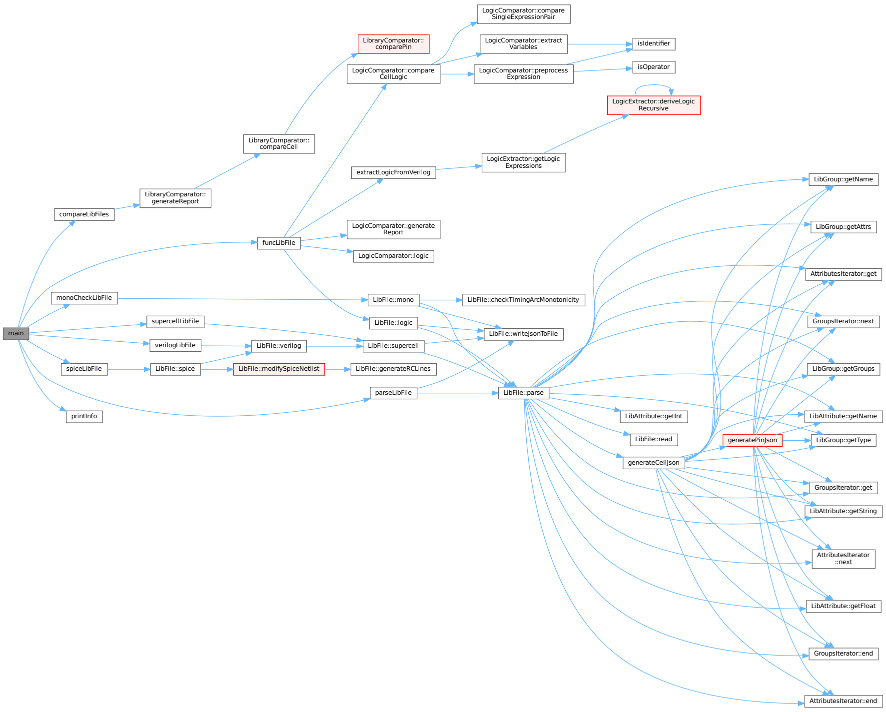
6.42.2 Function Documentation

6.42.2.1 main()

```
int main (
        int argc,
        char * argv[] )
```

Definition at line 48 of file main.cpp.

Here is the call graph for this function:



6.43 main.cpp

Go to the documentation of this file.

```
// ./src/main.cpp

#include "CLI/CLI.hpp"

#include "LibFileOperations.hpp"
#include "version.h"

int main(int argc, char *argv[]) {
  // Parse command line arguments
  std::vector<std::string> library_paths; // Support multiple files
  std::string log_file_name = "";

  // Command line parameter parsing using CLI11
  CLI::App app{APP_NAME};
  app.set_version_flag("-v,--version", APP_VERSION);

  // Add subcommand for parse mode
  CLI::App *parse_cmd =
      app.add_subcommand("parse", "Parse the Liberty file and write JSON to a file");
  parse_cmd->add_option("library_path", library_paths, "Specify the library file to process")
      ->check(CLI::ExistingFile)
      ->required();
  parse_cmd->add_option("-l,--log", log_file_name,
                        "Specify the log file name. Default: <basename>.parse.log");
  parse_cmd->callback([&] {
    printInfo();
    // Check if multi files
    if (library_paths.size() > 1) {
      spdlog::info("Running sequential parsing for {} files.", library_paths.size());
```

```cpp
      spdlog::info("Each library will write to its own log file.");
      // Sequential parsing
      for (const auto &library_path : library_paths) {
        parseLibFile(library_path, log_file_name = "");
      }
    } else {
      parseLibFile(library_paths[0], log_file_name);
    }
  });

  // Add subcommand for mono check mode
  bool is_slew = false;
  CLI::App *mono_cmd = app.add_subcommand("mono", "Check the monotonicity of timing arc values");
  mono_cmd->add_option("library_path", library_paths, "Specify the library file to process")
      ->check(CLI::ExistingFile)
      ->required();
  mono_cmd->add_option("-l,--log", log_file_name,
                       "Specify the log file name. Default: <basename>.mono.log");
  mono_cmd->add_flag("-s,--slew", is_slew,
                     "Specify that monotonicity checks also include input slew.");
  mono_cmd->callback([&] {
    printInfo();
    // Check if multi files
    if (library_paths.size() > 1) {
      spdlog::info("Running multi-threaded monotonicity check for {} files.", library_paths.size());
      spdlog::info("Each thread will write to its own log file.");

      // Sequential json file check
      for (const auto &library_path : library_paths) {
        if (!std::filesystem::exists(std::filesystem::path(library_path).stem().string() +
                                     ".json")) {
          spdlog::info("JSON file not found for '{}'. Parsing Liberty file first.", library_path);
          parseLibFile(library_path, log_file_name = "");
        }
      }
      spdlog::info("All JSON files prepared.");
      // Parallel monotonicity check
      std::vector<std::thread> threads;
      for (const auto &library_path : library_paths) {
        threads.emplace_back(monoCheckLibFile, library_path, log_file_name = "", is_slew);
      }
      for (auto &thread : threads) {
        thread.join();
      }
      spdlog::info("All threads completed.");
    } else {
      monoCheckLibFile(library_paths[0], log_file_name, is_slew);
    }
  });

  // Add subcommand for compare mode
  std::string ref_lib, comp_lib, report_file_name;
  CLI::App *compare_cmd = app.add_subcommand(
      "compare",
      "Compare the comparison library against the reference one and report differences");
  compare_cmd->add_option("--ref", ref_lib, "Specify the reference library file")
      ->check(CLI::ExistingFile)
      ->required();
  compare_cmd->add_option("--comp", comp_lib, "Specify the comparison library file")
      ->check(CLI::ExistingFile)
      ->required();
  double abstol = 0.002;
  compare_cmd->add_option("--abstol", abstol,
                          "Specify the absolute tolerance for comparison. Default: 0.002ns");
  double reltol = 0.02;
  compare_cmd->add_option("--reltol", reltol,
                          "Specify the relative tolerance for comparison. Default: 0.02/2.0%");
  compare_cmd->add_option("--report", report_file_name,
                          "Specify the report file name. Default: <comp_lib>.cmp.md");
```

```
compare_cmd->callback([&] {
  printInfo();
  compareLibFiles(ref_lib, comp_lib, reltol, abstol, report_file_name);
});

// Add subcommand for supercell generation
int chain_length = 1;
CLI::App *supercell_cmd =
    app.add_subcommand("supercell", "Generate supercells for the given Liberty file");
supercell_cmd->add_option("library_path", library_paths, "Specify the library file to process")
    ->check(CLI::ExistingFile)
    ->required();
supercell_cmd->add_option("-l,--log", log_file_name,
                          "Specify the log file name. Default: <basename>.supercell.log");
supercell_cmd->add_option("-c,--chain", chain_length,
                          "Specify the chain length for supercell generation. Default: 1");
std::vector<std::string> cell_names = {}; // "CMPE42D1" "AN2D0", "DFQD1"
supercell_cmd->add_option("--cells", cell_names,
                          "Specify the cell names to generate supercells for");
supercell_cmd->callback([&] {
  printInfo();
  // Check if multi files
  if (library_paths.size() > 1) {
    spdlog::info("Running multi-threaded supercell generation for {} files.",
                 library_paths.size());
    spdlog::info("Each library will write to its own log file.");

    // Sequential json file check
    for (const auto &library_path : library_paths) {
      if (!std::filesystem::exists(std::filesystem::path(library_path).stem().string() +
                                   ".json")) {
        spdlog::info("JSON file not found for '{}'. Parsing Liberty file first.", library_path);
        parseLibFile(library_path, log_file_name = "");
      }
    }
    spdlog::info("All JSON files prepared.");
    // Parallel supercell generation
    std::vector<std::thread> threads;
    for (const auto &library_path : library_paths) {
      threads.emplace_back(supercellLibFile, library_path, log_file_name = "", chain_length,
                           cell_names);
    }
    for (auto &thread : threads) {
      thread.join();
    }
    spdlog::info("All threads completed.");
  } else {
    supercellLibFile(library_paths[0], log_file_name, chain_length, cell_names);
  }
});

// Add subcommand for zlibboost
CLI::App *zlibboost_cmd =
    app.add_subcommand("zlibboost", "ZlibBoost - Multi-threaded Library Processing Tool");
std::string config_dir = "/home/songzx/Projects/zlibboost/config.tcl";
std::string python_dir = "/home/guocj/anaconda3/envs/myenv/bin/python";
std::string main_py_dir = "/home/songzx/Projects/zlibboost/zlibboost.py";
zlibboost_cmd->add_option(
    "-c, --config", config_dir,
    "Specify the configuration TCL file. Default: /home/songzx/Projects/zlibboost/config.tcl");
zlibboost_cmd->add_option(
    "--python", python_dir,
    "Specify the python directory. Default: /home/guocj/anaconda3/envs/myenv/bin/python");
zlibboost_cmd->add_option("--main", main_py_dir,
                          "Specify the main python script directory. Default: "
                          "/home/songzx/Projects/zlibboost/zlibboost.py");
zlibboost_cmd->callback([&] {
  printInfo();
  // Run the ZlibBoost tool
```

```
    std::string command = python_dir + " " + main_py_dir + " -c " + config_dir;
    spdlog::info("Running ZlibBoost with command: '{}'", command);
    int ret = std::system(command.c_str());
    if (ret == 0) {
      spdlog::info("ZlibBoost completed successfully.");
    } else {
      spdlog::error("ZlibBoost failed with return code: {}", ret);
    }
  });

  // Add subcommand for clear
  CLI::App *clear_cmd = app.add_subcommand(
      "clear", "Clear the log, JSON, map, markdown, Verilog, SPICE files in this directory");
  clear_cmd->callback([&] {
    printInfo();
    std::filesystem::path current_dir = std::filesystem::current_path();
    for (const auto &entry : std::filesystem::directory_iterator(current_dir)) {
      if (entry.path().extension() == ".log" || entry.path().extension() == ".json" ||
          entry.path().extension() == ".map" || entry.path().extension() == ".md" ||
          entry.path().extension() == ".v" || entry.path().extension() == ".spi") {
        spdlog::info("Removing file: '{}'", entry.path().string());
        std::filesystem::remove(entry.path());
      }
    }
    spdlog::info("All log, JSON, map, markdown files cleared.");
  });

  // Add subcommand for Verilog generation
  CLI::App *verilog_cmd =
      app.add_subcommand("verilog", "Generate Verilog file for given Liberty file");
  verilog_cmd->add_option("library_path", library_paths, "Specify the library file to process")
      ->check(CLI::ExistingFile)
      ->required();
  verilog_cmd->add_option("-l,--log", log_file_name,
                          "Specify the log file name. Default: <basename>.verilog.log");
  verilog_cmd->add_option("-c,--chain", chain_length,
                          "Specify the chain length for verilog generation. Default: 1");
  verilog_cmd->add_option("--cells", cell_names, "Specify the cell names to generate Verilog for");
  verilog_cmd->callback([&] {
    printInfo();
    // Check if multi files
    if (library_paths.size() > 1) {
      spdlog::info("Running multi-threaded Verilog generation for {} files.", library_paths.size());
      spdlog::info("Each library will write to its own log file.");

      // Sequential json file check
      for (const auto &library_path : library_paths) {
        if (!std::filesystem::exists(std::filesystem::path(library_path).stem().string() +
                                     ".json")) {
          spdlog::info("JSON file not found for '{}'. Parsing Liberty file first.", library_path);
          parseLibFile(library_path, log_file_name = "");
        }
      }
      spdlog::info("All JSON files prepared.");
      // Parallel Verilog generation
      std::vector<std::thread> threads;
      for (const auto &library_path : library_paths) {
        threads.emplace_back(verilogLibFile, library_path, log_file_name = "", chain_length,
                             cell_names);
      }
      for (auto &thread : threads) {
        thread.join();
      }
      spdlog::info("All threads completed.");
    } else {
      verilogLibFile(library_paths[0], log_file_name, chain_length, cell_names);
    }
  });

```

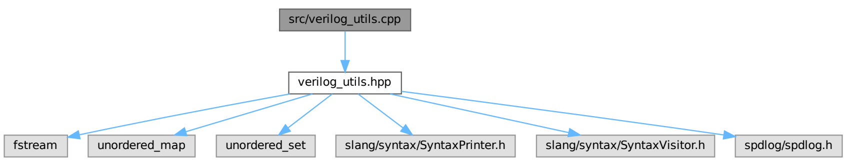
```
// Add subcommand for SPICE generation
CLI::App *spice_cmd = app.add_subcommand("spice", "Generate SPICE file for given Liberty file");
spice_cmd->add_option("library_path", library_paths, "Specify the library file to process")
    ->check(CLI::ExistingFile)
    ->required();
spice_cmd->add_option("-l,--log", log_file_name,
                      "Specify the log file name. Default: <basename>.spice.log");
spice_cmd->add_option("-c,--chain", chain_length,
                      "Specify the chain length for SPICE generation. Default: 1");
spice_cmd->add_option("--cells", cell_names, "Specify the cell names to generate SPICE for");
std::string verilog_lib_file =
    "/home/songzx/examples/mypdk/TSMC65/TSMC65NM_CLN65LP_STDIO_STDCELL/tcbn65lp_220a/"
    "0396011_20170308/TSMCHOME/digital/Front_End/verilog/tcbn65lp.v";
std::string spice_lib_file =
    "/home/songzx/examples/mypdk/TSMC65/TSMC65NM_CLN65LP_STDIO_STDCELL/tcbn65lp_220a/"
    "0396011_20170308/TSMCHOME/digital/Back_End/lpe_spice/tcbn65lp_200a/tcbn65lp_200a_lpe.spi";
spice_cmd->add_option("--vl", verilog_lib_file,
                      "Specify the location of the Verilog primitive library file");
spice_cmd->add_option(
    "--sl", spice_lib_file,
    "Specify the location of the SPICE library file to be included in the output");
spice_cmd->callback([&] {
  printInfo();
  // Check if multi files
  if (library_paths.size() > 1) {
    spdlog::info("Running multi-threaded SPICE generation for {} files.", library_paths.size());
    spdlog::info("Each library will write to its own log file.");

    // Sequential json file check
    for (const auto &library_path : library_paths) {
      if (!std::filesystem::exists(std::filesystem::path(library_path).stem().string() +
                                   ".json")) {
        spdlog::info("JSON file not found for '{}'. Parsing Liberty file first.", library_path);
        parseLibFile(library_path, log_file_name = "");
      }
    }
    spdlog::info("All JSON files prepared.");
    // Parallel SPICE generation
    std::vector<std::thread> threads;
    for (const auto &library_path : library_paths) {
      threads.emplace_back(spiceLibFile, library_path, log_file_name = "", chain_length,
                           cell_names, verilog_lib_file, spice_lib_file);
    }
    for (auto &thread : threads) {
      thread.join();
    }
    spdlog::info("All threads completed.");
  } else {
    spiceLibFile(library_paths[0], log_file_name, chain_length, cell_names, verilog_lib_file,
                 spice_lib_file);
  }
});

// Add subcommand for funtional equivalence check
CLI::App *func_cmd = app.add_subcommand(
    "func", "Check functional equivalence of two Liberty files or Verilog files");
std::string ref_file, comp_file;
func_cmd->add_option("--ref", ref_file, "Specify the reference Liberty or Verilog file")
    ->check(CLI::ExistingFile)
    ->required();
func_cmd->add_option("--comp", comp_file, "Specify the comparison Liberty or Verilog file")
    ->check(CLI::ExistingFile)
    ->required();
func_cmd->add_option("--cells", cell_names,
                     "Specify the cell names to check functional equivalence for");
func_cmd->add_option("--report", report_file_name,
                     "Specify the report file name. Default: <comp_lib>.cmp.md");

func_cmd->callback([&] {
```

```
  printInfo();
  funcLibFile(ref_file, comp_file, cell_names, report_file_name);
 });

 CLI11_PARSE(app, argc, argv);

 // End of program
 char hostname[256];
 gethostname(hostname, sizeof(hostname));

 auto now = std::chrono::system_clock::now();
 auto time_now = std::chrono::system_clock::to_time_t(now);
 std::stringstream ss;
 ss << std::put_time(std::localtime(&time_now), "%c");

 spdlog::info("ZlibValidation exited on '{}' at {}", hostname, ss.str());
 return 0;
}
```

6.44 src/verilog_utils.cpp File Reference

`#include "verilog_utils.hpp"`

Include dependency graph for verilog_utils.cpp:



Functions

- void getAST (const std::string &verilog_file, const std::string &cell)

6.44.1 Function Documentation

6.44.1.1 getAST()

```
void getAST (
        const std::string & verilog_file,
        const std::string & cell )
```

Definition at line 544 of file verilog_utils.cpp.

Here is the call graph for this function:

6.45 verilog_utils.cpp

Go to the documentation of this file.

```
// verilog_utils.cpp

#include "verilog_utils.hpp"

// Adding a generic handler method to record all visited nodes and increment the depth counter
void VerilogVisitor::handle(const slang::syntax::SyntaxNode &node) {
  std::string indent(depth_ * 2, ' ');
  spdlog::debug("{}Node: {}", indent, slang::syntax::toString(node.kind));

  // Increase depth, visit child nodes, then decrease depth
  depth_++;
  visitDefault(node);
  depth_--;
}

// Handle module declarations
void VerilogVisitor::handle(const slang::syntax::ModuleDeclarationSyntax &module) {
  std::string indent(depth_ * 2, ' ');

  if (module.header && module.header->name.valid()) {
    std::string_view moduleName = module.header->name.valueText();
    spdlog::debug("{}Module Name:{}", indent, moduleName);

    inTargetModule_ = !targetCell_.empty() && moduleName == targetCell_;
    if (!inTargetModule_) {
      return;
    }

    // If a specific module name is specified, check if it matches
    if (inTargetModule_) {
      spdlog::info("{}Found target module: {}", indent, targetCell_);
      // Print the module ports
      if (module.header->ports) {
        spdlog::info("{}Ports: {}", indent, module.header->ports->toString());
      }
    }
  }

  // Continue processing child nodes
  depth_++;
  visitDefault(module);
  depth_--;
}

// Handle port declarations
void VerilogVisitor::handle(const slang::syntax::PortDeclarationSyntax &portDecl) {
  if (!inTargetModule_) {
```

```
    return;
  }

  std::string indent = std::string(depth_ * 2, ' ');

  // Get port direction
  std::string direction = "unknown";
  if (portDecl.header) {
    // Try to convert the header to different types of port headers
    if (auto *varPort = portDecl.header->as_if<slang::syntax::VariablePortHeaderSyntax>()) {
      if (varPort->direction) {
        direction = varPort->direction.valueText();
      }
    } else if (auto *netPort = portDecl.header->as_if<slang::syntax::NetPortHeaderSyntax>()) {
      if (netPort->direction) {
        direction = netPort->direction.valueText();
      }
    }

    // Get port name
    for (const auto &declarator : portDecl.declarators) {
      if (declarator->name.valid()) {
        std::string_view portName = declarator->name.valueText();
        spdlog::info("{}Port Name: {} ({})", indent, portName, direction);
      }
    }
  }
}

// Handle hierarchical instantiation (module instance)
void VerilogVisitor::handle(const slang::syntax::HierarchyInstantiationSyntax &hierarchyInst) {
  if (!inTargetModule_) {
    return;
  }

  std::string indent(depth_ * 2, ' ');

  if (hierarchyInst.type.valid()) {
    std::string_view instType = hierarchyInst.type.valueText();
    spdlog::info("{}Hierarchy Instance Type: {}", indent, instType);

    // Safely print parameter values (if any)
    try {
      if (hierarchyInst.parameters) {
        spdlog::info("{}Parameters:", indent);
        for (const auto &param : hierarchyInst.parameters->parameters) {
          if (!param)
            continue; // Null pointer check

          if (auto *orderedParam = param->as_if<slang::syntax::OrderedParamAssignmentSyntax>()) {
            if (orderedParam->expr) {
              spdlog::info("{}  Parameter: {}", indent, orderedParam->expr->toString());
            }
          } else if (auto *namedParam = param->as_if<slang::syntax::NamedParamAssignmentSyntax>()) {
            if (namedParam->name.valid() && namedParam->expr) {
              spdlog::info("{}  Parameter {}: {}", indent, namedParam->name.valueText(),
                           namedParam->expr->toString());
            }
          }
        }
      }
    } catch (const std::exception &e) {
      spdlog::warn("{}Exception while processing parameters: {}", indent, e.what());
    }

    // Safely handle instances
    try {
      for (const auto &instance : hierarchyInst.instances) {
        if (!instance) {
```

```
        spdlog::warn("{}Invalid instance", indent);
        continue;
      } else if (!instance->decl) {
        spdlog::warn("{}Invalid instance declaration", indent);
        // continue;
      } else {
        try {
          if (instance->decl->name.valid()) {
            std::string_view instName = instance->decl->name.valueText();
            spdlog::info("{}Instance Name: {}", indent, instName);
          }
        } catch (...) {
          spdlog::debug("{}Error printing name", indent);
        }

        // Safely print dimensions
        try {
          if (!instance->decl->dimensions.empty()) {
            spdlog::info("{}  Dimensions: {}", indent, instance->decl->dimensions.toString());
          }
        } catch (...) {
          spdlog::debug("{}Error printing dimensions", indent);
        }
      }

      // Safely print port connections
      spdlog::info("{}  Port connections:", indent);
      for (const auto &conn : instance->connections) {
        if (!conn) {
          spdlog::warn("{}    Null connection", indent);
          continue; // Null pointer check
        }

        try {
          // Use as_if method for safe type conversion
          if (auto *ordered = conn->as_if<slang::syntax::OrderedPortConnectionSyntax>()) {
            if (ordered->expr) {
              spdlog::info("{}    Ordered connection: {}", indent, ordered->expr->toString());
            } else {
              spdlog::info("{}    Ordered connection: <empty>", indent);
            }
          } else if (auto *named = conn->as_if<slang::syntax::NamedPortConnectionSyntax>()) {
            if (named->name.valid()) {
              std::string exprStr = named->expr ? named->expr->toString() : "<empty>";
              spdlog::info("{}    .{}({})", indent, named->name.valueText(), exprStr);
            }
          } else if (conn->as_if<slang::syntax::EmptyPortConnectionSyntax>()) {
            spdlog::info("{}    <empty connection>", indent);
          } else if (conn->as_if<slang::syntax::WildcardPortConnectionSyntax>()) {
            spdlog::info("{}    .* (wildcard connection)", indent);
          } else {
            spdlog::info("{}    Unknown connection type: {}", indent,
                         slang::syntax::toString(conn->kind));
          }
        } catch (const std::exception &e) {
          spdlog::warn("{}Exception processing connection: {}", indent, e.what());
        } catch (...) {
          spdlog::warn("{}Unknown exception processing connection", indent);
        }
      }
    }
  } catch (const std::exception &e) {
    spdlog::warn("{}Exception while processing instances: {}", indent, e.what());
  }

  // Continue processing child nodes
  depth_++;
  visitDefault(hierarchyInst);
  depth_--;
```

```
  }
}
/*
// Handle primitive gate instantiation
void VerilogVisitor::handle(const slang::syntax::PrimitiveInstantiationSyntax &primitiveInst) {
if (!inTargetModule_) {
return;
}

std::string indent(depth_ * 2, ' ');

// Get primitive gate type
if (primitiveInst.type.valid()) {
std::string_view gateType = primitiveInst.type.valueText();
spdlog::info("{}Primitive Gate: {}", indent, gateType);

// Print delay information (if any)
if (primitiveInst.delay) {
spdlog::info("{}Delay: {}", indent, primitiveInst.delay->toString());
}

// Print strength information (if any)
if (primitiveInst.strength) {
spdlog::info("{}Strength: {}", indent, primitiveInst.strength->toString());
}

// Print each instance
for (const auto &instance : primitiveInst.instances) {
// Primitive gates usually don't have names, but if they do, print them
if (instance->decl && instance->decl->name.valid()) {
std::string_view instName = instance->decl->name.valueText();
spdlog::info("{}  Gate instance: {}", indent, instName);
}

// Print connections
spdlog::info("{}  Gate connections:", indent);
for (size_t i = 0; i < instance->connections.size(); ++i) {
const auto &conn = instance->connections[i];
// For gate-level instantiation, connections are usually ordered
if (auto *ordered = conn->as_if<slang::syntax::OrderedPortConnectionSyntax>()) {
if (ordered->expr) {
// The first is usually the output, the rest are inputs
std::string portType = (i == 0) ? "output" : "input";
spdlog::info("{}    {} {}: {}", indent, portType, i, ordered->expr->toString());
}
}
}
}

// Continue to traverse deeper when specific standard gate type
// Create a set of safe standard gate types
static const std::unordered_set<std::string_view> safeGateTypes = {
"and",   "or",      "nand",    "nor",      "xor",     "xnor",   "not",
"buf",   "bufif0",  "bufif1",  "notif0",   "notif1",  "pullup", "pulldown",
"cmos",  "rcmos",   "nmos",    "pmos",     "rnmos",   "rpmos",  "tran",
"rtran", "tranif0", "tranif1", "rtranif0", "rtranif1"};

if (safeGateTypes.find(gateType) != safeGateTypes.end()) {
// Continue processing child nodes
depth_++;
visitDefault(primitiveInst);
depth_--;
} else {
spdlog::warn("{}Skipping deeper traversal of primitive: {}", indent, gateType);
}

} else {
spdlog::warn("{}Primitive instantiation missing gate type", indent);
}
```

```
}
*/
// Handle specify block
void VerilogVisitor::handle(const slang::syntax::SpecifyBlockSyntax &specifyBlock) {
  if (!inTargetModule_) {
    return;
  }

  std::string indent(depth_ * 2, ' ');
  spdlog::info("{}Specify Block:", indent);

  // Iterate through all path declarations
  for (const auto &item : specifyBlock.items) {
    if (auto *pathDecl = item->as_if<slang::syntax::PathDeclarationSyntax>()) {
      if (pathDecl->desc) {
        std::string pathSrc;
        std::string pathDst;

        // Get path source
        if (!pathDecl->desc->inputs.empty()) {
          // Get the first input
          if (auto *identifier =
                  pathDecl->desc->inputs[0]->as_if<slang::syntax::IdentifierNameSyntax>()) {
            if (identifier->identifier.valid()) {
              pathSrc = identifier->identifier.valueText();
            }
          }
        }

        // Get path destination
        if (pathDecl->desc->suffix) {
          if (auto *simpleSuffix =
                  pathDecl->desc->suffix->as_if<slang::syntax::SimplePathSuffixSyntax>()) {
            if (!simpleSuffix->outputs.empty()) {
              if (auto *identifier =
                      simpleSuffix->outputs[0]->as_if<slang::syntax::IdentifierNameSyntax>()) {
                if (identifier->identifier.valid()) {
                  pathDst = identifier->identifier.valueText();
                }
              }
            }
          } else if (auto *edgeSuffix =
                           pathDecl->desc->suffix
                               ->as_if<slang::syntax::EdgeSensitivePathSuffixSyntax>()) {
              if (!edgeSuffix->outputs.empty()) {
                if (auto *identifier =
                        edgeSuffix->outputs[0]->as_if<slang::syntax::IdentifierNameSyntax>()) {
                  if (identifier->identifier.valid()) {
                    pathDst = identifier->identifier.valueText();
                  }
                }
              }
            }
          }

          // Construct path string
          std::string pathStr = pathSrc + " => " + pathDst;

          // Get path delay
          std::string delayStr = "(";
          for (size_t i = 0; i < pathDecl->delays.size(); ++i) {
            if (i > 0)
              delayStr += ", ";
            delayStr += pathDecl->delays[i]->toString();
          }
          delayStr += ")";

          spdlog::info("{}  Path: {} = {}", indent, pathStr, delayStr);
        }
      }
```

```
      // Can add handling for ConditionalPathDeclarationSyntax and IfNonePathDeclarationSyntax
      else if (auto *condPath = item->as_if<slang::syntax::ConditionalPathDeclarationSyntax>()) {
        if (condPath->path && condPath->predicate) {
          spdlog::info("{}  Conditional Path (if {})", indent, condPath->predicate->toString());
          // Recursively handle internal path
          handle(*condPath->path);
        }
      } else if (auto *ifNonePath = item->as_if<slang::syntax::IfNonePathDeclarationSyntax>()) {
        if (ifNonePath->path) {
          spdlog::info("{}  If-None Path", indent);
          // Recursively handle internal path
          handle(*ifNonePath->path);
        }
      }
    }

    // Continue processing child nodes
    depth_++;
    visitDefault(specifyBlock);
    depth_--;
  }
}

// Handle module declarations, only keep the target module
void CellExtractor::handle(const slang::syntax::ModuleDeclarationSyntax &module) {
  if (module.header && module.header->name.valid()) {
    std::string_view moduleName = module.header->name.valueText();

    // If it is the target module, mark as found, otherwise remove it
    if (!targetCell_.empty() && moduleName == targetCell_) {
      foundTarget_ = true;
      // Do not modify, keep this module
    } else {
      // Remove non-target modules
      remove(module);
    }
  }
}

// Get result
bool CellExtractor::foundTargetCell()const { return foundTarget_; }

// Handle module declarations
void CellPrinter::handle(const slang::syntax::ModuleDeclarationSyntax &module) {
  if (module.header && module.header->name.valid()) {
    std::string_view moduleName = module.header->name.valueText();

    // Check if it is the target module
    if (!targetCell_.empty() && moduleName == targetCell_) {
      foundTarget_ = true;

      // Print module definition
      out_ << "`timescale 1ns/10ps\n";
      out_ << module.toString();

      return; // Do not traverse further
    }
  }

  // Continue traversing other modules
  visitDefault(module);
}

void ModuleRewriter::handle(const slang::syntax::SyntaxNode &node) {
  std::string indent(depth_ * 2, ' ');
  logger_->debug("{}Node: {}", indent, slang::syntax::toString(node.kind));

  // Continue processing child nodes
  depth_++;
```

```
  visitDefault(node);
  depth_--;
}

void ModuleRewriter::handle(const slang::syntax::ModuleDeclarationSyntax &module) {
  logger_->debug("Processing module: {}", module.header->name.valueText());

  // Create intermediate wires for connections between instances
  for (int i = 0; i < instance_count_ - 1; i++) {
    auto &newNetNode = parse("\n  wire OP_" + std::to_string(i) + ";");
    insertAtBack(module.members, newNetNode);
    logger_->debug("Added intermediate wire: {}", newNetNode.toString());
  }

  // Get critical input port & output port from
  // the module name pattern: CELLNAME__X#__CRITICALPORT__OUTPUTPORT
  std::string criticalInputPort = "";
  std::string criticalOutputPort = "";
  if (!this->moduleName_.empty()) {
    size_t firstSep = this->moduleName_.find("__");
    if (firstSep != std::string::npos) {
      size_t secondSep = this->moduleName_.find("__", firstSep + 2);
      if (secondSep != std::string::npos) {
        size_t thirdSep = this->moduleName_.find("__", secondSep + 2);
        if (thirdSep != std::string::npos) {
          criticalInputPort = this->moduleName_.substr(secondSep + 2, thirdSep - (secondSep + 2));
          criticalOutputPort = this->moduleName_.substr(thirdSep + 2);
          logger_->debug("Extracted critical input port: {}", criticalInputPort);
          logger_->debug("Extracted output port: {}", criticalOutputPort);
        } else {
          logger_->warn("Can't find thirdSep. Invalid module name pattern: {}", this->moduleName_);
          return;
        }
      } else {
        logger_->warn("Can't find secondSep. Invalid module name pattern: {}", this->moduleName_);
        return;
      }
    } else {
      logger_->warn("Can't find firstSep. Invalid module name pattern: {}", this->moduleName_);
      return;
    }
  } else {
    logger_->warn("Module name is empty. Can't extract critical input port.");
    return;
  }

  // Add additional wires for intermediate outputs that aren't part of the chain
  if (this->outputPins_.size() > 1) {                // Only add if there are multiple output pins
    for (int i = 0; i < instance_count_ - 1; i++) { // Skip the first and last instances
      for (const auto &outputPin : this->outputPins_) {
        if (outputPin != criticalOutputPort) { // Found an intermediate output pin
          auto &newNetNode = parse("\n  wire P_" + std::to_string(i) + "__" + outputPin + ";");
          insertAtBack(module.members, newNetNode);
          logger_->debug("Added intermediate wire: {}", newNetNode.toString());
        }
      }
    }
  }

  std::string portList_str = module.header->ports->toString();
  logger_->debug("Port list: {}", portList_str);

  std::vector<std::string> allPorts;
  auto ansiPortList = module.header->ports->as_if<slang::syntax::AnsiPortListSyntax>();
  if (!ansiPortList) {
    logger_->warn("Port list is not ANSI style.");
    return;
  }
  portInfoMap_.clear(); // Clear the port info map
```

```
for (const auto portMember : ansiPortList->ports) {
  if (portMember->kind == slang::syntax::SyntaxKind::ImplicitAnsiPort) {
    const auto implicitPort = portMember->as_if<slang::syntax::ImplicitAnsiPortSyntax>();
    const auto &directionToken =
        implicitPort->header->as_if<slang::syntax::VariablePortHeaderSyntax>()->direction;
    const auto &nameToken = implicitPort->declarator->name;

    std::string_view portName = nameToken.valueText();
    std::string_view direction = directionToken.valueText();
    portInfoMap_[std::string(portName)] = std::string(direction); // Store port info in the map
    logger_->debug("Port Name: {}, Direction: {}", portName, direction);
  } else {
    logger_->warn("Port member is not ImplicitAnsiPort.");
  }
}

for (int i = 0; i < instance_count_; i++) {
  std::string instanceName = "I_" + this->cellName_ + "__X" + std::to_string(i) + "__" +
                             criticalInputPort + "__" + criticalOutputPort;
  std::string instanceCode = "\n  " + this->cellName_ + " " + instanceName + " (";
  std::vector<std::string> portConnections;

  for (const auto &pair : portInfoMap_) {
    std::string portName = pair.first;
    std::string direction = pair.second;
    std::string connectionName;

    if (portName == criticalInputPort) {
      if (i == 0) {
        connectionName = portName; // First instance: connect to module input port
      } else {
        connectionName =
            "OP_" + std::to_string(i - 1); // Intermediate instances: connect to previous OP wire
      }
    } else if (portName == criticalOutputPort) {
      if (i == instance_count_ - 1) {
        connectionName = portName; // Last instance: connect to module output port
      } else {
        connectionName = "OP_" + std::to_string(i); // Intermediate instances: connect to OP wire
      }
    } else if (direction == "output") { // Handle other output ports
      if (i < instance_count_ - 1) {
        connectionName = "P_" + std::to_string(i) + "__" +
                         portName; // Connect to intermediate P_i__portName wire
      } else {
        connectionName = portName; // Last instance: connect to module output port
      }
    } else { // For input ports (and inout?), connect directly to module port
      connectionName = portName;
    }
    portConnections.push_back("." + portName + "(" + connectionName + ")");
  }

  // Connect all ports with comma and space
  if (!portConnections.empty()) {
    instanceCode += portConnections[0];
    for (size_t i = 1; i < portConnections.size(); ++i) {
      instanceCode += ", " + portConnections[i];
    }
  }
  instanceCode += ");";

  // Blank line before the first instance
  if (i == 0) {
    instanceCode = "\n" + instanceCode;
  }

  // Insert the instance code
  auto &instanceNode = parse(instanceCode);
```

```
    insertAtBack(module.members, instanceNode);
  }
}

void getAST(const std::string &verilog_file, const std::string &cell) {
  try {
    spdlog::info("Starting get AST from Verilog file: '{}'", verilog_file);
    auto result = slang::syntax::SyntaxTree::fromFile(verilog_file);
    if (result) {
      spdlog::info("Successfully parsed Verilog file.");

      try {
        // First, use the visitor to print basic information
        VerilogVisitor visitor(cell);
        visitor.visit(result.value()->root());

        // If a target cell is specified, use the rewriter to extract the relevant code
        if (!cell.empty()) {
          // Create a rewriter to only keep the target cell related code
          CellExtractor extractor(cell);
          auto extractedTree = extractor.transform(result.value());

          if (extractor.foundTargetCell()) {
            // Use SyntaxPrinter to print the extracted code
            std::string extractedCode = slang::syntax::SyntaxPrinter::printFile(*extractedTree);

            // Save to a file named after the cell name
            std::string outputFile = cell + ".v";
            std::ofstream cellOut(outputFile);
            if (cellOut) {
              cellOut << extractedCode;
              cellOut.close();
              spdlog::info("Extracted '{}' cell code to '{}'", cell, outputFile);
            } else {
              spdlog::error("Failed to write extracted cell code to '{}'", outputFile);
            }
          } else {
            spdlog::warn("Target cell '{}' not found in the Verilog file", cell);
          }
        }

        // spdlog::info("Print full source code to 'full_source_code.v'");
        // // Optionally save the entire syntax tree
        // std::string fullOutput = slang::syntax::SyntaxPrinter::printFile(*result.value());
        // std::ofstream out("full_source_code.v");
        // out << fullOutput;
        // out.close();

        spdlog::info("Print target cell code to 'cell_code.v using toString()'");
        // Use CellPrinter to print the target cell's code
        std::ofstream cellOut("cell_code.v");
        CellPrinter cellPrinter(cell, cellOut);
        cellPrinter.visit(result.value()->root());
        cellOut.close();

      } catch (const std::exception &e) {
        spdlog::error("Exception during AST traversal: {}", e.what());
      } catch (...) {
        spdlog::error("Unknown exception during AST traversal");
      }
    } else {
      spdlog::error("Error parsing Verilog file.");
    }
  } catch (const std::exception &e) {
    spdlog::error("Exception during Verilog parsing: {}", e.what());
  } catch (...) {
    spdlog::error("Unknown exception during Verilog parsing");
  }
}
```

索引

~AttributesIterator
AttributesIterator, 24
~GroupsIterator
GroupsIterator, 33
~LibAttribute
LibAttribute, 36
~LibFile
LibFile, 42
~LibGroup
LibGroup, 65
~ValuesIterator
ValuesIterator, 108

abstol_
LibraryComparator, 76
all_pin_results_
LogicComparator, 87
APP_AUTHOR
version.h, 163
APP_CONTACT
version.h, 163
APP_NAME
version.h, 164
APP_VERSION
version.h, 164
APP_VERSION_MAJOR
version.h, 164
APP_VERSION_MINOR
version.h, 164
APP_VERSION_PATCH
version.h, 164
are_equivalent
PinComparisonResult, 105
attr_
AttributesIterator, 25
LibAttribute, 40
AttributesIterator, 23
~AttributesIterator, 24
attr_, 25
AttributesIterator, 23
attrs_, 25
end, 24
err_, 26
get, 24
next, 25
attrs_
AttributesIterator, 25

basename_
LibFile, 61
bool_
ValuesIterator, 110
BUILD_TIMESTAMP
version.h, 165

cell_name_
LogicComparator, 87
CellExtractor, 26
CellExtractor, 27
foundTarget_, 28
foundTargetCell, 28
handle, 28
targetCell_, 28
cellName_
ModuleRewriter, 103
CellPrinter, 29
CellPrinter, 30
foundTarget_, 31
handle, 30
out_, 31
targetCell_, 31
checkTimingArcMonotonicity
LibFile, 43

comp_compiles
PinComparisonResult, 105
comp_expr_processed
PinComparisonResult, 105
comp_expr_raw
PinComparisonResult, 106
comp_json_
LibraryComparator, 76
comp_lib_path_
LibraryComparator, 76
comp_outpin_map_
LogicComparator, 87
comp_truth_table
PinComparisonResult, 106
compareCell
LibraryComparator, 70
compareCellLogic
LogicComparator, 78
compareLibFiles
LibFileOperations.cpp, 200
LibFileOperations.hpp, 134
compareLut
LibraryComparator, 71
comparePin
LibraryComparator, 72
compareSingleExpressionPair
LogicComparator, 80
compareTimingArc
LibraryComparator, 73
comparison_possible
PinComparisonResult, 106

depth_
ModuleRewriter, 103
VerilogVisitor, 114
deriveLogicRecursive
LogicExtractor, 90

end
AttributesIterator, 24
GroupsIterator, 34
ValuesIterator, 109
err_
AttributesIterator, 26
GroupsIterator, 35
LibAttribute, 40
LibFile, 61
LibGroup, 67
ValuesIterator, 110
error_message
PinComparisonResult, 106
exprp_
ValuesIterator, 110
extractAndPrintNetlistInfo
LogicExtractor.cpp, 242
LogicExtractor.hpp, 156
extractLogicFromVerilog
LogicExtractor.cpp, 243
LogicExtractor.hpp, 157
extractVariables
LogicComparator, 81

filename_
LibFile, 62
filepath_
LibFile, 62
float_
ValuesIterator, 110
formatExpression
LogicExtractor, 92
foundTarget_
CellExtractor, 28
CellPrinter, 31
foundTargetCell
CellExtractor, 28
funcLibFile
LibFileOperations.cpp, 201
LibFileOperations.hpp, 136

GateInfo, 31
gateTypeName, 32
inputSignals, 32
kind, 32
outputSignal, 32
gateOutputDrivers_
LogicExtractor, 98
gateTypeName
GateInfo, 32

generateCellJson
json_utils.cpp, 168
json_utils.hpp, 121
generateLutJson
json_utils.cpp, 169
json_utils.hpp, 122
generatePinJson
json_utils.cpp, 171
json_utils.hpp, 124
generatePowerJson
json_utils.cpp, 174
json_utils.hpp, 127
generateRCLines
LibFile, 44
generateReport
LibraryComparator, 75
LogicComparator, 83
generateTimingJson
json_utils.cpp, 175
get
AttributesIterator, 24
GroupsIterator, 34
getAST
verilog_utils.cpp, 261
verilog_utils.hpp, 161
getAttrs
LibGroup, 65
getBoolean
LibAttribute, 37
getExtractedGates
LogicExtractor, 92
getFloat
LibAttribute, 37
getGroups
LibGroup, 65
getInt
LibAttribute, 37
getInternalWires
LogicExtractor, 93
getLogicExpressions
LogicExtractor, 93
getName
LibAttribute, 38
LibGroup, 66
getPrimaryInputs
LogicExtractor, 94
getPrimaryOutputs
LogicExtractor, 94
getString
LibAttribute, 38
getType
LibGroup, 66
getValues
LibAttribute, 39
group_
GroupsIterator, 35
LibGroup, 67
groups_
GroupsIterator, 35
GroupsIterator, 33
~GroupsIterator, 33
end, 34
err_, 35
get, 34
group_, 35
groups_, 35
GroupsIterator, 33
next, 34

handle
CellExtractor, 28
CellPrinter, 30
LogicExtractor, 95–98
ModuleRewriter, 102
VerilogVisitor, 113, 114

include/Iterators.hpp, 117, 118
include/json_utils.hpp, 119, 128
include/LibAttribute.hpp, 129, 130
include/LibFile.hpp, 131, 132
include/LibFileOperations.hpp, 133, 148
include/LibGroup.hpp, 149, 150
include/LibraryComparator.hpp, 151, 152
include/LogicComparator.hpp, 153, 154
include/LogicExtractor.hpp, 155, 158
include/verilog_utils.hpp, 160, 161
include/version.h, 163, 165

inputPins_
ModuleRewriter, 103
inputSignals
GateInfo, 32
instance_count_
ModuleRewriter, 103
int_
ValuesIterator, 110
inTargetModule_
LogicExtractor, 98
VerilogVisitor, 115
internalWires_
LogicExtractor, 99
isComplex
LibAttribute, 39
isIdentifier
LogicComparator.cpp, 226
isOperator
LogicComparator.cpp, 226

json
json_utils.hpp, 120
LibFile.hpp, 132
LibraryComparator.hpp, 152
json_utils.cpp
generateCellJson, 168
generateLutJson, 169
generatePinJson, 171
generatePowerJson, 174
generateTimingJson, 175
parseStringToVector, 177
json_utils.hpp
generateCellJson, 121
generateLutJson, 122
generatePinJson, 124
generatePowerJson, 127
json, 120
jsonname_
LibFile, 62

kind
GateInfo, 32

lib_json_
LibFile, 62
LibAttribute, 36
~LibAttribute, 36
attr_, 40
err_, 40
getBoolean, 37
getFloat, 37
getInt, 37
getName, 38
getString, 38
getValues, 39
isComplex, 39
LibAttribute, 36
LibFile, 40
~LibFile, 42
basename_, 61
checkTimingArcMonotonicity, 43
err_, 61
filename_, 62
filepath_, 62
generateRCLines, 44
jsonname_, 62
lib_json_, 62
LibFile, 42
libname_, 62
logger_, 63
loggername_, 63
logic, 45
modify, 47
modifySpiceNetlist, 47
mono, 48
parse, 50
process_, 63
read, 53
spice, 54
splitString, 56
supercell, 56
temperature_, 63
verilog, 59
voltage_, 63
writeJsonToFile, 60
LibFile.hpp
json, 132

LibFileOperations.cpp
compareLibFiles, 200
funcLibFile, 201
monoCheckLibFile, 203
parseLibFile, 205
printInfo, 208
spiceLibFile, 208
supercellLibFile, 210
verilogLibFile, 212
LibFileOperations.hpp
compareLibFiles, 134
funcLibFile, 136
monoCheckLibFile, 138
parseLibFile, 139
printInfo, 142
spiceLibFile, 142
supercellLibFile, 144
verilogLibFile, 146
LibGroup, 64
~LibGroup, 65
err_, 67
getAttrs, 65
getGroups, 65
getName, 66
getType, 66
group_, 67
LibGroup, 64
libname_
LibFile, 62
LibraryComparator, 67
abstol_, 76
comp_json_, 76
comp_lib_path_, 76
compareCell, 70
compareLut, 71
comparePin, 72
compareTimingArc, 73
generateReport, 75
LibraryComparator, 69
ref_json_, 76
ref_lib_path_, 76
reltol_, 77
LibraryComparator.hpp
json, 152
logger_
LibFile, 63
ModuleRewriter, 103
loggername_
LibFile, 63
logic
LibFile, 45
LogicComparator, 85
logicCache_
LogicExtractor, 99
LogicComparator, 77
all_pin_results_, 87
cell_name_, 87
comp_outpin_map_, 87
compareCellLogic, 78
compareSingleExpressionPair, 80
extractVariables, 81
generateReport, 83
logic, 85
LogicComparator, 78
preprocessExpression, 85
ref_outpin_map_, 88
LogicComparator.cpp
isIdentifier, 226
isOperator, 226
LogicExtractor, 88
deriveLogicRecursive, 90
formatExpression, 92
gateOutputDrivers_, 98
getExtractedGates, 92
getInternalWires, 93
getLogicExpressions, 93
getPrimaryInputs, 94
getPrimaryOutputs, 94
handle, 95–98
inTargetModule_, 98
internalWires_, 99
logicCache_, 99
LogicExtractor, 90
parsingComplete_, 99
portDirections_, 99
primaryInputs_, 99

primaryOutputs_, 100
targetCell_, 100
LogicExtractor.cpp
extractAndPrintNetlistInfo, 242
extractLogicFromVerilog, 243
LogicExtractor.hpp
extractAndPrintNetlistInfo, 156
extractLogicFromVerilog, 157

main
main.cpp, 255
main.cpp
main, 255
modify
LibFile, 47
modifySpiceNetlist
LibFile, 47
moduleName_
ModuleRewriter, 104
ModuleRewriter, 100
cellName_, 103
depth_, 103
handle, 102
inputPins_, 103
instance_count_, 103
logger_, 103
moduleName_, 104
ModuleRewriter, 102
outputPins_, 104
portInfoMap_, 104
mono
LibFile, 48
monoCheckLibFile
LibFileOperations.cpp, 203
LibFileOperations.hpp, 138

next
AttributesIterator, 25
GroupsIterator, 34
ValuesIterator, 109

out_
CellPrinter, 31
outputPins_
ModuleRewriter, 104
outputSignal
GateInfo, 32

parse
LibFile, 50
parseLibFile
LibFileOperations.cpp, 205
LibFileOperations.hpp, 139
parseStringToVector
json_utils.cpp, 177
parsingComplete_
LogicExtractor, 99
pin_name
PinComparisonResult, 106
PinComparisonResult, 104
are_equivalent, 105
comp_compiles, 105
comp_expr_processed, 105
comp_expr_raw, 106
comp_truth_table, 106
comparison_possible, 106
error_message, 106
pin_name, 106
ref_compiles, 107
ref_expr_processed, 107
ref_expr_raw, 107
ref_truth_table, 107
portDirections_
LogicExtractor, 99
portInfoMap_
ModuleRewriter, 104
preprocessExpression
LogicComparator, 85
primaryInputs_
LogicExtractor, 99
primaryOutputs_
LogicExtractor, 100
printInfo
LibFileOperations.cpp, 208
LibFileOperations.hpp, 142
process_
LibFile, 63

read
LibFile, 53
README.md, 166
ref_compiles
PinComparisonResult, 107
ref_expr_processed
PinComparisonResult, 107
ref_expr_raw
PinComparisonResult, 107
ref_json_
LibraryComparator, 76
ref_lib_path_
LibraryComparator, 76
ref_outpin_map_
LogicComparator, 88
ref_truth_table
PinComparisonResult, 107
reltol_
LibraryComparator, 77

spice
LibFile, 54
spiceLibFile
LibFileOperations.cpp, 208
LibFileOperations.hpp, 142
splitString
LibFile, 56
src/Iterators.cpp, 166
src/json_utils.cpp, 167, 178
src/LibAtrribute.cpp, 181
src/LibFile.cpp, 182
src/LibFileOperations.cpp, 199, 214
src/LibGroup.cpp, 219
src/LibraryComparator.cpp, 220
src/LogicComparator.cpp, 225, 227
src/LogicExtractor.cpp, 242, 244
src/main.cpp, 253, 256
src/verilog_utils.cpp, 261, 262
str_
ValuesIterator, 110
supercell
LibFile, 56
supercellLibFile
LibFileOperations.cpp, 210
LibFileOperations.hpp, 144

targetCell_
CellExtractor, 28
CellPrinter, 31
LogicExtractor, 100
VerilogVisitor, 115
temperature_
LibFile, 63

values_
ValuesIterator, 111
ValuesIterator, 108
~ValuesIterator, 108
bool_, 110
end, 109
err_, 110
exprp_, 110
float_, 110
int_, 110
next, 109
str_, 110
values_, 111
ValuesIterator, 108
vtype_, 111
verilog
LibFile, 59
verilog_utils.cpp
getAST, 261
verilog_utils.hpp
getAST, 161
verilogLibFile
LibFileOperations.cpp, 212
LibFileOperations.hpp, 146
VerilogVisitor, 111
depth_, 114
handle, 113, 114
inTargetModule_, 115
targetCell_, 115
VerilogVisitor, 112
version.h
APP_AUTHOR, 163
APP_CONTACT, 163

APP_NAME, 164
APP_VERSION, 164
APP_VERSION_MAJOR, 164
APP_VERSION_MINOR, 164
APP_VERSION_PATCH, 164
BUILD_TIMESTAMP, 165
voltage_
LibFile, 63
vtype_
ValuesIterator, 111

writeJsonToFile
LibFile, 60